[UNIVERSITY NAME]

[School / Department of …]

Study on the Parameter Optimization and Simulation of the Pedestrian Motion Tracking Model (Using Object Detection Deep Neural Network and ByteTrack Algorithm) Based on Reinforcement Learning

A thesis submitted in partial fulfilment of the requirements for the degree of

Master of [Engineering / Science]

by

[AUTHOR NAME]

Supervisor: [SUPERVISOR NAME]

[Month Year]

Declaration

I hereby declare that this thesis is my own work and that, to the best of my knowledge, it contains no material previously published or written by another person, nor material which has been accepted for the award of any other degree, except where due acknowledgement is made in the text. All software developed for this study, the simulation models, the data generation pipeline and the experiment scripts are described in the thesis and its appendices.

Signature: _____ Date: _____

Acknowledgements

[Acknowledgements to the supervisor, colleagues, family and funding bodies — to be written by the author.]

Table of Contents

List of Figures x

List of Tables xvii

List of Abbreviations xx

Chapter 1 Introduction 1

1.1 Research Background 1

1.1.1 Pedestrian Monitoring, Crowd Safety and Video Analytics 1

1.1.2 Tracking-by-Detection and the Role of Its Parameters 2

1.1.3 The Cost of Tuning: Ground Truth and Annotation 3

1.1.4 Simulation as a Source of Perfectly Labelled Data 4

1.2 Problem Statement 5

1.3 Research Objectives and Research Questions 10

1.4 Contributions 11

1.5 Scope and Limitations 13

1.6 Thesis Organisation 14

Chapter 2 Literature Review 17

2.1 Deep-Learning Object Detection 17

2.1.1 Two-Stage Region-Based Detectors 17

2.1.2 One-Stage, Anchor-Free and Transformer-Based Detectors 18

2.1.3 The YOLO Family of Real-Time Detectors 19

2.1.4 YOLOv8: Architecture, Training Objective and Post-Processing 21

2.1.5 Transfer Learning and Fine-Tuning for Pedestrian Detection 23

2.2 Multi-Object Tracking 23

2.2.1 Problem Formulation and Paradigms 23

2.2.2 Kalman Filtering, Assignment and the SORT Lineage 24

2.2.3 ByteTrack: Associating Every Detection Box 25

2.2.4 Successors of ByteTrack 26

2.2.5 Joint Detection-and-Tracking and Transformer-Based Trackers 27
2.2.6 Benchmarks and Evaluation Metrics 28
2.3 Pedestrian Dynamics Modelling and Simulation 29
2.3.1 Modelling Scales and Representations 29
2.3.2 The Social Force Model 30
2.3.3 Cellular-Automaton, Velocity-Based and Heuristic Models 31
2.3.4 The Fundamental Diagram 31
2.3.5 The AnyLogic Pedestrian Library and Its Python Interfaces 32
2.4 Synthetic Data and Simulation for Computer Vision 33
2.4.1 Virtual Worlds as Sources of Labelled Data 33
2.4.2 The Domain Gap: Randomisation and Adaptation 33
2.4.3 Synthetic Pedestrian and Crowd Datasets 34
2.5 Hyper-Parameter Optimisation 35
2.5.1 Problem Formulation 35
2.5.2 Grid Search and Random Search 36
2.5.3 Bayesian Optimisation and TPE 36
2.5.4 Evolution Strategies and CMA-ES 37
2.5.5 Multi-Fidelity Methods and Population-Based Training 37
2.5.6 Hyper-Parameter Optimisation of Multi-Object Trackers 37
2.6 Reinforcement Learning 38
2.6.1 Markov Decision Processes 38
2.6.2 Value-Based, Policy-Gradient and Actor-Critic Methods 39
2.6.3 Proximal Policy Optimisation 39
2.6.4 Contextual Bandits and One-Shot Decisions 40
2.6.5 Reinforcement Learning for Parameter Tuning and Visual Tracking 40
2.7 Summary and Research Gap 41
Chapter 3 Theoretical Foundations 44

3.1 Object Detection with YOLOv8 44
3.1.1 Dense One-Stage, Anchor-Free Prediction 44
3.1.2 Network Architecture 45
3.1.3 Box Representation with Distribution Focal Loss 47
3.1.4 Training Objective and Task-Aligned Assignment 48
3.1.5 Confidence Threshold and Non-Maximum Suppression 50
3.1.6 Transfer Learning and Domain Shift 50
3.2 Multi-Object Tracking by Detection and ByteTrack 52
3.2.1 Problem Definition 52
3.2.2 Kalman Filter Motion Model 52
3.2.3 Association Costs and the Assignment Problem 54
3.2.4 The BYTE Two-Stage Association 55
3.2.5 Track Life Cycle 57
3.2.6 The Tuned Parameter Vector 58
3.3 Pedestrian Dynamics: The Social Force Model 58
3.3.1 Equation of Motion and Driving Term 59
3.3.2 Pedestrian Interaction: Elliptical Specification II 60
3.3.3 Lateral Evasion 61
3.3.4 Body Contact, Sliding Friction and Damping 62
3.3.5 Walls and Obstacles 62
3.3.6 Kinematic Constraints and Numerical Integration 63
3.3.7 The Fundamental Diagram 63
3.4 Evaluation Metrics 64
3.4.1 CLEAR MOT Metrics 64
3.4.2 Identity Metrics 65
3.4.3 Higher Order Tracking Accuracy 65
3.4.4 Combination over Sequences and Use as Optimisation Signal 68

3.5 Reinforcement Learning 68
3.5.1 Markov Decision Processes 68
3.5.2 Contextual Bandits as One-Step MDPs 70
3.5.3 The Policy Gradient Theorem and Gaussian Policies 70
3.5.4 Actor-Critic Methods and Generalised Advantage Estimation 71
3.5.5 Proximal Policy Optimisation 72
3.5.6 Why PPO for a Continuous Seven-Dimensional Action 74
3.6 Hyper-Parameter Optimisation Baselines 75
3.6.1 The Black-Box Objective 75
3.6.2 Grid Search and Random Search 75
3.6.3 Tree-Structured Parzen Estimator 76
3.6.4 Covariance Matrix Adaptation Evolution Strategy 76
3.6.5 Optimising Versus Learning to Optimise 77
3.7 Chapter Summary 78
Chapter 4 System Design and Simulation Model 79
4.1 Overall Architecture and Closed Loop 79
4.2 AnyLogic Pedestrian Model Specification 81
4.2.1 Agent Types and Flowcharts 82
4.2.2 Space Markup, Logging and 3D Camera 83
4.2.3 Scenarios 84
4.3 Python Co-Simulation of the AnyLogic Scenarios 86
4.3.1 Social-Force Dynamics 86
4.3.2 Arrivals and Navigation 87
4.3.3 Model Verification and Calibration 88
4.4 Virtual Camera and Ground Truth 90
4.4.1 Camera Model 90
4.4.2 Rendering and Sensor Effects 92

4.4.3 Ground Truth and Ignore Regions 92
4.5 Synthetic Dataset 94
4.6 Detector 98
4.7 Tracker Integration 100
4.7.1 ByteTrack Implementation and Verification 100
4.7.2 Parameter Space 100
4.8 Evaluation Module 101
4.9 Reinforcement-Learning Environment Design 103
4.9.1 Formulation A: Contextual Parameter Selection 104
4.9.2 Formulation B: Adaptive Online Control 106
4.9.3 PPO Hyper-Parameters and Budget Matching 107
4.10 Baseline Optimisers and Fairness 108
4.11 Implementation and Computational Setup 110
4.12 Chapter Summary 111
Chapter 5 Experiments and Results 112
5.1 Experimental Setup and Protocol 112
5.2 Transfer Learning of the YOLOv8 Detector (RQ1) 113
5.2.1 Training Behaviour 114
5.2.2 Accuracy on the Unseen Test Split 115
5.2.3 Confidence, Visibility and the Two-Stage Association 118
5.3 Parameter Sensitivity and Scenario Dependence (RQ2) 120
5.3.1 One-at-a-Time Sensitivity 120
5.3.2 Response Surfaces 122
5.3.3 Scenario Dependence of the Optimum 125
5.4 Optimisation on the Training Objective 127
5.4.1 Baseline Optimisers 127
5.4.2 Training of the PPO Agents 129

5.5 Results on the Test Splits (RQ3, RQ4) 131
5.5.1 Overall Accuracy 131
5.5.2 Per-Condition Gains 134
5.5.3 Per-Sequence Gains and Significance 136
5.6 Analysis of the Learned Policies 140
5.6.1 Parameters Selected by PPO-Contextual 140
5.6.2 Behaviour of PPO-Adaptive on a Dynamic Sequence 144
5.7 Influence of the Detector on Tracking 146
5.8 Error Analysis 148
5.8.1 Decomposition of HOTA 148
5.8.2 The Trade-Off between FP/FN and Identity Switches 149
5.8.3 Qualitative Comparison 151
5.9 Computational Cost 152
5.10 Summary 154
References 155
Appendix A AnyLogic Model Specification and Code 168
A.1 Model Elements 168
A.2 Java Code of the Agent Type Main 169
A.3 Converting AnyLogic Exports to the Pipeline Format 171
A.4 Closed-Loop Connection with Alpyne 172
Appendix B Software Structure and Reproducibility 174

List of Figures

Figure 1.1. Closed-loop research framework of this thesis. Sixteen conditions (four scenario layouts at three densities, plus one dynamic-density condition per layout) are specified as AnyLogic scenarios and executed in the experiments by a Python social-force co-simulation of the same models. A virtual camera (1280 × 720 pixels, 15 frames per second (FPS)) renders the trajectories into H.264 video with pixel-exact ground truth. Frames of a dedicated detector-training split fine-tune a COCO-pretrained YOLOv8 detector offline; the fine-tuned YOLOv8s feeds the optimisation loop. Simulation, rendering and detection are computed once and cached, whereas NMS with ByteTrack, the evaluation against the ground truth (HOTA, MOTA, IDF1) and the PPO agent form the loop repeated for every parameter vector θ. The HPO baselines use the same loop with an optimiser in place of the PPO agent. 6

Figure 1.2. Organisation of the thesis into background (Chapters 1–2), foundations and design (Chapters 3–4), experiments (Chapter 5) and synthesis (Chapters 6–7)..... 15

Figure 2.1. Taxonomy of deep-learning object detectors (two-stage, one-stage anchor-based, one-stage anchor-free, transformer-based), including Mask R-CNN [68], Cascade R-CNN [69], CornerNet [73], FCOS [74], CenterNet [75], Deformable DETR [78] and RT-DETR [79]; years are those of the peer-reviewed venue, otherwise of the first public release. 19

Figure 2.2. Evolution of the YOLO family of real-time detectors (versions are evenly spaced, so the time axis is not to scale; years are those of the peer-reviewed venue, otherwise of the first public release)..... 20

Figure 2.3. Multi-object tracking paradigms: tracking-by-detection, joint detection and tracking, and query-based transformers..... 24

Figure 2.4. Classification of pedestrian dynamics models by scale (macroscopic, mesoscopic, microscopic) and representation, with key references [47], [121], [122], [123], [127], [128], [129], [130]..... 30

Figure 2.5. Families of hyper-parameter optimisation methods and how they use the evaluation feedback; in panel (d), A is the contextual formulation (HOTA gain over the default parameters on a clip) and B the adaptive formulation (CLEAR-type window reward relative to a default run)..... 36

Figure 3.1. Architecture of YOLOv8. (a) Overall architecture: backbone of stride-2 convolutions and C2f blocks ending in SPPF (left), PAN-FPN neck fusing P3, P4 and P5 (centre) and one Detect head per level (right). (b) C2f block with its bottlenecks. (c) Decoupled anchor-free Detect head, whose box branch predicts a 16-bin distance distribution per side (DFL) and whose classification branch outputs a sigmoid score; training uses the task-aligned assigner t = p^α u^β (p corresponds to s in Equation (3.7); u is CIoU clamped at zero), CIoU + DFL and BCE losses, and inference ends with class-agnostic NMS at threshold θ_nms (applied post hoc, Section 3.1.5). 46

Figure 3.2. Per-frame flowchart of ByteTrack as used in this thesis. Detections pass class-agnostic NMS and are split by score into D_high, D_low and discarded boxes; D_high is associated with the predicted tracked and lost tracks (cost 1 − IoU·s), D_low with the remaining tracked tracks (cost 1 − IoU) and the remaining high-score boxes with unconfirmed tracks (fixed limit 0.7). Matched tracks are updated, unmatched high-score boxes may start tracks, unmatched tracked tracks become lost and are removed after the buffer period, and confirmed tracks are output. Parameters in bold belong to the tuned vector θ and are shown with their configuration names (Table 4.6); the NMS is the post-hoc suppression with threshold θ_nms (nms_iou). 56

Figure 3.3. Schematic of the social force model as implemented in this thesis. (a) Pedestrian i is driven along the desired direction e_i towards its target by (v_i^0 e_i − v_i)/τ, is repelled by pedestrian j with a force parallel to the gradient of the elliptical distance b_ij and weighted by the anisotropy w(φ_ij) (shaded region, λ = 0.15), and is repelled by the nearest wall point W along n_iW; the inset shows body contact with compression and normal damping along n_ij and sliding friction along t_ij. (b) The equations per unit mass with the parameter values used (calibrated A = 2.0 m/s² and B = 0.30 m; fixed τ = 0.5 s, Δt = 0.5 s and λ = 0.15); the lateral evasion term, the speed cap and the noise term are not drawn..... 60

Figure 3.4. Worked example of HOTA at one threshold α. (a) Per-frame matching of two ground-truth and four predicted trajectories over ten frames, with each true positive labelled by its partner and misses (FN) and unmatched predictions (FP) marked. (b) Association sets of the pair c = (GT A, Pred 1): six TPA, four FNA and two FPA, so A(c) = 0.50. (c) DetA = 0.80, AssA ≈ 0.457 as the mean of the association accuracies over all true positives, HOTA_α = √(0.80 · 0.457) ≈ 0.60, and the final HOTA as the mean over α ∈ {0.05, …, 0.95}. 67

Figure 3.5. Proximal policy optimisation as used in this thesis. (a) The actor, a Gaussian policy with seven outputs, maps the observation s to an action a, which is clipped to u and mapped to θ by Equation (4.7) with the range bounds θ^min and θ^max; the environment applies NMS and ByteTrack with θ to cached YOLOv8 detections and evaluates the result against the simulator ground truth to give the reward r. Transitions are stored in a rollout buffer, advantages are computed by GAE from the critic V_ψ, and actor and critic are updated over ten epochs with the clipped surrogate and the value loss. (b) Clipped surrogate objective of one sample against the probability ratio ρ_t for positive (left) and negative (right) advantage, ε = 0.2; the dashed line is the unclipped objective ρ_t Â_t. 73

Figure 4.1. Architecture of the closed-loop system. Simulation and perception (pedestrian simulation, virtual camera, ground truth, YOLOv8 fine-tuning and inference) are computed once; only the tracking layer is re-run for every parameter vector θ proposed by the PPO agents or baseline optimisers..... 80

Figure 4.2. Pedestrian Library flowcharts: (a) corridor and street crossing, (b) bottleneck with an intermediate gate target, (c) station plaza with a probabilistic detour to the waiting area; top right: PedSource attributes and the trajectory logging event with its logged columns..... 83

Figure 4.3. Top views of the scenarios: walls and barriers (black), PedSource entrance lines (green), corner blocks and kiosks (green squares), the plaza's waiting area (dashed), sample trajectories at medium density (blue), camera position and height (triangle) and the part of the ground inside the camera's field of view (shaded, dotted outline). 86

Figure 4.4. Speed–density relations of the co-simulation compared with Weidmann's fundamental diagram (black): (a) calibration runs in the unidirectional 40 m × 3 m corridor for the nine combinations of the calibration grid (selected combination highlighted; lines are density-binned mean speeds, points the raw measurements of the selected combination); (b) measurement areas of the training sequences of the four scenarios (non-waiting pedestrians, sampled every second)..... 90

Figure 4.5. Virtual camera geometry: (a) side view with optical axis, image plane, projection rays and a pedestrian drawn schematically as a cylinder of radius r and height H; (b) image plane with amodal box, visible and occluded parts of a pedestrian and the visibility ratio; (c) pinhole projection equations and camera parameters..... 91

Figure 4.6. Rendered test frames at low (left) and high (right) density with ground-truth amodal boxes coloured by visibility: green $\nu \geq 0.5$, orange $0.1 \leq \nu < 0.5$, red $\nu < 0.1$ (ignored in the evaluation). Pedestrians with less than half of their silhouette inside the image ($\iota < 0.5$) are also ignored but not marked separately. 94

Figure 4.7. Dataset organisation. Top: sequences per condition and split (detector training / training / validation / test) and the 20 s training clips of one static condition, with the four HPO objective clips highlighted. Bottom: use of the splits; the detector-training and validation splits feed only YOLOv8 transfer learning, both training splits feed the HPO baselines (including the global TPE) and both PPO formulations, and the held-out test splits feed only the final evaluation of all methods. 96

Figure 4.8. Ground-truth statistics of the training and test sequences: (a) visibility ratio per scenario for pedestrians at least half inside the image, (b) height of evaluated boxes per scenario and (c) mean evaluated pedestrians per frame per scenario and density. 98

Figure 4.9. Evaluation and reward pipeline: ignore pre-processing, then CLEAR, Identity and HOTA on whole clips or sequences (contextual reward from HOTA), or streaming CLEAR summed over 1 s windows (adaptive reward). 102

Figure 4.10. RL formulations. (a) Formulation A, contextual: a descriptor of the first 4 s of detections is mapped to one θ for the whole 20 s clip, rewarded by the HOTA gain over the defaults ($\gamma = 0$). (b) Formulation B, adaptive: every 1 s the policy observes window statistics and the current θ and sets θ for the next window, rewarded by the windowed score difference to a default run ($\gamma = 0.9$). 104

Figure 5.1. Transfer learning of YOLOv8n, YOLOv8s and YOLOv8m on the detector-training split. (a) Box loss on the training data (solid) and the validation data (dashed); (b) validation mAP@0.5 and (c) validation mAP@0.5:0.95 per epoch. Colour shade indicates the model size, from light (YOLOv8n) to dark (YOLOv8m). Note the narrow vertical range of panel (b). 114

Figure 5.2. Gain from transfer learning on the validation split: (a) mAP@0.5 and (b) mAP@0.5:0.95 of the COCO-pretrained models evaluated zero-shot (grey) and of the transfer-learned models (blue) for YOLOv8n, YOLOv8s and YOLOv8m. 117

Figure 5.3. Detections of the COCO-pretrained YOLOv8s evaluated zero-shot (left) and of the transfer-learned YOLOv8s (right) on the same test frames of a high-density bottleneck sequence (top) and a high-density corridor sequence (bottom), after NMS at 0.7.

Green boxes have a confidence of at least 0.5 and enter ByteTrack's first association under the default parameters, orange boxes a confidence between 0.1 and 0.5 and enter the second association. The panel labels give the numbers of high- and low-score boxes and of pedestrians with visibility of at least 0.1 in the whole frame; the images are crops. 118

Figure 5.4. Confidence of the transfer-learned YOLOv8s on the test split (every third frame, NMS at 0.7, confidence ≥ 0.05, matching at IoU ≥ 0.5). (a) Histograms of the confidence of true positives (blue) and false positives (orange) on a logarithmic scale, with dotted lines at the default low-score threshold 0.1 and high-score threshold 0.5; (b) mean confidence of true positives and share of true positives with a confidence below 0.5 per visibility bin; (c) recall per visibility bin counting detections with confidence ≥ 0.05 (green) and ≥ 0.5 (blue)..... 119

Figure 5.5. One-at-a-time sensitivity of HOTA on the training split. Each panel varies one parameter over eleven values with the others at their defaults; coloured lines are the mean over the six clips of a scenario (two clips per density level), the dashed line the mean over all 24 clips, and the dotted vertical line the default value. 121

Figure 5.6. Response surfaces of the mean HOTA over three high-density training clips per scenario as a function of the high-score threshold τhigh and the first-association cost limit $\theta m1$, with the other parameters at their defaults: (a) corridor, (b) crossing, (c) bottleneck, (d) plaza. The star marks the best grid point and the circle the default; each panel has its own colour scale. 123

Figure 5.7. Response surfaces of the mean HOTA over three high-density training clips per scenario as a function of the track buffer $n$buf (frames at 30 FPS) and the new-track threshold τnew, with the other parameters at their defaults: (a) corridor, (b) crossing, (c) bottleneck, (d) plaza. The star marks the best grid point and the circle the default; each panel has its own colour scale. 124

Figure 5.8. Convergence of the baseline optimisers on the training objective. Best HOTA gain over the default found after a given number of objective evaluations (each evaluation scores four training clips), averaged over the 16 conditions and three seeds; shaded bands show the standard error of the mean over the 48 condition–seed runs. Pink: Random search; green: TPE; orange: CMA-ES. 128

Figure 5.9. Learning curves of the PPO agents. (a) PPO-contextual, budget-matched run: rolling mean (window 384 steps, one rollout) of the HOTA of the sampled actions on

the training clips, mean of three seeds (line) and range over seeds (band). (b) PPO-adaptive: rolling mean (50 episodes) of the episode return, the sum over windows of the score difference to a default run (Equation (4.10)), for the three seeds individually; the dotted line marks the return of the default parameters (zero)..... 130

Figure 5.10. Test accuracy of all methods. Top row: static-density test split (48 sequences); bottom row: dynamic-density test split (16 sequences). (a, d) HOTA, (b, e) MOTA, (c, f) IDF1, all in %. Dots: mean over three seeds; error bars: one standard deviation; dotted line: default parameters. 134

Figure 5.11. HOTA gain over the default (points) per test condition and method. Rows: the twelve static conditions (top block) and the four dynamic conditions (bottom block); columns: methods. Each cell is the difference between the seed-averaged combined HOTA of the method and that of the default over the test sequences of the condition; blue cells mark improvements, red cells deteriorations..... 135

Figure 5.12. Distribution of the per-sequence HOTA gain over the default (points) for each tuned method, pooled over the 48 static and 16 dynamic test sequences; each sequence's HOTA is averaged over the three seeds before the default is subtracted. Boxes: interquartile range with median; whiskers: 1.5 times the interquartile range; circles: outliers; dotted line: no gain. 137

Figure 5.13. Parameters selected by PPO-contextual on the test splits. Each panel shows one component of $\boldsymbol{\theta}$; rows are the four scenarios and the three markers per scenario the low, medium and high static density levels (light to dark blue). Dots are means and horizontal bars the range over the three seeds and the test sequences of the condition; the dotted vertical line marks the default value. The horizontal axes span the search ranges of Table 4.6. 142

Figure 5.14. PPO-adaptive (seed 0) on the dynamic-density test sequence bottleneck_dynamic_200 (90 s, 90 windows of 1 s). Top: number of evaluated pedestrians in view per frame. Middle: four of the seven parameters chosen by the policy ($\tau\mathrm{high}$, $\tau\mathrm{new}$, $\theta m1$ and the NMS IoU threshold); the first window uses the default. Bottom: window score $(\mathrm{TP} - \mathrm{FP} - 5\,\mathrm{IDSW})/\mathrm{GT}$ of Equation (4.10) for the policy run and for the default run. 144

Figure 5.15. Influence of the detector on tracking accuracy on the static test split. Grouped bars of HOTA (left) and IDF1 (right) for the zero-shot YOLOv8s and the fine-

tuned YOLOv8n, YOLOv8s and YOLOv8m, each tracked with the default parameters, the global TPE parameters and the PPO-contextual policy (mean over three seeds); TPE and the policy were tuned or trained with the fine-tuned YOLOv8s. 146

Figure 5.16. Decomposition of HOTA on the static test split for Default, TPE and the two PPO agents. (a) HOTAα over the localisation threshold α, mean over the 48 test sequences (and the three seeds of each tuned method). (b) DetA against AssA, one point per test sequence and seed (one point per sequence for the deterministic default). 148

Figure 5.17. Default (top) and PPO-contextual (bottom) on two high-density static test sequences, at the frame at the 90th percentile of the per-frame reduction of FP plus missed pedestrians: (a, c) bottleneck_high_100 at t = 2.8 s (frame 42), (b, d) corridor_high_101 at t = 2.7 s (frame 40). Green: true-positive track boxes (IoU ≥ 0.5); red: false-positive track boxes; magenta: ground-truth boxes of missed pedestrians. Counts per frame are given in the panel titles. 151

List of Tables

Table 1.1. The seven tuned parameters of the detection–tracking pipeline, their search ranges and their default (reference) values. The buffer is specified in frames at 30 FPS, as in ByteTrack, and corresponds to 5–75 frames at the 15 FPS of the virtual camera. 8

Table 2.1. Principal versions of the YOLO family (results as reported by the authors; datasets, input sizes and hardware differ, so the figures are not directly comparable). Years are those of the peer-reviewed venue, or of the first public release for preprints and software. "–": not quoted here. 20

Table 2.2. Detector post-processing and ByteTrack parameters with their defaults in the ByteTrack paper [21], the official code [22] and Ultralytics [24], and their search ranges and defaults in this thesis (Chapter 4); the seven parameters with a range are tuned, the other two are fixed ("–": not specified in the cited source). 26

Table 2.3. Representative multi-object trackers (TbD: tracking-by-detection; JDT: joint detection and tracking; results as reported with different detectors; "–": not quoted here; BoT: the strong person re-identification baseline adopted by StrongSORT; AFLink: appearance-free link model [110]). 27

Table 2.4. Comparison of pedestrian dynamics model families. 31

Table 2.5. Representative synthetic datasets for pedestrian and crowd analysis. 34

Table 2.6. Comparison of hyper-parameter optimisation methods. 37

Table 2.7. Positioning of closely related studies relative to this thesis. 41

Table 3.1. Parameters of the social force model of the pedestrian co-simulation (forces per unit mass). 63

Table 3.2. Tracking metrics used in this thesis. 68

Table 4.1. Geometry, flows, arrival rates and camera poses of the four scenarios (HFOV: horizontal field of view; 1280 × 720 pixels at 15 FPS). 85

Table 4.2. Splits of the synthetic dataset. 95

Table 4.3. Size of the dataset splits used for tracking (evaluated boxes: visibility ≥ 0.1 and at least half inside the image). 97

Table 4.4. Crowding statistics per static condition (training and test sequences; visibility over pedestrians at least half inside the image). 97

Table 4.5. YOLOv8 transfer-learning and inference settings. 98

Table 4.6. Tuned parameter vector θ of the detection–tracking pipeline. 101

Table 4.7. Scene descriptor of formulation A. 104

Table 4.8. Observation of formulation B. 106

Table 4.9. PPO hyper-parameters (Stable-Baselines3). 107

Table 4.10. Optimisation methods, budgets and test-time selection rules. 109

Table 4.11. Hardware and software. 110

Table 5.1. Detection accuracy on the validation split (Ultralytics validation, every tenth frame, labels with visibility ≥ 0.2) of the COCO-pretrained models evaluated zero-shot and of the fine-tuned models, with number of epochs and training time. 115

Table 5.2. Detection accuracy on the test split per model and visibility band (post-hoc NMS at 0.7; COCO-style AP with 101-point interpolation; AP averaged over IoU 0.50:0.95; recall and precision at confidence ≥ 0.25 and IoU 0.5; pedestrians at least half inside the image; pedestrians outside the band ignored). 116

Table 5.3. Best parameter vector per condition found by TPE on the training objective (mean HOTA over the four HPO clips of a condition; for the global row, over the first HPO clip of each of the 16 conditions), seed with the best objective, ordered by the best objective value. Default and best objective in per cent; gain in points computed from the values rounded to three decimals. The last row gives the default parameters. 125

Table 5.4. Gain of the baseline optimisers on the training objective fq (mean HOTA over the four objective clips of a condition), mean ± standard deviation over 16 conditions × 3 seeds. The grid search evaluates all 37 points once and has no convergence curve. . 129

Table 5.5. Tracking accuracy on the static-density test split (48 sequences, 43,200 frames), combined metrics. Tuned methods: mean ± standard deviation over three seeds (DetA and AssA: mean); counts are seed means rounded to integers. Default and grid search are deterministic single runs. 131

Table 5.6. Tracking accuracy on the dynamic-density test split (16 sequences, 21,600 frames), combined metrics; conventions as in Table 5.5. The per-condition baselines use the parameters tuned for the dynamic condition of the scenario. 133

Table 5.7. Per-sequence HOTA gain over the default (seed-averaged), two-sided Wilcoxon signed-rank test against zero and Holm-adjusted p-values within each split (family-wise $\alpha = 0.05$). All listed results are significant. 137

Table 5.8. Pairwise comparison of the PPO agents with the baselines and with each other: per-sequence difference of the seed-averaged HOTA gains (points; positive = PPO agent better), two-sided Wilcoxon signed-rank test, Holm adjustment over all pairs within a split. Bold: significant at family-wise $\alpha = 0.05$..... 138

Table 5.9. Mean parameters selected by PPO-contextual (deterministic policy, budget-matched run) per test condition, averaged over test sequences and three seeds, compared with the default $\boldsymbol{\theta}0$ and the search ranges. 140

Table 5.10. Tracking accuracy on the static test split (48 sequences, combined metrics) with different detectors. TPE (global) and PPO-contextual were tuned or trained with the fine-tuned YOLOv8s and are applied without re-tuning; PPO-contextual values are means over three seeds. 147

Table 5.11. Measured run times. Detection times are means over the processed videos; training times are wall-clock times. 152

Table A.1. Elements of the AnyLogic model PedTrackSim (agent types Main and Ped). 168

Table A.2. Pedestrian Library blocks and their key properties. 169

Table B.1. Library modules (package ptrack). 174

Table B.2. Experiment and analysis scripts (code/experiments, code/figures). 174

List of Abbreviations

Abbreviation	Meaning
AP	Average precision
API	Application programming interface
AssA	Association accuracy (HOTA sub-metric)
AUC	Area under the curve
BCE	Binary cross-entropy
BO	Bayesian optimisation
CA	Cellular automaton
CIoU	Complete intersection over union
CLEAR	Classification of Events, Activities and Relationships (CLEAR-MOT metrics)
CMA-ES	Covariance matrix adaptation evolution strategy
CNN	Convolutional neural network
COCO	Microsoft Common Objects in Context (dataset)
CPU	Central processing unit
CSP	Cross Stage Partial (network design)
CSV	Comma-separated values (file format)
CUDA	Compute Unified Device Architecture
DDPG	Deep deterministic policy gradient
DetA	Detection accuracy (HOTA sub-metric)
DETR	Detection Transformer
DFL	Distribution focal loss
DIoU	Distance intersection over union
DQN	Deep Q-network
EI	Expected improvement
EMA	Exponential moving average
FL	Focal loss
FN	False negative
FNA	False negative associations (HOTA)
FP	False positive
FP16	16-bit (half-precision) floating point
FPA	False positive associations (HOTA)
FPN	Feature pyramid network
FPS	Frames per second
Frag	Number of trajectory fragmentations
GAE	Generalised advantage estimation
GELAN	Generalised efficient layer aggregation network
GM-PHD	Gaussian mixture probability hypothesis density (filter)
GPU	Graphics processing unit
GSI	Gaussian-smoothed interpolation
GT	Ground truth
GTA	Grand Theft Auto (video game used for synthetic data)
HFOV	Horizontal field of view
HOTA	Higher Order Tracking Accuracy
HPO	Hyper-parameter optimisation
ID	Identity (identifier)
IDF1	Identification F1 score
IDFN	Identification false negatives
IDFP	Identification false positives
IDP	Identification precision
IDR	Identification recall
IDSW	Identity switches
IDTP	Identification true positives
IoU	Intersection over union
JDE	Joint detection and embedding

Abbreviation	Meaning
JDT	Joint detection and tracking
JSON	JavaScript Object Notation (file format)
JTA	Joint Track Auto (synthetic dataset)
KITTI	Karlsruhe Institute of Technology and Toyota Technological Institute (benchmark)
LAPJV	Jonker–Volgenant algorithm for the linear assignment problem
LocA	Localisation accuracy (HOTA sub-metric)
mAP	Mean average precision
MCMC	Markov chain Monte Carlo
MDP	Markov decision process
ML	Mostly lost (trajectories)
MLP	Multilayer perceptron
MOT	Multi-object tracking
MOTA	Multiple object tracking accuracy
MOTP	Multiple object tracking precision
MT	Mostly tracked (trajectories)
MTA	Multi Camera Track Auto (synthetic dataset)
NMS	Non-maximum suppression
NSA	Noise scale adaptive (Kalman filter)
OC-SORT	Observation-centric SORT
ORCA	Optimal reciprocal collision avoidance
PAN	Path aggregation network
PBT	Population-based training
PDE	Partial differential equation
PNG	Portable Network Graphics (image format)
PPO	Proximal policy optimisation
R-CNN	Region-based convolutional neural network
R-ELAN	Residual efficient layer aggregation network
RAM	Random-access memory
Re-ID	Re-identification
RL	Reinforcement learning
RMSE	Root-mean-square error
RPN	Region proposal network
RQ	Research question
SAC	Soft actor-critic
SGD	Stochastic gradient descent
SiLU	Sigmoid linear unit
SMAC	Sequential model-based algorithm configuration
SORT	Simple Online and Realtime Tracking
SPP	Spatial pyramid pooling
SPPF	Spatial pyramid pooling – fast
SSD	Single Shot MultiBox Detector
STAL	Small-target-aware label assignment
TbD	Tracking-by-detection
TOOD	Task-aligned one-stage object detection
TP	True positive
TPA	True positive associations (HOTA)
TPE	Tree-structured Parzen Estimator
TRPO	Trust region policy optimisation
VOC	PASCAL Visual Object Classes (benchmark)
YOLO	You Only Look Once

Chapter 1 Introduction

Video cameras are now among the most widespread sensors in public space, and automatically extracting pedestrian trajectories from their images is a basic capability for crowd management, transport planning and public safety. This thesis studies how the parameters of a modern pedestrian tracking pipeline, consisting of a YOLOv8 object detector and the ByteTrack multi-object tracker, can be optimised by reinforcement learning when the training signal comes not from manually annotated real video but from a pedestrian simulation whose virtual camera delivers video together with perfect ground truth. This chapter motivates the study (Section 1.1), states the problem precisely (Section 1.2), sets out the objectives and research questions (Section 1.3), summarises the contributions (Section 1.4), defines the scope and limitations (Section 1.5) and outlines the organisation of the thesis (Section 1.6).

1.1 Research Background

1.1.1 Pedestrian Monitoring, Crowd Safety and Video Analytics

Railway stations, transport interchanges, street crossings, public squares and large events bring thousands of people together in confined spaces. Under normal conditions such crowds organise themselves remarkably well. Experiments and simulations have documented lane formation in counter-flows, oscillating flow directions at bottlenecks and stripe formation in crossing flows [1], [2]. When density rises beyond critical levels, however, orderly motion can break down. Simulations of escape situations reproduce clogging at exits and the “faster-is-slower” effect [3]. The analysis of the 2010 Love Parade disaster in Duisburg showed that fatal crowd turbulence can arise from amplifying feedbacks and cascading effects in an overcrowded system, without malicious intent on the part of anyone involved. The same analysis calls for preventive measures that allow earlier intervention, and such situations are best prevented when the state of the crowd, meaning its density, speed and flow, is observed early enough for operators to act [4].

Cameras are a natural sensor for this purpose, because many public spaces are already equipped with them. Real-time video analytics at scale has been described as a defining application of edge computing, because camera data are voluminous and live analysis must meet strict latency requirements [5]. In smart cities and urban digital twins, live sensor data are combined with virtual models of the physical environment to support monitoring and

planning [6], [7]. For pedestrian applications, the essential output of video analytics is a set of trajectories, the position of every person over time under a persistent identity. Trajectories yield the density, speed and flow of a facility, allow empirical fundamental diagrams to be estimated [8] and supply the basis for calibrating microscopic pedestrian models [9]. Detections of You Only Look Once (YOLO) detectors in surveillance video have even been used to calibrate social-force simulations built in AnyLogic [10] and to initialise evacuation simulations [11]. Every such analysis is only as good as the multi-object tracking (MOT) that produced its trajectories.

1.1.2 Tracking-by-Detection and the Role of Its Parameters

The dominant paradigm for pedestrian MOT is tracking-by-detection. An object detector first localises the pedestrians in each frame independently, and a tracker then links the detections over time into trajectories [12], [13]. One-stage detectors of the YOLO family have become the usual first stage of such pipelines because they combine competitive accuracy with real-time throughput [14], [15]. YOLOv8, the version adopted in this thesis, is an anchor-free detector distributed with weights pre-trained on the Microsoft Common Objects in Context (COCO) dataset [16], [17]. For association, the family of trackers descending from Simple Online and Realtime Tracking (SORT) pairs a constant-velocity Kalman filter with optimal assignment of detections to predicted tracks [18], [19], [20]. ByteTrack extended this scheme with an observation about low-confidence detections, which most trackers discard: such detections often belong to occluded pedestrians and can be recovered in a second association stage [21]. At the time of its publication, ByteTrack achieved state-of-the-art results on the MOT17 and MOT20 benchmarks with motion and overlap cues alone, reaching a multiple object tracking accuracy (MOTA) of 80.3 and a Higher Order Tracking Accuracy (HOTA) of 63.1 on the MOT17 test set, and 77.8 MOTA and 61.3 HOTA on MOT20 [21]. Later trackers have surpassed these figures, but ByteTrack remains a transparent, fast and strong baseline for crowded pedestrian scenes.

Behind the simplicity of tracking-by-detection lies a considerable number of hand-set decision thresholds. The detector's raw output is filtered by a confidence threshold and by non-maximum suppression (NMS). The intersection-over-union (IoU) threshold of NMS decides whether two overlapping boxes describe one pedestrian or two. ByteTrack then divides the remaining detections into a high-score and a low-score set, applies a separate cost limit in each of its two association stages, starts new tracks only above a further score

threshold, and keeps unmatched tracks alive for a fixed number of frames before deleting them [21], [22]. These parameters act at different points of the pipeline, but they interact strongly. A low NMS IoU threshold removes duplicate boxes, but in a dense crowd it also suppresses pedestrians who genuinely overlap. A low high-score threshold lets occluded pedestrians into the first association, but it also admits false positives that can capture the matches of existing tracks and, if they also exceed the new-track threshold, start spurious tracks. A long track buffer bridges occlusions, but it raises the risk that a lost identity is later transferred to a different person.

The best compromise is expected to depend on the detector and on the scene: on crowd density, on the degree of occlusion, on how large pedestrians appear in the image and on the confidence distribution of the particular detector. Whether, and how strongly, this holds for the pipeline studied here is one of the research questions of this thesis (Section 1.3). The authors of SORT already observed that changing the detector alone changed tracking accuracy by up to 18.9% [20]. The MOT20 benchmark was assembled from very crowded scenes, with mean densities of up to 246 pedestrians per frame, in order to test trackers under conditions far denser than those of earlier benchmarks [23]. A telling sign that no single setting suits every detector and scene is that the recommended ByteTrack defaults differ markedly between implementations. In the official code's MOT evaluation setting, which pairs ByteTrack with a YOLOX detector, the high-score threshold is 0.6 and the first-association limit 0.9, whereas the widely used Ultralytics configuration for its own YOLO detectors sets them to 0.25 and 0.8 [21], [22], [24]. The official evaluation code even overrides the high-score threshold for individual MOT17 and MOT20 sequences and the track buffer for individual MOT17 sequences [22].

1.1.3 The Cost of Tuning: Ground Truth and Annotation

In practice, the parameters of a deployed tracker are often not tuned for the camera at hand. Tracking hyper-parameters are commonly chosen by hand, and systematic optimisation is rarely applied in multi-object tracking [25], [26]. Tuning methods are not what is missing. Grid search, random search, Bayesian optimisation with the Tree-structured Parzen Estimator (TPE) and the covariance matrix adaptation evolution strategy (CMA-ES) are mature hyper-parameter optimisation (HPO) methods, and standard toolkits provide them [27], [28], [29], [30]. What is missing is a usable objective function. Tracking accuracy measures such as MOTA [31], the identification F1 score (IDF1) [32]

and HOTA [33] compare tracker output with ground-truth (GT) trajectories. Such ground truth requires a bounding box and a consistent identity for every pedestrian in every frame.

Annotating crowded video in this way is laborious. The MOTChallenge benchmarks needed a dedicated annotation protocol with visibility levels to label pedestrians consistently [34], [35], and the effort per frame grows with the number of people in view. Privacy is a further obstacle, because recording and annotating identifiable people in public space is increasingly restricted, a motivation cited explicitly by the authors of synthetic person datasets [36], [37] and by a review of synthetic data for video surveillance [38]. Practitioners are therefore likely to fall back on the published default thresholds, which were chosen on benchmark sequences whose camera geometry and crowd density may have little in common with the target scene.

Even where some annotated data exist, two difficulties remain. First, classical HPO returns one static configuration per run, so if the best parameters depend on the scene, every camera and crowd condition needs its own optimisation with its own annotated data. Second, the conditions in front of a single camera change over time: a station concourse may be nearly empty at midday and crowded at rush hour. A tuning method that observes the scene and adjusts the parameters, once per scene or continuously, would address both difficulties, but training such a method demands even more labelled feedback than a single optimisation run.

1.1.4 Simulation as a Source of Perfectly Labelled Data

Simulation offers a way out of this dilemma. A virtual world knows the position, extent and identity of every object it renders, so annotations come at no extra cost and are exact. Several lines of work demonstrate the value of such data. Virtual KITTI showed that cloned virtual driving scenes provide automatic ground truth for detection and tracking, together with a controlled environment for studying the effect of imaging conditions [39]. Game-engine datasets showed that body-pose detectors and trackers trained on the synthetic people of JTA, and pedestrian detectors and trackers trained on MOTSynth, generalise to real benchmarks, although on the very crowded MOT20 benchmark MOTSynth-trained models still needed fine-tuning on real data [37], [40]. Scene-specific pedestrian detectors have even been trained for a new surveillance camera from virtual pedestrians alone, without any real pedestrian data [41], [42].

The appearance gap between rendered and real images can be narrowed by domain randomisation [43], [44]. For a detector pre-trained on real images, it can also be narrowed by fine-tuning on the target domain. This form of transfer learning exploits the fact that early convolutional features are general across tasks [45], [46].

The game-engine pedestrian datasets JTA, MOTSynth and MTA, however, were generated once, with engines whose crowd behaviour does not come from validated pedestrian models, and they serve as static training sets or benchmarks [36], [37], [40]. Pedestrian dynamics simulation offers complementary strengths. The social force model treats each pedestrian as a particle driven towards a destination and repelled by other pedestrians and by walls [47], [48], and it reproduces the self-organised phenomena observed in real crowds. AnyLogic is a multimethod simulation tool [49] whose Pedestrian Library provides a continuous-space pedestrian model together with a graphical space markup, a flowchart of pedestrian behaviour blocks and 2D and 3D views [50]. The literature treats this model as social-force-based, although its equations are not published [50], [51]. Python interfaces for reinforcement-learning experiments complement it [52].

Combined with a virtual camera, such a simulator can generate any number of video sequences for any layout, crowd density and camera placement, each with a pixel-exact record of every agent. The tracking accuracy of a detector–tracker pipeline thereby becomes a quantity that can be computed on demand, thousands of times, for any parameter setting, which is exactly the feedback that reinforcement learning (RL) requires. RL algorithms such as proximal policy optimisation (PPO) learn policies that map observations to actions so as to maximise the expected cumulative (discounted) reward [53], [54]. Earlier work has shown that RL can choose tracker hyper-parameters per sequence or per frame in single-object tracking [55], and that it can tune the parameters of a simulator to benefit a downstream vision model [56]. As Chapter 2 shows, however, to the author's knowledge no study has yet closed the loop between a pedestrian simulator, a virtual camera, a transfer-learned detector, a multi-object tracker and an RL agent that re-tunes the joint detector–tracker parameters. This thesis builds and studies such a loop.

1.2 Problem Statement

The system studied in this thesis is a closed loop of simulation, detection, tracking and reward, illustrated in Figure 1.1. Pedestrian scenarios are simulated, and a virtual camera placed in each simulated scene renders the moving pedestrians as surveillance video while

recording pixel-exact ground truth. A YOLOv8 detector, pre-trained on COCO and transfer-learned on automatically labelled frames of the rendered video, detects the virtual pedestrians in every frame. ByteTrack links the detections into trajectories, which are scored against the simulator's ground truth. The resulting score is the reward from which an RL agent learns to set the detector post-processing and tracker parameters, so that the workflow of video generation, detection, tracking and reward supplies the feedback for every parameter setting that the agent proposes.

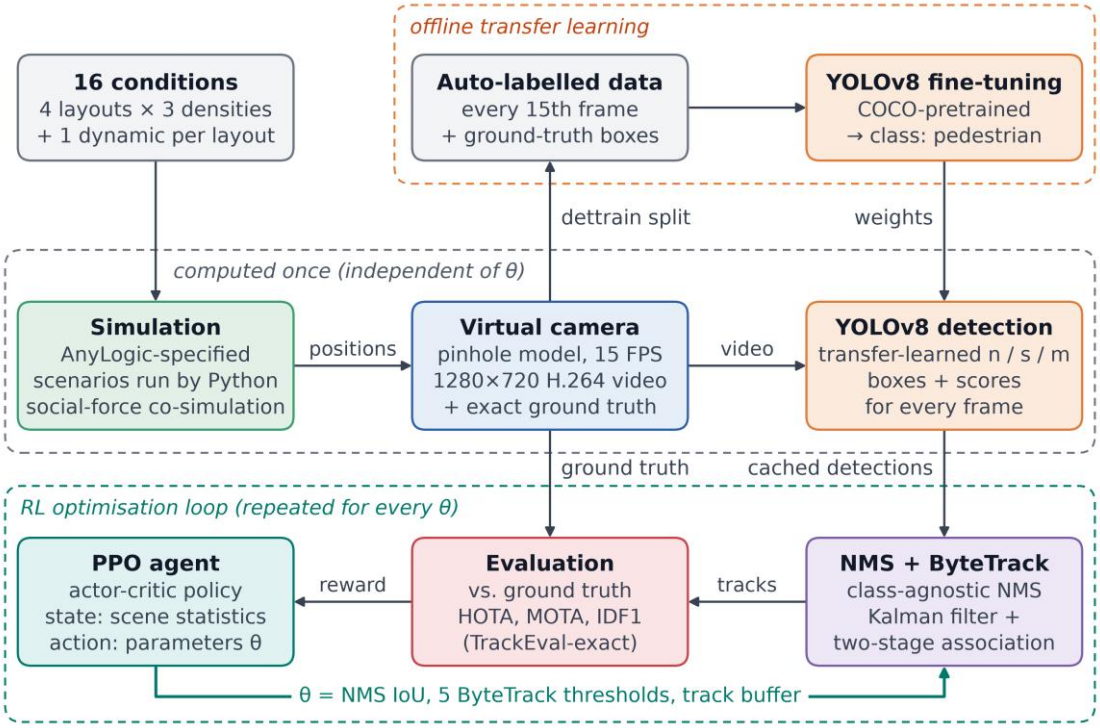


Figure 1.1. Closed-loop research framework of this thesis. Sixteen conditions (four scenario layouts at three densities, plus one dynamic-density condition per layout) are specified as AnyLogic scenarios and executed in the experiments by a Python social-force co-simulation of the same models. A virtual camera (1280 × 720 pixels, 15 frames per second (FPS)) renders the trajectories into H.264 video with pixel-exact ground truth. Frames of a dedicated detector-training split fine-tune a COCO-pretrained YOLOv8 detector offline; the fine-tuned YOLOv8s feeds the optimisation loop. Simulation, rendering and detection are computed once and cached, whereas NMS with ByteTrack, the evaluation against the ground truth (HOTA, MOTA, IDF1) and the PPO agent form the loop repeated for every parameter vector θ. The HPO baselines use the same loop with an optimiser in place of the PPO agent.

The scenarios are specified as AnyLogic Pedestrian Library models, but the reported experiments were run with a headless Python co-simulation that implements the same scenarios, pedestrian parameters and camera poses. The reason is practical: the 256 sequences of the dataset had to be generated reproducibly, in batch and by script, with pixel-exact instance-identity ground truth, on a 32-thread workstation. The co-simulation is also transparent. Every force term and navigation rule can be calibrated and reported,

whereas the equations of the Pedestrian Library are not documented [50], [51]. The Java code templates of the AnyLogic model, a converter for AnyLogic trajectory exports (`anylogic_to_ptrack.py`) and a template for an Alpyne bridge are provided with the thesis, but they were not used for the reported experiments, and the thesis does not establish that trajectories from the AnyLogic runtime and from the co-simulation are equivalent (Sections 4.1 and 4.3, Appendix A). Simulation, rendering and neural-network inference depend only on the scenario, the density level and the seed (and on the fixed detector weights), not on the tuned parameters. They are therefore computed once and cached, and only the inexpensive post-processing and tracking stages are re-run for every candidate parameter vector.

Let a video sequence V with ground-truth trajectories $G(V)$ be drawn from a distribution $p(V)$ over scenarios, crowd densities and random realisations. After transfer learning, the detector weights are fixed; all parameter-optimisation and RL experiments use the fine-tuned YOLOv8s, while YOLOv8n, YOLOv8m and the zero-shot model are examined in the detector study of Chapter 5. For every frame, the detector produces a loosely suppressed set of scored candidate boxes. The rest of the pipeline is controlled by the seven-dimensional parameter vector

$$\boldsymbol{\theta} = \left(\theta_{\text{nms}},\ \tau_{\text{high}},\ \tau_{\text{low}},\ \tau_{\text{new}},\ \theta_{m1},\ \theta_{m2},\ n_{\text{buf}}\right) \in \Theta = \prod_{i=1}^{7}\left[\theta_i^{\min},\ \theta_i^{\max}\right] \subset \mathbb{R}^7, \qquad (1.1)$$

whose components and their lower and upper bounds $\theta_i^{\min}$ and $\theta_i^{\max}$ are listed in Table 1.1. The search space Θ is thus the box spanned by the search ranges of Table 1.1, with θ_{nms} snapped to its grid of step 0.05, because the NMS stage is evaluated only at eleven pre-computed thresholds (Section 4.6). The first component is the NMS IoU threshold of the detector post-processing. The other six are ByteTrack parameters that govern the split of detections by confidence, the two association stages, the creation of new tracks and the deletion of lost tracks. The search ranges are wide enough to contain the published defaults of both the official and the Ultralytics implementation. The default values in the last column of Table 1.1 serve as the reference configuration against which every optimised setting is compared. They combine the two published sets: the NMS threshold of 0.7, the low-score threshold of 0.1, the second-association cost limit of 0.5 and the buffer of 30 are common to both implementations; the first-association cost limit of 0.8 follows Ultralytics (official code: 0.9); and the high-score threshold of 0.5 lies between the

two published values of 0.25 and 0.6, with the new-track threshold of 0.6 set 0.1 above it, as in the official code.

Table 1.1. The seven tuned parameters of the detection–tracking pipeline, their search ranges and their default (reference) values. The buffer is specified in frames at 30 FPS, as in ByteTrack, and corresponds to 5–75 frames at the 15 FPS of the virtual camera.

Parameter (implementation name)	Symbol	Pipeline stage	Search range	Default
NMS IoU threshold (`nms_iou`)	θ_{nms}	Detector post-processing	[0.40, 0.90], grid step 0.05	0.70
High-score threshold (`track_high_thresh`)	τ_{high}	Split of detections by score	[0.20, 0.80]	0.50
Low-score threshold (`track_low_thresh`)	τ_{low}	Split of detections by score	[0.05, 0.40]	0.10
New-track threshold (`new_track_thresh`)	τ_{new}	Track initialisation	[0.20, 0.90]	0.60
First-association cost limit (`match_thresh`)	θ_{m1}	First association	[0.50, 0.95]	0.80
Second-association cost limit (`second_match_thresh`)	θ_{m2}	Second association	[0.20, 0.90]	0.50
Lost-track buffer (`track_buffer`)	n_{buf}	Track termination	[10, 150] frames at 30 FPS (0.33–5 s)	30

Let $F(V;\boldsymbol{\theta})$ denote the trajectories that the pipeline produces on sequence V under parameters $\boldsymbol{\theta}$. The primary quality measure is HOTA, which balances detection, association and localisation accuracy in a single score, whereas MOTA over-weights detection and IDF1 over-weights association [33]. MOTA and IDF1 are reported as secondary measures. The problem is studied in three variants of increasing flexibility.

Static parameters. The classical formulation looks for one parameter vector that performs best on average:

$$\boldsymbol{\theta}^{*} = \operatorname*{argmax}_{\boldsymbol{\theta}\in\Theta} \mathbb{E}_{V\sim p(V)}[\mathrm{HOTA}(F(V;\boldsymbol{\theta}),\, G(V))]. \qquad (1.2)$$

Equation (1.2) can be solved either globally, with one vector for all scenes, or separately for each scenario–density condition. The per-condition solution is useful only if the condition of a new sequence is known when it is deployed. Grid search, random search, TPE and CMA-ES approximate the per-condition version of Equation (1.2) by replacing the expectation with the mean HOTA over four fixed, simulator-labelled training clips of the condition (Equation (3.59)) and maximising this mean by repeated evaluation. A global

TPE run treats the global version in the same way, with one training clip of each of the 16 conditions.

Contextual (per-scene) parameters. In the contextual formulation (formulation A), a policy π observes a scene descriptor $\boldsymbol{c}(V)$ and chooses the parameter vector for that scene:

$$\pi^* = \operatorname*{argmax}_{\pi} \mathbb{E}_{V\sim p(V)}[\mathrm{HOTA}(F(V;\pi(\boldsymbol{c}(V))),\, G(V))]. \qquad (1.3)$$

The descriptor is computed only from the detector output of the first four seconds of the sequence, so the policy needs no ground truth and no condition label at deployment. Because each scene requires a single decision, Equation (1.3) is a contextual bandit problem [57], [58]. This thesis solves it with PPO.

Adaptive (online) parameters. In the adaptive formulation (formulation B), the parameters are re-chosen at decision points $t = 1, \ldots, T$, one per second of video. At each decision point the policy observes $\boldsymbol{o}_t$, a summary of the recent behaviour of the detector and the tracker, and chooses the parameter vector $\boldsymbol{\theta}_t$, which is applied to the following one-second window of the running tracker without a reset:

$$\boldsymbol{\theta}_t = \pi(\boldsymbol{o}_t), \qquad \pi^* = \operatorname*{argmax}_{\pi} \mathbb{E}_{V\sim p(V)}\left[\sum_{t=1}^{T} \gamma^{\,t-1} r_t\right]. \qquad (1.4)$$

Here r_t is the tracking reward of the window governed by $\boldsymbol{\theta}_t$, computed against the ground truth relative to a default-parameter run of the same sequence, and γ is a discount factor; for a stochastic policy, the expectation also extends over its actions. Because a parameter change affects which tracks exist in later windows, Equation (1.4) defines a sequential decision problem rather than a sequence of independent decisions. It is treated as a Markov decision process over observations, although the Markov property holds only approximately, because $\boldsymbol{o}_t$ summarises the state of the tracker and of the crowd only partially (Section 3.5.1). This variant targets scenes whose crowd density changes over time.

All three variants share features that make them hard. The objective is piecewise constant in $\boldsymbol{\theta}$, because the tracking output changes only when a parameter change flips a discrete decision, such as a suppression, a match or a track birth; its gradient is therefore zero almost everywhere and undefined at the jumps. The objective is also noisy, because it depends on the random realisation of the crowd, and costly, because every evaluation requires a complete tracking run over a clip or sequence and a full evaluation against the

ground truth (HOTA, or the windowed CLEAR counts of the online reward), which is cheap compared with simulation and detection but must be repeated thousands of times. A further requirement shapes the whole study: no real annotations are available, so the ground truth needed for optimisation and training must come from the simulator, and the resulting method must not need ground truth at deployment. Finally, the detector has to be adapted before any of these variants is meaningful. A COCO-pretrained detector has never seen the rendered pedestrians, and its confidence scores in the synthetic domain determine what every score threshold in Table 1.1 actually means. Transfer learning on the synthetic frames is therefore a prerequisite of the parameter optimisation, not a separate topic.

1.3 Research Objectives and Research Questions

The aim of this thesis is to determine whether, and in what form, reinforcement learning driven by simulator ground truth can optimise the parameters of a YOLOv8 + ByteTrack pedestrian tracking pipeline. To this end the thesis pursues five objectives:

1. To build a simulation-based data-generation environment in which four pedestrian scenarios (corridor, crossing, bottleneck and station plaza), at three crowd densities and with time-varying density, are specified as AnyLogic Pedestrian Library models, executed by a calibrated Python social-force co-simulation and filmed by a virtual camera that delivers compressed video with pixel-exact ground truth.
2. To adapt a COCO-pretrained YOLOv8 detector to the synthetic pedestrian domain by transfer learning, and to quantify its benefit over the zero-shot detector in terms of both detection and tracking accuracy.
3. To implement ByteTrack and the standard tracking metrics in a verified, fast form, and to characterise how sensitive tracking accuracy is to each of the seven parameters across scenarios and densities.
4. To formulate detector–tracker parameter optimisation as a contextual and as an adaptive (sequential) RL problem, and to train PPO agents for both.
5. To compare the RL agents with classical HPO methods on test sequences that were never used for training or tuning, with the training budget of the contextual agent matched to that of random search, TPE and CMA-ES.

These objectives lead to four research questions (RQ1–RQ4).

RQ1. To what extent can transfer learning on automatically labelled simulated pedestrians close the domain gap of a COCO-pretrained YOLOv8 detector, measured by detection accuracy and by the downstream tracking accuracy of ByteTrack?

RQ2. How sensitive is the HOTA of the YOLOv8 + ByteTrack pipeline to its seven parameters, and do the best settings differ between scenarios and crowd densities to an extent that justifies scene-adaptive tuning?

RQ3. Can a PPO policy that observes only detector-derived scene statistics select per-scene parameters that match or exceed random search, TPE and CMA-ES at an equal evaluation budget, how close does it come to a high-budget grid search, and does it generalise to unseen test sequences?

RQ4. Does online re-tuning of the parameters every second improve tracking accuracy over fixed parameters when crowd density changes during a sequence?

RQ1 concerns the perception stage and is a precondition for the others, RQ2 establishes whether the optimisation problem is worth solving, and RQ3 and RQ4 address the two RL formulations directly.

1.4 Contributions

This thesis makes six contributions.

Simulation environment with virtual camera and pixel-exact ground truth. Four pedestrian scenarios are specified as AnyLogic Pedestrian Library models (space markup, a PedSource–PedGoTo–PedSink flowchart with optional waiting areas, and a 3D camera), accompanied by Java code templates, a converter for AnyLogic trajectory exports and a Python bridge template. The experiments use a headless Python co-simulation that implements the same scenarios with an elliptical social-force model calibrated against Weidmann's speed–density relation [59]. A virtual pinhole camera renders articulated pedestrian figures with sensor effects and H.264 compression. For every pedestrian in every frame, an instance-identity buffer gives the number of visible pixels, and the silhouette drawn alone gives the tight amodal box, the silhouette area and the fraction of the body inside the image; the visibility ratio is the ratio of visible to silhouette pixels. The resulting dataset contains 256 sequences with disjoint random seeds for detector training, tuning, validation and testing.

Transfer-learned YOLOv8 for synthetic pedestrians. COCO-pretrained YOLOv8n, YOLOv8s and YOLOv8m models are fully fine-tuned on automatically labelled frames as single-class pedestrian detectors and compared with the zero-shot COCO person detector. NMS keep-masks pre-computed for IoU thresholds from 0.40 to 0.90 in steps of 0.05

(eleven thresholds), and applied post hoc to the loosely suppressed detector output (an approximation, Section 4.6), allow the NMS threshold to be optimised jointly with the tracker parameters without re-running the network.

Verified tracker and metric implementations. ByteTrack is re-implemented in NumPy with its hard-coded thresholds exposed as parameters. Compared with the official code, run with lightweight substitutes for its compiled dependencies `lap` and `cython_bbox`, the re-implementation produced identical track identities and boxes in twelve test runs when both computed IoU with the same convention. Its only documented deviation from the original is that convention: it uses continuous coordinates without the "+1 pixel" box size of `cython_bbox`; switching between the two conventions changed HOTA by at most about 1.3 points in those runs. HOTA, the CLEAR-MOT metrics (CLEAR: Classification of Events, Activities and Relationships) [31] and the identity metrics are re-implemented and were verified to be bit-exact against the reference TrackEval implementation on a 1,350-frame sequence [60]. Together they make thousands of evaluations affordable while keeping the results comparable with the literature.

Two RL formulations of detector–tracker parameter optimisation. In the contextual formulation, a nine-dimensional scene descriptor is mapped to one seven-dimensional parameter vector per clip, and the reward is the HOTA gain over the default configuration. In the adaptive formulation, the parameters are re-chosen every second from window statistics of the running tracker, and the reward is the per-window difference between the policy run and a default-parameter run of $(\mathrm{TP} - \mathrm{FP} - 5\,\mathrm{IDSW})/\mathrm{GT}$, where TP, FP and IDSW are the true positives, false positives and identity switches of the window under a streaming CLEAR-MOT matching and GT is the number of evaluated ground-truth objects in the window. Both are implemented as Gymnasium environments and solved with the Stable-Baselines3 implementation of PPO [61], [62].

Systematic comparison with classical optimisation. The contextual PPO agent is compared at a matched nominal budget of 12,800 clip evaluations (13,056 executed, because Stable-Baselines3 completes its last rollout; Section 4.9.3), the cost of one run of a per-condition baseline over all 16 conditions, with random search and with TPE and CMA-ES implemented with Optuna [30]. The PPO agents are further compared with a globally tuned TPE configuration and with a high-budget grid search over $3^7 = 2{,}187$ configurations per condition. The adaptive agent solves a sequential problem that none of the baselines addresses, so its training budget is not matched. Every stochastic method is

run with three seeds, and all methods are evaluated on the held-out static and dynamic-density test splits, never used for training or tuning, with paired Wilcoxon signed-rank tests against the default configuration, Holm-corrected within each split [63], [64]. The per-condition baselines are deliberately given the true condition label of each test sequence, an advantage the RL policies do not have.

Reproducible code. The complete pipeline, from simulation and rendering through detector training, tracking and evaluation to HPO and RL, is provided with the thesis (Appendix B), with one script per experiment, fixed random seeds and documented software versions.

1.5 Scope and Limitations

The scope of this thesis is set by its methodological choice to replace real annotation with simulation, and several limitations follow from it.

Synthetic data only. All training, tuning and evaluation use synthetic video, so the results characterise the method within the rendered domain and do not establish performance on real surveillance footage. The simulation-to-real gap has two components. In appearance, the rendered figures, textures and lighting are simpler than real people and scenes, so a detector fine-tuned on them is not expected to transfer to real images unchanged. In behaviour, the social-force co-simulation was calibrated against Weidmann's speed–density relation for unidirectional walking (RMSE 0.108 m/s, Section 4.3.3), and in the scenarios density rises with the arrival rate while the motion stays smooth, but it does not reproduce the full variety of real behaviour, such as walking groups, spontaneous stops outside the modelled waiting area or running. The findings most likely to transfer are qualitative: which parameters matter, how their optimal values shift with density, and whether scene-adaptive tuning pays off. Chapter 6 discusses which of the findings can be expected to transfer to real surveillance video.

Co-simulation rather than the AnyLogic runtime. The scenarios are specified as AnyLogic models, but the reported experiments were executed with the Python co-simulation that implements them. The Java code templates, the converter `anylogic_to_ptrack.py` and the Alpyne bridge are provided, but they were not used for the reported experiments; their documentation describes them as templates that must be checked against the AnyLogic version in use. AnyLogic's internal pedestrian model is not documented [50], [51], so the co-simulation cannot reproduce it exactly, and trajectories

from the AnyLogic runtime would differ in detail from those of the co-simulation, even though the pipeline can process both. The thesis therefore does not establish that AnyLogic-runtime and co-simulation trajectories are equivalent, and its results characterise the co-simulation.

Single fixed camera, pedestrians only. Each scenario is observed by one static camera with a fixed pose. Camera motion, multi-camera tracking and changes of viewpoint during a sequence are not considered. Pedestrians are the only object class; vehicles, cyclists and children below the simulated body-height range are not modelled.

Restricted parameter space. The detector weights are fixed during parameter optimisation, and only the seven parameters of Table 1.1 are tuned; the network and the Kalman filter noise model are not. ByteTrack is used without an appearance (re-identification) model, so association relies on motion and overlap alone. Tuning the NMS threshold post hoc on a loosely suppressed candidate set approximates, rather than exactly reproduces, running the detector's own NMS at that threshold.

Computational setting. All experiments run on a single workstation, and conclusions about the relative efficiency of the optimisers apply to the evaluation budgets affordable there.

1.6 Thesis Organisation

The thesis has seven chapters, arranged in four parts as shown in Figure 1.2.

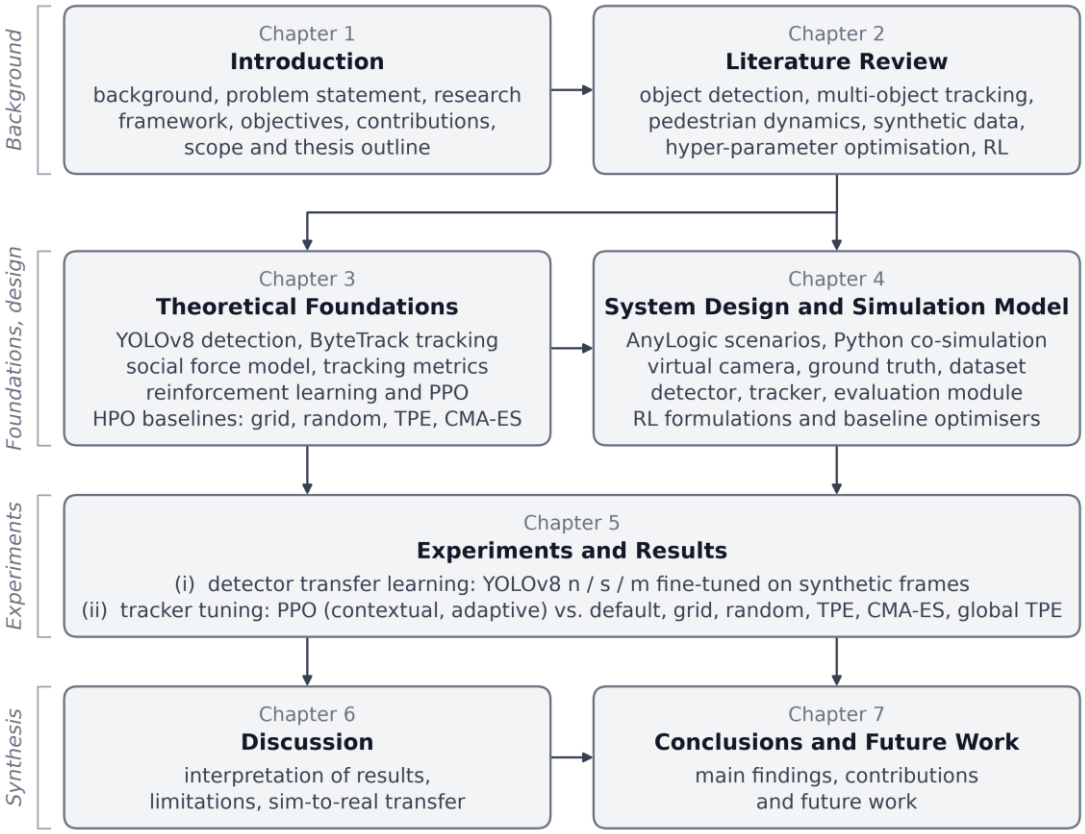


Figure 1.2. Organisation of the thesis into background (Chapters 1–2), foundations and design (Chapters 3–4), experiments (Chapter 5) and synthesis (Chapters 6–7).

The background part consists of the present chapter and Chapter 2, which reviews the literature on object detection with emphasis on the YOLO family and transfer learning, on multi-object tracking and its metrics, on pedestrian dynamics and the AnyLogic Pedestrian Library, on synthetic data, on hyper-parameter optimisation and on reinforcement learning, and which closes by deriving the research gap.

The foundations and design part consists of Chapters 3 and 4. Chapter 3 presents the theory on which the system rests: YOLOv8 detection, the ByteTrack algorithm with its Kalman filter and two-stage association, the social force model, the HOTA, CLEAR-MOT and identity metrics, reinforcement learning with PPO, and the classical HPO baselines, namely grid search, random search, TPE and CMA-ES. Chapter 4 describes the system design: the AnyLogic scenario models and their Python co-simulation, the virtual camera and ground-truth generation, the dataset, the detector transfer learning, the tracker integration and evaluation module, the two RL formulations and the baseline optimisers.

The experimental part, Chapter 5, comprises the detector transfer-learning study, which compares zero-shot and fine-tuned YOLOv8 models, and the tracking-parameter

optimisation study, which covers the sensitivity analysis and the comparison of the contextual and adaptive PPO agents (labelled PPO-contextual and PPO-adaptive in Chapter 5) with grid search, random search, TPE and CMA-ES on static and dynamic-density test sequences.

The synthesis part consists of Chapters 6 and 7. Chapter 6 interprets the results with respect to the research questions and discusses the limitations of the study, in particular the transfer from simulation to real video. Chapter 7 summarises the main findings and contributions and outlines directions for future work. Appendix A documents the AnyLogic model specification and the accompanying code templates needed to rebuild the scenarios, and Appendix B describes the software structure and the scripts that reproduce every experiment.

Chapter 2 Literature Review

The system studied in this thesis forms a closed loop. A pedestrian simulation, specified as AnyLogic Pedestrian Library models and executed for the experiments by a Python co-simulation of the same models (Chapter 4), generates virtual crowds; a virtual surveillance camera renders them as video; a YOLOv8 detector, pre-trained on COCO and transfer-learned on automatically labelled synthetic frames, localises the pedestrians; ByteTrack associates the detections into trajectories; the trajectories are scored against the simulator's ground truth; and a reinforcement-learning (RL) agent trained with proximal policy optimisation (PPO) uses these scores to re-tune the detector's NMS threshold and the tracker parameters. Each link belongs to a different research community, and this chapter is organised accordingly. Section 2.1 reviews deep-learning object detection, Section 2.2 multi-object tracking (MOT), Section 2.3 pedestrian dynamics and the AnyLogic environment, Section 2.4 synthetic data for computer vision, Section 2.5 hyper-parameter optimisation (HPO) and Section 2.6 reinforcement learning. Section 2.7 states the research gap addressed by this thesis.

2.1 Deep-Learning Object Detection

Object detection localises every instance of a set of categories in an image with a bounding box, a class label and a confidence score. In a tracking-by-detection pipeline the detector is the first perceptual stage and its errors propagate into the tracker: the authors of SORT reported that changing the detector alone improved tracking by up to 18.9% [20].

2.1.1 Two-Stage Region-Based Detectors

R-CNN opened the era of deep detection by applying a convolutional neural network (CNN) to bottom-up region proposals, reaching 53.3% mAP on PASCAL VOC 2012, more than 30% higher in relative terms than the previous best [65]. It also showed that when labelled data are scarce, supervised pre-training on ImageNet followed by domain-specific fine-tuning gives a large boost, an early demonstration of transfer learning in detection. Fast R-CNN shared convolutional computation across the whole image and, with VGG16, trained 9 times and tested 213 times faster than R-CNN [66]. Faster R-CNN replaced external proposals with a Region Proposal Network (RPN) that shares full-image features with the detector, running at 5 frames per second (FPS) with 300 proposals per image [67]. Mask R-CNN added a mask branch in parallel with box recognition at small

extra cost [68], and Cascade R-CNN chains detectors trained with increasing IoU thresholds, each refining the hypotheses of the previous stage [69]. Feature Pyramid Networks (FPN) then addressed scale variation with a top-down pathway and lateral connections that make every pyramid level semantically strong [70], a design that survives in the necks of modern YOLO models. Two-stage detectors remain accurate, but their sequential structure limits throughput, a drawback when every frame of a video must be processed inside an optimisation loop.

2.1.2 One-Stage, Anchor-Free and Transformer-Based Detectors

One-stage detectors predict boxes and classes densely in a single pass. SSD predicts offsets for default boxes of several scales and aspect ratios on several feature maps, reaching 72.1% mAP on VOC 2007 at 58 FPS with 300 × 300 inputs [71]. RetinaNet traced the accuracy gap between one- and two-stage detectors to the extreme foreground–background imbalance of dense prediction and introduced the focal loss [72],

$$\mathrm{FL}(p_t) = -\alpha_t(1-p_t)^{\gamma}\log(p_t), \qquad (2.1)$$

which down-weights easy examples ($\gamma = 2$, $\alpha = 0.25$ worked best); with nine anchors per location on a ResNet-101-FPN backbone it reached 39.1 AP on COCO test-dev. Anchors, however, introduce dataset-specific hyper-parameters. Anchor-free detectors avoid them by predicting a pair of box corners [73], per-pixel distances to the box edges [74] or object centre points [75]. YOLOX made the YOLO detector anchor-free and added a decoupled head and SimOTA label assignment, with YOLOX-L reaching 50.0% AP on COCO at 68.9 FPS [76]; YOLOv8 later adopted the anchor-free, decoupled-head design [15]. Transformer-based detectors form a fourth family, which treats detection as set prediction: DETR combines a transformer encoder–decoder, learned object queries and a bipartite-matching loss, removing anchors and non-maximum suppression (NMS) while matching an optimised Faster R-CNN on COCO [77]. Deformable DETR attends only to a few sampling points around a reference and outperforms DETR with ten times fewer training epochs [78], and RT-DETR was the first real-time end-to-end transformer detector [79]. NMS removal has since reached the YOLO family, as YOLOv10 trains NMS-free models through consistent dual assignments [80]. The four families are summarised in Figure 2.1.

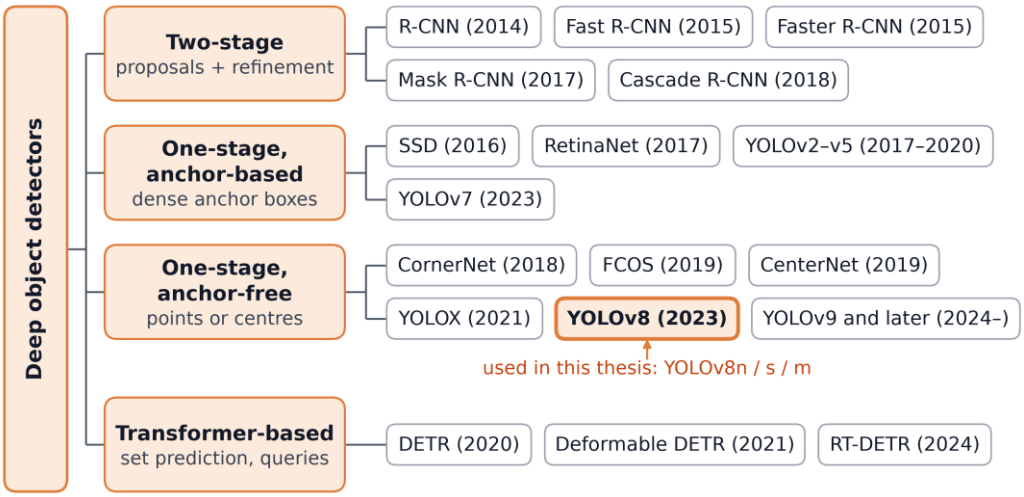


Figure 2.1. Taxonomy of deep-learning object detectors (two-stage, one-stage anchor-based, one-stage anchor-free, transformer-based), including Mask R-CNN [68], Cascade R-CNN [69], CornerNet [73], FCOS [74], CenterNet [75], Deformable DETR [78] and RT-DETR [79]; years are those of the peer-reviewed venue, otherwise of the first public release.

2.1.3 The YOLO Family of Real-Time Detectors

YOLO treats detection as one regression from image pixels to boxes and class probabilities [14]. The image is divided into an $S \times S$ grid whose cells each predict B boxes and C class probabilities; for PASCAL VOC ($S = 7, B = 2, C = 20$) a network of 24 convolutional and 2 fully connected layers outputs a $7 \times 7 \times 30$ tensor, reaching 63.4 mAP at 45 FPS. Reasoning over the whole image gave fewer background false positives but more localisation errors than contemporary detectors.

Later versions kept the single-pass design while absorbing ideas from the wider literature, as shown in Figure 2.2. YOLOv2 bundled a set of improvements and YOLO9000 could detect more than 9000 categories [81]. YOLOv3 introduced the Darknet-53 backbone, three prediction scales with three of nine anchor clusters each, and logistic class predictors [82]. YOLOv4 combined a CSPDarknet53 backbone, a neck of spatial pyramid pooling (SPP) [83] and a path aggregation network (PAN), and the YOLOv3 head, and added a “bag of freebies” (training-only refinements such as Mosaic augmentation and the CIoU loss) and a “bag of specials” (cheap inference-time modules) [84]. Its backbone rests on the cross-stage partial (CSP) design, which integrates feature maps from the beginning and end of each stage and cuts computation by 20% without loss of accuracy [85], and its neck on PANet’s bottom-up path augmentation, which brings low-level localisation signals to the top of the hierarchy [86].

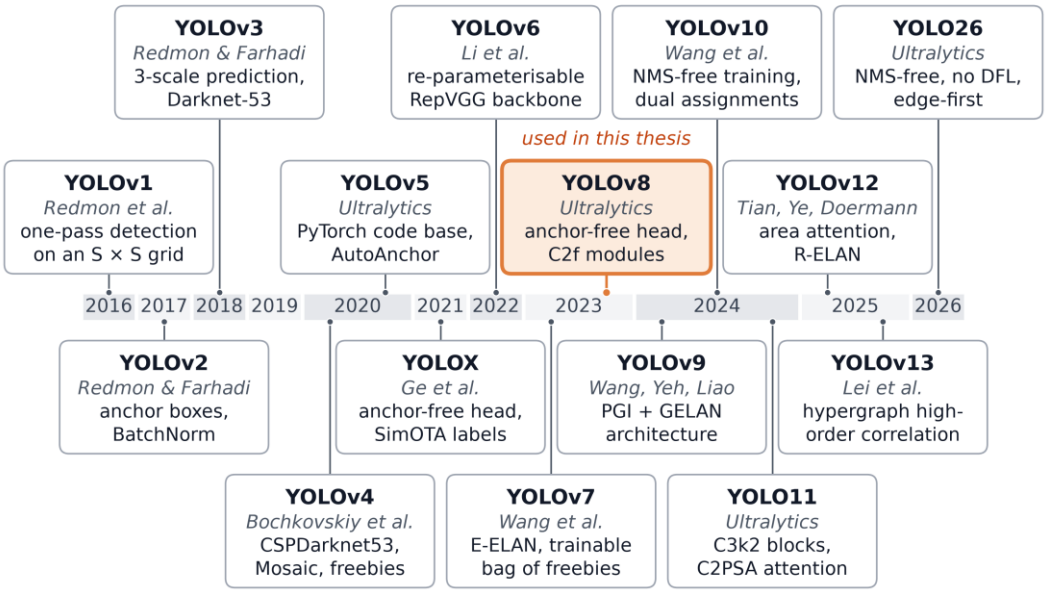


Figure 2.2. Evolution of the YOLO family of real-time detectors (versions are evenly spaced, so the time axis is not to scale; years are those of the peer-reviewed venue, otherwise of the first public release).

Since 2020 development has diversified across Ultralytics YOLOv5 [87], YOLOX [76], the industry-oriented YOLOv6 [88], YOLOv7 with its trainable bag-of-freebies [89] and Ultralytics YOLOv8 [17]. Recent versions target information loss and post-processing: YOLOv9 introduces Programmable Gradient Information and the GELAN architecture [90], YOLOv10 removes NMS [80], YOLO11 refines the backbone and neck so that YOLO11m exceeds YOLOv8m with 22% fewer parameters [91], YOLOv12 makes the design attention-centric with area attention and R-ELAN blocks at CNN-like speed [92], YOLOv13 adds hypergraph-based correlation enhancement [93], and YOLO26, released in January 2026, offers native NMS-free end-to-end inference and removes Distribution Focal Loss [94]. Table 2.1 summarises the principal versions; Terven et al. review the family up to YOLOv8 [15].

Table 2.1. Principal versions of the YOLO family (results as reported by the authors; datasets, input sizes and hardware differ, so the figures are not directly comparable). Years are those of the peer-reviewed venue, or of the first public release for preprints and software. "–": not quoted here.

Version (year)	Main architectural or training contribution	Result reported by the authors
YOLOv1 (2016) [14]	Single regression on an $S \times S$ grid	63.4 mAP, VOC 2007, 45 FPS
YOLOv2 (2017) [81]	Set of improvements; joint detection–classification training	76.8 mAP, VOC 2007, 67 FPS
YOLOv3 (2018) [82]	Darknet-53; three prediction scales; logistic classifiers	57.9 mAP@50, COCO, 51 ms
YOLOv4 (2020) [84]	CSPDarknet53, SPP + PAN; bag of freebies/specials	43.5% AP, COCO, about 65 FPS

Version (year)	Main architectural or training contribution	Result reported by the authors
YOLOv5 (2020) [87]	Ultralytics PyTorch implementation	–
YOLOX (2021) [76]	Anchor-free; decoupled head; SimOTA	50.0% AP (L), COCO, 68.9 FPS
YOLOv6 (2022) [88]	Industrial design; quantisation	43.5% AP (S), COCO, 495 FPS (T4)
YOLOv7 (2023) [89]	Trainable bag-of-freebies	56.8% AP, COCO, ≥30 FPS
YOLOv8 (2023) [17]	C2f, SPPF, PAN-FPN; anchor-free split head; CIoU + DFL	37.3–53.9 mAP50-95 (n–x)
YOLOv9 (2024) [90]	Programmable Gradient Information; GELAN	–
YOLOv10 (2024) [80]	NMS-free consistent dual assignments	1.8× faster than RT-DETR-R18 [79] at similar AP (S)
YOLO11 (2024) [91]	Improved backbone and neck	39.5–54.7 mAP50-95 (n–x)
YOLOv12 (2025) [92]	Attention-centric: area attention, R-ELAN	40.5% mAP (N), COCO, 1.62 ms (T4)
YOLOv13 (2025) [93]	Hypergraph correlation enhancement (HyperACE, FullPAD)	+3.0 mAP over YOLO11-N, +1.5 over YOLOv12-N (N), COCO
YOLO26 (2026) [94]	NMS-free dual head; no DFL; MuSGD optimiser; STAL	40.9–57.5 mAP50-95 (n–x; one-to-many head; NMS-free head 40.1–56.9)

YOLOv8 is adopted here because its architecture and losses are fully specified in official code, its scaled variants span 3.2 to 68.2 million parameters (37.3 to 53.9 COCO mAP50-95), allowing a model size to be chosen whose one-off inference over all videos is affordable (detections are computed once and cached, so the optimisation loop only selects pre-computed NMS results and re-runs ByteTrack and the evaluation; Chapter 4), and the same framework provides integrated ByteTrack and BoT-SORT tracking [17], [24]. Newer versions are more accurate, and some (YOLOv10, YOLO26) are NMS-free and therefore lack the NMS threshold tuned here; the optimisation method proposed in this thesis is not tied to a particular version.

2.1.4 YOLOv8: Architecture, Training Objective and Post-Processing

The YOLOv8 backbone stacks strided convolutions and C2f blocks, cross-stage partial bottlenecks with two convolutions, and ends in a fast spatial pyramid pooling (SPPF) layer, the Ultralytics form of SPP [15], [17], [83]. The neck follows a top-down path (upsampling, concatenation, C2f) and then a bottom-up path (strided convolution, concatenation, C2f), the PAN-FPN design that combines FPN and PANet [70], [86]. The head predicts at strides 8, 16 and 32 (P3–P5) and is anchor-free and split: the box branch outputs 4×16 values, sixteen discrete bins per box edge, separately from the classification branch.

Classification is trained with binary cross-entropy and box regression with two terms. The Complete-IoU (CIoU) loss penalises poor overlap, centre distance and aspect-ratio mismatch [95]:

$$\mathcal{L}_{\mathrm{CIoU}} = 1 - \mathrm{IoU} + \frac{\rho^2(\boldsymbol{b}, \boldsymbol{b}^{gt})}{c^2} + \alpha v, \qquad v = \frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}} - \arctan\frac{w}{h}\right)^2, \qquad \alpha = \frac{v}{(1-\mathrm{IoU}) + v}, \qquad (2.2)$$

where ρ is the distance between box centres and c the diagonal of the smallest enclosing box. It descends from Generalised IoU, proposed because a plain IoU loss gives no gradient for non-overlapping boxes [96]. The Distribution Focal Loss (DFL) treats each edge as a discrete distribution and concentrates probability on the two bins y_i, y_{i+1} nearest the target y [97]:

$$\mathrm{DFL}(S_i, S_{i+1}) = -((y_{i+1} - y)\log S_i + (y - y_i)\log S_{i+1}). \qquad (2.3)$$

Positive samples are chosen by the task-aligned assigner introduced with TOOD [98], which the Ultralytics implementation runs with top-k = 10, $\alpha = 0.5$ and $\beta = 6.0$; the default loss gains are 7.5 (box), 0.5 (classification) and 1.5 (DFL) [17].

At inference, predictions are filtered by a confidence threshold (default 0.25) and by NMS with an IoU threshold (default 0.7) [17]. In crowds, where occlusion is the main difficulty of human detection [99], the NMS threshold is a genuine trade-off: a low value suppresses correct detections of overlapping people, while a high value retains duplicates that the tracker may turn into spurious tracks. Variants such as DIoU-NMS change the suppression criterion but not the trade-off [95]. The confidence threshold also interacts with the tracker: ByteTrack exploits low-score detections (Section 2.2.3), so a confidence threshold above its low-score threshold narrows its second association, and one at or above its high-score threshold, as the Ultralytics predict-mode default of 0.25 would be, leaves the second association empty, which is why Ultralytics lowers the default confidence to 0.1 in tracking mode [24]. Detector post-processing and tracker parameters are therefore coupled. In this thesis the NMS threshold and the tracker parameters are tuned jointly, while the confidence threshold is fixed at the low value of 0.05 so that the tracker receives the low-score boxes it needs.

2.1.5 Transfer Learning and Fine-Tuning for Pedestrian Detection

Transfer learning addresses training and deployment data that differ in feature space or distribution. Pan and Yang define a domain as a feature space with a marginal distribution $P(X)$ and a task as a label space with a predictive function, and distinguish inductive, transductive and unsupervised settings [45]; a later survey of more than forty approaches concludes that the best method depends on the application [100]. Here the source domain is COCO, with 91 object types and 2.5 million labelled instances in 328k images [16], which supplies the pre-trained weights; the target domain is frames rendered by the virtual camera from the simulated scenes, whose labels the simulator produces automatically. The setting is therefore inductive transfer by fine-tuning.

Fine-tuning is preferred to training from scratch for well-documented reasons. Early CNN layers learn general features while later layers become task-specific, transferability declines with task distance but still beats random initialisation, and transferred initialisation improves generalisation even after fine-tuning [46]. Across 16 networks and 12 datasets, ImageNet accuracy correlates with transfer accuracy at $r = 0.99$ for fixed features and $r = 0.96$ for fine-tuning [101]. Pedestrian benchmarks show where detectors struggle: the Caltech evaluation found performance disappointing for low-resolution and partially occluded pedestrians [102], CityPersons showed that annotation diversity improves generalisation under heavy occlusion and at small scale [103], and CrowdHuman provides about 470K human instances in its training and validation sets, with about 22.6 persons per image [99]. A virtual surveillance camera shares the elevated viewpoint, small object size and occlusion of these benchmarks but renders avatars rather than people; this appearance gap is discussed in Section 2.4.

2.2 Multi-Object Tracking

2.2.1 Problem Formulation and Paradigms

MOT estimates the trajectories of a time-varying number of targets while preserving their identities. Reviews organise the field around appearance and motion modelling, interaction and occlusion handling, and inference [13], or decompose trackers into detection, feature extraction and motion prediction, affinity computation and association [12]. Three paradigms dominate, as illustrated in Figure 2.3: *tracking-by-detection*, in which an independent detector feeds a separate association step; *joint detection and*

tracking, in which one network outputs detections together with association cues; and *query-based transformers*, in which learned track queries carry identities across frames.

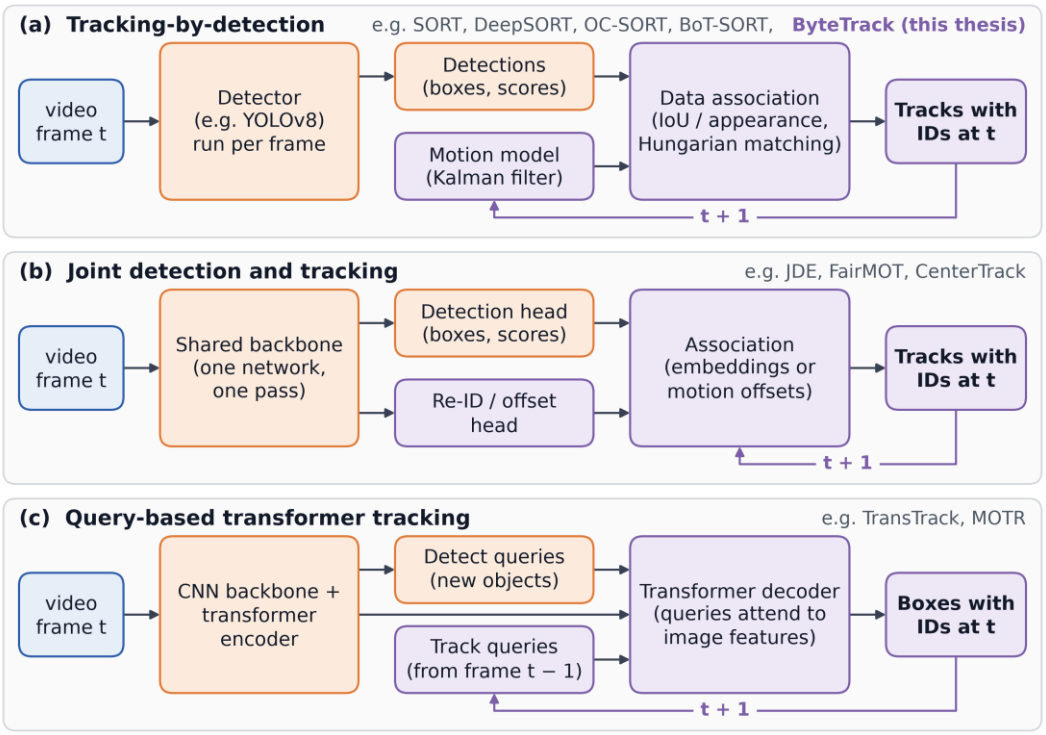


Figure 2.3. Multi-object tracking paradigms: tracking-by-detection, joint detection and tracking, and query-based transformers.

2.2.2 Kalman Filtering, Assignment and the SORT Lineage

Tracking-by-detection rests on two classical tools. The Kalman filter alternately predicts a state with a linear model and corrects it with a measurement, giving the minimum-mean-square-error estimate for linear systems with Gaussian noise (and the best linear estimator otherwise) [18]:

$$\hat{\boldsymbol{x}}_{t|t-1} = \boldsymbol{F}\hat{\boldsymbol{x}}_{t-1|t-1}, \qquad \boldsymbol{P}_{t|t-1} = \boldsymbol{F}\boldsymbol{P}_{t-1|t-1}\boldsymbol{F}^{\top} + \boldsymbol{Q}, \tag{2.4}$$

$$\boldsymbol{K}_t = \boldsymbol{P}_{t|t-1}\boldsymbol{H}^{\top}\left(\boldsymbol{H}\boldsymbol{P}_{t|t-1}\boldsymbol{H}^{\top} + \boldsymbol{R}\right)^{-1}, \qquad \hat{\boldsymbol{x}}_{t|t} = \hat{\boldsymbol{x}}_{t|t-1} + \boldsymbol{K}_t\left(\boldsymbol{z}_t - \boldsymbol{H}\hat{\boldsymbol{x}}_{t|t-1}\right). \tag{2.5}$$

The second is the linear assignment problem, solved exactly by the Hungarian method [19] and Munkres' algorithm [104], and also exactly, but faster in practice, by the Jonker–Volgenant algorithm (LAPJV) [105], which both the ByteTrack code and Ultralytics use with a cost limit per association: the matching threshold in the first association and fixed values of 0.5 and 0.7 in the second and unconfirmed associations (Table 2.2).

SORT combined these tools in a minimal tracker [20]. Each target has state $[u, v, s, r, \dot{u}, \dot{v}, \dot{s}]^{\top}$ (box centre, area and a constant aspect ratio) with constant-velocity

dynamics; detections are matched to predicted boxes by IoU distance, matches below a minimum IoU are rejected, and tracks end after a single missed frame in the reported experiments. SORT ran at 260 Hz but lost identities through occlusion. DeepSORT added a CNN appearance descriptor pre-trained on the MARS re-identification (Re-ID) dataset, Mahalanobis gating, a matching cascade favouring recently seen tracks, three-frame tentative initialisation and deletion after 30 unmatched frames; on MOT16 identity switches fell from 1423 to 781 [106].

2.2.3 ByteTrack: Associating Every Detection Box

ByteTrack observes that trackers usually discard low-score detections even though many belong to occluded or blurred objects [21]. Its association method, BYTE, splits the detections by score s into a high-score set ($s > \tau_{\text{high}}$) and a low-score set ($\tau_{\text{low}} < s < \tau_{\text{high}}$); boxes with $s \leq \tau_{\text{low}}$ are discarded. The first association matches high-score boxes to all Kalman-predicted tracks, including recently lost ones, by IoU or Re-ID similarity with the Hungarian algorithm; outside its MOT20 setting the official code fuses overlap and score into the cost $1 - \text{IoU} \cdot s$ [22]. The second association matches low-score boxes by IoU alone, since their appearance is unreliable, to the remaining tracks that are still in the tracked state; lost tracks are excluded in the official code. Newly started, unconfirmed tracks are then matched to the remaining high-score boxes with a fixed cost limit of 0.7 [22]. Unmatched low-score boxes are discarded as background, unmatched tracks are kept as lost for 30 frames, and new tracks start only from unmatched high-score boxes. With a YOLOX-X detector ByteTrack reached 80.3 MOTA, 77.3 IDF1 and 63.1 HOTA on MOT17 at 29.6 FPS, and inserting BYTE into nine other trackers raised their IDF1 by 1 to 10 points [21].

ByteTrack's behaviour is governed by a few interpretable thresholds, listed in Table 2.2 with the YOLOv8 post-processing parameters. Their defaults differ markedly between sources: the original code uses a high-score threshold of 0.6, whereas Ultralytics uses 0.25 for both the high-score and new-track thresholds [21], [22], [24]. This is not an error. Score thresholds are meaningful only relative to a detector's score distribution, which changes with its architecture, training data and scene. The official evaluation code itself overrides the defaults for individual benchmark sequences, setting the track buffer to 14 frames for MOT17-05 and MOT17-06 and to 25 for MOT17-13 and MOT17-14, and the high-score threshold to 0.65 for MOT17-01 and MOT17-06, 0.7 for MOT17-12, 0.67 for MOT17-14 and 0.3 for MOT20-06 and MOT20-08 [22], direct evidence of manual, scene-dependent

tuning. A detector fine-tuned on synthetic frames that observes crowds of changing density cannot be expected to share the optimal thresholds of a YOLOX-X detector trained for MOT17. This dependence on detector and scene is the practical motivation for automatic, adaptive tuning.

Table 2.2. Detector post-processing and ByteTrack parameters with their defaults in the ByteTrack paper [21], the official code [22] and Ultralytics [24], and their search ranges and defaults in this thesis (Chapter 4); the seven parameters with a range are tuned, the other two are fixed ("–": not specified in the cited source).

Parameter	Role in the pipeline	Paper	Official code	Ultralytics	This thesis (range; default)
Confidence threshold	Discards detections before tracking	–	0.01	0.25 (predict) / 0.1 (track mode)	fixed (0.05)
NMS IoU threshold	Suppresses overlapping detections	–	0.7	0.7	[0.40, 0.90]; 0.70
High-score threshold	Splits high- and low-score sets	0.6	0.6	0.25	[0.20, 0.80]; 0.50
Low-score threshold	Lower bound of second association	–	0.1	0.1	[0.05, 0.40]; 0.10
New-track threshold	Minimum score to start a track	–	high-score + 0.1	0.25	[0.20, 0.90]; 0.60
First-association cost limit (matching threshold)	Cost limit of first association	IoU < 0.2 rejected	0.9	0.8	[0.50, 0.95]; 0.80
Second-association cost limit	IoU-distance limit for low-score boxes	–	0.5 (hard-coded)	0.5 (hard-coded)	[0.20, 0.90]; 0.50
Unconfirmed-track cost limit	Cost limit for newly started tracks	–	0.7 (hard-coded)	0.7 (hard-coded)	fixed (0.7)
Lost-track buffer	Frames a lost track is kept	30	30 (scaled by frame rate)	30	[10, 150]; 30 (at 30 FPS, scaled by frame rate)

2.2.4 Successors of ByteTrack

Several later tracking-by-detection methods refine the SORT/ByteTrack design; Table 2.3 compares them with their predecessors and with the joint and transformer-based trackers of Section 2.2.5. OC-SORT notes that the linear Kalman filter accumulates error while a target is unobserved and adds observation-centric re-updating, a motion-direction consistency cost and a recovery association, remaining purely motion-based at more than 700 FPS on a CPU [107]. Deep OC-SORT adds adaptively weighted appearance matching [108]. BoT-SORT fuses motion and appearance, compensates for camera motion and estimates box width and height directly instead of aspect ratio [109]. StrongSORT upgrades every DeepSORT component, including camera-motion compensation and a

noise-scale-adaptive Kalman filter, replaces the matching cascade with global assignment and adds offline linking and Gaussian-smoothed interpolation [110].

Table 2.3. Representative multi-object trackers (TbD: tracking-by-detection; JDT: joint detection and tracking; results as reported with different detectors; "–": not quoted here; BoT: the strong person re-identification baseline adopted by StrongSORT; AFLink: appearance-free link model [110]).

Tracker	Paradigm	Motion cue	Appearance cue	Association	MOT17 test (MOTA / IDF1 / HOTA)
SORT [20]	TbD	Kalman filter	None	IoU + Hungarian	– (MOT15: 33.4 MOTA)
DeepSORT [106]	TbD	Kalman + Mahalanobis gate	CNN Re-ID	Cascade + IoU	– (MOT16: 61.4 MOTA)
ByteTrack [21]	TbD	Kalman filter	Optional	High- then low-score boxes	80.3 / 77.3 / 63.1
OC-SORT [107]	TbD	Observation-centric Kalman	None	With recovery stage	78.0 / 77.5 / 63.2
BoT-SORT [109]	TbD	Kalman (w, h) + camera compensation	Re-ID	Fused IoU and Re-ID	80.5 / 80.2 / 65.0
StrongSORT++ [110]	TbD	NSA Kalman + camera compensation	BoT + EMA	Global + AFLink + GSI	79.6 / 79.5 / 64.4
Deep OC-SORT [108]	TbD	OC-SORT	Adaptive Re-ID	OC-SORT	79.4 / 80.6 / 64.9
FairMOT [111]	JDT	–	64-d Re-ID branch	Joint detection + embedding	73.7 / 72.3 / –
CenterTrack [112]	JDT	Predicted offsets	None	Point-based	67.3 / – / –
TransTrack [113]	Transformer	Track queries	Query features	Query-based	74.5 / – / –

The successors gain accuracy mainly through appearance models, camera-motion compensation and offline post-processing. A fixed virtual camera has no ego-motion, offline interpolation is incompatible with online re-tuning, and rendered avatars may vary less in appearance than real people, the condition targeted by DanceTrack, on whose uniform-appearance, non-linear-motion videos state-of-the-art trackers performed far worse than on MOT17/MOT20 [114]. ByteTrack thus offers a strong, fast, online baseline whose few parameters have clear meanings, which makes it a suitable object of parameter optimisation.

2.2.5 Joint Detection-and-Tracking and Transformer-Based Trackers

Tracktor showed that a detector's box regression can propagate objects between frames without tracking-specific training [115]. JDE embedded the appearance model in a single-shot detector and achieved near-real-time MOT [116]; FairMOT balanced the detection and Re-ID branches on an anchor-free backbone and found low-dimensional

embeddings preferable [111]; and CenterTrack predicts associations from an image pair and the previous detections [112]. Transformer trackers extend DETR to video: TransTrack uses previous-frame features as queries [113], MOTR propagates track queries through the whole video without heuristic association and outperformed ByteTrack by 6.5% HOTA on DanceTrack [117], MOTRv2 bootstraps it with a pretrained detector [118], and MOTIP recasts association as in-context identity prediction [119]. End-to-end trackers have fewer hand-set thresholds, but they require identity-labelled training video and cannot be adjusted at run time through interpretable parameters, so the modular paradigm suits a study of run-time tuning better.

2.2.6 Benchmarks and Evaluation Metrics

The MOTChallenge benchmarks standardised pedestrian tracking evaluation: MOT15 with 11 training and 11 test sequences [120], MOT16 with 14 new, crowded sequences [34], MOT17 with refined MOT16 labels and three public detectors [35], and MOT20 with mean densities reaching 246 pedestrians per frame [23]. DanceTrack targets uniform appearance and non-linear motion [114].

Three metric families are in use. The CLEAR MOT metrics define tracking accuracy [31]

$$\mathrm{MOTA} = 1 - \frac{\sum_t (m_t + fp_t + mme_t)}{\sum_t g_t}, \qquad (2.6)$$

where m_t, fp_t and mme_t are misses, false positives and identity mismatches in frame t and g_t the number of ground-truth objects. MOTA lies in $(-\infty, 1]$ and becomes negative when the errors outnumber the ground-truth objects. Because misses and false positives usually outnumber identity switches, MOTA is dominated by detection. IDF1 is computed from a single one-to-one matching of whole trajectories and therefore rewards consistent identities [32]:

$$\mathrm{IDF1} = \frac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}}. \qquad (2.7)$$

HOTA was proposed because MOTA over-weights detection and IDF1 over-weights association [33]:

$$\mathrm{HOTA}_\alpha = \sqrt{\mathrm{DetA}_\alpha \cdot \mathrm{AssA}_\alpha}, \qquad \mathrm{DetA}_\alpha = \frac{|\mathrm{TP}_\alpha|}{|\mathrm{TP}_\alpha| + |\mathrm{FN}_\alpha| + |\mathrm{FP}_\alpha|}, \qquad (2.8)$$

$$\mathrm{AssA}_\alpha = \frac{1}{|\mathrm{TP}_\alpha|} \sum_{c \in \mathrm{TP}_\alpha} \frac{|\mathrm{TPA}(c)|}{|\mathrm{TPA}(c)| + |\mathrm{FNA}(c)| + |\mathrm{FPA}(c)|}, \qquad \mathrm{HOTA}$$

$$= \frac{1}{19} \sum_{\alpha \in \{0.05, 0.10, \ldots, 0.95\}} \mathrm{HOTA}_\alpha\,, \qquad (2.9)$$

where α is the localisation (IoU) threshold of a match and, for each true positive c, the sets $\mathrm{TPA}(c)$, $\mathrm{FNA}(c)$ and $\mathrm{FPA}(c)$ collect, over the whole sequence, the true-positive, false-negative and false-positive associations of its ground-truth and predicted identities. TrackEval is the official implementation of these metrics [60].

The metric choice shapes reward design. An agent rewarded with MOTA would favour detection-related parameters and neglect association, whereas HOTA, a geometric mean of detection and association accuracy, penalises neglect of either. HOTA and IDF1, however, are defined over whole sequences, IDF1 through a single trajectory matching and HOTA through association counts accumulated over all frames, so neither can be split into per-frame or per-window rewards. An agent that re-tunes parameters online therefore needs a CLEAR-type surrogate whose error counts can be summed per window [31]. The adaptive formulation of this thesis uses $(\mathrm{TP} - \mathrm{FP} - 5\,\mathrm{IDSW})/\mathrm{GT}$ per window, relative to a default-parameter run: a window-level MOTA in which identity switches weigh five times as much, to offset MOTA's bias towards detection. All three metrics require ground-truth trajectories, which are costly to annotate in real video but available for every agent in every frame of a simulation, the property on which the closed loop of this thesis depends.

2.3 Pedestrian Dynamics Modelling and Simulation

2.3.1 Modelling Scales and Representations

The realism of the simulated crowd determines how representative the tracking problem is. Pedestrian models are classified by scale and representation, as shown in Figure 2.4. Macroscopic models treat the crowd as a continuum; Hughes derived coupled nonlinear partial differential equations for two-dimensional pedestrian flow [121]. Mesoscopic models describe the crowd through distributions of individual properties such as speed: Henderson found measured crowd speed distributions to agree well with the Maxwell–Boltzmann theory of gases [122], and gas-kinetic models evolve a phase-space density of pedestrians [123]. Microscopic models represent each pedestrian as an agent and include force-based, cellular-automaton, discrete-step, velocity-based and heuristic models.

Reviews compare these families against crowd phenomena such as bottleneck, crossing and counterflow behaviour [124], survey individual behaviour models and self-organised crowd patterns [2], classify evacuation models into seven approaches [125] and stress the need for empirical calibration data [126]. Only microscopic models suit the present purpose, because a camera observes, and a tracker follows, individual bodies.

level of detail: aggregate → individual

Level	Model family	Formulation	Space / time	Key references
Macroscopic crowd as fluid	Continuum / fluid-dynamic	PDE: conservation law + eikonal route choice	continuous / continuous	*Hughes 2002*
Mesoscopic crowd as gas	Gas-kinetic	kinetic equation for the phase-space density	continuous / continuous	*Henderson 1971; Hoogendoorn & Bovy 2000*
Microscopic individual pedestrians	**Social force** (this thesis†)	ODE: Newtonian driving + repulsive forces	continuous / continuous*	*Helbing & Molnár 1995*
	Cellular automata	transition rules, floor fields on a cell lattice	discrete / discrete	*Burstedde et al. 2001*
	Discrete-step (Optimal Steps)	next position on a circle of step-length radius	continuous / discrete	*Seitz & Köster 2012*
	Velocity-based (ORCA)	collision-free velocity from a linear program	continuous / discrete	*van den Berg et al. 2011*
	Heuristic	behavioural heuristics (visual distance)	continuous / discrete	*Moussaïd et al. 2011*

** integrated numerically with a small time step*
† scenarios specified as AnyLogic models, run by a Python social-force co-simulation (Chapter 4)

Figure 2.4. Classification of pedestrian dynamics models by scale (macroscopic, mesoscopic, microscopic) and representation, with key references [47], [121], [122], [123], [127], [128], [129], [130].

2.3.2 The Social Force Model

The social force model describes pedestrians as if driven by "social forces" representing their internal motivation: an acceleration towards a desired velocity, repulsion from other pedestrians and boundaries, and attraction [47]. In the escape-panic formulation [3],

$$m_i \frac{d\boldsymbol{v}_i}{dt} = m_i \frac{v_i^0 \boldsymbol{e}_i^0 - \boldsymbol{v}_i}{\tau_i} + \sum_{j \neq i} \boldsymbol{f}_{ij} + \sum_{W} \boldsymbol{f}_{iW}\,, \qquad (2.10)$$

$$\boldsymbol{f}_{ij} = \left\{ A_i \exp\left[\frac{r_{ij} - d_{ij}}{B_i}\right] + k\, g(r_{ij} - d_{ij}) \right\} \boldsymbol{n}_{ij} + \kappa\, g(r_{ij} - d_{ij})\, \Delta v_{ji}^t\, \boldsymbol{t}_{ij}, \qquad (2.11)$$

where $v_i^0 \boldsymbol{e}_i^0$ is the desired velocity, τ_i a relaxation time, r_{ij} the sum of body radii, d_{ij} the distance between centres, and $g(x)$ is non-zero only on contact. With $m = 80$ kg, $\tau = 0.5$ s, $A = 2 \times 10^3$ N and $B = 0.08$ m, the model reproduced clogging at exits and the "faster-is-slower" effect [3], and experiments later confirmed self-organisation phenomena it predicts [1]. Calibration is a recurring theme: Johansson et al. fitted social force

parameters to video-tracking trajectories with an evolutionary algorithm [9], an early coupling of tracking and pedestrian-model optimisation, though in the opposite direction to this thesis. A review describes the model as one of the most widespread continuous pedestrian models [48], while critics note that force models can be hard to calibrate [130].

2.3.3 Cellular-Automaton, Velocity-Based and Heuristic Models

Cellular automata discretise space into cells. In the floor-field model each pedestrian moves at most one cell per step, guided by a field that diffuses, decays and is reinforced by motion, which alone produces lane formation [127]; in evacuation, intermediate coupling to the floor fields minimises evacuation time [131]. Grid-constrained motion is efficient but looks unnatural on video, and the Optimal Steps Model keeps CA-like rules while letting pedestrians step to any point on a circle of step-length radius [128]. Velocity-based models choose collision-free velocities directly; ORCA gives each agent half the responsibility for every pairwise avoidance and was demonstrated with 25,000 agents [129]. Moussaïd et al. instead proposed two vision-based heuristics for speed and direction that reproduce lane formation, stop-and-go waves and crowd turbulence [130].

2.3.4 The Fundamental Diagram

Simulated crowds are usually validated against the fundamental diagram, the empirical density–speed relation. Weidmann's review condensed 25 studies into a planar diagram with a maximal density of 5.4 ped/m^2 and a free walking speed of 1.34 m/s [8], [59]. Seyfried et al. found a linear relation between speed and the length required per pedestrian, $d = 0.36 + 1.06\,v$, and good agreement between one- and two-dimensional diagrams for densities of 1 to 5 ped/m^2 [8]. Matching the fundamental diagram makes speeds realistic for a given density, a necessary though not sufficient condition for realistic motion and occlusion patterns at the densities chosen for the scenarios. Table 2.4 compares the model families.

Table 2.4. Comparison of pedestrian dynamics model families.

Model family	Scale	Mechanism	Representative works	Relevance to this thesis
Continuum	Macroscopic	Nonlinear PDEs for density and velocity	[121]	No individual trajectories to render or track
Social force	Microscopic	Driving, repulsive and contact forces	[3], [47]	Used in this thesis (Chapter 4); reported in an AnyLogic-based calibration study to underlie the

Model family	Scale	Mechanism	Representative works	Relevance to this thesis
				AnyLogic Pedestrian Library [10]; smooth trajectories
Cellular automaton	Microscopic	Grid transitions guided by floor fields	[127], [131]	Efficient, but grid motion looks unnatural on video
Discrete-step	Microscopic	Optimal step on a circle	[128]	Realistic stepping in continuous space
Velocity-based	Microscopic	Collision-free velocity selection	[129]	Scales to very large crowds
Heuristic	Microscopic	Vision-based behavioural rules	[130]	Collective patterns from few parameters

2.3.5 The AnyLogic Pedestrian Library and Its Python Interfaces

AnyLogic is a multi-method environment supporting agent-based, system dynamics and discrete-event modelling in one model [49], [132]. Its Pedestrian Library simulates pedestrians in continuous space reacting to walls, obstacles and each other, with flows assembled from blocks such as `PedSource`, `PedGoTo` and `PedSink`, and results viewable in two and three dimensions [50]. The movement equations are not documented. Siebers et al. call such embedded libraries a black box, as opposed to white-box implementations with full parameter control [51], and the library's motion model is reported in an AnyLogic-based calibration study to be social-force based [10].

The library has been applied widely to transport facilities, for example to size ticket windows in a Hangzhou subway station [133] and to simulate panic during metro-platform evacuation in Beijing [134]. Three studies combine AnyLogic with YOLO detectors. Niu et al. detected pedestrians' heads with an improved YOLOv5s, at an average accuracy of 94.01%, and used the positions to initialise an evacuation model [11]. Li et al. built density matrices from YOLOv5 detections in surveillance video and calibrated the social force parameters of an AnyLogic model of a metro station (Pinglonglu East Station, Suzhou), obtaining a desired speed of 1.37 m/s and a relaxation time of 0.53 s with an error below 5% after eight iterations [10]. Geng et al. monitored passenger flow at key nodes of existing metro stations with YOLOv8 and coupled the data with an AnyLogic simulation to predict congestion after the stations' conversion into transfer stations [135]. In all three, information flows from real video into the simulator and none uses the simulator to generate video; the first two involve no tracking. A review of pedestrian optimisation models concludes that calibration needs more attention [136].

Two official interfaces connect AnyLogic to Python. Alpyne runs models exported from an AnyLogic RL experiment and lets Python code reset runs, apply actions and read outputs [52]; Pypeline lets a running model call Python, for example to test trained policies, at some computational cost [137]. The Gymnasium API (`reset`, `step`, observation and action spaces) is the standard way to expose such a pipeline to RL libraries [62].

2.4 Synthetic Data and Simulation for Computer Vision

2.4.1 Virtual Worlds as Sources of Labelled Data

Training on rendered images predates deep learning: pedestrian classifiers trained in virtual scenarios performed similarly to real-trained ones on the Daimler benchmark [138]. Hattori et al. synthesised scene-specific training data for a newly installed surveillance camera without any real pedestrian data, using scene geometry and simulated pedestrian motion, and outperformed models trained on limited real data [41]; the journal version renders geometrically and photometrically accurate pedestrians and adds pose estimation [42]. The two studies of Hattori et al. are closest to rendering simulated pedestrians through a fixed virtual camera, but they train a detector (and, in the journal version, a pose estimator) and neither evaluate nor tune trackers.

Game engines later became the main source of synthetic data. Virtual KITTI cloned driving sequences into virtual worlds with automatic ground truth for detection, tracking and other tasks, and found a small real-to-virtual gap [39]; Virtual KITTI 2 improved realism [139]. Game data with one third of CamVid outperformed the full real training set for segmentation [140], the VIPER benchmark provides more than 250K frames with tracking ground truth [141], SYNTHIA improved segmentation when mixed with real data [142], and a vehicle detector trained only on GTA-based data outperformed its real-trained counterpart on KITTI [143]. Simulators such as CARLA [144] and AirSim [145] and tools such as Unity Perception [146] have made such data generation routine.

2.4.2 The Domain Gap: Randomisation and Adaptation

Models trained on synthetic data often degrade on real data. Domain randomisation varies rendering so widely that reality appears as "just another variation"; a detector trained on non-realistic textures localised real objects to within 1.5 cm [43]. Randomised synthetic data followed by real fine-tuning outperforms real data alone [44], and structured randomisation adds scene context [147]. Domain adaptation aligns the domains instead:

SimGAN refines simulator images adversarially while keeping their labels [148], CyCADA combines pixel- and feature-level adaptation [149], and Domain Adaptive Faster R-CNN adds image- and instance-level domain classifiers [150]. A third strategy learns the simulator's parameters: Learning to Simulate tunes them with RL to maximise downstream accuracy [56], and Meta-Sim learns scene-graph attributes that reduce the distribution gap [151]. Broader treatments cover synthetic data for deep learning [152] and sim-to-real transfer of RL agents [153].

In this thesis the detector is fine-tuned and evaluated, and the RL agent trained and tested, within the same simulator, so the gap does not affect the internal validity of comparisons between optimisation methods. It does, however, limit how directly tuned parameter values transfer to real cameras.

2.4.3 Synthetic Pedestrian and Crowd Datasets

Synthetic datasets of people have multiplied, as Table 2.5 shows. JTA provides about 500,000 GTA V frames for joint detection and tracking [40]. MOTSynth, motivated by privacy concerns and annotation cost, offers 768 clips with 40,780,800 pedestrian instances; Tracktor trained only on it reached 56.9 MOTA on MOT17, exceeding a COCO-pretrained, MOT17-fine-tuned version, although MOT20 still required real fine-tuning [37]. Other datasets address multi-camera tracking [36], crowd counting [154], human keypoints [155], detection in simulated crowds [156], human body analysis [157] and controlled re-identification studies [158]. Crowd simulation has also been used to animate synthetic pedestrians: LCrowdV couples procedural crowd simulation with procedural rendering to generate labelled crowd videos that improved pedestrian detection and crowd-behaviour classification [159], and a game-engine crowd simulator produces automatically annotated video for evaluating trackers [160], extended in CrowdSim2 to validate the IOU-Tracker, DeepSORT and Deep-TAMA trackers [161]. A review of surveillance applications notes that synthetic data now mainly serve to reduce annotation effort, mitigate bias and respect privacy [38].

Table 2.5. Representative synthetic datasets for pedestrian and crowd analysis.

Dataset	Generator	Primary task	Scale (as reported)	Key finding
Virtual KITTI [39]	Unity	Detection, MOT, depth, flow	50 videos, 21,260 frames	Small real-to-virtual gap
VIPER [141]	Commercial game	Detection, tracking, flow	More than 250K frames	Exact ground truth, realistic statistics
JTA [40]	GTA V	Pose detection and tracking	About 500,000 frames	Virtual-only training generalises

Dataset	Generator	Primary task	Scale (as reported)	Key finding
MOTSynth [37]	GTA V	Pedestrian detection and tracking	768 clips, 1,382,400 frames	56.9 MOTA on MOT17, synthetic-only
MTA [36]	GTA V	Multi-camera tracking	Over 2,400 identities, 6 cameras	Privacy-free multi-camera data
GCC [154]	GTA V	Crowd counting	15,212 images	Pre-training improves real counting
PeopleSansPeople [155]	Unity	Human keypoints	Parameterised generator	Few-shot keypoint AP 44.43 with synthetic pre-training vs 6.40 real-only
LCrowdV [159]	Procedural crowd simulation and rendering	Crowd behaviour, pedestrian detection	Arbitrary number of labelled videos	Improves detection and behaviour classification
CrowdSim2 [156], [161]	Unity crowd simulator	People and vehicle detection; people tracking	Varying weather and density	Controlled detector and tracker benchmarking

Two limitations of these datasets matter here. Most pedestrian datasets come from game engines whose crowd behaviour is designed for games rather than derived from calibrated pedestrian models, the crowd-simulation-based generators above being partial exceptions, and they are static, generated once for training or benchmarking rather than on demand inside an optimisation loop. Simulation has also served as a controlled test bed: Virtual KITTI isolated the effect of weather and imaging conditions on trackers "all other things being equal" [39], and PersonX isolated the effect of viewpoint on re-identification [158]. Digital twins couple a physical and a virtual system through data integration [6] and have been applied to urban planning with mobility simulation [7]. Without such a connection, the simulated surveillance scene of this thesis is not a digital twin, but it plays the role of a virtual counterpart of a camera installation in which detector and tracker parameters can be studied under controlled conditions.

2.5 Hyper-Parameter Optimisation

2.5.1 Problem Formulation

Tuning the pipeline is an HPO problem: find

$$\boldsymbol{\theta}^* = \operatorname{argmax}_{\boldsymbol{\theta}\in\Theta} f(\boldsymbol{\theta}), \qquad (2.12)$$

where $\boldsymbol{\theta}$ collects the seven tuned parameters of Table 2.2 and f is a tracking metric such as HOTA. The objective has no gradient, each evaluation requires detection and tracking over many frames, and its value is noisy because it depends on the crowd realisation. The automated machine-learning literature surveys methods for such problems

[162]; as shown in Figure 2.5, they differ mainly in how they use the feedback from earlier evaluations.

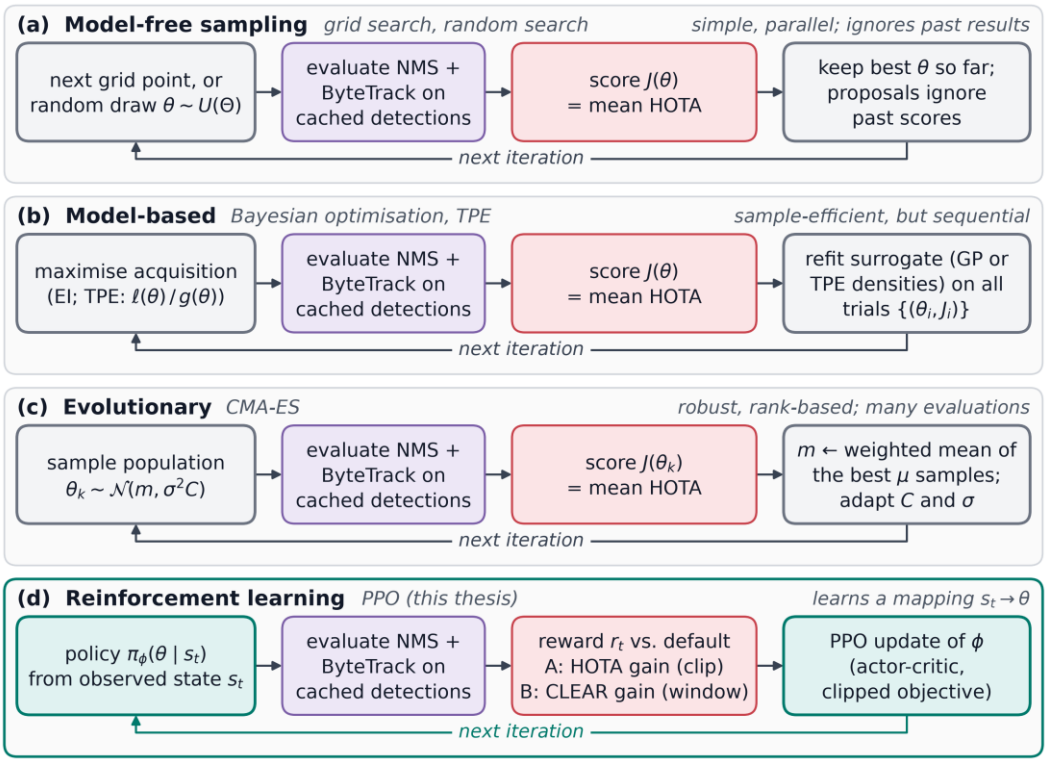


Figure 2.5. Families of hyper-parameter optimisation methods and how they use the evaluation feedback; in panel (d), A is the contextual formulation (HOTA gain over the default parameters on a clip) and B the adaptive formulation (CLEAR-type window reward relative to a default run).

2.5.2 Grid Search and Random Search

Grid search evaluates every combination on a lattice; its cost grows exponentially with dimension and most trials vary unimportant parameters. Bergstra and Bengio showed that random search finds equally good or better configurations in a fraction of the time, because for most datasets only a few hyper-parameters matter, although different ones matter on different datasets [28]. They proposed random search as the natural baseline for adaptive methods, which justifies its use here.

2.5.3 Bayesian Optimisation and TPE

Model-based methods fit a surrogate of f to past evaluations and choose the next point with an acquisition function such as expected improvement. Snoek et al. modelled performance with a Gaussian process, showed that kernel choice and the treatment of its hyper-parameters are crucial, and matched expert tuning [163]. The Tree-structured Parzen Estimator (TPE) models $p(\boldsymbol{\theta} \mid y)$ with separate densities for good and bad trials, which handles conditional search spaces naturally [27]. TPE is the default sampler of Optuna,

whose define-by-run API, pruning and support for samplers such as TPE and CMA-ES make it the toolkit for the TPE, global TPE and CMA-ES baselines of this thesis [30].

2.5.4 Evolution Strategies and CMA-ES

The Covariance Matrix Adaptation Evolution Strategy (CMA-ES) samples candidates from $\boldsymbol{m} + \sigma\,\mathcal{N}(\mathbf{0}, \boldsymbol{C})$, ranks them and moves the mean towards the best, while adapting the covariance through rank-μ and evolution-path updates and the step size separately [29]. Because it uses only rankings, it is invariant to order-preserving transformations of the objective and adapts to ill-conditioned, non-separable problems [164]. It has been proposed for tuning deep networks because of these invariances and its suitability for parallel evaluation [165], and it suits the coupled continuous thresholds of Table 2.2.

2.5.5 Multi-Fidelity Methods and Population-Based Training

Multi-fidelity methods evaluate many configurations cheaply and invest fully only in promising ones [162]; Optuna prunes unpromising trials with a variant of asynchronous successive halving [30]. Population Based Training (PBT) optimises a population of models and hyper-parameters jointly, yielding a schedule of settings rather than one configuration [166], an idea related to the online re-tuning formulation of this thesis. Table 2.6 compares the families.

Table 2.6. Comparison of hyper-parameter optimisation methods.

Method	Use of feedback	Search model	Output	Limitation for scene-adaptive tuning
Grid search	None	Fixed lattice	One configuration	Exponential cost
Random search [28]	None	Random sampling	One configuration	No learning from past trials
Gaussian-process BO [163]	All past trials	Surrogate of $p(y \mid \boldsymbol{\theta})$	One configuration	Re-run for every new scene
TPE [27]	All past trials	Densities of good and bad trials	One configuration	Re-run for every new scene
CMA-ES [29]	Rankings per generation	Adapted Gaussian distribution	One configuration	Re-run for every new scene
Multi-fidelity pruning [30]	Intermediate results	Depends on sampler	One configuration	Low-fidelity scores may mislead
PBT [166]	Population performance	Exploit and explore	Schedule over training	Tied to one training run
RL policy (this thesis)	Rewards from experience	Policy and value networks	Mapping from state to parameters	Requires prior training

2.5.6 Hyper-Parameter Optimisation of Multi-Object Trackers

HPO has rarely been applied to trackers themselves. Madrigal et al. compared four HPO tools (MCMC, SMAC, TPE and Spearmint) for a multi-object tracker evaluated on

PETS09 and ETH and showed that tuning changes results enough to affect competition rankings [167], and Fleck and Zöllner tuned GM-PHD trackers with TPE, using MOTA as the objective, in simulation and on MOT20 [25]. Most recently, Adžemović observed that tracking-by-detection hyper-parameters are conventionally chosen by hand; with HOTA as the objective, TPE and a multi-fidelity greedy coordinate search improved on both hand-tuned configurations and published results for SORT, ByteTrack, SparseTrack and MoveSORT-KF on DanceTrack and SportsMOT, in all eight tracker–dataset combinations [26]. These studies return one static configuration per tracker and dataset. Run-time adaptation has been studied without a policy learned from a tracking reward: Chau et al. learned satisfactory tracker parameters offline for clusters of video context, such as occlusion level and density, and switched to them online when the context changed [168], and Ma et al. adjusted the high-score threshold of ByteTrack dynamically from the current detections [169].

Classical HPO returns the configuration that is best on average over the evaluation data. If the best parameters depend on the scene, every new scene requires a new optimisation, and parameters cannot change within a video. A policy mapping an observed scene state to parameters removes this limitation at the cost of training, which motivates the RL approach reviewed next.

2.6 Reinforcement Learning

2.6.1 Markov Decision Processes

RL studies an agent that learns through interaction to maximise cumulative reward. The interaction is formalised as a Markov decision process (MDP) with states s, actions a, rewards and dynamics $p(s', r \mid s, a)$ [53], [170]. The agent seeks a policy $\pi(a \mid s)$ maximising the expected discounted return

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \qquad 0 \le \gamma < 1, \qquad (2.13)$$

and the value functions $V^{\pi}(s)$ and $Q^{\pi}(s, a)$ satisfy the Bellman equations that underlie dynamic programming and learning methods [170]. In this thesis the state summarises the scene and recent pipeline behaviour, the action is the vector of the seven tuned parameters of Table 2.2, and the reward derives from the metrics of Section 2.2.6.

2.6.2 Value-Based, Policy-Gradient and Actor-Critic Methods

Value-based methods learn Q and act greedily. The deep Q-network (DQN) learned Atari games from raw pixels at the level of an expert human player across 49 games, stabilised by experience replay and a target network [171]. DQN needs discrete actions, however, which would force the continuous thresholds onto a grid that grows exponentially with their number. Policy-gradient methods instead parameterise the policy directly. REINFORCE ascends $\alpha\,(r-b)\,\nabla_{\boldsymbol{\phi}}\ln\pi(a \mid s;\boldsymbol{\phi})$, and the baseline b reduces variance without bias [172]; actor-critic methods learn a value function to serve as this baseline [53]. Generalised Advantage Estimation (GAE) defines $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ and

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l\, \delta_{t+l}, \qquad (2.14)$$

where $\lambda = 0$ gives the low-variance but biased one-step estimate and $\lambda = 1$ the unbiased but high-variance Monte Carlo estimate [173]. TRPO constrains each update by the Kullback–Leibler divergence between successive policies to obtain near-monotonic improvement [174]. Off-policy alternatives reuse a replay buffer: DDPG extends DQN to continuous actions [175] and Soft Actor-Critic adds a maximum-entropy objective for stability [176].

2.6.3 Proximal Policy Optimisation

PPO keeps the benefits of TRPO with a first-order objective [54]. With $r_t(\boldsymbol{\phi}) = \pi_{\boldsymbol{\phi}}(a_t \mid s_t)/\pi_{\boldsymbol{\phi}_{\mathrm{old}}}(a_t \mid s_t)$,

$$L^{\mathrm{CLIP}}(\boldsymbol{\phi}) = \hat{\mathbb{E}}_t\big[\min\big(r_t(\boldsymbol{\phi})\hat{A}_t,\ \mathrm{clip}(r_t(\boldsymbol{\phi}),1-\epsilon,1+\epsilon)\hat{A}_t\big)\big], \qquad (2.15)$$

a pessimistic bound that removes the incentive to move the ratio outside $[1-\epsilon, 1+\epsilon]$. A shared network adds a value loss weighted by c_1 and an entropy bonus weighted by c_2, advantages come from truncated GAE, and several epochs of minibatch gradient ascent are run per batch. In the authors' MuJoCo ablation, $\epsilon = 0.2$ scored best (0.82) against 0.76 for $\epsilon = 0.1$ and -0.39 without clipping or penalty; their settings (horizon 2048, step size 3×10^{-4}, 10 epochs, minibatch 64, $\gamma = 0.99$, $\lambda = 0.95$) are the PPO defaults of Stable-Baselines3 [54], [61]. PPO is adopted here for three reasons: it handles continuous, multi-dimensional actions with a Gaussian policy whose actions, in the unit cube $[-1,1]^7$, map linearly onto the bounded parameter ranges (Chapter 4); its default settings are well

established [61]; and it collects experience from many parallel environments, which suits a pipeline whose evaluations are expensive but independent of each other.

2.6.4 Contextual Bandits and One-Shot Decisions

When an action does not affect future states, the MDP reduces to a contextual bandit: the learner observes a context, acts, and sees only that action's reward [57]. LinUCB, which selects actions by an upper confidence bound under a linear payoff model, achieved a 12.5% click lift over a context-free bandit in news recommendation [58]. This setting matches the contextual formulation of this thesis, which chooses one parameter vector per scene from scene features. That formulation is solved with PPO on one-step episodes (one clip, one action) with $\gamma = 0$, so that the return equals the immediate reward; PPO then acts as a policy-gradient contextual bandit whose critic serves as a learned, context-dependent baseline [172]. LinUCB-style algorithms were not used because they assume a finite set of arms and a payoff linear in the context features [58], whereas the action here is a continuous seven-dimensional vector whose effect on HOTA is non-linear and coupled (Section 2.1.4). The adaptive formulation, re-tuning every second with each choice affecting the tracks carried forward, is a genuinely sequential problem requiring the full MDP machinery.

2.6.5 Reinforcement Learning for Parameter Tuning and Visual Tracking

RL has served as a configuration optimiser in neural architecture search [177], in learned optimisation algorithms [178] and in simulator tuning [56]. In tracking, the closest work is that of Dong et al., who argued that tracker hyper-parameters should be chosen adaptively per sequence rather than fixed offline and applied blindly; their continuous deep Q-learning agent raised a Siamese tracker's AUC on OTB-2013 from 0.597 to 0.629 at a small speed cost [55]. Huang et al. used Q-learning to decide at which feature layer a tracker may stop [179], and ADNet tracked a single object through learned translation, scaling and stop actions [180]. Outside vision, deep RL has chosen the parameters of a radar multi-target tracker adaptively to the scene [181], and Q-learning has selected the Gaussian-process hyper-parameters of an extended-object tracker [182]. In video MOT, RL has mostly replaced tracker components: Xiang et al. modelled each target's lifetime as an MDP and learned the association policy [183], collaborative deep RL let each object choose among update, ignore, block and delete actions [184], and MARLMOT learned track management with a reward computed from ground-truth tracks, optimising MOTA

directly [185]. MARLMOT exploits ground truth in the reward, but it works without vision on given detections and replaces the tracker rather than tuning a detector–tracker pipeline. It was evaluated on a simulated scenario and on the MOT15 training set, which makes its benchmark results non-comparable [12]; its authors also note that the SORT baseline's parameters had been tuned manually on the whole training data and that SORT's performance depends strongly on them [185]. Table 2.7 positions these studies against this thesis.

Table 2.7. Positioning of closely related studies relative to this thesis.

Study	Video source	Detector	Tracker	Objective	Adaptation method	Adaptive
Johansson et al. [9]	Real video	Tracking software	–	Fit to trajectories	Evolutionary (tunes simulator)	No
Li et al. [10]	Real video	YOLOv5	–	Density matrices	Newton downhill (tunes simulator)	No
Niu et al. [11]	Real video	Improved YOLOv5s	–	–	None	No
Gaidon et al. [39]	Virtual worlds	Deep detectors	MOT trackers	CLEAR metrics	None (evaluation)	No
Fabbri et al. [37]	GTA V	Trained on synthetic data	Tracktor and others	MOT metrics	None (training data)	No
Ruiz et al. [56]	Parameterised simulator	Downstream model	–	Downstream accuracy	RL (tunes simulator)	No
Dong et al. [55]	Real video	–	Siamese single-object	Tracking accuracy	Continuous deep Q-learning	Yes
Rosello and Kochenderfer [185]	Simulated scenario; MOT15 training set	Given detections	Learned policy	MOTA from ground truth	Multi-agent RL	n/a (replaces tracker)
Stephan et al. [181]	Radar (no video)	Radar processing chain	Radar multi-target tracker	Multi-target tracking performance	Deep RL (tunes tracker)	Yes
This thesis	AnyLogic-specified scenarios run by a Python co-simulation + virtual camera	YOLOv8, transfer-learned	ByteTrack	HOTA gain (contextual) or CLEAR-type window reward (adaptive) from simulator ground truth	PPO vs grid, random, TPE, CMA-ES	Yes

2.7 Summary and Research Gap

The review has covered six bodies of work. One-stage anchor-free detectors such as YOLOv8 deliver real-time accuracy and are routinely adapted by fine-tuning COCO-

pretrained weights, yet their output depends on confidence and NMS thresholds whose best values change with density and occlusion. ByteTrack showed that exploiting low-score detections preserves identities, but its behaviour hinges on thresholds whose recommended defaults differ markedly between implementations, a sign that no single setting is universally appropriate. Tracking metrics have matured from MOTA through IDF1 to HOTA, but all require dense ground truth that is expensive for real video. Social-force-type models produce realistic microscopic trajectories, and the AnyLogic Pedestrian Library, whose motion model is reported in an AnyLogic-based calibration study to be social-force based [10], adds 3D visualisation and RL-oriented Python interfaces. Virtual worlds have proved useful for training and controlled evaluation of vision models. Classical HPO returns one static configuration per run and has rarely been applied to trackers, whereas RL can learn context-dependent policies and has tuned the hyper-parameters of a single-object visual tracker and of non-visual trackers.

The gaps below rest on this review and on a targeted search, last run in October 2026, of arXiv, Crossref, OpenAlex and the web. The search strings combined "AnyLogic" with "YOLO", "detection", "tracking" or "camera"; "pedestrian simulation" or "crowd simulation" with "synthetic", "video" or "tracking"; "reinforcement learning" with "multi-object tracking", "tracker", "hyperparameter" or "threshold"; and "ByteTrack" with "hyperparameter optimization", "adaptive threshold" or "reinforcement". Forward citation searches started from the closest works [10], [11], [41], [55], [167], and the reference list of the most recent study of tracker HPO was checked [26]. Where a combination is said not to exist, this means that the search did not find it; no search of this kind can be exhaustive. Despite the progress reviewed, a gap remains at the intersection of these fields:

1. **AnyLogic has been connected to detectors only in the real-to-simulation direction, and simulation-to-video studies stop short of tuning a detector–tracker pipeline.** AnyLogic studies use YOLO detections from real video to initialise, calibrate or inform simulations [10], [11], [135]. The nearest precursors in the other direction render simulated pedestrians for a fixed surveillance camera but train a detector (and, in the journal version, a pose estimator), with no tracker [41], [42], or use a game-engine crowd simulator to evaluate trackers without tuning them [160], [161]. The search found no study that uses a calibrated pedestrian-dynamics model to generate surveillance video with exact ground truth for evaluating and improving a detector–tracker pipeline.
2. **Synthetic pedestrian data are static and mostly not derived from calibrated pedestrian-dynamics models.** Large synthetic tracking datasets are generated once with game engines [36], [37], [40] and serve as training data or fixed benchmarks

rather than as an interactive environment queried inside an optimisation loop. Partial exceptions animate the pedestrians with crowd simulation [156], [159], [161] or simulated pedestrian motion [41], but they too produce data for training or benchmarking rather than for online optimisation.

3. **Detector and tracker parameters are tuned manually, separately or statically.** The official ByteTrack code hard-codes per-sequence overrides of its track buffer and high-score threshold [22], and SORT's parameters were tuned manually on the MOT15 training data [185]. The few HPO studies of trackers return one static configuration per tracker and dataset [25], [26], [167], and run-time adaptation has relied on context clusters learned offline or on a threshold computed from the current detections [168], [169]. RL-based tuning of tracker parameters has addressed a single-object visual tracker [55] and non-visual trackers [181], [182], and RL in video MOT has replaced tracker components rather than tuning the joint detector–tracker parameter vector [183], [184], [185].
4. **Simulator ground truth has not been exploited for dense, scene-adaptive rewards.** Prior RL approaches to tracking relied on limited labelled benchmarks [55], [183], [184] or worked without vision [185]; a simulator that records every agent's identity and position can supply HOTA, MOTA and IDF1, or window-level CLEAR-type counts, as rewards for any scene, density or camera placement.

This thesis addresses the gap with a closed loop of simulation, detection, tracking, reward and RL re-tuning. Scenes specified as AnyLogic Pedestrian Library models (executed for the experiments by a Python co-simulation of the same models; Chapter 4) are rendered through a virtual surveillance camera; a COCO-pretrained YOLOv8 is transfer-learned on the automatically labelled frames; ByteTrack builds trajectories; HOTA, MOTA and IDF1 are computed against the simulator's ground truth; and a PPO agent learns to set the NMS IoU threshold and the ByteTrack high-score, low-score and new-track thresholds, first- and second-association cost limits and lost-track buffer (seven in total). Two formulations are studied. The contextual one chooses one parameter vector per scene from scene features and is rewarded by the HOTA gain over the default parameters. The adaptive one re-tunes the parameters online every second and is rewarded per window by the CLEAR-type quantity $(\mathrm{TP} - \mathrm{FP} - 5\,\mathrm{IDSW})/\mathrm{GT}$ relative to a default-parameter run (Section 2.2.6). The agent is compared with the HPO baselines of Section 2.5: grid search, random search, TPE and CMA-ES. The design and implementation of this system are described in the following chapters.

Chapter 3 Theoretical Foundations

Chapter 1 introduced the system of this thesis as a closed loop (Figure 1.1). A pedestrian model, specified in AnyLogic and executed for the experiments by a headless Python co-simulation of the same scenarios, moves virtual pedestrians through a scene; a virtual camera renders them as video; a YOLOv8 detector transfer-learned from COCO localises them; ByteTrack links the detections into trajectories; and a reinforcement-learning agent re-tunes the detector post-processing and the tracker according to how well the trajectories agree with the simulator's ground truth. This chapter supplies the theory needed to state precisely what is optimised and why the problem has its particular structure. Section 3.1 treats the detector and transfer learning, Section 3.2 tracking by detection and ByteTrack, ending with the seven-dimensional parameter vector that all optimisers act on, Section 3.3 the social force model, Section 3.4 the tracking metrics, Section 3.5 reinforcement learning and Section 3.6 the black-box optimisers used as baselines. Every symbol is defined in the section in which it is used, and the notation of the parameter vector is the one used in Chapter 4. Where two fields claim the same letter, the secondary use carries a subscript (for example λ_{GAE}, γ_{TPE} and c_{vf}) or a different symbol (p_0 for the initial-state distribution of a Markov decision process). The most important of the remaining conventional reuses are pointed out where they occur, for example: s for a detection score and a reinforcement-learning state, A for an interaction strength and, with a superscript, an advantage, α for an exponent of the detector's label assigner and the HOTA localisation threshold, μ for the CMA-ES parent number and, in bold, a policy mean, and H for the image height and, in bold, the Kalman observation matrix.

3.1 Object Detection with YOLOv8

The detector's boxes and confidence scores are the only information about the virtual pedestrians that the tracker receives. Two aspects of the detector are therefore central to this thesis: what its confidence score measures, because the tracker's thresholds act on it, and how overlapping boxes are suppressed, because the suppression threshold is one of the tuned parameters.

3.1.1 Dense One-Stage, Anchor-Free Prediction

An object detector maps an image $I \in \mathbb{R}^{3\times H\times W}$ to a set of scored boxes $\mathcal{D} = \{(\boldsymbol{b}_k, s_k)\}_{k=1}^{N}$, where $\boldsymbol{b}_k = (x_1, y_1, x_2, y_2)_k$ holds the corner coordinates of an axis-aligned

box and $s_k \in [0,1]$ is a confidence score; with a single class the label is implicit. Two-stage detectors first propose regions and then classify them [67], whereas one-stage detectors predict a score and a box at every position of their feature maps in a single pass and reduce this dense output afterwards [14], [71], [72]. Anchor-free one-stage detectors regress the box directly from a point instead of refining predefined anchor boxes, which removes the anchor shapes as data-dependent hyper-parameters [76]. YOLOv8 is of this type [15], [17]. Its predictions are attached to anchor points, the cell centres of three feature maps with strides $S_l \in \{8,16,32\}$ pixels:

$$\mathcal{P}_l = \{((u + 1/2)\, S_l,\ (v + 1/2)\, S_l) : u = 0, \dots, W/S_l - 1,\ v = 0, \dots, H/S_l - 1\}, \quad l \in \{3,4,5\}. \qquad (3.1)$$

A square 960×960 training image, matching the 960-pixel input size used in this thesis, yields $120^2 + 60^2 + 30^2 = 18{,}900$ anchor points, each emitting one candidate box; at inference the 1280×720 frames are letterboxed to 960×544 pixels, which gives $120 \cdot 68 + 60 \cdot 34 + 30 \cdot 17 = 10{,}710$ anchor points. Detection is thus a dense regression and classification problem followed by a set-reduction step (Section 3.1.5) that turns thousands of candidates into a few dozen detections, and it is in this step that the first tunable parameter enters the pipeline.

3.1.2 Network Architecture

Figure 3.1 shows the architecture, which consists of a backbone, a neck and a head [15], [17]. Every "Conv" unit comprises a convolution, batch normalisation and the SiLU activation. The backbone halves the resolution five times with stride-2 3×3 convolutions (levels P1/2 to P5/32), inserts a C2f block after the second to fifth down-sampling step and ends with an SPPF block. The bottleneck repeats of the base configuration (3, 6, 6, 3) are scaled by a depth multiple of 0.33 for the n and s models and 0.67 for the m model, and the channel widths by 0.25, 0.50 and 0.75. YOLOv8n, YOLOv8s and YOLOv8m, the three sizes fine-tuned in this thesis, have 3.2, 11.2 and 25.9 million parameters and reach 37.3, 44.9 and 50.2 $\text{mAP}_{50\text{-}95}$ on COCO [17].

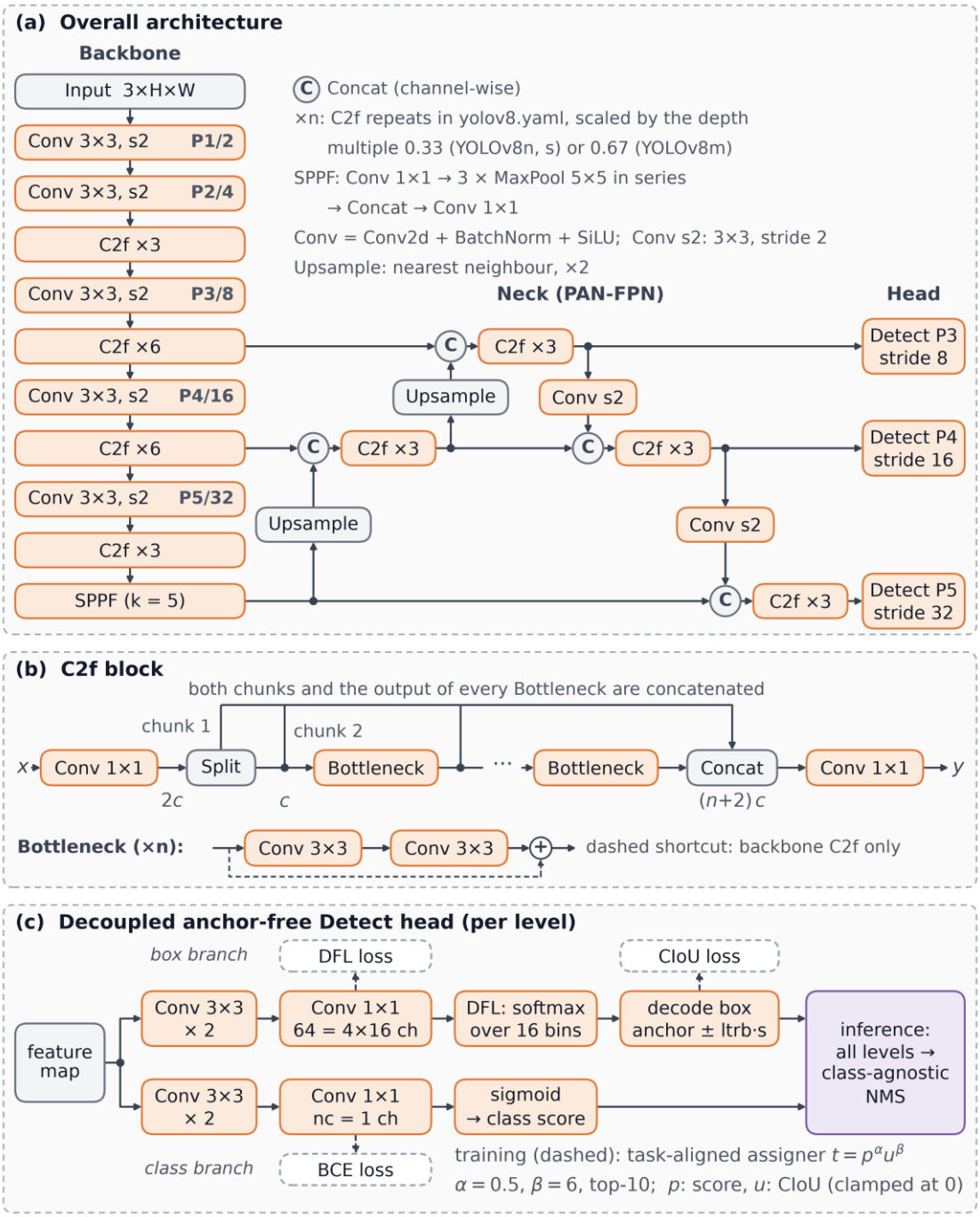


Figure 3.1. Architecture of YOLOv8. (a) Overall architecture: backbone of stride-2 convolutions and C2f blocks ending in SPPF (left), PAN-FPN neck fusing P3, P4 and P5 (centre) and one Detect head per level (right). (b) C2f block with its bottlenecks. (c) Decoupled anchor-free Detect head, whose box branch predicts a 16-bin distance distribution per side (DFL) and whose classification branch outputs a sigmoid score; training uses the task-aligned assigner t = p^α u^β (p corresponds to s in Equation (3.7); u is CIoU clamped at zero), CIoU + DFL and BCE losses, and inference ends with class-agnostic NMS at threshold θ_nms (applied post hoc, Section 3.1.5).

The C2f block (Figure 3.1b) is a cross-stage partial design [85]. A 1 × 1 convolution projects the input to $2c$ channels, the output width of the block (in the neck this projection reduces the number of channels), and the result is split into two chunks of c channels; the

first is kept, the second passes through n bottlenecks of two 3×3 convolutions each (with a residual shortcut in the backbone, without one in the neck), and the two chunks together with all n bottleneck outputs are concatenated to $(n + 2)c$ channels and fused by a 1×1 convolution. Unlike the CSP layer of YOLOv5, which passes only the end of the chain to the fusion layer, C2f exposes features of every depth and thus provides many gradient paths of different lengths at small cost [15], [85]. The SPPF block applies three 5×5 stride-1 max-pooling operations in series between two 1×1 convolutions and concatenates the four intermediate maps. Since two stride-1 pooling windows of sizes k_1 and k_2 compose to one of size $k_1 + k_2 - 1$, the serial outputs equal pooling with windows of 5, 9 and 13, so SPPF reproduces more cheaply the parallel spatial pyramid pooling of YOLOv3 and YOLOv4, which concatenates max-pooling outputs with kernel sizes 1, 5, 9 and 13 at stride 1 [84]. The neck is a PAN-FPN [70], [86]: a top-down path upsamples and fuses deep features into the P4 and P3 maps, giving high-resolution maps semantic context, and a bottom-up path of stride-2 convolutions carries precise localisation signals back to P4 and P5. For an elevated surveillance camera the P3 level matters most, because distant pedestrians are only a few tens of pixels tall.

The head is decoupled and anchor-free. At each level a box branch (two 3×3 convolutions and a 1×1 convolution) outputs $4R$ channels, with $R = 16$ distance bins (`reg_max`), and a separate classification branch outputs $n_c = 1$ channel followed by a sigmoid. Decoupling follows the observation that classification and localisation favour different features [76]. YOLOv8 has no separate objectness output, so the sigmoid class score is the detection confidence s_k [17].

3.1.3 Box Representation with Distribution Focal Loss

YOLOv8 represents the distance from an anchor point $\boldsymbol{p} = (p_x, p_y)$ to each box side $k \in \{\mathrm{l}, \mathrm{t}, \mathrm{r}, \mathrm{b}\}$, in units of the stride, as a discrete distribution over R bins [97]. From the logits $z_{k,i}$ of the box branch,

$$P_k(i) = \frac{\exp(z_{k,i})}{\sum_{j=0}^{R-1} \exp\left(z_{k,j}\right)}, \qquad \hat{d}_k = \sum_{i=0}^{R-1} i \; P_k(i), \tag{3.2}$$

and the decoded box is

$$\widehat{\boldsymbol{b}} = \left(p_x - S_l \hat{d}_{\mathrm{l}},\; p_y - S_l \hat{d}_{\mathrm{t}},\; p_x + S_l \hat{d}_{\mathrm{r}},\; p_y + S_l \hat{d}_{\mathrm{b}}\right). \tag{3.3}$$

A side can thus lie at most $(R-1)S_l$ pixels from its anchor point, 120 pixels on P3 and 480 on P5. For a continuous target distance y with neighbouring integers $y_i = \lfloor y \rfloor$ and $y_{i+1} = y_i + 1$, the Distribution Focal Loss (DFL) is

$$\mathcal{L}_{\mathrm{DFL}} = -((y_{i+1} - y)\log P(y_i) + (y - y_i)\log P(y_{i+1})), \qquad (3.4)$$

a cross-entropy with target mass $y_{i+1} - y$ on bin y_i and $y - y_i$ on bin y_{i+1}. If the prediction equals this target, the expectation of Equation (3.2) is $y_i(y_{i+1} - y) + y_{i+1}(y - y_i) = y$, so the loss drives the expected distance to the target while concentrating mass around it [97]. The distribution can also express uncertainty about an edge, for example the lower edge of a pedestrian whose legs are hidden, while its expectation still gives a sub-bin estimate.

3.1.4 Training Objective and Task-Aligned Assignment

Box quality is measured throughout this thesis by the intersection over union,

$$\mathrm{IoU}(\boldsymbol{b}, \boldsymbol{b}') = \frac{|\boldsymbol{b} \cap \boldsymbol{b}'|}{|\boldsymbol{b} \cup \boldsymbol{b}'|} = \frac{|\boldsymbol{b} \cap \boldsymbol{b}'|}{|\boldsymbol{b}| + |\boldsymbol{b}'| - |\boldsymbol{b} \cap \boldsymbol{b}'|}, \qquad (3.5)$$

where $|\cdot|$ denotes area; the same quantity governs NMS, the association cost of ByteTrack and the matching in every evaluation metric. Because $1 - \mathrm{IoU}$ has no gradient for disjoint boxes, the Generalised IoU [96] and later the Distance- and Complete-IoU losses were introduced [95]. YOLOv8 uses CIoU:

$$\mathcal{L}_{\mathrm{CIoU}} = 1 - \mathrm{IoU} + \frac{\| \boldsymbol{c} - \boldsymbol{c}^{gt} \|^2}{c^2} + \alpha_v v, \qquad v = \frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}} - \arctan\frac{w}{h}\right)^2, \qquad \alpha_v = \frac{v}{(1 - \mathrm{IoU}) + v}, \qquad (3.6)$$

where $\boldsymbol{c}$ and $\boldsymbol{c}^{gt}$ are the box centres, c the diagonal of the smallest enclosing box and v an aspect-ratio inconsistency with weight α_v. The centre term supplies a gradient for disjoint boxes, and the aspect term is relevant for the tall, narrow boxes of pedestrians.

Which anchor points are trained as positives for a ground-truth box g is decided by the task-aligned assigner introduced with TOOD [17], [98]. For an anchor point $\boldsymbol{p}$ inside g, the alignment metric is

$$t_{\boldsymbol{p}g} = s_{\boldsymbol{p}g}^{\alpha}\, u_{\boldsymbol{p}g}^{\beta}, \qquad (3.7)$$

where s_{pg} is the score currently predicted for the class of g and u_{pg} the overlap of the predicted box with g (CIoU clamped at zero in the Ultralytics implementation). YOLOv8 uses $\alpha = 0.5$ and $\beta = 6.0$ [17], whereas TOOD used $\alpha = 1$ and $\beta = 6$ [98]. The ten anchor points with the largest t become positives of g, an anchor point claimed by several boxes goes to the one it overlaps most, and the large β makes localisation quality the dominant criterion. The classification targets are soft,

$$\hat{t}_{\boldsymbol{p}} = t_{\boldsymbol{p}g} \frac{\max_{\boldsymbol{p}'} u_{\boldsymbol{p}'g}}{\max_{\boldsymbol{p}'} t_{\boldsymbol{p}'g}}, \qquad (3.8)$$

with maxima over the positives of g, so the best-aligned positive receives the best overlap achieved for that object as its target, the other positives proportionally smaller targets, and all negative anchor points the target zero. The classification branch is trained with binary cross-entropy (BCE), which YOLOv3 introduced to the YOLO family [82],

$$\mathcal{L}_{\text{cls}} = -\frac{1}{\sum_{\boldsymbol{p}} \hat{t}_{\boldsymbol{p}}} \sum_{\boldsymbol{p}} \left(\hat{t}_{\boldsymbol{p}} \log \hat{s}_{\boldsymbol{p}} + (1 - \hat{t}_{\boldsymbol{p}}) \log(1 - \hat{s}_{\boldsymbol{p}}) \right), \qquad (3.9)$$

and the total loss is

$$\mathcal{L} = \lambda_{\text{box}} \mathcal{L}_{\text{CIoU}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}} + \lambda_{\text{dfl}} \mathcal{L}_{\text{DFL}}, \qquad (3.10)$$

with default gains 7.5, 0.5 and 1.5, the box terms being averaged over the positives weighted by their soft targets [17].

Equations (3.7) to (3.9) have a consequence that is central to this thesis. Because the score is trained towards the achieved overlap rather than towards one, the YOLOv8 confidence is an IoU-aware estimate of detection quality, not a pure probability that a pedestrian is present. A partially occluded pedestrian, whose box can be localised only approximately, is therefore expected to receive a lower score even when the network has little doubt that a person is present; this is exactly the population of detections that ByteTrack's low-score association exploits (Section 3.2.4). It also explains why score thresholds do not transfer between detectors: the score distribution depends on the training objective, the data and the scene, and fine-tuning shifts it.

3.1.5 Confidence Threshold and Non-Maximum Suppression

Because several neighbouring anchor points are trained as positives for each pedestrian, the dense output contains many overlapping candidates. A confidence filter first removes weak candidates,

$$\mathcal{D}^{\text{conf}} = \{(\hat{\boldsymbol{b}}_{\boldsymbol{p}}, \hat{s}_{\boldsymbol{p}}) : \hat{s}_{\boldsymbol{p}} > \tau_{\text{conf}}\}, \qquad (3.11)$$

and greedy non-maximum suppression (NMS), used for example in R-CNN [65], removes duplicates: the highest-scoring remaining candidate is moved to the output set $\mathcal{D}^{\text{nms}}$, all remaining candidates whose IoU with it exceeds the threshold θ_{nms} are deleted, and the step repeats. Equivalently, a box $\boldsymbol{b}_k$ is retained if and only if

$$\text{IoU}(\boldsymbol{b}_k, \boldsymbol{b}_j) \leq \theta_{\text{nms}} \quad \text{for every } \boldsymbol{b}_j \in \mathcal{D}^{\text{nms}} \text{ with } s_j > s_k, \qquad (3.12)$$

a recursive definition, because membership of $\mathcal{D}^{\text{nms}}$ follows from the same rule applied to higher-scoring boxes. The Ultralytics defaults are $\tau_{\text{conf}} = 0.25$ and $\theta_{\text{nms}} = 0.7$ [17]; with one class, suppression is class-agnostic.

In crowds θ_{nms} embodies a genuine trade-off, because two pedestrians side by side, or one partly behind another as seen from an oblique camera, easily produce boxes with an IoU above 0.5. A low threshold deletes the weaker of two correct detections, a miss no tracker can recover; a high threshold lets duplicates survive, which may start spurious tracks or be matched to a track in place of the correct detection. Occlusion is the central difficulty of crowded human detection [99]; NMS-free designs avoid the hand-set threshold [77], [80], but YOLOv8 retains it, and its best value depends on density, viewpoint and the tracker's thresholds. Because ByteTrack's second association consumes boxes scoring between a low and a high threshold (Section 3.2.4), the detector runs with $\tau_{\text{conf}} = 0.05$, delegating score filtering to the tracker, while θ_{nms} is tuned in [0.40,0.90]. So that the network need not be re-run for every candidate threshold, its own NMS runs once at the permissive threshold 0.9 and the tuned suppression is applied post hoc to the cached candidates; Section 4.6 describes this design and the rare configurations in which it differs from suppression of the raw output (Equation (4.6)).

3.1.6 Transfer Learning and Domain Shift

Using the definitions of Pan and Yang introduced in Section 2.1.5, a domain is a feature space with a marginal distribution, $\{\mathcal{X}, P(X)\}$, and a task a label space with a predictive function, $\{\mathcal{Y}, f(\cdot)\}$; transfer learning improves the target predictive function

using knowledge from a source domain [45], [100]. The source here is COCO [16], for which the pre-trained weights predict 80 classes, and the target consists of rendered frames labelled automatically with one class. Fine-tuning minimises the empirical target risk by stochastic gradient descent started at the source weights $\boldsymbol{w}_S$:

$$\boldsymbol{w}_T^* = \operatorname*{argmin}_{\boldsymbol{w}} \frac{1}{|\mathcal{S}_T|} \sum_{(I,Y)\in\mathcal{S}_T} \mathcal{L}\,(I, Y; \boldsymbol{w}), \qquad \boldsymbol{w}^{(0)} = \boldsymbol{w}_S, \qquad (3.13)$$

with $\mathcal{S}_T$ the target training set and $\mathcal{L}$ the loss of Equation (3.10). Since a finite number of gradient steps explores a neighbourhood of the starting point, the source solution acts as an implicit prior. It is a useful one because early layers learn general features such as edges and colour blobs while later layers become task-specific, and transferred initialisation improves generalisation even after full fine-tuning [46]; better source models also tend to transfer better [101]. All trainable layers are fine-tuned in this thesis (Ultralytics keeps the fixed projection of the DFL decoding frozen); only the last 1×1 convolution of the classification branch, whose output dimension changes from 80 to one, is newly initialised.

The source–target difference can be analysed through the factorisation $P(X, Y) = P(X)\,P(Y \mid X)$. The marginal $P(X)$ shifts because renderings of articulated figures with palette colours, procedural ground textures, a fixed elevated viewpoint, small objects and compression artefacts differ systematically from COCO photographs, a covariate shift [45]. The conditional $P(Y \mid X)$ shifts because COCO boxes, the tight boxes of the annotated instance masks [16], enclose the visible extent of a person, whereas the labels here are amodal boxes, clipped to the image, that enclose the whole body; for a partially occluded pedestrian the same pixels correspond to a different correct box, a shift that no rendering realism removes and that only target labels can teach. Moreover, dense crowds of small, mutually occluding people are atypical of general-purpose data, as the dedicated CrowdHuman benchmark with about 22.6 persons per image illustrates [99]. A zero-shot baseline that keeps only the COCO “person” class measures the combined effect of these shifts. The per-sequence variation of illumination gain and colour tint, together with fixed blur, sensor noise and H.264 compression, provides a mild form of domain randomisation [43], [44]. The gap of concern is from COCO to the synthetic domain, in which everything is evaluated; the gap to real video is a limitation discussed in Chapter 6. The decisive consequence for the pipeline is that fine-tuning changes the score distribution, so the tracker’s thresholds must be tuned for the fine-tuned detector.

3.2 Multi-Object Tracking by Detection and ByteTrack

3.2.1 Problem Definition

In frame t of a video of T frames, a set $\mathcal{G}_t$ of ground-truth objects with persistent identities is present, the detector supplies a set $\mathcal{D}_t$ of scored candidate boxes, and the tracker outputs hypotheses $\mathcal{H}_t$, each a track identity with a box. In this thesis $\mathcal{D}_t$ denotes the cached detector output, namely the candidates scoring above 0.05 that survive the network's own NMS at the permissive threshold 0.9 (Section 3.1.5). Ideally every object is covered by exactly one hypothesis whose identity never changes, and every hypothesis covers an object. Tracking by detection decomposes the problem into detection, motion prediction, affinity computation and association [12], [13]. The tuned pipeline first applies post-hoc NMS with the threshold θ_{nms} and then ByteTrack, which is online and in this thesis uses motion and overlap only. Given the cached candidates, the output is therefore a deterministic function of the parameters, the inner part of the composition in Equation (4.1), before the metric is applied,

$$\mathcal{H}_{1:T} = \text{Track}(\mathcal{D}_{1:T};\ \boldsymbol{\theta}) = \text{ByteTrack}\big(\text{NMS}_{\theta_{\text{nms}}}(\mathcal{D}_{1:T});\ \boldsymbol{\theta}_{\text{trk}}\big), \qquad (3.14)$$

where $\text{NMS}_{\theta_{\text{nms}}}$ is the greedy suppression of Equation (3.12), applied frame by frame, and $\boldsymbol{\theta} = (\theta_{\text{nms}}, \boldsymbol{\theta}_{\text{trk}})$ is defined in Section 3.2.6, $\boldsymbol{\theta}_{\text{trk}}$ being its six tracker components. This function is piecewise constant in $\boldsymbol{\theta}$. For a finite sequence the output changes only at finitely many critical values: where a score or an association cost crosses a threshold, where the quantised NMS threshold or the floored buffer length of Equation (3.25) jumps, or where the optimal assignment of Equation (3.21) changes with θ_m. At such a value the output changes discontinuously, as a track is born, lost or matched differently. Every tracking metric is therefore a step function of $\boldsymbol{\theta}$ with zero gradient almost everywhere, which is the fundamental reason why the parameters are tuned by black-box optimisation and reinforcement learning rather than by gradient descent.

3.2.2 Kalman Filter Motion Model

ByteTrack inherits the motion model of DeepSORT [21], [106]. Each track carries the state

$$\boldsymbol{x} = (x, y, a, h, \dot{x}, \dot{y}, \dot{a}, \dot{h})^{\top}, \qquad (3.15)$$

with box centre (x, y), aspect ratio $a = w/h$, height h in pixels and velocities per frame. Motion is constant-velocity and a detection supplies the measurement $\boldsymbol{z} = (x, y, a, h)^{\top}$:

$$\boldsymbol{F} = \begin{pmatrix} \boldsymbol{I}_4 & \boldsymbol{I}_4 \\ \boldsymbol{0} & \boldsymbol{I}_4 \end{pmatrix}, \qquad \boldsymbol{H} = (\boldsymbol{I}_4 \quad \boldsymbol{0}). \qquad (3.16)$$

The bold observation matrix $\boldsymbol{H}$ is unrelated to the image height H of Section 3.1.1. The Kalman filter alternates prediction and correction [18]:

$$\hat{\boldsymbol{x}}_{t|t-1} = \boldsymbol{F}\hat{\boldsymbol{x}}_{t-1|t-1}, \qquad \boldsymbol{P}_{t|t-1} = \boldsymbol{F}\boldsymbol{P}_{t-1|t-1}\boldsymbol{F}^{\top} + \boldsymbol{Q}_t, \qquad (3.17)$$

$$\boldsymbol{S}_t = \boldsymbol{H}\boldsymbol{P}_{t|t-1}\boldsymbol{H}^{\top} + \boldsymbol{R}_t, \quad \boldsymbol{K}_t = \boldsymbol{P}_{t|t-1}\boldsymbol{H}^{\top}\boldsymbol{S}_t^{-1}, \quad \hat{\boldsymbol{x}}_{t|t} = \hat{\boldsymbol{x}}_{t|t-1} + \boldsymbol{K}_t\big(\boldsymbol{z}_t - \boldsymbol{H}\hat{\boldsymbol{x}}_{t|t-1}\big), \quad \boldsymbol{P}_{t|t} = \boldsymbol{P}_{t|t-1} - \boldsymbol{K}_t\boldsymbol{S}_t\boldsymbol{K}_t^{\top}. \qquad (3.18)$$

The noise covariances are diagonal with standard deviations proportional to the current height, with weights $\sigma_p = 1/20$ for position and $\sigma_v = 1/160$ for velocity:

$$\boldsymbol{Q}_t = \operatorname{diag}\big((\sigma_p h)^2, (\sigma_p h)^2, 10^{-4}, (\sigma_p h)^2, (\sigma_v h)^2, (\sigma_v h)^2, 10^{-10}, (\sigma_v h)^2\big), \qquad \boldsymbol{R}_t = \operatorname{diag}\big((\sigma_p h)^2, (\sigma_p h)^2, 10^{-2}, (\sigma_p h)^2\big). \qquad (3.19)$$

A new track starts from the measured box with zero velocity and covariance $\operatorname{diag}\big((2\sigma_p h)^2, (2\sigma_p h)^2, 10^{-4}, (2\sigma_p h)^2, (10\sigma_v h)^2, (10\sigma_v h)^2, 10^{-10}, (10\sigma_v h)^2\big)$. Three properties of this model matter for tuning. First, the noise scales with h, which makes the filter invariant to apparent size, and the aspect ratio is treated as almost constant, which suits upright walkers. Second, the noise is specified per frame and does not scale with the frame interval: at the 15 frames per second of the virtual camera a pedestrian moves twice as far per frame as at 30 frames per second, the rate assumed by the track-buffer convention of Equation (3.25), so the filter is relatively over-confident and the association gates must absorb larger prediction errors. Third, ByteTrack associates using only the predicted mean, unlike the Mahalanobis gating of DeepSORT [106]; a lost track coasts at its last velocity with zero height velocity, and its growing covariance affects only the gain at re-activation. Since linear extrapolation accumulates error during occlusions [107], the lost-track buffer of Section 3.2.5 cannot usefully be made arbitrarily long.

3.2.3 Association Costs and the Assignment Problem

For a track with predicted box $\widehat{\boldsymbol{b}}_i$ and a detection $(\boldsymbol{b}_j, s_j)$, ByteTrack uses a score-fused cost in the first association, as in the official and the Ultralytics implementations, and the plain IoU distance in the second [21], [22], [24]:

$$c_{ij}^{(1)} = 1 - \mathrm{IoU}(\widehat{\boldsymbol{b}}_i, \boldsymbol{b}_j)\, s_j, \qquad c_{ij}^{(2)} = 1 - \mathrm{IoU}(\widehat{\boldsymbol{b}}_i, \boldsymbol{b}_j). \qquad (3.20)$$

Given a cost matrix $\boldsymbol{C} \in \mathbb{R}^{n\times m}$ and a cost limit θ_m, the association solves the linear assignment problem

$$\min_{\boldsymbol{X}\in\{0,1\}^{n\times m}} \sum_{i,j} (c_{ij} - \theta_m)\, x_{ij} \quad \text{s.t.} \quad \sum_j x_{ij} \le 1, \quad \sum_i x_{ij} \le 1, \quad x_{ij} = 0 \text{ if } c_{ij} > \theta_m. \qquad (3.21)$$

Each admissible pair contributes the non-positive amount $c_{ij} - \theta_m$, so the solution maximises the total margin $\sum_{i,j} (\theta_m - c_{ij})\, x_{ij}$ over admissible pairs and never matches a pair above the limit. It does not necessarily maximise the number of matches: with two tracks A and B, two detections 1 and 2, $c_{A1} = 0$, $c_{A2} = c_{B1} = c \le \theta_m$ and $c_{B2} > \theta_m$, the single match of A with 1 beats the two matches of A with 2 and B with 1 whenever $\theta_m < 2c$. Equation (3.21) is a linear assignment problem, classically solved by the Hungarian method [19], [104]. The official code solves it with the Jonker–Volgenant shortest-augmenting-path algorithm on a matrix padded with dummy entries of cost $\theta_m/2$, which makes leaving a track and a detection unassigned cost θ_m and is equivalent to Equation (3.21) [22], [105]. The implementation used here applies SciPy's exact linear-assignment solver, itself a modified Jonker–Volgenant algorithm [186], to the clipped matrix $\min(c_{ij} - \theta_m, 0)$ and keeps only the pairs with $c_{ij} \le \theta_m$, which again yields an optimal solution of Equation (3.21).

The cost limits translate into overlap requirements: a first-stage match needs $\mathrm{IoU} \cdot s \ge 1 - \theta_{m1}$ and a second-stage match $\mathrm{IoU} \ge 1 - \theta_{m2}$. With the defaults $\theta_{m1} = 0.8$ and $\theta_{m2} = 0.5$, a detection scoring 0.9 can be matched at an IoU of 0.22, one scoring 0.6 needs 0.33, and a low-score detection needs 0.5, so score fusion makes the gate adaptive to detection confidence. For two equal boxes offset horizontally by a fraction δ of their width,

$$\mathrm{IoU}(\delta) = \frac{(1-\delta)\, wh}{2wh - (1-\delta)\, wh} = \frac{1-\delta}{1+\delta}, \qquad 0 \le \delta \le 1, \qquad (3.22)$$

so an IoU of 0.5 tolerates a displacement of one third of the box width and an IoU of 0.2 two thirds. Because pedestrian boxes are narrow, these tolerances are small in absolute terms: the limits must be loose enough to absorb prediction errors at low frame rates yet tight enough to keep a track from jumping to a neighbour. Raising θ_{m1} trades fragmentation for identity swaps between adjacent pedestrians, a balance that depends on density and motion.

3.2.4 The BYTE Two-Stage Association

ByteTrack associates almost every detection rather than only the confident ones [21]. The detections of frame t that survive the post-hoc NMS, $\mathcal{D}_t^{\text{nms}}$, are split into

$$\mathcal{D}_t^{\text{high}} = \{(\boldsymbol{b}, s) \in \mathcal{D}_t^{\text{nms}} : s > \tau_{\text{high}}\}, \qquad \mathcal{D}_t^{\text{low}} = \{(\boldsymbol{b}, s) \in \mathcal{D}_t^{\text{nms}} : \tau_{\text{low}} < s < \tau_{\text{high}}\}, \qquad (3.23)$$

and boxes with $s \leq \tau_{\text{low}}$ are discarded, as are, in the official code, boxes with s exactly equal to τ_{high}, because both inequalities are strict [22]. Figure 3.2 shows the per-frame procedure. (i) Tracked and lost tracks are propagated with Equation (3.17). (ii) $\mathcal{D}_t^{\text{high}}$ is matched to tracked and lost tracks with cost $c^{(1)}$ and limit θ_{m1}; matched lost tracks are re-activated with their old identity. (iii) $\mathcal{D}_t^{\text{low}}$ is matched to the still unmatched tracked tracks (not lost ones) with cost $c^{(2)}$ and limit θ_{m2}; unmatched low-score boxes are discarded and still unmatched tracked tracks become lost. (iv) Unconfirmed tracks born in the previous frame are matched to the remaining high-score boxes with cost $c^{(1)}$ and the fixed limit 0.7; unmatched ones are removed. (v) Each remaining high-score box with $s \geq \tau_{\text{new}}$ starts a new unconfirmed track. (vi) Lost tracks unmatched for longer than the buffer period are removed. All matched tracks receive the correction of Equation (3.18).

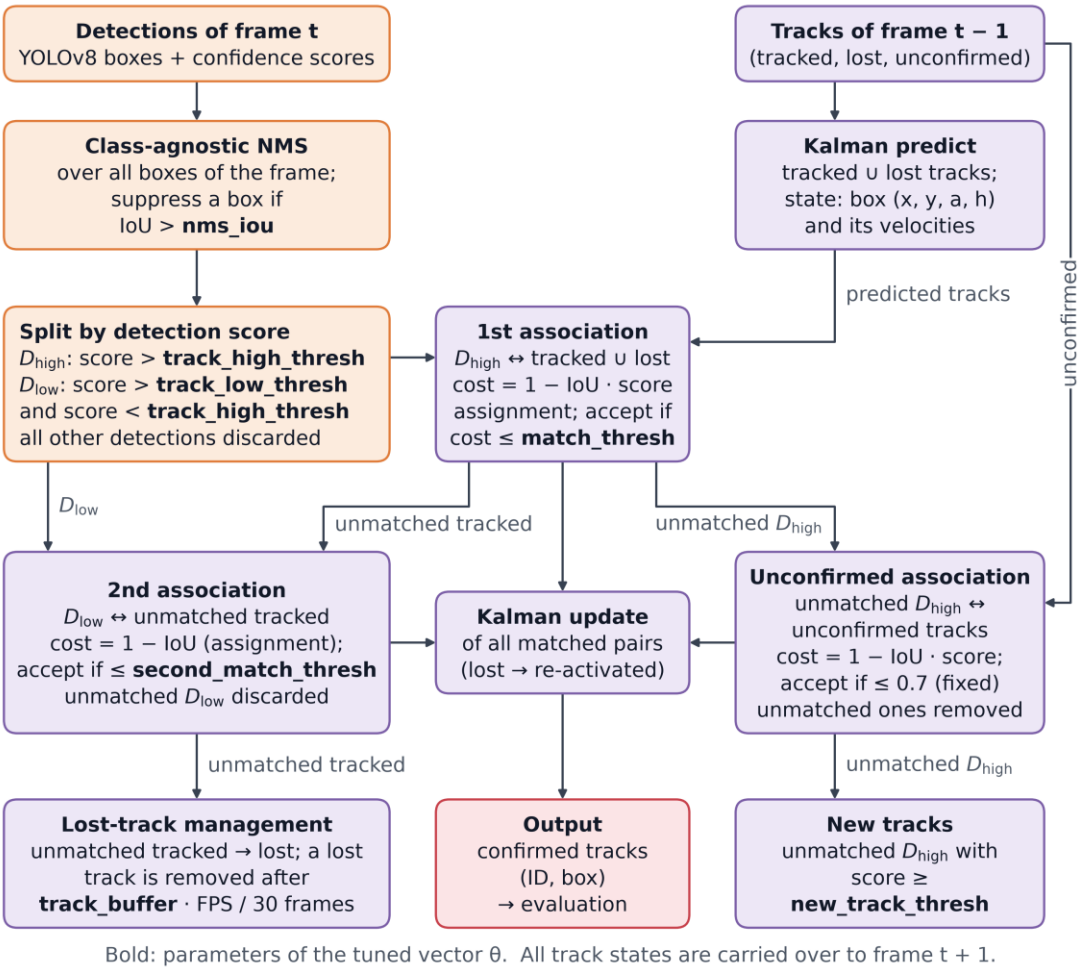


Figure 3.2. Per-frame flowchart of ByteTrack as used in this thesis. Detections pass class-agnostic NMS and are split by score into D_high, D_low and discarded boxes; D_high is associated with the predicted tracked and lost tracks (cost 1 − IoU·s), D_low with the remaining tracked tracks (cost 1 − IoU) and the remaining high-score boxes with unconfirmed tracks (fixed limit 0.7). Matched tracks are updated, unmatched high-score boxes may start tracks, unmatched tracked tracks become lost and are removed after the buffer period, and confirmed tracks are output. Parameters in bold belong to the tuned vector θ and are shown with their configuration names (Table 4.6); the NMS is the post-hoc suppression with threshold θ_nms (nms_iou).

Bayes' rule shows why low-score boxes help. Let E be the event that a detection corresponds to a real pedestrian and M the event that it passes the second-stage gate of an existing tracked track. For a detection with score s,

$$\frac{P(E \mid s, M)}{P(\bar{E} \mid s, M)} = \frac{P(M \mid E, s)}{P(M \mid \bar{E}, s)} \cdot \frac{P(E \mid s)}{P(\bar{E} \mid s)}. \qquad (3.24)$$

A true detection of a tracked pedestrian overlaps that pedestrian's prediction with high probability, whereas a background false positive does so only by coincidence. The likelihood ratio is therefore large, and a low-score box that continues a track can be more trustworthy than an isolated high-score box. BYTE exploits this asymmetry by letting low-

score boxes continue tracks but never start them. Occlusion is the typical case: as a pedestrian passes behind another, the visible part shrinks, the IoU-aware score falls (Section 3.1.4) and the detection drops below τ_{high}; without the second stage the track would be lost and the pedestrian possibly re-detected under a new identity, costing an identity switch and a fragmentation. Only IoU is used in this stage because appearance cues of occluded boxes are unreliable [21]. The mechanism has a failure mode: when two pedestrians overlap heavily, a low-score box on the occluder, or a duplicate left by permissive NMS, may be attached to the occluded pedestrian's track. The thresholds τ_{low} and θ_{m2} govern this risk and are scene-dependent, which is why the official code's hard-coded values of 0.1 and 0.5 [22] are exposed as tunable parameters here.

3.2.5 Track Life Cycle

A track is unconfirmed, tracked (confirmed), lost or removed. A new track is unconfirmed, except in the first frame of a sequence, and is confirmed only if matched again in the next frame, so single-frame false positives never reach the output. An unmatched tracked track becomes lost; it keeps its identity, continues to be predicted and is re-activated by a first-stage match with a high-score box, so low-score boxes can bridge gaps of tracked tracks but cannot revive lost ones. A lost track is removed after more than

$$N_{\text{lost}} = \left\lfloor \frac{f}{30}\, n_{\text{buf}} \right\rceil \qquad (3.25)$$

unmatched frames, where f is the frame rate and n_{buf} the track buffer in frames at 30 frames per second (configuration name `track_buffer`). The buffer thus corresponds to $n_{\text{buf}}/30$ seconds at any frame rate: at 15 frames per second the default of 30 keeps a lost track for 15 frames or one second, and the search range [10,150] (Table 4.6) spans 5 to 75 frames, or 0.33 to 5 seconds. As in the official code, a lost track that is marked removed remains in the lost pool for one further frame, in which a first-stage match can still re-activate it, so the longest gap that can be bridged is $N_{\text{lost}} + 1$ frames, 16 at the default [22]. In addition, when a tracked and a lost track overlap with an IoU above 0.85, the one that has existed for the shorter time is discarded, and only confirmed tracked tracks with an area above 10 square pixels and a width-to-height ratio of at most 1.6 are output. A lost track produces no output, so its missed frames count as false negatives until it is re-activated. The buffer length therefore expresses a trade-off: a long buffer bridges long occlusions, but a drifting prediction may later be re-activated by another pedestrian's

detection, an identity switch, whereas a short buffer fragments trajectories into several identities.

3.2.6 The Tuned Parameter Vector

Detector post-processing and tracker together expose seven tuned parameters, which form the vector $\boldsymbol{\theta} = (\theta_{\text{nms}}, \tau_{\text{high}}, \tau_{\text{low}}, \tau_{\text{new}}, \theta_{m1}, \theta_{m2}, n_{\text{buf}})$: the NMS threshold of Equation (3.12); the high-score threshold, which splits the surviving detections into $\mathcal{D}^{\text{high}}$ and $\mathcal{D}^{\text{low}}$ in Equation (3.23); the low-score threshold, at or below which boxes are discarded; the new-track threshold of step (v) in Section 3.2.4; the cost limits of the first and second association, that is, the limits θ_m of Equation (3.21) for the costs $c^{(1)}$ and $c^{(2)}$ of Equation (3.20); and the lost-track buffer of Equation (3.25). The components θ_{m1} and θ_{m2} are scalars, and $\boldsymbol{\theta}_{\text{trk}}$ denotes the six components after θ_{nms}. The search ranges, the configuration names and the default vector $\boldsymbol{\theta}_0$, which combines published values and is the reference of every comparison, are design decisions and are listed in Table 4.6 (Section 4.7.2). All optimisers, including the policies, act in the unit cube $\boldsymbol{u} \in [-1,1]^7$, which Equation (4.7) maps affinely onto the parameter box $\Theta = [\boldsymbol{\theta}^{\min}, \boldsymbol{\theta}^{\max}]$; θ_{nms} is then mapped to the nearest of the eleven grid values $0.40, 0.45, \ldots, 0.90$, and n_{buf} enters Equation (3.25) through the floor.

The parameters are structurally coupled. New tracks arise only from $\mathcal{D}^{\text{high}}$, so the effective birth threshold is $\max(\tau_{\text{high}}, \tau_{\text{new}})$ and τ_{new} has no effect below τ_{high}. The ranges of τ_{low} and τ_{high} overlap (Table 4.6), so an optimiser can empty $\mathcal{D}^{\text{low}}$ and reduce BYTE to a single-stage association in the style of SORT [20]. Duplicates surviving permissive NMS create spurious tracks only if they exceed the birth threshold and remain unmatched, coupling θ_{nms} with τ_{high} and τ_{new}. Finally, the minimum IoU of a first-stage match, $(1 - \theta_{m1})/s$, depends on the scores admitted by τ_{high}. The response of a tracking metric to $\boldsymbol{\theta}$ is therefore non-separable, contains plateaus on which some parameters have no effect, and is piecewise constant (Section 3.2.1), properties that shape the choice of optimisers in Sections 3.5 and 3.6.

3.3 Pedestrian Dynamics: The Social Force Model

The tracking problem is only as representative as the motion that generates it. The AnyLogic Pedestrian Library moves pedestrians in continuous space, reacting to walls, obstacles and each other [50], but does not expose its equations and is treated in the

literature as a social-force-based black box [51]. Because the experiments run on a Python co-simulation of the AnyLogic scenarios (Chapter 4), the model of the co-simulation is stated here explicitly. The co-simulation does not reproduce the library's undisclosed equations. It shares with the AnyLogic models the scenario geometry, flowchart semantics, pedestrian attribute distributions, arrival processes and camera poses. Its dynamics combine the social force model of Helbing and Molnár [47], the elliptical interaction of Johansson, Helbing and Shukla [9] and the body forces of Helbing, Farkas and Vicsek [3]; the lateral evasion rule, the contact damping and the bound on backward walking are additions made in this thesis and are not taken from these sources, whereas the speed cap of $1.3\, v_i^0$ follows Helbing and Molnár. All forces are per unit mass; Table 3.1 lists the parameter values, and Chapter 4 describes navigation, arrivals and calibration.

3.3.1 Equation of Motion and Driving Term

The velocity $\boldsymbol{v}_i$ and position $\boldsymbol{x}_i$ of pedestrian i obey

$$\frac{d\boldsymbol{v}_i}{dt} = \frac{v_i^0\, \boldsymbol{e}_i - \boldsymbol{v}_i}{\tau} + \sum_{j \neq i} \left(\boldsymbol{f}_{ij} + \boldsymbol{f}_{ij}^{\mathrm{ev}} + \boldsymbol{f}_{ij}^{\mathrm{c}} \right) + \sum_{W} \boldsymbol{f}_{iW} + \boldsymbol{\xi}_i, \qquad \frac{d\boldsymbol{x}_i}{dt} = \boldsymbol{v}_i, \qquad (3.26)$$

with, in order, the driving term, social interaction, lateral evasion, body contact, wall interaction and a noise term $\boldsymbol{\xi}_i$ of standard deviation 0.05 m/s^2 per component. The driving term relaxes the velocity towards the desired velocity within $\tau = 0.5$ s; in free flow $\boldsymbol{v}_i(t) = v_i^0 \boldsymbol{e}_i + (\boldsymbol{v}_i(0) - v_i^0 \boldsymbol{e}_i) e^{-t/\tau}$, which closes 95% of the gap within 1.5 s. Desired speeds are uniform on [1.0,1.6] m/s. Following AnyLogic's target-line semantics, the desired direction points to the closest point $\boldsymbol{q}_i$ of the current target line,

$$\boldsymbol{e}_i = \frac{\boldsymbol{q}_i - \boldsymbol{x}_i}{\| \boldsymbol{q}_i - \boldsymbol{x}_i \|}, \qquad (3.27)$$

while route-level navigation is described in Chapter 4. Figure 3.3 illustrates the geometry.

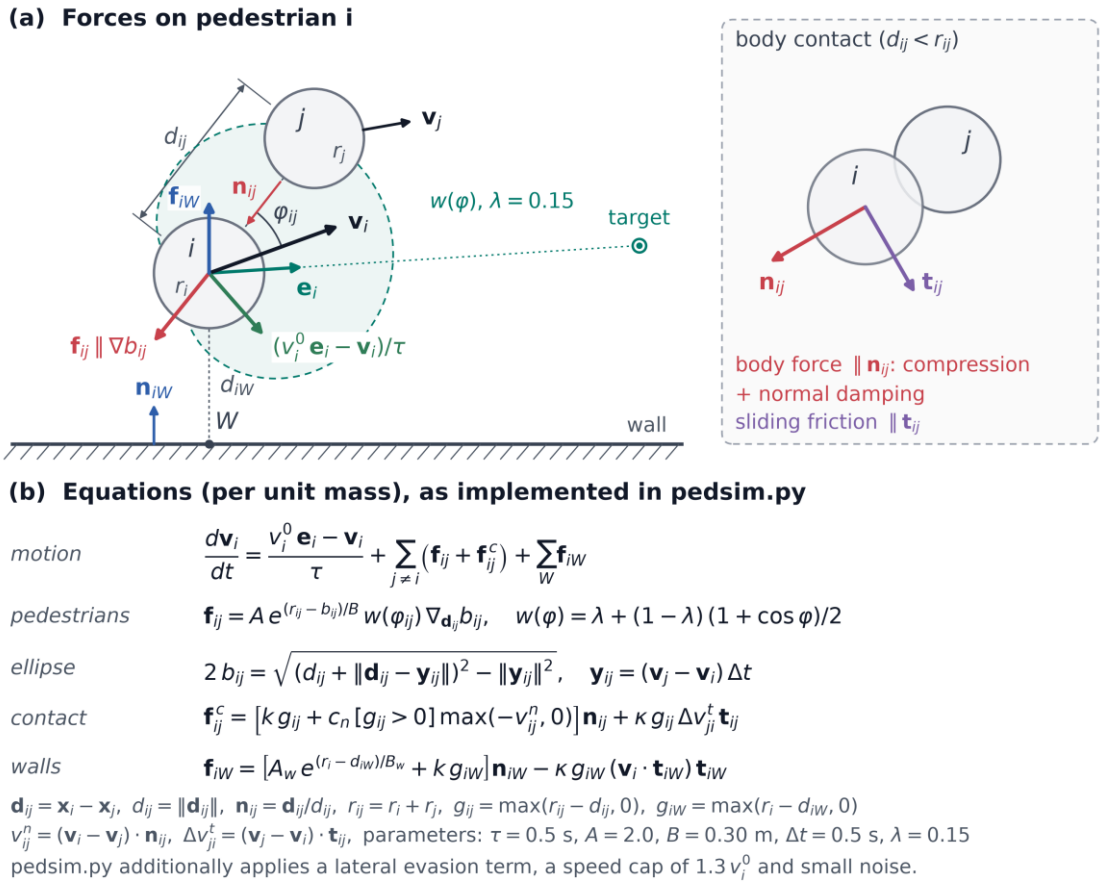


Figure 3.3. Schematic of the social force model as implemented in this thesis. (a) Pedestrian i is driven along the desired direction e_i towards its target by (v_i^0 e_i − v_i)/τ, is repelled by pedestrian j with a force parallel to the gradient of the elliptical distance b_ij and weighted by the anisotropy w(φ_ij) (shaded region, λ = 0.15), and is repelled by the nearest wall point W along n_iW; the inset shows body contact with compression and normal damping along n_ij and sliding friction along t_ij. (b) The equations per unit mass with the parameter values used (calibrated A = 2.0 m/s² and B = 0.30 m; fixed τ = 0.5 s, Δt = 0.5 s and λ = 0.15); the lateral evasion term, the speed cap and the noise term are not drawn.

3.3.2 Pedestrian Interaction: Elliptical Specification II

In the circular specification, repulsion depends only on the distance $d_{ij} = \| \boldsymbol{x}_i - \boldsymbol{x}_j \|$ (marked in Figure 3.3). Such a potential does not anticipate encounters: pedestrians on a head-on course come close before they are pushed apart abruptly, producing unrealistic oscillations in counter-flow. Johansson et al. calibrated several specifications against video-tracked trajectories; the velocity-dependent elliptical specification II reached the best fit, although its advantage over the simple circular specification was moderate [9]. Their elliptical specification II combines the relative position $\boldsymbol{d}_{ij} = \boldsymbol{x}_i - \boldsymbol{x}_j$ with the relative displacement $\boldsymbol{y}_{ij} = (\boldsymbol{v}_j - \boldsymbol{v}_i)\,\Delta t$ over a look-ahead time $\Delta t = 0.5$ s:

$$b_{ij} = \frac{1}{2}\sqrt{(\| \boldsymbol{d}_{ij} \| + \| \boldsymbol{d}_{ij} - \boldsymbol{y}_{ij} \|)^2 - \| \boldsymbol{y}_{ij} \|^2}. \qquad (3.28)$$

Geometrically, b_{ij} is the semi-minor axis of the ellipse through $\boldsymbol{x}_i$ whose foci are $\boldsymbol{x}_j$ and $\boldsymbol{x}_j + \boldsymbol{y}_{ij}$, the position j will occupy relative to i after the look-ahead time: $\| \boldsymbol{d}_{ij} \| + \| \boldsymbol{d}_{ij} - \boldsymbol{y}_{ij} \|$ is twice the semi-major axis and $\| \boldsymbol{y}_{ij} \|$ the focal distance. Without relative motion $b_{ij} = d_{ij}$, recovering the circular case; for an approaching pedestrian the equipotentials stretch along the relative motion, so repulsion sets in earlier. The force follows from the potential $V(b) = AB\exp\big((r_{ij} - b)/B\big)$ by differentiation with respect to $\boldsymbol{d}_{ij}$, weighted by an anisotropy factor:

$$\boldsymbol{f}_{ij} = F_{ij}\, \nabla_{\boldsymbol{d}_{ij}} b_{ij}, \qquad F_{ij} = A \exp\left(\frac{r_{ij} - b_{ij}}{B}\right) w(\varphi_{ij}), \qquad (3.29)$$

$$\nabla_{\boldsymbol{d}_{ij}} b_{ij} = \frac{\| \boldsymbol{d}_{ij} \| + \| \boldsymbol{d}_{ij} - \boldsymbol{y}_{ij} \|}{4\, b_{ij}} \left(\frac{\boldsymbol{d}_{ij}}{\| \boldsymbol{d}_{ij} \|} + \frac{\boldsymbol{d}_{ij} - \boldsymbol{y}_{ij}}{\| \boldsymbol{d}_{ij} - \boldsymbol{y}_{ij} \|} \right), \qquad (3.30)$$

where Equation (3.30) follows from differentiating b_{ij}^2 in Equation (3.28) with $\boldsymbol{y}_{ij}$ held fixed. Here $r_{ij} = r_i + r_j$ is the sum of the radii (diameters uniform on [0.4,0.5] m), and $A = 2.0$ m/s^2 and $B = 0.30$ m result from the calibration in Chapter 4. The anisotropy, introduced by Helbing and Molnár as a field-of-view weighting [47], takes the continuous form used by Johansson et al. [9]:

$$w(\varphi_{ij}) = \lambda + (1 - \lambda)\, \frac{1 + \cos\varphi_{ij}}{2}, \qquad \cos\varphi_{ij} = -\boldsymbol{n}_{ij} \cdot \hat{\boldsymbol{v}}_i, \qquad (3.31)$$

where $\boldsymbol{n}_{ij} = \boldsymbol{d}_{ij} / \| \boldsymbol{d}_{ij} \|$ points from j to i and $\hat{\boldsymbol{v}}_i$ is the walking direction (the direction of $\boldsymbol{v}_i$, or $\boldsymbol{e}_i$ below 0.1 m/s). The weight is one for a pedestrian straight ahead and $\lambda = 0.15$ for one directly behind; this λ is unrelated to λ_{GAE} and λ_{p} in Sections 3.5 and 3.6. With the anisotropy the interaction is no longer a strict potential force and is not reciprocal, an intended property, since pedestrians are not physical particles [47], [48].

3.3.3 Lateral Evasion

The elliptical force still acts mainly along the line between two pedestrians, so in an exactly head-on encounter both decelerate without deflecting and may remain blocked until noise breaks the symmetry. Real pedestrians sidestep early, behaviour that heuristic models describe with simple rules [130]. An evasion term therefore acts when j lies ahead ($\cos\varphi_{ij} > 0$) and the two approach each other, $\chi_{ij} = \mathbf{1}\big[(\boldsymbol{v}_i - \boldsymbol{v}_j) \cdot (\boldsymbol{x}_j - \boldsymbol{x}_i) > 0\big] = 1$:

$$\boldsymbol{f}_{ij}^{\mathrm{ev}} = k_{\mathrm{ev}}\, F_{ij}\, \max(\cos\varphi_{ij}, 0)\, \chi_{ij}\, \sigma_{ij}\, \boldsymbol{e}_i^{\perp}, \qquad (3.32)$$

with F_{ij} from Equation (3.29), the right-hand normal $\boldsymbol{e}_i^{\perp}$ of the desired direction, gain $k_{\mathrm{ev}} = 0.8$, and $\sigma_{ij} = +1$ (step right) if j is on the left or almost straight ahead and -1 (step left) if j is on the right. The keep-right convention breaks the symmetry of head-on encounters consistently and favours lane formation. Chapter 4 reports the verification showing that, without the elliptical specification and this term, counter-flow produced pedestrians bouncing back and forth.

3.3.4 Body Contact, Sliding Friction and Damping

When bodies overlap, physical forces act in addition [3]. With overlap $g_{ij} = \max(0, r_{ij} - \| \boldsymbol{d}_{ij} \|)$ and tangential unit vector $\boldsymbol{t}_{ij}$,

$$\boldsymbol{f}_{ij}^{\mathrm{c}} = \Big(k\, g_{ij} + c_n\, \mathbf{1}[g_{ij} > 0] \max\big(0, -(\boldsymbol{v}_i - \boldsymbol{v}_j) \cdot \boldsymbol{n}_{ij}\big) \Big) \boldsymbol{n}_{ij} + \kappa\, g_{ij} \big((\boldsymbol{v}_j - \boldsymbol{v}_i) \cdot \boldsymbol{t}_{ij} \big) \boldsymbol{t}_{ij}, \qquad (3.33)$$

comprising body compression, a normal damping term that opposes the approaching normal velocity, and sliding friction. The coefficients $k = 1500$ s^{-2} and $\kappa = 3000$ m^{-1}s^{-1} equal the values $k = 1.2 \times 10^5$ kg s^{-2} and $\kappa = 2.4 \times 10^5$ kg m^{-1}s^{-1} of Helbing et al. divided by their mass of 80 kg [3], and $c_n = 60$ s^{-1}. The damping does not make contacts inelastic in the discrete scheme. It acts on both bodies and is integrated explicitly, so one step of Equation (3.35) multiplies the approaching relative normal velocity by $1 - 2c_n \Delta t_{\mathrm{s}}$, compression aside. With $c_n \Delta t_{\mathrm{s}} = 1$ this factor is -1: the damping over-corrects and reverses an approach instead of removing it, and because the compression term only adds to the reversed velocity, a pair subject to the contact term alone leaves the contact at least as fast as it approached. A value of $c_n = 30$ s^{-1}, or an implicit treatment of the damping, would remove the approaching velocity in one step. Contacts are rare in normal walking but matter at the bottleneck gate and at high density, where the compression term keeps bodies from interpenetrating, an artefact the camera would show clearly.

3.3.5 Walls and Obstacles

Walls, barriers and obstacle edges are line segments. With d_{iW} the distance to the nearest point of segment W, $\boldsymbol{n}_{iW}$ the unit vector from that point to the pedestrian, $\boldsymbol{t}_{iW}$ the tangent and $g_{iW} = \max(0, r_i - d_{iW})$,

$$\boldsymbol{f}_{iW} = A_w \exp\left(\frac{r_i - d_{iW}}{B_w}\right) \boldsymbol{n}_{iW} + k\, g_{iW}\, \boldsymbol{n}_{iW} - \kappa\, g_{iW}\, (\boldsymbol{v}_i \cdot \boldsymbol{t}_{iW})\, \boldsymbol{t}_{iW}, \qquad (3.34)$$

with $A_w = 5.0$ m/s^2 and $B_w = 0.12$ m, a stronger but shorter-ranged repulsion than between pedestrians.

3.3.6 Kinematic Constraints and Numerical Integration

The speed is capped at $1.3\,v_i^0$, and voluntary backward walking is excluded by enforcing $\boldsymbol{v}_i \cdot \boldsymbol{e}_i \geq -0.1$ m/s (except while a pedestrian waits). Equation (3.26) is integrated with the semi-implicit Euler method and time step $\Delta t_s = 1/60$ s,

$$\boldsymbol{v}_i^{n+1} = \Pi(\boldsymbol{v}_i^n + \Delta t_s\, \boldsymbol{a}_i^n), \qquad \boldsymbol{x}_i^{n+1} = \boldsymbol{x}_i^n + \Delta t_s\, \boldsymbol{v}_i^{n+1}, \qquad (3.35)$$

where $\boldsymbol{a}_i^n$ is the right-hand side of Equation (3.26) and Π applies the constraints. Updating positions with the new velocity gives better long-term behaviour than the explicit Euler method for the oscillatory contact springs at no extra cost.

Table 3.1. Parameters of the social force model of the pedestrian co-simulation (forces per unit mass).

Term	Symbol	Value
Relaxation time	τ	0.5 s
Desired speed	v_i^0	uniform on [1.0, 1.6] m/s
Body diameter	$2r_i$	uniform on [0.4, 0.5] m
Interaction strength and range	A, B	2.0 m/s^2, 0.30 m (calibrated)
Look-ahead time of the elliptical specification	Δt	0.5 s
Anisotropy	λ	0.15
Lateral evasion gain	k_{ev}	0.8
Body compression, sliding friction, normal damping	k, κ, c_n	1500 s^{-2}, 3000 m^{-1}s^{-1}, 60 s^{-1}
Wall strength and range	A_w, B_w	5.0 m/s^2, 0.12 m
Speed cap and backward-speed limit	–	$1.3\,v_i^0$, 0.1 m/s
Noise standard deviation	$\boldsymbol{\xi}_i$	0.05 m/s^2
Integration time step	Δt_s	1/60 s

3.3.7 The Fundamental Diagram

At the macroscopic level a pedestrian model is validated by its fundamental diagram, the relation between density and speed or flow. The most widely cited planar diagram is Weidmann's synthesis of 25 investigations [8], [59]:

$$v(\rho) = v_f \left[1 - \exp\left(-\gamma_W \left(\frac{1}{\rho} - \frac{1}{\rho_{\max}}\right)\right)\right], \qquad J(\rho) = \rho\, v(\rho), \qquad (3.36)$$

with free speed $v_f = 1.34$ m/s, maximal density $\rho_{\max} = 5.4$ ped/m^2, shape parameter $\gamma_W = 1.913$ ped/m^2 and specific flow J. Equation (3.36) gives about 1.30 m/s at 0.5 ped/m^2, 1.06 m/s at 1 ped/m^2 and 0.61 m/s at 2 ped/m^2, with a maximal flow of about 1.22

ped/(m s) near 1.75 ped/m^2. Chapter 4 uses this relation to calibrate A and B. It matters for a second reason as well: density fixes the mean spacing between pedestrians, roughly $1/\sqrt{\rho}$, which is 1 m at 1 ped/m^2 and about 0.7 m at 2 ped/m^2, and bodies at such spacings overlap in the image of an oblique, elevated camera. The density regimes of the model thus become the occlusion regimes faced by detector and tracker, and an unrealistic fundamental diagram would also produce unrealistic occlusion statistics.

3.4 Evaluation Metrics

Tracking metrics define the rewards of the agents, the objective of the baseline optimisers and the final evaluation. They compare the tracker output with the simulator's ground truth using the IoU of Equation (3.5). In analogy with the MOTChallenge evaluation, which excludes non-evaluated objects [34], [35], this thesis treats ground-truth objects with visibility below 0.1 or with less than half of their silhouette inside the image as "ignore" objects (Chapter 4): they are not evaluated, and tracker boxes matched to them with an IoU of at least 0.5 are removed. The metrics are computed by a re-implementation of TrackEval [60], verified against it in Section 4.8. Table 3.2 summarises the metrics.

3.4.1 CLEAR MOT Metrics

The CLEAR MOT metrics establish correspondences frame by frame [31]. In each frame, ground truth $\mathcal{G}_t$ and hypotheses $\mathcal{H}_t$ are matched one-to-one by an optimal assignment that maximises total similarity subject to an IoU of at least 0.5, with priority, in TrackEval, for pairs matched in the previous frame. Matches $\mathcal{M}_t$ are true positives, unmatched ground truth are false negatives and unmatched hypotheses false positives; an identity switch (IDSW) is counted when an object is matched to a different identity than at its most recent match. Then

$$\mathrm{MOTA} = 1 - \frac{\sum_t (\mathrm{FN}_t + \mathrm{FP}_t + \mathrm{IDSW}_t)}{\sum_t |\mathcal{G}_t|}, \qquad \mathrm{MOTP} = \frac{\sum_t \sum_{(g,h)\in\mathcal{M}_t} \mathrm{IoU}(g,h)}{\sum_t |\mathcal{M}_t|}. \qquad (3.37)$$

MOTA lies in $(-\infty, 1]$; MOTP is computed, as in TrackEval, as the mean IoU of the matches, whereas the original formulation averaged a distance error [31]. A fragmentation (Frag) is counted whenever the tracked status of a ground-truth trajectory resumes after an interruption, and a trajectory is mostly tracked (MT) if matched in more than 80% of its

frames and mostly lost (ML) if matched in less than 20%. Because misses and false positives usually far outnumber identity switches, each of which is counted only once, MOTA is dominated by detection quality.

3.4.2 Identity Metrics

The identity metrics measure how long identities remain correct by matching whole trajectories one-to-one [32]. Let n_ω and n_η be the numbers of frames in which ground-truth trajectory $\omega \in \Omega$ and hypothesis trajectory $\eta \in \mathcal{E}$ are present, and $m_{\omega\eta}$ the number of frames in which both are present with an IoU of at least 0.5. Pairing ω with η yields $m_{\omega\eta}$ identity true positives, $n_\omega - m_{\omega\eta}$ identity false negatives and $n_\eta - m_{\omega\eta}$ identity false positives, and unpaired trajectories contribute all their frames as errors. Among all one-to-one pairings $\mathcal{J} \subset \Omega \times \mathcal{E}$, the chosen one minimises the errors,

$$\mathcal{J}^* = \operatorname*{argmin}_{\mathcal{J}} \left[\left(\sum_{\omega \in \Omega} n_\omega - \sum_{(\omega,\eta)\in\mathcal{J}} m_{\omega\eta} \right) + \left(\sum_{\eta \in \mathcal{E}} n_\eta - \sum_{(\omega,\eta)\in\mathcal{J}} m_{\omega\eta} \right) \right], \qquad (3.38)$$

where the two brackets are IDFN and IDFP; it is found by solving an assignment problem on a $(|\Omega| + |\mathcal{E}|)^2$ matrix with dummy entries. Since $\text{IDFN} + \text{IDFP} = \sum_\omega n_\omega + \sum_\eta n_\eta - 2\,\text{IDTP}$, this maximises the identity true positives. Then

$$\text{IDP} = \frac{\text{IDTP}}{\text{IDTP} + \text{IDFP}}, \qquad \text{IDR} = \frac{\text{IDTP}}{\text{IDTP} + \text{IDFN}}, \qquad \text{IDF1} = \frac{2\,\text{IDTP}}{2\,\text{IDTP} + \text{IDFP} + \text{IDFN}}. \qquad (3.39)$$

Because each trajectory is credited to only one hypothesis, a switch in the middle of a long trajectory can cost up to half of its frames, so IDF1 strongly rewards long, consistent identities.

3.4.3 Higher Order Tracking Accuracy

As discussed in Section 2.2.6, HOTA was designed to balance detection and association, which MOTA and IDF1 weight unevenly [33]. In the TrackEval implementation reproduced here [60], ground truth and predictions are matched once per frame by an optimal assignment that maximises the product of the IoU and a global alignment score, which measures how consistently the two identities overlap over the whole sequence. For each localisation threshold α (unrelated to the assigner exponent α of

Section 3.1.4), the matched pairs with an IoU of at least α are true positives, and the remaining objects and predictions are false negatives and false positives:

$$\mathrm{DetA}_\alpha = \frac{|\mathrm{TP}|}{|\mathrm{TP}| + |\mathrm{FN}| + |\mathrm{FP}|}. \qquad (3.40)$$

For each true positive c, $\mathrm{TPA}(c)$ contains the true positives with the same ground-truth and prediction identities, $\mathrm{FNA}(c)$ the detections of the same object that are missed or matched to another prediction identity, and $\mathrm{FPA}(c)$ the predictions of the same identity that are unmatched or matched to another object. Then

$$\mathcal{A}(c) = \frac{|\mathrm{TPA}(c)|}{|\mathrm{TPA}(c)| + |\mathrm{FNA}(c)| + |\mathrm{FPA}(c)|}, \qquad \mathrm{AssA}_\alpha = \frac{1}{|\mathrm{TP}|}\sum_{c\in\mathrm{TP}} \mathcal{A}\,(c), \qquad (3.41)$$

$$\mathrm{HOTA}_\alpha = \sqrt{\mathrm{DetA}_\alpha \cdot \mathrm{AssA}_\alpha}, \qquad \mathrm{HOTA} = \frac{1}{19}\sum_{\alpha\in\{0.05,0.10,\ldots,0.95\}} \mathrm{HOTA}_\alpha\,. \qquad (3.42)$$

DetA and AssA are likewise averaged over α, and the localisation accuracy is the mean IoU of the true positives,

$$\mathrm{LocA} = \frac{1}{19}\sum_{\alpha} \frac{1}{|\mathrm{TP}_\alpha|}\sum_{c\in\mathrm{TP}_\alpha} \mathrm{IoU}\,(c). \qquad (3.43)$$

DetA further decomposes into detection recall and precision, and AssA into association recall and precision [33].

Figure 3.4 works through a toy example in which every matched pair exceeds α. Two ground-truth trajectories over ten frames are covered by four predicted trajectories, giving 16 true positives, 2 false negatives and 2 false positives, hence $\mathrm{DetA}_\alpha = 0.80$. For the pair (GT A, Pred 1), six true positives share both identities, four detections of GT A are missed or matched to Pred 2, and two detections of Pred 1 are matched to GT B, so $\mathcal{A}(c) = 6/12 = 0.50$. Averaging over all true positives gives $\mathrm{AssA}_\alpha = 7.31/16 \approx 0.457$ and $\mathrm{HOTA}_\alpha = \sqrt{0.80 \cdot 0.457} \approx 0.60$.

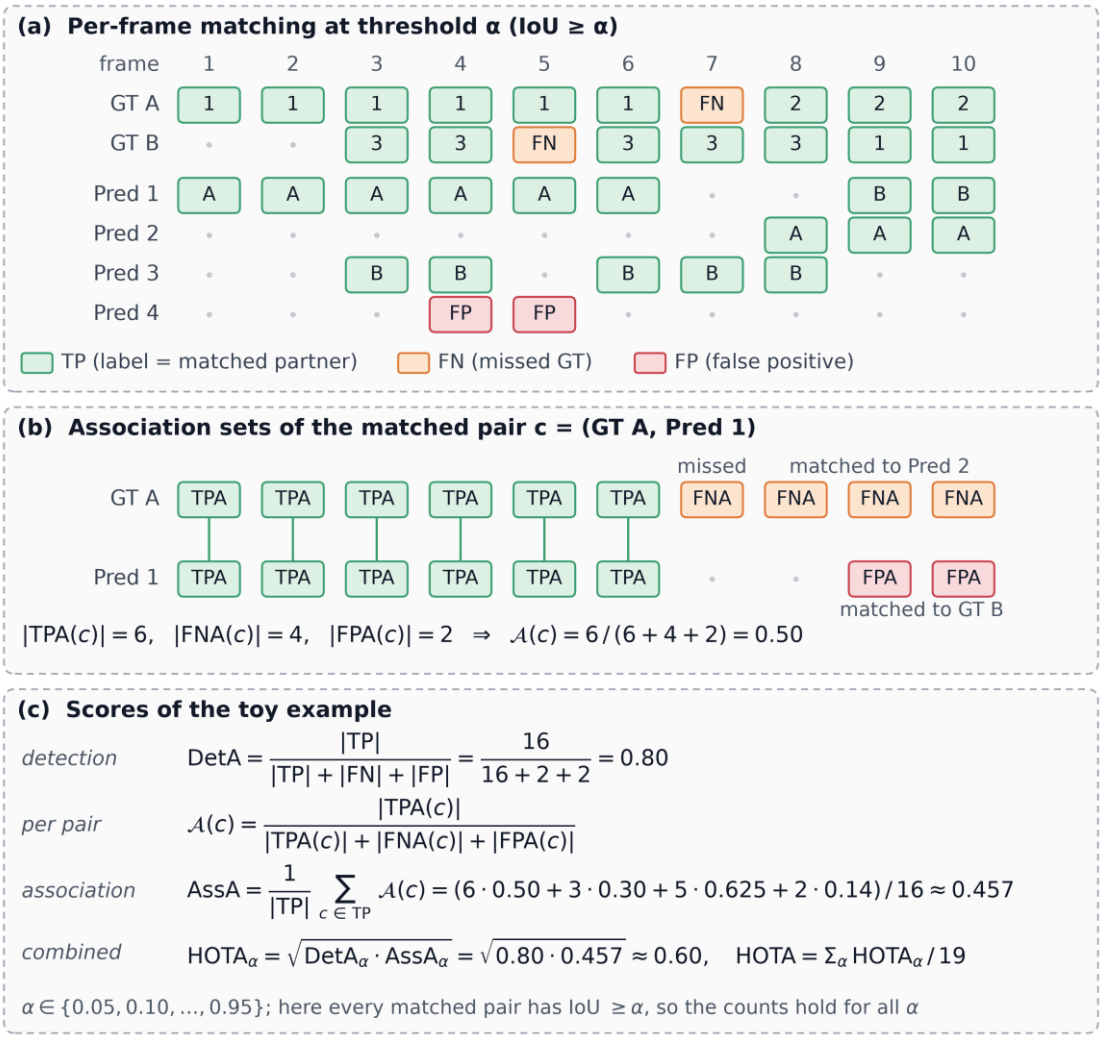


Figure 3.4. Worked example of HOTA at one threshold α. (a) Per-frame matching of two ground-truth and four predicted trajectories over ten frames, with each true positive labelled by its partner and misses (FN) and unmatched predictions (FP) marked. (b) Association sets of the pair c = (GT A, Pred 1): six TPA, four FNA and two FPA, so A(c) = 0.50. (c) DetA = 0.80, AssA ≈ 0.457 as the mean of the association accuracies over all true positives, HOTA_α = √(0.80 · 0.457) ≈ 0.60, and the final HOTA as the mean over α ∈ {0.05, …, 0.95}.

For comparison, the same example yields $\mathrm{MOTA} = 1 - (2 + 2 + 2)/18 \approx 0.67$, each identity change being counted once, and $\mathrm{IDF1} = 2 \cdot 11/(2 \cdot 11 + 7 + 7) \approx 0.61$, the best pairing (GT A with Pred 1, GT B with Pred 3) crediting 11 of 18 ground-truth detections. MOTA hardly notices the fragmented identities, whereas HOTA penalises them through every true positive with incomplete association. As a geometric mean averaged over thresholds, HOTA drops if detection, association or localisation is neglected, which makes it the natural objective for a parameter vector that affects all three at once.

3.4.4 Combination over Sequences and Use as Optimisation Signal

Results over several sequences are combined as in TrackEval's COMBINED row rather than by averaging sequence scores; Section 4.8 states the rule (Equation (4.8)). Where a decision applies to a whole clip, as in the contextual formulation and the baseline optimisers, the clip's HOTA is used. Where decisions are made every second, as in the adaptive formulation, HOTA is unsuitable, because its association term depends on entire trajectories and the HOTA of a one-second window is neither additive over windows nor attributable to that window's decision. The CLEAR counts, in contrast, are additive over frames, so the adaptive reward uses the window score $(\mathrm{TP} - \mathrm{FP} - 5\,\mathrm{IDSW})/\mathrm{GT}$, computed by a streaming CLEAR matching that keeps the identity history across windows, minus the same score of a default-parameter run of the same sequence (Equation (4.10)). Since $\mathrm{TP} = \mathrm{GT} - \mathrm{FN}$, the window score equals the window's MOTA minus $4\,\mathrm{IDSW}/\mathrm{GT}$, a MOTA-like score in which an identity switch weighs five times as much as a miss.

Table 3.2. Tracking metrics used in this thesis.

Metric	Definition	Range	Emphasis	Use in this thesis
MOTA	Equation (3.37)	$(-\infty, 1]$	Detection errors; identity switches counted once	Evaluation; basis of the adaptive reward
MOTP	Equation (3.37)	$[0.5,1]$	Localisation of matched boxes	Evaluation
IDSW, Frag	Section 3.4.1	≥ 0	Identity changes, interruptions	Evaluation; IDSW in the adaptive reward
MT, ML	Section 3.4.1	counts	Coverage of whole trajectories	Evaluation
IDF1, IDP, IDR	Equation (3.39)	$[0,1]$	Long-term identity consistency	Evaluation
HOTA, DetA, AssA, LocA	Equations (3.40) to (3.43)	$[0,1]$	Balance of detection, association and localisation	Contextual reward, HPO objective, main evaluation metric

3.5 Reinforcement Learning

3.5.1 Markov Decision Processes

Reinforcement learning formalises sequential decision making as a Markov decision process (MDP),

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, p_0), \qquad (3.44)$$

with state space $\mathcal{S}$, action space $\mathcal{A}$, transition probabilities $P(s' \mid s, a)$, reward function $r(s, a)$, discount factor $\gamma \in [0,1]$ and initial-state distribution p_0 [53], [170]. The state is written s, as is conventional; it is unrelated to the detection score s_k of Sections 3.1

and 3.2. A policy $\pi_{\boldsymbol{\phi}}(a \mid s)$ with parameters $\boldsymbol{\phi}$ selects actions, and the return from step t of an episode of length T is

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k \, r_{t+k}, \qquad (3.45)$$

where r_t is the reward received after action a_t in state s_t. The value, action-value and advantage functions are

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid s_t = s], \qquad Q^{\pi}(s,a) = \mathbb{E}_{\pi}[G_t \mid s_t = s, a_t = a], \qquad A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s), \qquad (3.46)$$

where the superscripted advantage A^{π} is unrelated to the interaction strength A of Section 3.3, and the value function satisfies the Bellman expectation equation

$$V^{\pi}(s) = \mathbb{E}_{a \sim \pi(\cdot \mid s)}\big[r(s,a) + \gamma \, \mathbb{E}_{s' \sim P(\cdot \mid s,a)} \, V^{\pi}(s')\big]. \qquad (3.47)$$

The objective is the expected return from the initial states,

$$J(\boldsymbol{\phi}) = \mathbb{E}_{s_0 \sim p_0}[V^{\pi_{\boldsymbol{\phi}}}(s_0)]. \qquad (3.48)$$

Here the environment is the tracking pipeline, wrapped in the Gymnasium interface [62]. Simulation, rendering and detection do not depend on the action and are cached. For each action $\boldsymbol{u} \in [-1,1]^7$, mapped to $\boldsymbol{\theta}$ by Equation (4.7), suppression is selected from keep-masks pre-computed for the eleven grid thresholds, and tracking and evaluation are recomputed; the state summarises the scene and the pipeline's recent behaviour, and the reward derives from the metrics of Section 3.4. The contextual formulation reduces to a one-step problem (Section 3.5.2).

The adaptive formulation is a multi-step MDP over the one-second windows of a sequence. Its observation s_t is 19-dimensional: twelve statistics of the window just processed, such as detection counts, association match rates, track births and deaths and crowding, together with the current $\boldsymbol{\theta}$ in unit coordinates (Section 4.9.2). The action $\boldsymbol{u}_t$ sets $\boldsymbol{\theta}$ for the next window of 15 frames and is applied to the running tracker without a reset, so tracks, Kalman states and identities persist and a decision also affects later windows. The reward r_t is the window score of Section 3.4.4 for the policy run minus the same score of a default-parameter run of the same sequence; the default run's window scores are precomputed once per sequence and do not depend on the actions, so subtracting them leaves the optimal policy unchanged, but because the two runs carry different tracks

after the first decision, r_t reflects the accumulated effect of all earlier decisions and not only that of the latest one. An episode is a whole training sequence of 60 or 90 windows, discounted with $\gamma = 0.9$ (Section 3.5.4). The Markov property holds only approximately, because the true state includes every track's Kalman state and the configuration of the crowd, of which the agent observes a compact summary; the problem is treated as an MDP over observations, a standard approximation.

3.5.2 Contextual Bandits as One-Step MDPs

When every episode consists of a single decision, transitions and discounting become irrelevant and the objective reduces to

$$J(\boldsymbol{\phi}) = \mathbb{E}_{s\sim p_0}\, \mathbb{E}_{a\sim\pi_{\boldsymbol{\phi}}(\cdot|s)}[r(s,a)], \qquad (3.49)$$

the contextual bandit problem, in which the learner observes a context, chooses an action and sees only that action's reward [57], [58]. The contextual formulation of this thesis has this form: the context is a nine-dimensional descriptor computed from the detections of the first four seconds of a 20-second clip, the action is the parameter vector for the clip, and the reward is $r = 100\,(\mathrm{HOTA}(\boldsymbol{\theta}) - \mathrm{HOTA}(\boldsymbol{\theta}_0))$ with the default parameters $\boldsymbol{\theta}_0$. Subtracting the default score leaves the optimal action of every context unchanged, because the subtracted term does not depend on the action, but removes much of the reward variation caused by clip difficulty. It is a clip-dependent, action-independent baseline; it introduces no bias because the policy conditions only on s, and it reduces the variance of the gradient estimate of Section 3.5.3. The factor 100 expresses the reward in HOTA points. The optimal policy is

$$\pi^*(s) = \operatorname*{argmax}_{\boldsymbol{\theta}\in\Theta} \mathbb{E}[\mathrm{HOTA}(\boldsymbol{\theta}) - \mathrm{HOTA}(\boldsymbol{\theta}_0) \mid s], \qquad (3.50)$$

so the trained policy is an amortised optimiser: instead of one black-box optimisation per condition, it learns a mapping intended to return a near-optimal parameter vector for any observed context in one forward pass. Bandit feedback provides no derivative with respect to the action, so the policy must explore, which a stochastic Gaussian policy does by construction.

3.5.3 The Policy Gradient Theorem and Gaussian Policies

The policy gradient theorem gives the gradient of J without differentiating the environment [53], [172]:

$$\nabla_{\boldsymbol{\phi}} J(\boldsymbol{\phi}) \propto \mathbb{E}_{s\sim d^{\pi},\, a\sim\pi_{\boldsymbol{\phi}}(\cdot|s)}\left[\nabla_{\boldsymbol{\phi}} \log \pi_{\boldsymbol{\phi}}(a \mid s)\,(Q^{\pi_{\boldsymbol{\phi}}}(s,a) - b(s))\right], \qquad (3.51)$$

where d^{π} is the on-policy state distribution and $b(s)$ any action-independent baseline. The baseline does not bias the gradient [172], because $\mathbb{E}_{a\sim\pi}\left[\nabla_{\boldsymbol{\phi}} \log \pi_{\boldsymbol{\phi}}(a \mid s)\right] = \nabla_{\boldsymbol{\phi}} \int \pi_{\boldsymbol{\phi}}(a \mid s)\, da = 0$, but it reduces variance, and $b = V^{\pi}$ turns the bracket into the advantage A^{π} of Equation (3.46) [53], [173]. Equation (3.51) is why policy gradients suit this problem: the derivative falls on the smooth log-density of the policy, not on the reward, which is a step function of the parameters.

For continuous actions the policy is a diagonal Gaussian, $\boldsymbol{a} = \boldsymbol{\mu}_{\boldsymbol{\phi}}(s) + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$ with $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, a network output as mean and state-independent learned standard deviations $\boldsymbol{\sigma} = \exp(\ell_{\sigma})$, as in Stable-Baselines3 [61]. Its log-likelihood is

$$\log \pi_{\boldsymbol{\phi}}(\boldsymbol{a} \mid s) = -\sum_{k=1}^{7} \left[\frac{(a_k - \mu_{\boldsymbol{\phi},k}(s))^2}{2\sigma_k^2} + \log \sigma_k + 1/2 \log 2\pi \right], \qquad (3.52)$$

whose gradient with respect to the mean is $(a_k - \mu_k)/\sigma_k^2$ times the network gradient of μ_k: weighted by the advantage, a better-than-expected action pulls the mean towards itself and a worse one pushes it away, while the gradient with respect to $\log \sigma_k$, proportional to $(a_k - \mu_k)^2/\sigma_k^2 - 1$, narrows the distribution when good actions lie near the mean. Sampled actions are clipped to $[-1,1]^7$ before Equation (4.7) is applied, the log-likelihood is evaluated for the unclipped sample, and the deterministic mean is used at deployment. Seen from the parameter space, the expected reward of the Gaussian policy, $\mathbb{E}_{\boldsymbol{\varepsilon}}[r(s, \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon})]$, is a Gaussian-smoothed version of the discontinuous reward and is differentiable in $\boldsymbol{\mu}$, which lets the policy progress across the plateaus and steps described in Section 3.2.6.

3.5.4 Actor-Critic Methods and Generalised Advantage Estimation

Actor-critic methods learn a value function $V_{\boldsymbol{\psi}}(s)$, the critic, alongside the policy, the actor, to estimate advantages [53]. Generalised advantage estimation (GAE) sums exponentially weighted temporal-difference residuals [173]:

$$\delta_t = r_t + \gamma V_{\boldsymbol{\psi}}(s_{t+1}) - V_{\boldsymbol{\psi}}(s_t), \qquad \hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma \lambda_{\mathrm{GAE}})^l\, \delta_{t+l}, \qquad (3.53)$$

with $V_{\boldsymbol{\psi}}(s_T) = 0$ at the end of an episode. The parameter $\lambda_{\mathrm{GAE}} \in [0,1]$ trades bias against variance: $\lambda_{\mathrm{GAE}} = 0$ gives the low-variance but critic-biased estimate δ_t, and

$\lambda_{\mathrm{GAE}} = 1$ gives the unbiased but noisier $G_t - V_{\boldsymbol{\psi}}(s_t)$ [173]. In the contextual bandit, $\gamma = 0$ and $\lambda_{\mathrm{GAE}} = 1$, so $\hat{A} = r - V_{\boldsymbol{\psi}}(s)$ and the critic learns the expected gain of the current policy over the defaults in that context, a learned state-dependent baseline. In the adaptive formulation, $\gamma = 0.9$ gives an effective horizon of $1/(1-\gamma) = 10$ decisions, about ten seconds, long enough to credit a parameter change with its delayed effects on track survival and identity switches, and $\lambda_{\mathrm{GAE}} = 0.95$ follows common practice [54], [61].

3.5.5 Proximal Policy Optimisation

Trust region policy optimisation (TRPO) stabilises policy updates by maximising an importance-sampled surrogate subject to a bound on the Kullback–Leibler divergence between successive policies [174]; proximal policy optimisation (PPO) achieves a similar effect with a first-order method [54]. With the probability ratio

$$\rho_t(\boldsymbol{\phi}) = \frac{\pi_{\boldsymbol{\phi}}(a_t \mid s_t)}{\pi_{\boldsymbol{\phi}_{\mathrm{old}}}(a_t \mid s_t)}, \qquad (3.54)$$

between the current policy and the one that collected the data (the ratio ρ_t is unrelated to the density ρ of Section 3.3.7), the clipped surrogate objective is

$$L^{\mathrm{CLIP}}(\boldsymbol{\phi}) = \hat{\mathbb{E}}_t\left[\min(\rho_t(\boldsymbol{\phi})\hat{A}_t,\ \mathrm{clip}(\rho_t(\boldsymbol{\phi}), 1-\epsilon, 1+\epsilon)\hat{A}_t)\right], \qquad (3.55)$$

with $\epsilon = 0.2$. Panel (b) of Figure 3.5 plots one sample's objective against the ratio. For a positive advantage it grows up to $1+\epsilon$ and is flat beyond, so there is no incentive to raise a good action's probability by more than that factor; for a negative advantage it is flat below $1-\epsilon$. Because of the minimum, L^{CLIP} is a pessimistic bound on the unclipped surrogate, and samples whose ratio has left the interval in the favourable direction contribute no gradient. Several epochs of minibatch updates on the same batch are therefore relatively safe, which matters when every sample requires a full tracking run and its evaluation.

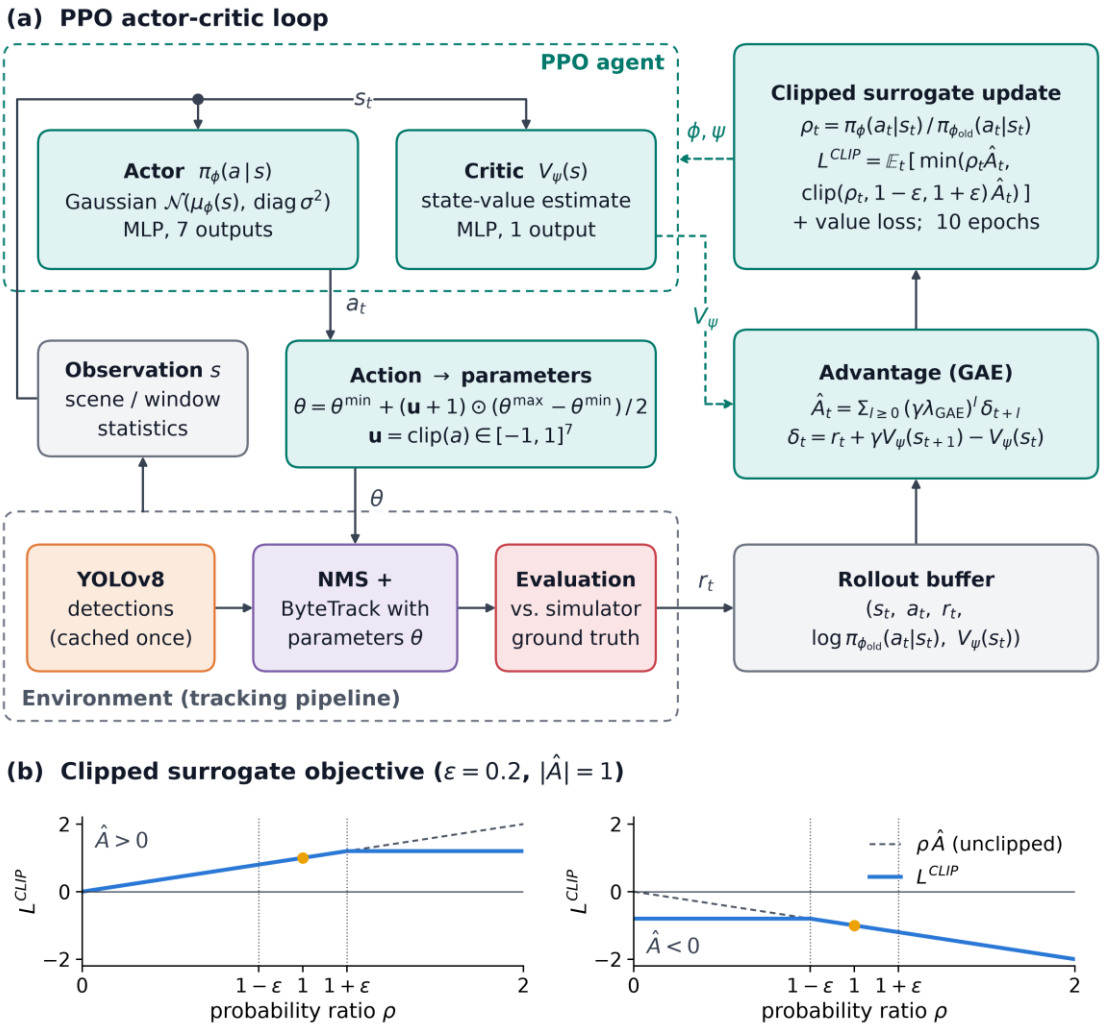


Figure 3.5. Proximal policy optimisation as used in this thesis. (a) The actor, a Gaussian policy with seven outputs, maps the observation s to an action a, which is clipped to u and mapped to θ by Equation (4.7) with the range bounds θ^min and θ^max; the environment applies NMS and ByteTrack with θ to cached YOLOv8 detections and evaluates the result against the simulator ground truth to give the reward r. Transitions are stored in a rollout buffer, advantages are computed by GAE from the critic V_ψ, and actor and critic are updated over ten epochs with the clipped surrogate and the value loss. (b) Clipped surrogate objective of one sample against the probability ratio ρ_t for positive (left) and negative (right) advantage, ε = 0.2; the dashed line is the unclipped objective ρ_t Â_t.

The critic is trained by regression on value targets, and exploration can be encouraged by the policy entropy:

$$L^{\mathrm{VF}}(\boldsymbol{\psi}) = \hat{\mathbb{E}}_t\big[(V_{\boldsymbol{\psi}}(s_t) - \hat{V}_t^{\mathrm{targ}})^2\big], \qquad \hat{V}_t^{\mathrm{targ}} = \hat{A}_t + V_{\boldsymbol{\psi}_{\mathrm{old}}}(s_t), \qquad (3.56)$$

$$\mathcal{H}[\pi_{\boldsymbol{\phi}}] = \sum_{k=1}^{7} (\log\sigma_k + 1/2\log(2\pi e)). \qquad (3.57)$$

The combined objective, maximised over both parameter sets, is

$$L(\boldsymbol{\phi}, \boldsymbol{\psi}) = L^{\mathrm{CLIP}}(\boldsymbol{\phi}) - c_{\mathrm{vf}} L^{\mathrm{VF}}(\boldsymbol{\psi}) + c_{\mathrm{ent}} \mathcal{H}[\pi_{\boldsymbol{\phi}}], \qquad (3.58)$$

with the value coefficient $c_{\mathrm{vf}} = 0.5$ and the entropy coefficient $c_{\mathrm{ent}} = 0$, so exploration is controlled by the learned standard deviations alone. Actor and critic are separate multilayer perceptrons without shared parameters. Stable-Baselines3 updates all parameters with a single Adam optimiser [187], normalises advantages per minibatch and clips the norm of the joint gradient at 0.5 [61]. Because actor and critic share no parameters and Adam normalises gradient magnitudes per parameter, $c_{\mathrm{vf}} = 0.5$ matters mainly through this global gradient-norm clipping, which couples the actor and critic updates.

Panel (a) of Figure 3.5 shows the training loop. Parallel environments collect a fixed number of transitions with the current policy; the transitions, with their log-probabilities and value estimates, fill a rollout buffer; advantages and value targets are computed with Equation (3.53); and actor and critic are updated for a fixed number of epochs (ten in this thesis) of minibatch gradient ascent on Equation (3.58) with Adam before the buffer is discarded. Network sizes, rollout lengths and learning rates of the two formulations are given in Chapter 4.

3.5.6 Why PPO for a Continuous Seven-Dimensional Action

Several properties of the problem favour PPO. First, the action is continuous and seven-dimensional. Value-based methods such as DQN maximise over a discrete action set [171], and three levels per parameter already give $3^7 = 2{,}187$ actions, five levels $5^7 = 78{,}125$, at coarse resolution; a Gaussian policy represents the action natively, and its mean is directly the deployed parameter vector. Second, the policy gradient needs only reward values (Section 3.5.3), which suits the piecewise-constant reward of Section 3.2.1. Third, PPO has well-tested default hyper-parameters and a reference implementation [54], [61], so the comparison with the baselines is less likely to be decided by tuning effort spent on the optimiser. Fourth, on-policy collection from parallel environments matches the cost structure of the problem, which lies in tracking runs that parallelise across processor cores. Off-policy methods such as DDPG and SAC can be more sample-efficient [175], [176], and the closest prior work on tracker hyper-parameters used continuous deep Q-learning [55]; these remain possible extensions. Finally, one algorithm for both formulations keeps their results comparable.

3.6 Hyper-Parameter Optimisation Baselines

3.6.1 The Black-Box Objective

The baselines treat tuning as black-box optimisation [162]. For a condition q, defined by scenario and density level or by the dynamic variant of a scenario, the objective is the mean HOTA over the set $\mathcal{C}_q$ of four fixed training clips,

$$f_q(\boldsymbol{\theta}) = \frac{1}{4}\sum_{c\in\mathcal{C}_q} \mathrm{HOTA}\left(\mathrm{Track}(\mathcal{D}_c;\boldsymbol{\theta}),\ \mathcal{G}_c\right), \qquad \boldsymbol{\theta}_q^* = \operatorname*{argmax}_{\boldsymbol{\theta}\in\Theta} f_q(\boldsymbol{\theta}), \qquad (3.59)$$

where $\mathcal{D}_c$ are the cached candidate detections of clip c (Section 3.2.1) and $\mathcal{G}_c$ its ground truth. The objective inherits the piecewise-constant structure of Section 3.2.1 and, because it depends on the particular clips, is a noisy estimate of performance on unseen sequences of the condition. Random search, TPE and CMA-ES each receive 200 evaluations of f_q per condition and seed in the unit cube of Equation (4.7). Since 16 conditions (twelve static, four dynamic) times 200 evaluations times four clips make 12,800 clip evaluations per seed, the contextual agent is trained for the same nominal number (13,056 executed, because Stable-Baselines3 completes its last rollout; Section 4.9.3), so the comparison is budget-matched to within about 2%. The essential difference lies in the output: Equation (3.59) yields one constant vector per condition, which must be selected with the condition label at test time, whereas the policy of Equation (3.50) yields a vector for any observed context.

3.6.2 Grid Search and Random Search

Grid search evaluates all L^d points of a lattice with L levels in d dimensions; with three levels (lower bound, centre, upper bound) and $d = 7$ it needs $3^7 = 2{,}187$ evaluations per condition, more than ten times the other budgets, and serves as a high-budget reference. However many points it evaluates, it explores only L values of each parameter. Random search samples $\boldsymbol{u}$ uniformly from the cube and so explores n distinct values of every parameter with n samples, except for the quantised θ_{nms} and n_{buf} [28]. If the near-optimal region occupies a fraction ζ of the volume, at least one of n samples hits it with probability

$$P_{\mathrm{hit}} = 1 - (1-\zeta)^n, \qquad (3.60)$$

independently of dimension: with $n = 200$, a 1% region is hit with probability of about 0.87 and a 5% region with probability above 0.9999. The fraction ζ itself, however, shrinks exponentially with the number of influential parameters, so random search is

efficient when the effective dimensionality is low [28], which the couplings of Section 3.2.6 may or may not permit.

3.6.3 Tree-Structured Parzen Estimator

Sequential model-based optimisation fits a model to past evaluations and selects the next point by maximising an acquisition function such as the expected improvement [163]. The Tree-structured Parzen Estimator (TPE) models the densities of good and bad configurations rather than the objective [27]. In the original formulation, which minimises a loss y, a threshold y^* is chosen such that a fraction γ_{TPE} of the observations lies below it, and

$$l(\boldsymbol{\theta}) = p(\boldsymbol{\theta} \mid y < y^*), \qquad g(\boldsymbol{\theta}) = p(\boldsymbol{\theta} \mid y \geq y^*), \qquad \mathrm{EI}_{y^*}(\boldsymbol{\theta}) \propto \left(\gamma_{\mathrm{TPE}} + (1 - \gamma_{\mathrm{TPE}})\frac{g(\boldsymbol{\theta})}{l(\boldsymbol{\theta})}\right)^{-1}. \qquad (3.61)$$

Maximising the expected improvement thus amounts to maximising $l(\boldsymbol{\theta})/g(\boldsymbol{\theta})$: the next configuration should be likely among good trials and unlikely among bad ones. For maximising HOTA the roles are mirrored. In Optuna [30], l and g are Parzen estimators with Gaussian kernels on the observed points. By default the good set comprises the best $\min(\lceil 0.1n \rceil, 25)$ of n trials, and 24 candidates are drawn from l, of which the one with the largest ratio is evaluated [188]; in this thesis the first 20 trials are random. The multivariate variant used here estimates joint densities and can represent correlations between coupled parameters. Because sampling concentrates where l is large, TPE converges quickly but may over-exploit a region that looked good on a noisy objective.

3.6.4 Covariance Matrix Adaptation Evolution Strategy

CMA-ES maintains a multivariate normal search distribution with mean $\boldsymbol{m}$, step size σ and covariance $\boldsymbol{C}$, and adapts them from the ranking of sampled candidates [29], [164]. Each generation draws λ_{p} candidates,

$$\boldsymbol{u}_k = \boldsymbol{m} + \sigma\, \boldsymbol{y}_k, \qquad \boldsymbol{y}_k \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{C}), \qquad k = 1, \ldots, \lambda_{\mathrm{p}}, \qquad (3.62)$$

ranks them and moves the mean towards a weighted average of the μ best steps $\boldsymbol{y}_{i:\lambda_{\mathrm{p}}}$, with positive decreasing weights w_i summing to one (the parent number μ is unrelated to the bold policy mean of Section 3.5.3):

$$\boldsymbol{m}' = \boldsymbol{m} + c_m \sigma \sum_{i=1}^{\mu} w_i \, \boldsymbol{y}_{i:\lambda_\mathrm{p}}. \qquad (3.63)$$

The covariance receives a rank-one update from the evolution path $\boldsymbol{p}_c$, which accumulates successive mean shifts, and a rank-μ update from the selected steps,

$$\boldsymbol{p}_c \leftarrow (1 - c_c)\boldsymbol{p}_c + h_\sigma \sqrt{c_c(2 - c_c)\mu_\mathrm{eff}} \, \frac{\boldsymbol{m}' - \boldsymbol{m}}{c_m \sigma}, \qquad \boldsymbol{C}$$

$$\leftarrow (1 - c_1 - c_\mu)\boldsymbol{C} + c_1 \, \boldsymbol{p}_c \boldsymbol{p}_c^\top + c_\mu \sum_{i=1}^{\mu} w_i \, \boldsymbol{y}_{i:\lambda_\mathrm{p}} \boldsymbol{y}_{i:\lambda_\mathrm{p}}^\top, \qquad (3.64)$$

and the step size is adapted with a conjugate path that lengthens when consecutive steps are aligned and shortens when they cancel:

$$\boldsymbol{p}_\sigma \leftarrow (1 - c_\sigma)\boldsymbol{p}_\sigma + \sqrt{c_\sigma(2 - c_\sigma)\mu_\mathrm{eff}} \, \boldsymbol{C}^{-1/2} \frac{\boldsymbol{m}' - \boldsymbol{m}}{c_m \sigma}, \qquad \sigma$$

$$\leftarrow \sigma \exp\left(\frac{c_\sigma}{d_\sigma} \left(\frac{\| \boldsymbol{p}_\sigma \|}{\mathbb{E} \| \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}) \|} - 1 \right) \right). \qquad (3.65)$$

Here $\mu_\mathrm{eff} = 1/\sum_i w_i^2$, the constants c_c, c_σ, c_1, c_μ, c_m and d_σ have recommended defaults, and $h_\sigma \in \{0,1\}$ stalls $\boldsymbol{p}_c$ when $\| \boldsymbol{p}_\sigma \|$ is large; Equation (3.64) is the basic form with positive weights [29]. The default population $\lambda_\mathrm{p} = 4 + \lfloor 3 \ln d \rfloor$ is 9 for $d = 7$, with $\mu = 4$ [29], so 200 evaluations correspond to about 22 generations. The CMA-ES mean is initialised at the centre of the cube, whereas the first trial is drawn by independent random sampling, Optuna's default of one start-up trial [188]; Optuna applies the initial step size in normalised $[0,1]$ coordinates, where the chosen 0.2 corresponds to 0.4 in $\boldsymbol{u}$. Being rank-based, CMA-ES is invariant to order-preserving transformations of the objective, and its full covariance can learn correlated directions such as a joint shift of the high-score and new-track thresholds [164], [165]. With 200 evaluations, however, the 28 free entries of a 7×7 covariance are estimated only roughly, and selection on a noisy objective slows adaptation.

3.6.5 Optimising Versus Learning to Optimise

The baselines and the agents answer different questions: per-condition optimisation asks which constant vector is best for a known condition, global TPE seeks one vector for all conditions, the contextual policy learns which vector suits an observed scene, and the adaptive policy learns how to change the vector as the scene evolves. A per-condition

optimum selected with the true condition label is a strong reference for any strategy that keeps the parameters constant within a condition, and it uses information that the policies, which observe only detector and tracker statistics, lack. Whether a learned policy can match it without the label, and whether online adaptation helps when density changes, are the empirical questions of Chapter 5.

3.7 Chapter Summary

This chapter has laid the theoretical basis of the pipeline. YOLOv8 was described as a dense, anchor-free detector whose DFL box representation, CIoU loss and task-aligned assignment produce an IoU-aware confidence score, reduced to detections by a confidence filter and greedy NMS, and fine-tuning from COCO was analysed as transfer under covariate and conditional shift that changes the score distribution. ByteTrack was formalised as a Kalman-filter tracker with two cost-limited assignment stages whose use of low-score boxes follows from a likelihood-ratio argument, and the seven parameters of $\boldsymbol{\theta}$ were defined with their roles and couplings. The social force model of the co-simulation was stated with all force terms and parameters, with the Weidmann fundamental diagram as calibration target and link between density and occlusion. The CLEAR MOT, identity and HOTA metrics were defined, and the use of HOTA for clip-level and of a CLEAR-based score for per-window decisions was justified. Reinforcement learning was developed from the MDP and the contextual bandit to PPO, whose suitability for a continuous seven-dimensional action and a discontinuous reward was argued, and grid search, random search, TPE and CMA-ES were presented as baselines returning one vector per condition. The recurring conclusion is that the tracking metric is a discontinuous, coupled and scene-dependent function of $\boldsymbol{\theta}$; Chapter 4 describes the experimental system built to address this problem.

Chapter 4 System Design and Simulation Model

This chapter describes how the methods introduced in Chapter 3 are assembled into a closed-loop system. Simulated pedestrians are filmed by a virtual camera, detected by a transfer-learned YOLOv8 network, tracked by ByteTrack and scored against the simulator's ground truth, and a reinforcement-learning (RL) agent uses the scores to re-tune the parameters of the detection–tracking pipeline. Section 4.1 presents the architecture, Section 4.2 the specification of the AnyLogic pedestrian model and Section 4.3 the Python co-simulation that generated the data, Sections 4.4 and 4.5 the virtual camera and the synthetic dataset, Sections 4.6 to 4.8 the detector, tracker and evaluation module, Sections 4.9 and 4.10 the RL environments and the baseline optimisers, and Section 4.11 the computational setup. Where Chapter 3 already states a model, this chapter cites its equations and records only what is specific to the implementation. Results follow in Chapter 5.

4.1 Overall Architecture and Closed Loop

The system has four layers, shown in Figure 4.1. The simulation layer generates pedestrian trajectories in four scenarios. A virtual camera renders them as 1280 × 720 pixel frames at 15 frames per second (FPS), the frames are encoded as H.264 video, and an exact per-frame ground truth of amodal boxes and visibility ratios is stored with each video. In the perception layer, frames of a dedicated detector-training split are exported with automatically generated labels to fine-tune COCO-pretrained YOLOv8 models. The fine-tuned network is then run on every frame of every tuning and test video, and its loosely suppressed outputs are cached together with pre-computed non-maximum suppression (NMS) masks. The tracking layer applies NMS and ByteTrack with a parameter vector $\boldsymbol{\theta}$ to the cached detections and scores the tracks against the ground truth with HOTA, MOTA and IDF1. The optimisation layer closes the loop: a PPO agent (Section 4.9) or a baseline optimiser (Section 4.10) proposes $\boldsymbol{\theta}$, receives the score as a reward or objective value, and proposes the next $\boldsymbol{\theta}$.

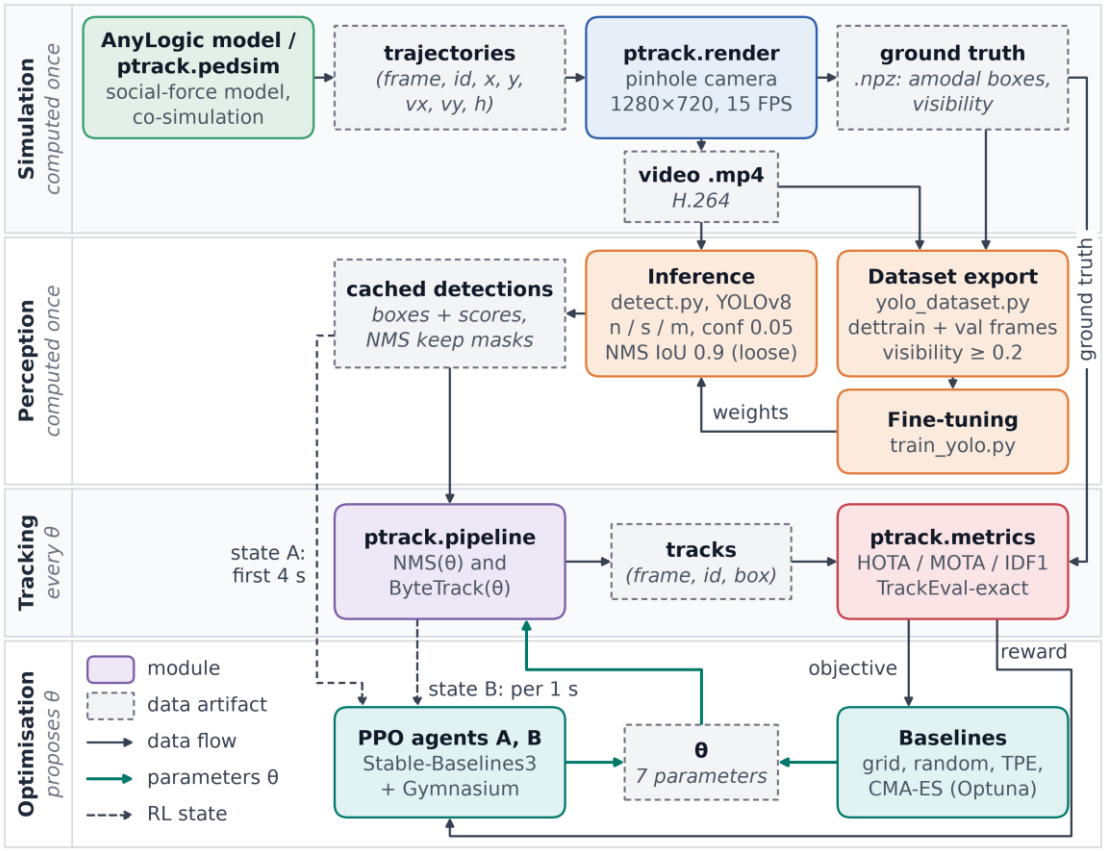


Figure 4.1. Architecture of the closed-loop system. Simulation and perception (pedestrian simulation, virtual camera, ground truth, YOLOv8 fine-tuning and inference) are computed once; only the tracking layer is re-run for every parameter vector θ proposed by the PPO agents or baseline optimisers.

The modelling toolbox of the study is AnyLogic 8, a multimethod simulation environment [49], with its Pedestrian Library [50]. The four scenarios are specified as AnyLogic models (Section 4.2, with the full specification and Java code templates in Appendix A). The experiments reported in this thesis, however, were executed with a headless Python co-simulation that implements the same scenarios, pedestrian attributes, arrival processes and camera poses. The module `ptrack.pedsim` integrates the social-force dynamics, and `ptrack.render` plays the role of the AnyLogic 3D camera. The reason is practical: the 256 sequences of the dataset had to be generated reproducibly, in batch and by script, without a graphical user interface, on a 32-thread workstation. The many thousands of pipeline evaluations of the RL agents and the hyper-parameter optimisation (HPO) baselines do not call the simulator in either case, because simulation is memoised (see below). The co-simulation is also transparent. The Pedestrian Library's motion equations are not documented, so the library behaves as a black box [51], whereas every force term and navigation rule of the co-simulation can be calibrated and reported (Section 4.3). The link to AnyLogic is kept at the data level. The script

`anylogic_to_ptrack.py` converts the trajectory CSV written by the AnyLogic logging event to the pipeline's format and renders the virtual-camera video and ground truth with `ptrack.render`, after which the same detection, tracking and evaluation code applies, and the template `alpyne_bridge.py` shows how an AnyLogic model exported for Alpyne [52] can be driven from Python to generate sequences on demand. All numerical results in Chapter 5 were nevertheless produced with the co-simulation, which Chapter 6 discusses as a limitation. The perception layer, by contrast, is not emulated at all: a real YOLOv8 network is fine-tuned on the rendered frames and run on every frame of every video.

The loop is computationally feasible because the optimised parameters act only after the neural network. A sequence or clip c is defined by its scenario, density level and seed. The simulator produces trajectories S_c, the renderer the video $V_c = \mathcal{R}(S_c)$ and the ground truth G_c, and the detector the loosely suppressed detections $\widehat{D}_c = \mathcal{Y}(V_c)$, that is, the candidates with a confidence above 0.05 that survive NMS at an IoU threshold of 0.9, at most 1,000 per frame (Section 4.6). Only the last stages depend on $\boldsymbol{\theta}$:

$$J_c(\boldsymbol{\theta}) = \mathcal{M}\big(\mathcal{B}\big(\mathcal{N}(\widehat{D}_c;\, \theta_{\mathrm{nms}});\, \boldsymbol{\theta}_{\mathrm{trk}}\big),\; G_c\big), \qquad (4.1)$$

where $\mathcal{N}$ is NMS with IoU threshold θ_{nms}, $\mathcal{B}$ is ByteTrack with the six remaining parameters $\boldsymbol{\theta}_{\mathrm{trk}}$, and $\mathcal{M}$ is the metric. A tracker parameter cannot change where a simulated person walks or what the camera sees. Simulation, rendering, encoding and GPU inference are therefore computed once per sequence and re-used by every optimiser. The memoisation of simulation, rendering and inference is exact; the only approximation is the post-hoc NMS, which applies $\mathcal{N}$ to $\widehat{D}_c$ instead of running the detector's own NMS at θ_{nms} (Section 4.6). An evaluation of J_c costs only a look-up of cached detections, ByteTrack and the metric computation on the CPU, so thousands of evaluations can run in parallel. The argument holds equally for AnyLogic-generated pedestrians: the AnyLogic RL experiment of Appendix A has no actions, so each run would be generated once and served to many episodes.

4.2 AnyLogic Pedestrian Model Specification

This section is a model specification with Java code templates (Appendix A); no AnyLogic run supplied data for this thesis, and the experiments use the co-simulation of Section 4.3, which implements this specification.

4.2.1 Agent Types and Flowcharts

The specified AnyLogic model PedTrackSim has two agent types. The top-level agent `Main` holds the space markup, the flowchart, the logging event and the parameters of a run: `scenario` (corridor, crossing, bottleneck or plaza), `arrivalRate` (total arrival rate in pedestrians per second), `seed` (the fixed seed of the experiment), `dynamicDensity` together with `rateLow` and `rateHigh` (the time-varying rate of the dynamic-density runs), `spaceHeightM` (the height of the space, needed to flip AnyLogic's downward screen axis) and `captureFrames`, `captureX` and `captureY` (the optional capture of the 3D window and its screen position). The agent type `Ped` extends the library's pedestrian with the body height `h` and two clothing-colour indices `shirt` and `pants`, which only the camera needs, and with the last logged position `lastX`, `lastY`, from which the logging event computes finite-difference velocities. Pedestrians are placed in a custom population `peds`, over which the logging event iterates.

Flows are built from standard Pedestrian Library blocks (Figure 4.2). `PedSource` generates pedestrians at a given rate on an entrance target line, with one source per entrance receiving a share of the total rate. It assigns a diameter from uniform(0.4, 0.5) m and a comfortable speed from uniform(1.0, 1.6) m/s, and its on-enter action draws the body height from a normal distribution with mean 1.70 m and standard deviation 0.09 m, clipped to [1.50, 1.95] m, together with random clothing indices that affect only the rendering. `PedGoTo` in "reach target" mode sends pedestrians to a line or area, `PedSelectOutput` splits a flow by probabilities, `PedWait` holds pedestrians in an area for a random delay, and `PedSink` removes them at the exit. In the corridor and the crossing pedestrians walk directly to the exit (Figure 4.2a). In the bottleneck a first `PedGoTo` targets the gate line so that every route passes the gate (Figure 4.2b). In the plaza 25 % of the pedestrians are sent to the waiting area in front of the ticket machines and held there for uniform(10, 30) s before walking to their exit (Figure 4.2c).

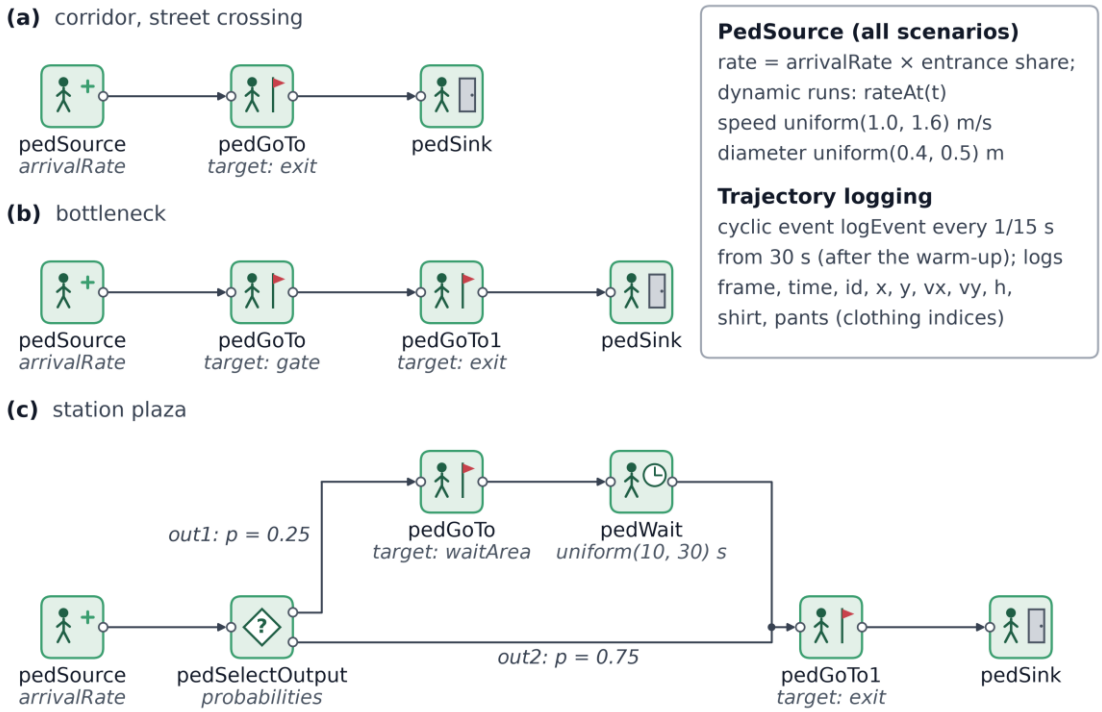


Figure 4.2. Pedestrian Library flowcharts: (a) corridor and street crossing, (b) bottleneck with an intermediate gate target, (c) station plaza with a probabilistic detour to the waiting area; top right: PedSource attributes and the trajectory logging event with its logged columns.

4.2.2 Space Markup, Logging and 3D Camera

Walls model fences, barriers and the outlines of kiosks and corner blocks; target lines model entrances, exits and the bottleneck gate; and a rectangular area models the plaza's waiting zone. The model uses AnyLogic's default scale of 10 pixels per metre, and the logging code converts AnyLogic's downward screen y-axis to the upward metric frame used in this thesis. Trajectories are written by a cyclic timeout event `logEvent` whose first occurrence at 30 s discards the warm-up in which the empty space fills, and which recurs every 1/15 s, matching the camera's 15 FPS. Each occurrence writes one line per pedestrian with frame index, time, identifier, position, finite-difference velocity, body height and clothing indices, and can optionally capture the 3D window to a PNG file for encoding with FFmpeg. The 3D view uses a `Camera` object at the pose of Table 4.1 and a 1280 × 720 pixel 3D window. In the dynamic-density runs the total arrival rate follows the schedule

$$Q(t) = Q_{\text{low}} + \left(Q_{\text{high}} - Q_{\text{low}}\right) s(t - t_w), \qquad s(t')$$

$$= \begin{cases} 0, & t' < 15 \text{ s} \\ (t' - 15 \text{ s})/20 \text{ s}, & 15 \text{ s} \le t' < 35 \text{ s} \\ 1, & 35 \text{ s} \le t' < 60 \text{ s} \\ 1 - (t' - 60 \text{ s})/15 \text{ s}, & 60 \text{ s} \le t' < 75 \text{ s} \\ 0, & t' \ge 75 \text{ s} \end{cases} \qquad (4.2)$$

with model time t, warm-up $t_w = 30$ s, recording time $t' = t - t_w$ and the scenario's low and high rates Q_{low} and Q_{high} (Table 4.1). The rate stays low for 15 s, ramps up over 20 s, stays high for 25 s, ramps down over 15 s and stays low until the end of the 90 s recording, a pattern resembling a short rush-hour pulse. For closed-loop use the specification also contains an AnyLogic RL experiment. It copies `scenario`, `arrivalRate` and `seed` to `Main`, observes the number of written frames and the trajectory file path, and stops when the warm-up plus the recording length has elapsed, after 90 s of model time for a static and 120 s for a dynamic-density sequence. It defines no action, because the agent acts on the perception pipeline, not on the pedestrians. The code of the logging event, rate schedule and frame capture is listed in Appendix A.

4.2.3 Scenarios

The four scenarios cover the movement patterns that most affect detection and tracking: counter-flow, crossing flows, a merging flow with queueing, and origin–destination movement around obstacles. These are also canonical test cases of pedestrian dynamics [1], [124]. Table 4.1 lists their parameters. The corridor is a fenced 40 m × 6 m walkway with a bidirectional flow. The crossing joins two 8 m wide streets bounded by four planted corner blocks, and its four flows meet at right angles in the central square. In the bottleneck a single flow enters along the whole western side of a 24 m × 16 m hall and must pass a 1.8 m gate in a barrier at $x = 14$ m, so a queue forms at higher rates. The plaza is a 32 m × 22 m station concourse with four 4 m openings, two kiosks and a waiting area; each pedestrian leaves through one of the three other openings with equal probability, and a quarter first stop in the waiting area. Three static density levels per scenario are defined by the total arrival rate, and the dynamic variant applies Equation (4.2) between the scenario's low and high rates.

Table 4.1. Geometry, flows, arrival rates and camera poses of the four scenarios (HFOV: horizontal field of view; 1280 × 720 pixels at 15 FPS).

Scenario	Area (m)	Walls and obstacles	Flows	Arrival rate low / medium / high (ped/s)	Camera position → look-at (m)	HFOV
Corridor	40 × 6	Fences at $y = 0, y = 6$	$x = 0 \leftrightarrow x = 40$; two sources, 50 % each	0.8 / 1.6 / 2.6	(13, −6.5, 7.5) → (20, 3, 0)	70°
Crossing	24 × 24	Four 8 × 8 m corner blocks	W ↔ E, S ↔ N; four sources, 25 % each	1.0 / 2.0 / 3.2	(3, 3, 9) → (13, 13, 0)	70°
Bottleneck	24 × 16	Hall walls at $y = 0, y = 16$; barrier at $x = 14$, gate $7.1 \leq y \leq 8.9$	$x = 0 \rightarrow$ gate $\rightarrow x = 24$; one source	0.8 / 1.5 / 2.4	(6, −5, 9) → (13, 8, 0)	70°
Plaza	32 × 22	Perimeter walls with four 4 m openings; kiosks (9.5–12, 5.5–8), (19.5–22, 13.5–16); waiting area (23–28.5, 3–6)	Each opening → one of the other three; 25 % wait U(10, 30) s	0.8 / 1.6 / 2.4	(3, −4, 10) → (16, 11, 0)	72°

Figure 4.3 shows the layouts with sample trajectories. In the corridor the opposing flows use the full width. In the crossing the flows stay within the streets and interpenetrate in the square. In the bottleneck the trajectories converge on the gate and fan out behind it, and a few pedestrians pressed against the barrier slide along it towards the opening. In the plaza nearly straight diagonals connect the openings, bend around the kiosks and form a dense tangle in the waiting area. Each camera, mounted 7.5 m to 10 m high and looking down at roughly 27° to 33°, covers the central part of its scene, where interactions are densest, a typical surveillance geometry in which pedestrians occlude each other along the viewing direction.

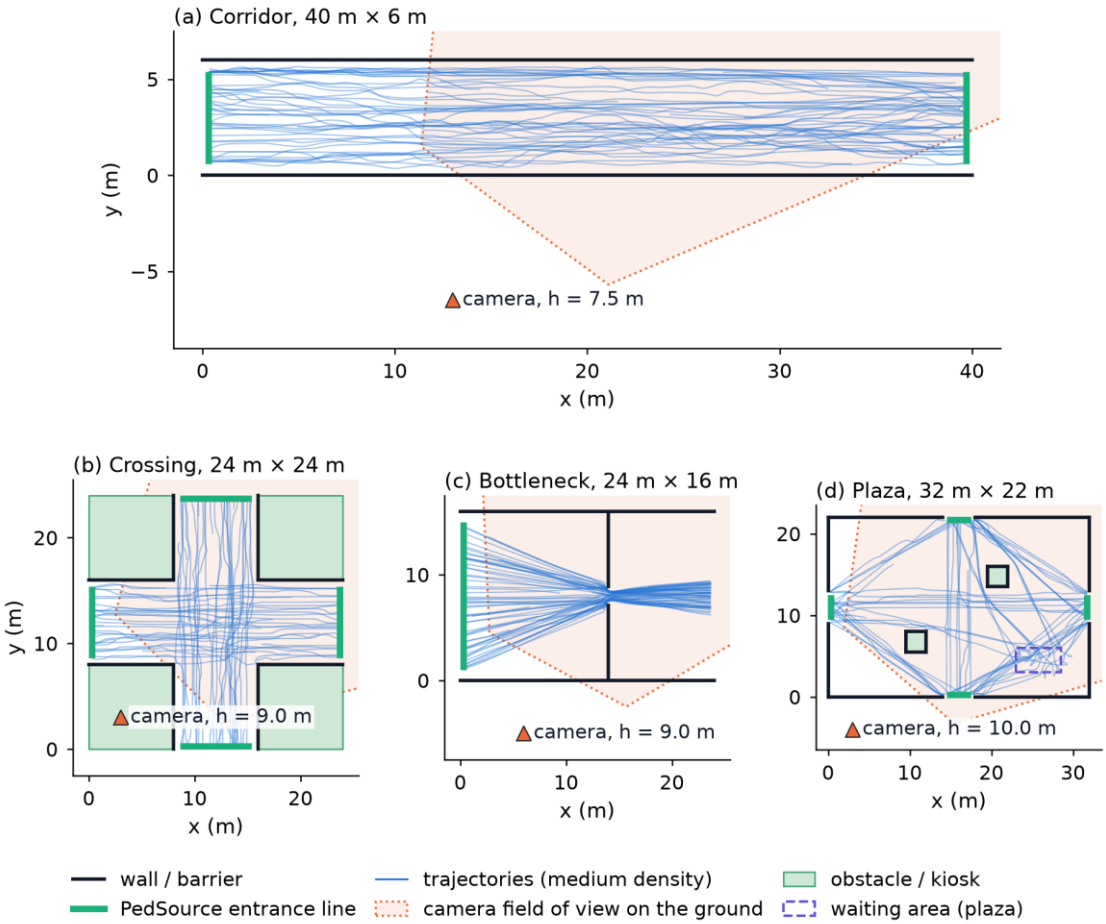


Figure 4.3. Top views of the scenarios: walls and barriers (black), PedSource entrance lines (green), corner blocks and kiosks (green squares), the plaza's waiting area (dashed), sample trajectories at medium density (blue), camera position and height (triangle) and the part of the ground inside the camera's field of view (shaded, dotted outline).

4.3 Python Co-Simulation of the AnyLogic Scenarios

The data of this thesis were not generated by AnyLogic. They were generated by the headless Python co-simulation described in this section, which implements the scenarios, flows, pedestrian attributes and camera poses specified in Section 4.2.

4.3.1 Social-Force Dynamics

The module `ptrack.pedsim` integrates the social force model of Section 3.3: the equation of motion (3.26) with the elliptical specification II of Equations (3.28) to (3.31), the lateral evasion term (3.32), the body contacts with normal damping (3.33) and the wall forces (3.34), with the parameter values of Table 3.1 and all forces per unit mass. The lateral evasion, the contact damping and the bound on backward walking are additions of this thesis to the cited models [3], [9], [47]; the speed cap of 1.3 times the desired speed follows Helbing and Molnár [47]. In the implementation, pairwise terms are evaluated as

dense NumPy arrays, which is efficient for the at most 115 pedestrians present in a scene at once (reached in a high-density corridor sequence of the detector-training split; the peak in the test split is 114). While a pedestrian waits, the desired speed in the driving term of Equation (3.26) is set to zero; the speed cap of Equation (4.3) keeps using the comfortable speed v_i^0. The keep-right rule of the evasion term uses a small tolerance: $\sigma_{ij} = +1$ (step right) unless the sine of the bearing of j relative to $\boldsymbol{e}_i$ is below -0.05, that is, unless j is clearly on the right. The fluctuation $\boldsymbol{\xi}_i$ is drawn independently in every step.

Equation (3.26) is integrated by the semi-implicit Euler scheme of Equation (3.35) with $\Delta t_\mathrm{s} = 1/60$ s, and the constraint operator Π is applied in two stages:

$$\begin{aligned} \boldsymbol{v}' &= \boldsymbol{v}_i^n + \Delta t_\mathrm{s}\, \boldsymbol{a}_i^n, \qquad \boldsymbol{v}'' = \boldsymbol{v}' \min\left(1,\, \frac{1.3\, v_i^0}{\| \boldsymbol{v}' \|}\right), \\ \boldsymbol{v}_i^{n+1} &= \boldsymbol{v}'' + \max(0,\, -v_b - \boldsymbol{v}'' \cdot \boldsymbol{e}_i)\, \boldsymbol{e}_i, \qquad \boldsymbol{x}_i^{n+1} = \boldsymbol{x}_i^n + \Delta t_\mathrm{s}\, \boldsymbol{v}_i^{n+1}. \end{aligned} \tag{4.3}$$

The speed is first capped at 1.3 times the comfortable speed, and the velocity component against $\boldsymbol{e}_i$ is then limited to $v_b = 0.1$ m/s, except while a pedestrian waits. Every fourth step is recorded, giving 15 FPS. For the renderer the simulator also keeps a heading, exponentially smoothed with weight 0.15 per step towards the velocity direction whenever the speed exceeds 0.15 m/s, and a gait phase that advances by one cycle per 0.75 body heights walked.

4.3.2 Arrivals and Navigation

Arrivals reproduce `PedSource`. In every step the number of new arrivals at source s is drawn from a Poisson distribution with mean $Q\, w_s\, \Delta t_\mathrm{s}$, where Q is the total arrival rate and w_s the source's share, and the arrivals are queued. Up to three queued pedestrians per source and step are placed at random points of the entrance line, but only where the new body keeps 0.05 m of clearance to all others, and they start at 80 % of their comfortable speed. As in AnyLogic, the inflow of a crowded scene is thus limited by the free space at the entrance.

Navigation follows the library's target-line semantics. Each pedestrian follows an ordered route of targets and aims at the closest point of the current one (Equation (3.27)). An exit line counts as reached within 0.35 m, where the pedestrian is removed as by `PedSink`. Waypoints count within 1.0 m and the waiting position within 0.4 m, after which the drawn waiting time elapses. An intermediate gate line counts as passed only when actually crossed, detected by a change of the side of the line. In the plaza, routes around the

kiosks are planned on a visibility graph [189] whose nodes are the kiosk corners offset by 0.75 m. Edges are blocked by kiosks enlarged by 0.35 m, and shortest paths are found with Dijkstra's algorithm [190], computed once when the pedestrian enters, for the leg to the waiting point and the leg on to the exit. In the crossing and the bottleneck a simpler rule stands in for AnyLogic's navigation mesh: if a wall blocks the straight line to the aimed point, for instance after the crowd has pushed a pedestrian into a side arm or against the barrier, the pedestrian first heads for a hub point, either the centre of the crossing at (12, 12) or a point just upstream of the gate at (13.2, 8.0).

4.3.3 Model Verification and Calibration

The model need not reproduce a particular site, but it must yield plausible speeds, densities and occlusions and must not produce motion that real crowds do not show, because the tracker's constant-velocity Kalman filter is sensitive to such artefacts. The most instructive verification finding concerned an earlier version with the circular specification, in which repulsion depends only on distance, and without the evasion term. In counter-flow, 28 % to 74 % of its pedestrians "bounced", that is, walked backwards at some point. The mechanism is simple. In a head-on encounter a distance-based repulsion points straight against the desired direction, so it decelerates and finally reverses the pedestrian instead of deflecting it. A tracker evaluated on such data would face motion reversals that real pedestrians rarely make. This finding motivated two changes. The elliptical specification makes repulsion set in earlier for approaching pedestrians, so encounters are anticipated, but it deflects them laterally only in oblique encounters: in an exactly head-on encounter $\boldsymbol{y}_{ij}$ is parallel to $\boldsymbol{d}_{ij}$, so the gradient of Equation (3.30) is parallel to $\boldsymbol{d}_{ij}$ as well and has no lateral component. Head-on encounters are therefore resolved by the evasion term of Equation (3.32). The bound on backward walking in Equation (4.3) complements these measures in the final model.

The interaction parameters were then calibrated on the grid $A \in \{0.8, 1.2, 2.0\}$ m/s^2 × $B \in \{0.3, 0.5, 0.7\}$ m against four criteria. The first is the root-mean-square error (RMSE) between the simulated speed–density relation and Weidmann's fundamental diagram, Equation (3.36) [59]. Weidmann's relation does not distinguish unidirectional from bidirectional flows; it was applied here in the simplest setting, a 40 m × 3 m unidirectional corridor without crossing or counter-flow conflicts, which was chosen so that the interaction parameters are tested without the conflicts and capacity limits of the scenarios. The corridor was fed at eight inflow rates between 0.5 and 5.0 ped/s, and density and mean

speed were measured every 0.25 s in a 10 m section between 30 s and 80 s of simulated time. The other three criteria were measured in runs of the corridor, crossing and bottleneck at high density (60 s after the warm-up, two seeds each). The second criterion is the share of pedestrians who ever walk backwards, meaning a velocity component along their overall trip direction below −0.3 m/s while not waiting. The third is the share of stuck pedestrians, with a 10 s average speed below 0.15 m/s while not waiting. The fourth is the number of pedestrians still upstream of the barrier ($x < 14$ m) at the end of the bottleneck run, a measure of the queue in front of the gate.

The selected combination $A = 2.0$ m/s^2, $B = 0.3$ m has the lowest RMSE of the grid, 0.108 m/s. It lies, however, at a corner of the grid (the largest A and the smallest B), so the grid does not bracket the optimum, and $A = 1.2$ m/s^2, $B = 0.5$ m is effectively tied with an RMSE of 0.109 m/s. The behavioural criteria decide between the two: $A = 1.2, B = 0.5$ leaves 4.3 % of the pedestrians stuck at the bottleneck and 46 pedestrians upstream of the barrier, against 1.8 % and 32 for the selected combination. The selected combination's backward-walking shares are 0.0 % (corridor), 1.2 % (crossing) and 0.8 % (bottleneck), and its stuck shares 0.4 % (crossing) and 1.8 % (bottleneck). The remaining backward events occur where the momentary desired direction legitimately departs from the trip direction, for example during detours via a hub point or when a pedestrian is pushed aside at the gate. Larger ranges B kept pedestrians farther apart, which produced more stuck pedestrians and longer queues at the gate; with $A = 2.0$ and $B = 0.7$, 24.7 % of the pedestrians were stuck at the bottleneck.

In the calibration corridor (Figure 4.4a) all nine combinations, apart from the sparsest measurements, walk somewhat slower than Weidmann's free speed at low density, and all slow down as density increases. The range B separates them most clearly. Long ranges keep pedestrians far apart, which makes them too slow at every density and limits the density reached in the corridor: $A = 2.0, B = 0.7$ never exceeds about 0.9 ped/m^2 and has the largest error (RMSE 0.313 m/s), followed by $A = 1.2, B = 0.7$ (0.200 m/s). Short ranges follow the curve more closely, but with weak repulsion they stay too fast above about 1 ped/m^2 ($A = 0.8, B = 0.3$: 0.161 m/s). The selected combination lies about 0.1 m/s below the curve between 0.25 and 0.75 ped/m^2 and up to about 0.2 m/s above it at the highest densities. Figure 4.4b checks the thesis scenarios themselves. Every second of every training sequence, the density and mean speed of the non-waiting pedestrians were sampled in one measurement area per scenario: 14–26 m × 0.5–5.5 m in the corridor, the

central square (8–16 m × 8–16 m) of the crossing, the area upstream of the gate (9–13.5 m × 4–12 m) in the bottleneck and 4–28 m × 8–14 m in the plaza. Most measurements lie below about 0.6 ped/m^2. Plaza points cluster at very low density near the free speed. Corridor and crossing points reach about 0.6 ped/m^2 at 1.0 to 1.3 m/s, somewhat below Weidmann's curve, and points from the area in front of the bottleneck gate fall clearly below it, down to 0.4 to 0.8 m/s, as density increases. This is expected rather than a calibration error, because counter-flow, crossing flows and the queue upstream of a bottleneck are governed by conflicts and by the gate's capacity, not by the free unidirectional walking in which the relation was applied for calibration [1]. What matters for this thesis is that density, and with it occlusion, increases with the arrival rate in every scenario while the motion remains smooth and forward-directed.

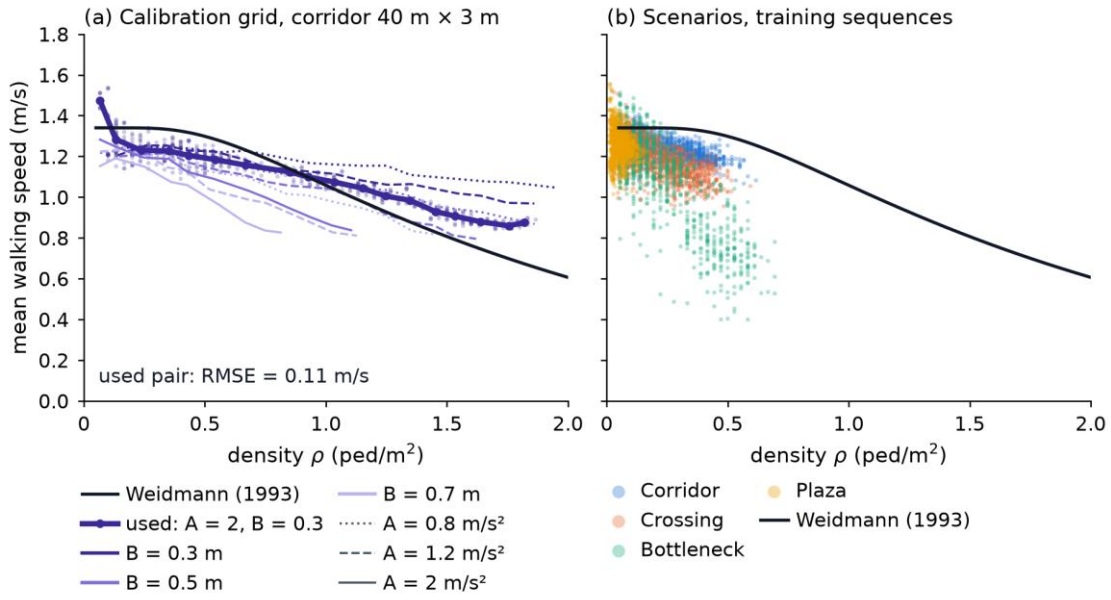


Figure 4.4. Speed–density relations of the co-simulation compared with Weidmann's fundamental diagram (black): (a) calibration runs in the unidirectional 40 m × 3 m corridor for the nine combinations of the calibration grid (selected combination highlighted; lines are density-binned mean speeds, points the raw measurements of the selected combination); (b) measurement areas of the training sequences of the four scenarios (non-waiting pedestrians, sampled every second).

4.4 Virtual Camera and Ground Truth

4.4.1 Camera Model

The camera of `ptrack.render` is an ideal pinhole camera without distortion or roll [191] (Figure 4.5). For the camera centre $\boldsymbol{C}$ and look-at point $\boldsymbol{L}$ of Table 4.1, the optical axis is $\boldsymbol{z}_c = (\boldsymbol{L} - \boldsymbol{C})/\| \boldsymbol{L} - \boldsymbol{C} \|$, the image x-axis is the normalised $\boldsymbol{x}_c = \boldsymbol{z}_c \times \boldsymbol{e}_z$ with the

vertical world axis $\boldsymbol{e}_z$, and $\boldsymbol{y}_c = \boldsymbol{z}_c \times \boldsymbol{x}_c$ points down in the image. A world point $\boldsymbol{P}$ projects to

$$\begin{pmatrix} x_c \\ y_c \\ z_c \end{pmatrix} = \boldsymbol{R}\,(\boldsymbol{P} - \boldsymbol{C}), \qquad \boldsymbol{R} = \begin{pmatrix} \boldsymbol{x}_c^\top \\ \boldsymbol{y}_c^\top \\ \boldsymbol{z}_c^\top \end{pmatrix},$$
$$u = f\,\frac{x_c}{z_c} + c_x, \qquad v = f\,\frac{y_c}{z_c} + c_y, \qquad f = \frac{W/2}{\tan(\mathrm{HFOV}/2)}, \tag{4.4}$$

with the image width $W = 1280$ pixels and the principal point $(c_x, c_y) = (640{,}360)$, the centre of the 1280 × 720 image. A 70° field of view gives $f \approx 914$ pixels and a vertical field of view of about 43°, and the plaza's 72° gives $f \approx 881$ pixels. A 1.70 m pedestrian at 15 m depth is thus about 100 pixels tall.

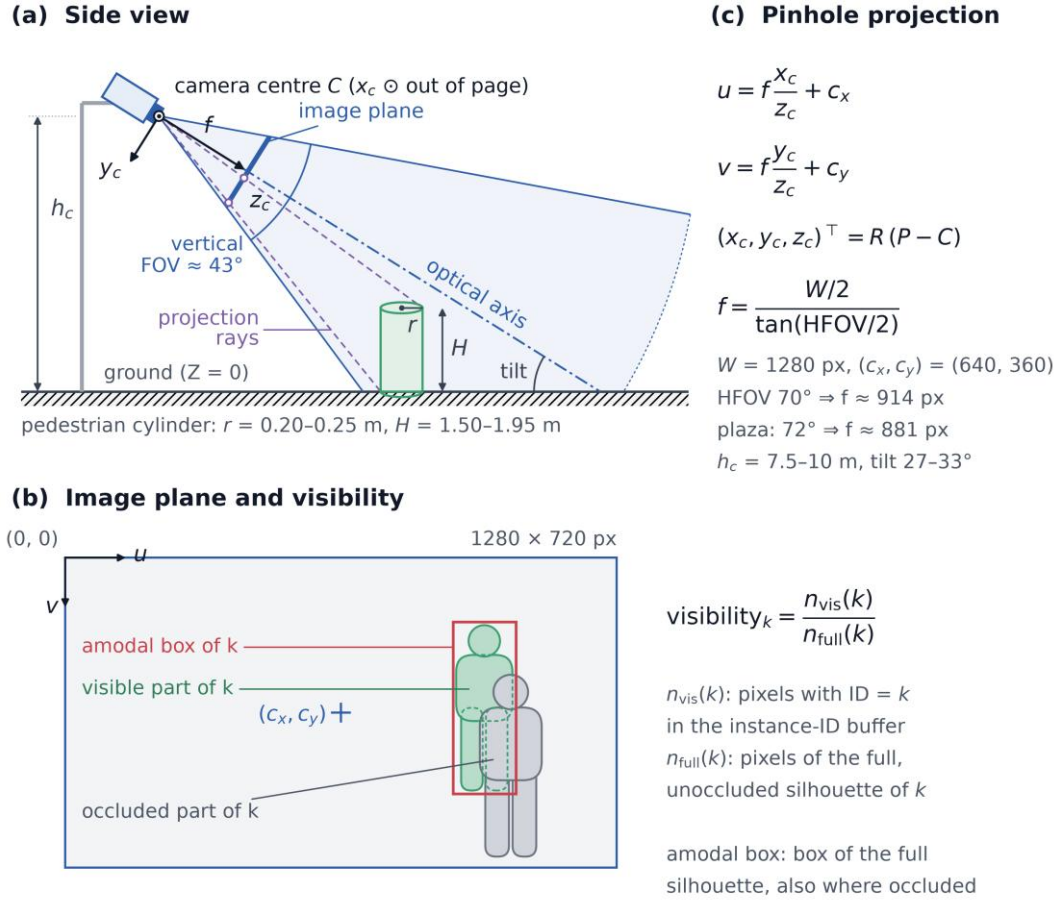


Figure 4.5. Virtual camera geometry: (a) side view with optical axis, image plane, projection rays and a pedestrian drawn schematically as a cylinder of radius r and height H; (b) image plane with amodal box, visible and occluded parts of a pedestrian and the visibility ratio; (c) pinhole projection equations and camera parameters.

4.4.2 Rendering and Sensor Effects

The background is rendered once per sequence by casting a ray through every pixel onto the ground plane. The walkable area carries paving tiles of 0.6 m (1.0 m on the crossing) with grout lines and a pseudo-random brightness per tile, and lawn surrounds it. The scenario's base colour varies per sequence by up to ±8 % per channel. Barriers are drawn as semi-transparent panels 0.9 m high and corner blocks and kiosks as planters 0.5 m high; the crossing has zebra markings and the plaza ticket machines. Pedestrians are articulated three-dimensional figures with 15 joints (feet, knees, hips, pelvis, shoulders, elbows, hands, neck and head) placed at fractions of the body height. The legs swing with a speed-dependent amplitude, the arms swing in opposition, and the gait phase advances with the distance walked, one cycle of two steps corresponding to about 0.75 times the body height, so the feet neither slide nor run in place. The heading is the smoothed velocity direction. After projection, limbs are drawn as thick anti-aliased segments whose width scales with height over depth, the torso as a polygon and the head as a circle with hair, with far-side limbs drawn first and darker. Clothing, skin and hair colours come from palettes, a soft elliptical shadow anchors each figure, and pedestrians are drawn far to near (the painter's algorithm), so nearer figures occlude farther ones.

A clean rendering would make detection unrealistically easy and confidence scores unrealistically well separated, so sensor effects are added. Each frame is blurred with a Gaussian kernel of standard deviation 0.7 pixels and multiplied by a per-sequence illumination gain from U(0.85, 1.12), a per-sequence colour tint from U(0.94, 1.06) per channel and a vignetting factor $1 - 0.18\,(\tilde{u}^2 + \tilde{v}^2)$, where $\tilde{u} = (u - c_x)/c_x$ and $\tilde{v} = (v - c_y)/c_y$ normalise each image axis by its half-size. The factor is therefore 0.82 at the midpoints of the image edges and 0.64 in the corners. Gaussian noise with a standard deviation of 2.5 grey levels is then added; it is taken from a bank of 12 noise fields drawn once per sequence, one of which is picked at random for each frame. This moderate appearance randomisation follows the spirit of domain randomisation [43], [44], although the aim here is a controlled testbed rather than transfer to real video. Finally, the frames are encoded with H.264 [192] (libx264, constant rate factor 23) through FFmpeg, so the detector always sees the block and blur artefacts of compressed surveillance video.

4.4.3 Ground Truth and Ignore Regions

Exact, automatically generated ground truth is a principal motivation for using simulation [37], [39]. While the colour image is drawn, every pedestrian is drawn again

with the same primitives and depth order, without anti-aliasing, into a 16-bit instance-identifier buffer. Each buffer pixel then holds the nearest pedestrian's identifier, and pixel counts give each pedestrian's visible area. Each full silhouette is also drawn alone on a canvas that is not cut at the image border, which gives the tight amodal box and the silhouette area. The visibility ratio and inside fraction of pedestrian k are

$$v_k = \frac{|\{\text{pixels with ID} = k \text{ in the instance buffer}\}|}{|\text{silhouette}_k|}, \qquad \iota_k = \frac{|\text{silhouette}_k \cap \text{image}|}{|\text{silhouette}_k|}. \qquad (4.5)$$

Because the instance buffer covers only the image, v_k reflects both occlusion and truncation at the border. For every pedestrian with at least one silhouette pixel in the image, each frame's ground truth stores the identifier, amodal box, v_k, ι_k, world position and depth.

Almost completely hidden pedestrians cannot reasonably be expected to be detected, yet detecting them should not be penalised either. The evaluation therefore distinguishes evaluated from ignored objects, as MOTChallenge does [34], [35], and adopts TrackEval's pre-processing logic [60]. A ground-truth object is ignored if $v_k < 0.1$ or $\iota_k < 0.5$. Tracker boxes matched to ignored objects by a Hungarian assignment at IoU ≥ 0.5 are removed before scoring, and all other ground-truth objects, including heavily occluded ones, are evaluated. Figure 4.6 shows rendered frames with boxes coloured by visibility. At low density most pedestrians are isolated and fully visible. At high density partial occlusions concentrate where the crowd is dense and far from the camera: at the far end of the corridor, in the queue at the bottleneck gate and around the plaza's waiting area. Ignored objects are mostly pedestrians truncated at the image border: over the training and test sequences, 2.4 % of the ground-truth objects are ignored, and 97 % of these because less than half of the silhouette lies inside the image ($\iota_k < 0.5$).

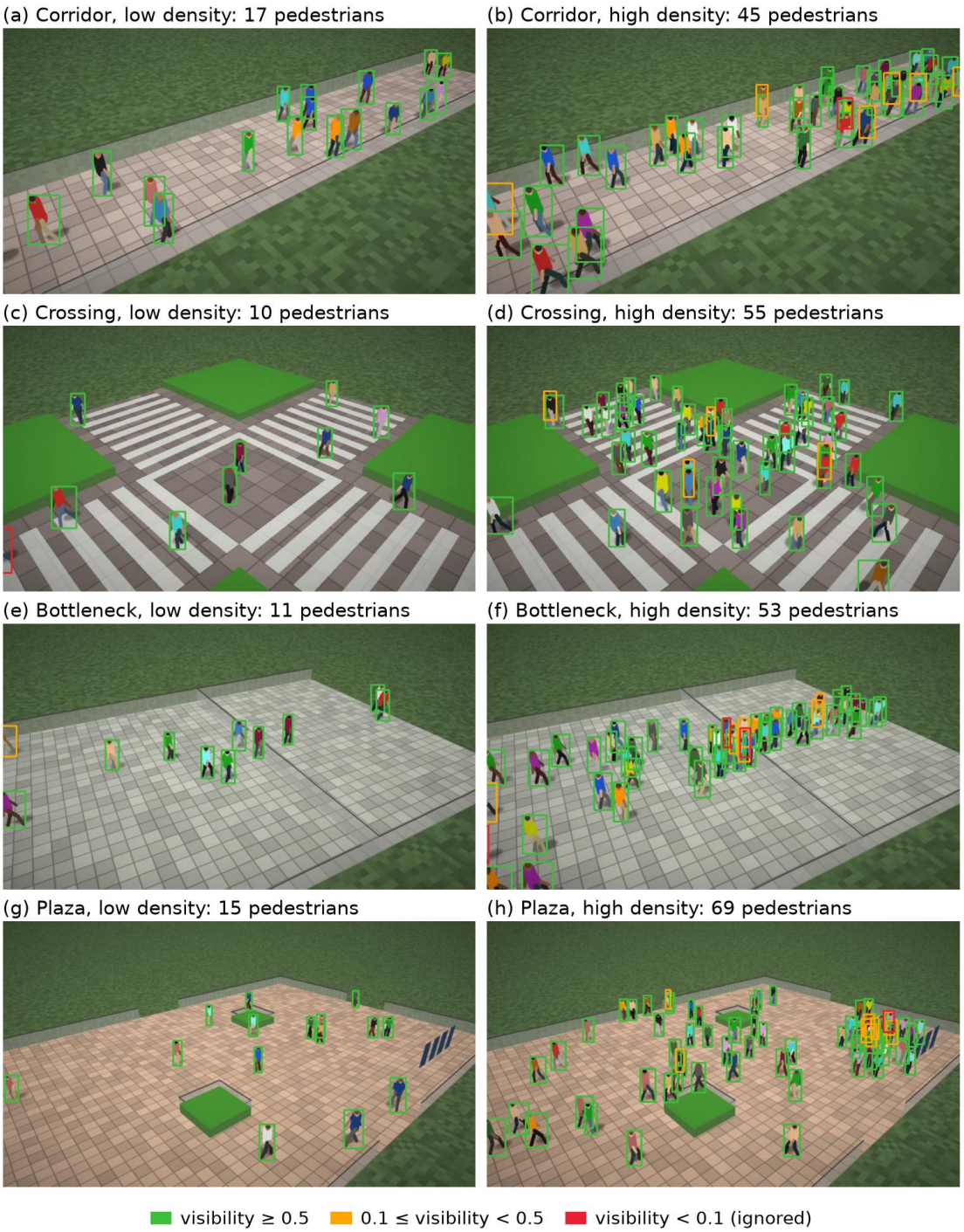


Figure 4.6. Rendered test frames at low (left) and high (right) density with ground-truth amodal boxes coloured by visibility: green $\nu \geq 0.5$, orange $0.1 \leq \nu < 0.5$, red $\nu < 0.1$ (ignored in the evaluation). Pedestrians with less than half of their silhouette inside the image ($\iota < 0.5$) are also ignored but not marked separately.

4.5 Synthetic Dataset

The dataset contains 256 sequences in splits with distinct purposes and disjoint seeds (Table 4.2 and Figure 4.7). The twelve static conditions combine the four scenarios with

three density levels, and each scenario also has a dynamic-density condition. Every sequence starts with a discarded 30 s warm-up and is recorded at 15 FPS, so a 60 s sequence has 900 frames.

Table 4.2. Splits of the synthetic dataset.

Split	Sequences	Duration	Seeds	Total	Purpose
Detector training	6 per static condition	60 s	300–305	72	YOLOv8 training images only (1 frame/s)
Training	6 per static condition	60 s	0–5	72	Tuning: HPO clips, RL training clips, sensitivity analysis
Validation	2 per static condition	30 s	50, 51	24	YOLOv8 validation (every 10th frame)
Test	4 per static condition	60 s	100–103	48	Final evaluation only
Dynamic training	6 per scenario	90 s	210–215	24	Tuning under time-varying density
Dynamic test	4 per scenario	90 s	200–203	16	Final evaluation only

Each split serves exactly one stage, which prevents leakage between stages. The detector is trained only on the detector-training split (4,320 images at one frame per second) and validated on every tenth frame of the validation split (1,080 images); it never sees a training, test or dynamic sequence. All tuning (HPO, RL and sensitivity analysis) uses only the training and dynamic-training splits, and the test splits are used once, for the final evaluation in Chapter 5. Each sequence's simulation seed is derived from its sequence seed (Table 4.2) and its scenario and density indices, so conditions sharing a sequence seed are statistically independent. The renderer's appearance parameters derive from the sequence seed, so test sequences have illumination settings unseen in training, drawn from the same distributions.

For tuning, each 60 s training sequence is cut into three 20 s clips of 300 frames and each 90 s dynamic training sequence into four, giving 216 static and 96 dynamic clips. The per-condition HPO objective uses four fixed clips taken from different sequences and positions to limit their correlation: the first clip of the first training sequence, the second of the second, the third of the third and the first of the fourth (for dynamic conditions, clips one to four of the first four sequences), as drawn at the top right of Figure 4.7. The contextual RL agent trains on all 312 clips, and the adaptive agent uses the 96 full training sequences as episodes.

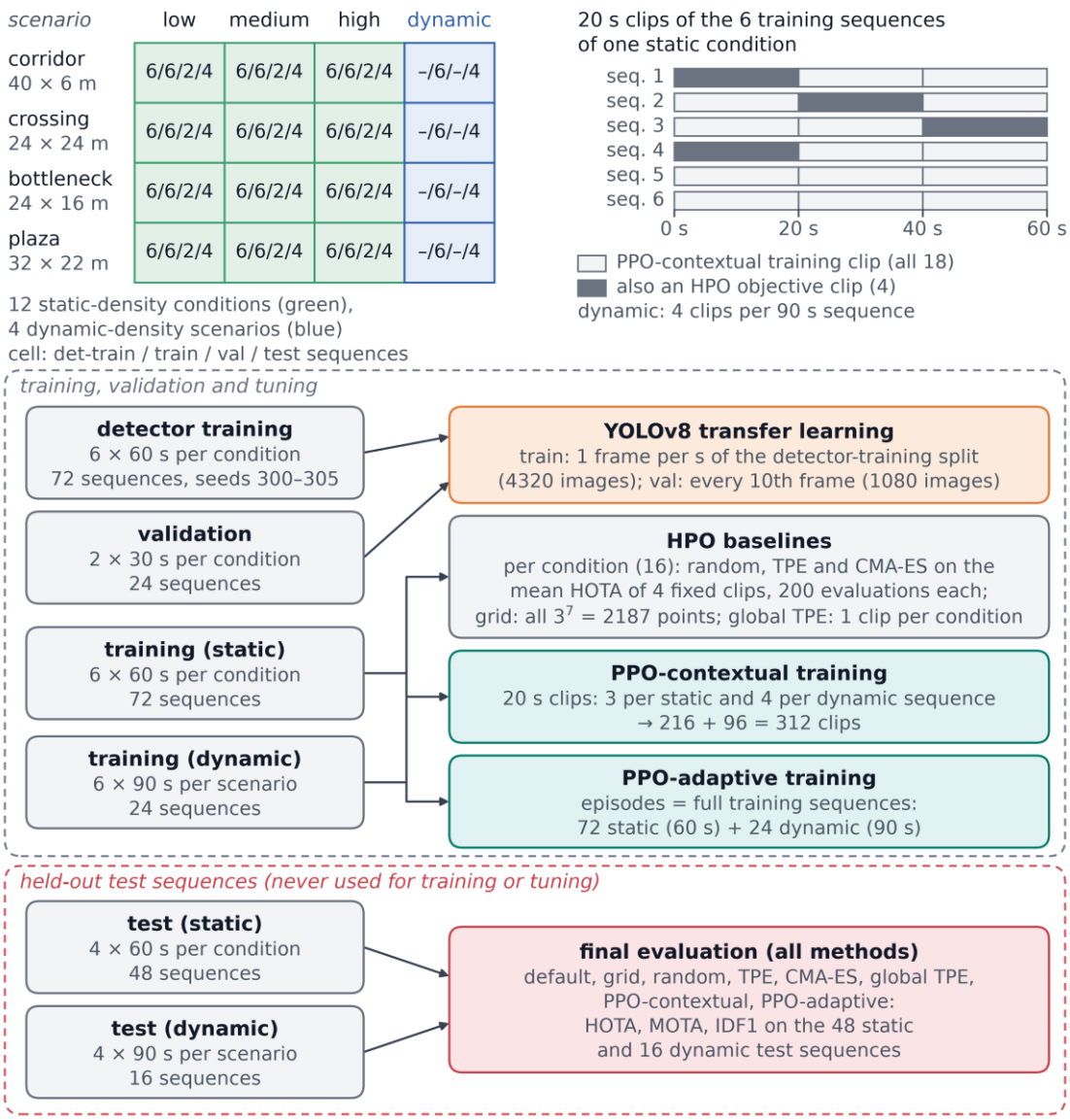


Figure 4.7. Dataset organisation. Top: sequences per condition and split (detector training / training / validation / test) and the 20 s training clips of one static condition, with the four HPO objective clips highlighted. Bottom: use of the splits; the detector-training and validation splits feed only YOLOv8 transfer learning, both training splits feed the HPO baselines (including the global TPE) and both PPO formulations, and the held-out test splits feed only the final evaluation of all methods.

Tables 4.3 and 4.4 summarise the size and crowding of the dataset; they describe the data, not experimental results, and are taken from the dataset statistics file of the code base. Evaluated boxes are ground-truth objects that are not ignored (Section 4.4.3), identities are distinct evaluated pedestrians per sequence summed over sequences, and the per-condition statistics cover the training and test sequences.

Table 4.3. Size of the dataset splits used for tracking (evaluated boxes: visibility ≥ 0.1 and at least half inside the image).

Split	Sequences	Frames	Duration (s)	Evaluated boxes	Identities
Training	72	64,800	4,320	2,240,814	9,797
Validation	24	10,800	720	366,860	1,991
Test	48	43,200	2,880	1,495,737	6,485
Dynamic training	24	32,400	2,160	1,125,504	4,126
Dynamic test	16	21,600	1,440	742,200	2,738

Table 4.4. Crowding statistics per static condition (training and test sequences; visibility over pedestrians at least half inside the image).

Scenario	Density	Mean evaluated pedestrians per frame	Maximum per frame	Mean visibility	Share with visibility < 0.5
Corridor	Low	17.0	29	0.942	2.5 %
Corridor	Medium	37.3	52	0.881	6.5 %
Corridor	High	54.8	75	0.838	10.1 %
Crossing	Low	16.3	30	0.969	0.8 %
Crossing	Medium	33.4	56	0.943	2.2 %
Crossing	High	54.6	76	0.913	3.7 %
Bottleneck	Low	14.6	25	0.949	2.4 %
Bottleneck	Medium	26.8	42	0.902	5.5 %
Bottleneck	High	49.9	69	0.813	14.6 %
Plaza	Low	18.8	33	0.955	2.5 %
Plaza	Medium	35.8	52	0.925	4.4 %
Plaza	High	55.9	82	0.883	8.6 %

Figure 4.8 shows the underlying distributions. In every scenario visibility is dominated by fully visible pedestrians, with a long, flat tail of partial occlusions reaching almost zero. The bottleneck, with its queue in front of the gate, and the corridor, whose elongated geometry lines pedestrians up along the viewing direction, have the heaviest tails (10.0 % and 7.7 % of the pedestrians below a visibility of 0.5), and the crossing has the lightest (2.8 %). The plaza and the crossing have the largest fully visible shares (67 % and 65 %, against 54 % in the bottleneck and 49 % in the corridor). Box heights reflect the camera geometry. Plaza pedestrians are the smallest, because that camera is highest and its area largest; the bottleneck shows a narrow band of medium heights, and the corridor and crossing extend to taller boxes near the camera. The number of evaluated pedestrians per frame grows from low to high density in every scenario, to between 50 and 56 on average at high density, with maxima of 69 to 82 per frame. The scenarios and density levels thus span the conditions under which the best tracker parameters are expected to differ.

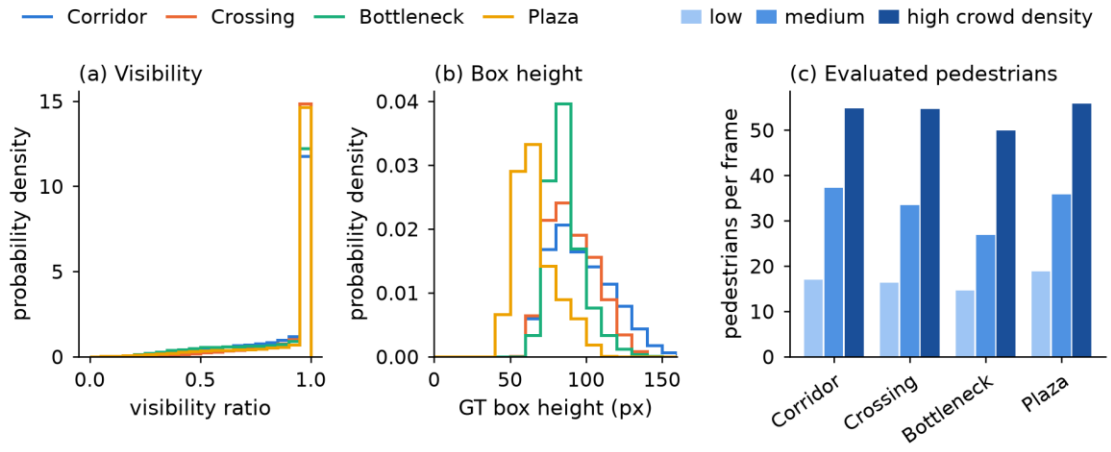


Figure 4.8. Ground-truth statistics of the training and test sequences: (a) visibility ratio per scenario for pedestrians at least half inside the image, (b) height of evaluated boxes per scenario and (c) mean evaluated pedestrians per frame per scenario and density.

4.6 Detector

The detector is an Ultralytics YOLOv8 model [17] in the n, s and m variants, initialised from the official COCO-pretrained weights [16] and fully fine-tuned on the detector-training frames. Transfer learning lets features that already encode generic shapes and persons adapt to the rendered appearance, the camera perspective and the compression artefacts [46]. The task has a single class, “pedestrian”. Labels are the amodal ground-truth boxes clipped to the image, for pedestrians with visibility of at least 0.2 and boxes at least 10 pixels high, so that the detector is not trained to hallucinate pedestrians hidden almost entirely. As a consequence, pedestrians with visibility between 0.1 and 0.2 are evaluated in tracking but are not positive training examples. Amodal boxes are used because the Kalman filter models full-body motion and the evaluation ground truth is amodal. Table 4.5 lists the settings. The input size of 960 pixels instead of the default 640 preserves resolution for small, distant pedestrians, and mosaic augmentation [84] is switched off in the last five epochs so that training ends on undistorted frames. The best checkpoint on the validation split is kept, and the number of epochs is reported in Chapter 5. As a reference for the domain gap, the COCO weights are also evaluated zero-shot with the class “person”. YOLOv8s is the detector of all tuning and RL experiments; the other variants and the zero-shot model appear in the detector study of Chapter 5.

Table 4.5. YOLOv8 transfer-learning and inference settings.

Setting	Transfer learning	Inference for tracking
Models	YOLOv8n/s/m, COCO initialisation, all layers trained	Fine-tuned best checkpoint; zero-shot COCO weights as reference

Setting	Transfer learning	Inference for tracking
Classes	"Pedestrian"	"Pedestrian" (fine-tuned) or COCO "person" (zero-shot)
Data	Detector-training split (1 frame/s); validation split (every 10th frame)	Every frame of the training, test and dynamic sequences
Labels	Amodal boxes clipped to the image, visibility ≥ 0.2, height ≥ 10 px	–
Input size, batch	960 px, 16 (YOLOv8m: 8, limited by GPU memory)	960 px, 16
Optimisation	SGD, initial learning rate 0.01, cosine schedule, seed 0	–
Augmentation	Mosaic (off in the last 5 epochs), Ultralytics defaults otherwise	–
Post-processing	Ultralytics defaults	Confidence ≥ 0.05, NMS IoU 0.9, ≤ 1,000 detections, FP16

Inference is configured to discard as little information as possible, leaving the remaining decisions to the tuned parameters. Every frame is processed with a confidence floor of 0.05, a loose NMS IoU threshold of 0.9 and at most 1,000 detections, and the resulting boxes and scores, the loosely suppressed detections $\widehat{D}_c$ of Equation (4.1), are cached. The floor equals the lower bound of ByteTrack's tunable low-score threshold, so it never removes a detection the tracker could use. Because changing the NMS threshold would normally require running the network again, class-agnostic greedy NMS (Equation (3.12)) is instead applied after inference to the loosely suppressed detections for the eleven thresholds $\eta_j \in \{0.40, 0.45, \ldots, 0.90\}$, and the keep masks are stored. At evaluation time θ_{nms} is mapped to the nearest grid value:

$$D_t(\boldsymbol{\theta}) = \mathrm{NMS}_{\eta_{j^*}}\left(\mathrm{NMS}_{0.9}(\hat{S}_t)\right) \approx \mathrm{NMS}_{\eta_{j^*}}(\hat{S}_t), \qquad j^* = \operatorname*{argmin}_j \left|\eta_j - \theta_{\mathrm{nms}}\right|, \qquad (4.6)$$

where $\hat{S}_t$ are the candidate boxes of frame t above the confidence floor. This post-hoc NMS approximates running the detector's own NMS at η_{j^*}, and the approximation is disclosed. The two sides of Equation (4.6) differ in rare configurations, for example when a box removed at 0.9 by a second box would have survived at the lower threshold because that second box is itself suppressed by a third. Moreover, the cap of 1,000 detections acts after the loose rather than the final NMS, and the threshold is quantised to steps of 0.05. Boxes overlapping by more than 0.9 almost always belong to the same pedestrian, so the differences are expected to be small. In exchange, the NMS threshold can be tuned jointly with the tracker at no extra inference cost.

4.7 Tracker Integration

4.7.1 ByteTrack Implementation and Verification

ByteTrack [21] is used in a NumPy re-implementation of the official `byte_tracker.py` that follows Section 3.2: the constant-velocity Kalman filter of Equations (3.15) to (3.19), the score split of Equation (3.23), the association costs of Equation (3.20) with the limits θ_{m1} and θ_{m2}, the fixed limit of 0.7 for unconfirmed tracks, and the removal of lost tracks after N_{lost} frames, Equation (3.25), so that at 15 FPS the default buffer $n_{\text{buf}} = 30$ retains lost tracks for 15 frames, or 1 s. Duplicates between tracked and lost tracks (IoU distance below 0.15) are resolved in favour of the longer-lived track, and only boxes larger than 10 square pixels with a width-to-height ratio of at most 1.6 are output. The official code solves the assignments with the Jonker–Volgenant solver `lap.lapjv` and a cost limit [105]; the re-implementation obtains the same matching with SciPy's exact linear-assignment solver, a modified Jonker–Volgenant algorithm [186], on the clipped costs of Equation (3.21).

With lightweight substitutes for its compiled dependencies (`lap`, `cython_bbox`), the official tracker and the re-implementation produced identical track identities and boxes in 12 test runs (four sequences, three parameter settings, 300 frames each) when both computed IoU with the same convention. The only documented deviation is the IoU convention. The official code inherits the "+1 pixel" box size of `cython_bbox`, whereas the re-implementation, like Ultralytics [24], uses continuous coordinates; in the verification runs, switching between the two conventions changed HOTA by at most about 1.3 points. Thresholds that the official code hard-codes or derives from the high-score threshold are exposed as parameters, named after Ultralytics' `bytetrack.yaml` where an equivalent exists; `second_match_thresh` and `unconfirmed_match_thresh` have no equivalent there and are introduced here. All parameters can be changed while the tracker runs, as the adaptive controller requires (Section 4.9.2).

4.7.2 Parameter Space

The tuned vector $\boldsymbol{\theta} = (\theta_{\text{nms}}, \tau_{\text{high}}, \tau_{\text{low}}, \tau_{\text{new}}, \theta_{m1}, \theta_{m2}, n_{\text{buf}})$ of Section 3.2.6 has the seven components of Table 4.6. The NMS threshold trades duplicate detections against missed neighbours in crowds. The score thresholds decide which detections enter each association and which may start tracks, the cost limits set how much overlap a match

requires, and the track buffer sets how long an occluded pedestrian's track survives. The tentative-track limit, score fusion and output filters stay fixed.

Table 4.6. Tuned parameter vector θ of the detection–tracking pipeline.

Parameter	Symbol (configuration name)	Range	Default $\boldsymbol{\theta}_0$
NMS IoU threshold	θ_{nms} (nms_iou)	[0.40, 0.90], grid step 0.05	0.70
High-score threshold	τ_{high} (track_high_thresh)	[0.20, 0.80]	0.50
Low-score threshold	τ_{low} (track_low_thresh)	[0.05, 0.40]	0.10
New-track threshold	τ_{new} (new_track_thresh)	[0.20, 0.90]	0.60
First-association cost limit	θ_{m1} (match_thresh)	[0.50, 0.95]	0.80
Second-association cost limit	θ_{m2} (second_match_thresh)	[0.20, 0.90]	0.50
Lost-track buffer	n_{buf} (track_buffer)	[10, 150] frames at 30 FPS (0.33–5 s)	30

The default $\boldsymbol{\theta}_0$, the reference of all rewards and comparisons, is assembled from the published values of Table 2.2. Where the official code and Ultralytics agree, it takes their common values: an NMS threshold of 0.7, a low-score threshold of 0.1, a second-association limit of 0.5 and a buffer of 30. Where they disagree, the first-association limit follows Ultralytics (0.8 rather than the official 0.9); the high-score threshold, published as 0.6 in the paper and the official code and as 0.25 in Ultralytics, takes the intermediate value 0.5; and the new-track threshold, $\tau_{\text{new}} = \tau_{\text{high}} + 0.1 = 0.6$, follows the official rule. Inconsistent combinations are not excluded: if $\tau_{\text{low}} \geq \tau_{\text{high}}$ the second association is simply empty, and tracks start only above $\max(\tau_{\text{high}}, \tau_{\text{new}})$ (Section 3.2.6). All optimisers act in the unit cube:

$$\boldsymbol{\theta} = \boldsymbol{\theta}^{\min} + \frac{\boldsymbol{u}+\mathbf{1}}{2} \odot \left(\boldsymbol{\theta}^{\max} - \boldsymbol{\theta}^{\min}\right), \qquad \boldsymbol{u} \in [-1,1]^7, \qquad (4.7)$$

with the range bounds $\boldsymbol{\theta}^{\min}$, $\boldsymbol{\theta}^{\max}$ of Table 4.6 and the element-wise product $\odot$; θ_{nms} is then snapped to the grid of Section 4.6. The normalisation gives all parameters a common scale. The default does not lie at the centre of the cube (its low-score threshold and buffer are near the lower bounds), so neither the initial PPO policy nor CMA-ES, both centred, starts from the default.

4.8 Evaluation Module

The module `ptrack.metrics` re-implements the CLEAR-MOT [31], Identity [32] and HOTA [33] metrics of TrackEval [60] as defined in Section 3.4, so results are comparable with MOTChallenge (Figure 4.9). After the ignore pre-processing, the IoU between

remaining ground-truth and tracker boxes is computed per frame. CLEAR matches boxes per frame at IoU ≥ 0.5, preferring to continue previous matches, and yields MOTA, MOTP, identity switches, fragmentations and the mostly-tracked and mostly-lost counts. Identity computes a global one-to-one assignment of trajectories and yields IDF1. HOTA averages $\mathrm{HOTA}_\alpha = \sqrt{\mathrm{DetA}_\alpha \, \mathrm{AssA}_\alpha}$ over $\alpha \in \{0.05, \ldots, 0.95\}$ and reports LocA. HOTA is the primary metric, because it balances detection and association [33]; MOTA and IDF1 are reported alongside.

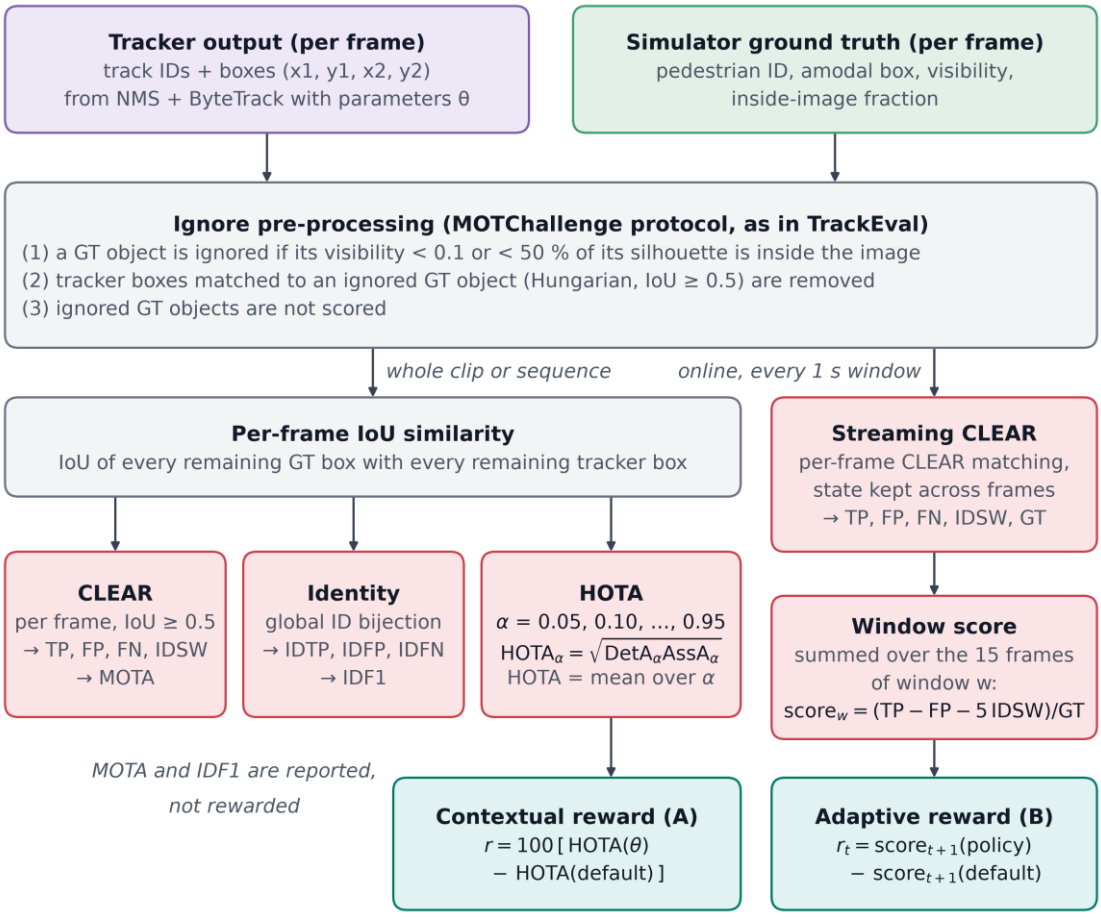


Figure 4.9. Evaluation and reward pipeline: ignore pre-processing, then CLEAR, Identity and HOTA on whole clips or sequences (contextual reward from HOTA), or streaming CLEAR summed over 1 s windows (adaptive reward).

Since every reward and comparison depends on these metrics, the re-implementation was verified against TrackEval on a 1,350-frame sequence tracked from deliberately noisy detections. HOTA, DetA, AssA, LocA, MOTA, identity switches, fragmentations, mostly-tracked and mostly-lost counts and IDF1 were bit-identical, including 1,957 identity switches and 10,837 fragmentations. The independent py-motmetrics package agreed on

MOTA and IDF1 to within 0.2 points but counted 1,834 identity switches, a known definitional difference.

Results over several sequences are combined as in TrackEval's COMBINED row rather than by averaging sequence scores. Counts (true positives, misses, false positives, identity switches, fragmentations, track categories, identity counts and the HOTA counts per α) are summed and the ratios recomputed, and association and localisation accuracies are averaged per α with true-positive weights:

$$\mathrm{AssA}_{\alpha}^{\mathrm{comb}} = \frac{\sum_c \mathrm{TP}_{\alpha,c}\ \mathrm{AssA}_{\alpha,c}}{\sum_c \mathrm{TP}_{\alpha,c}}, \qquad \mathrm{DetA}_{\alpha}^{\mathrm{comb}} = \frac{\sum_c \mathrm{TP}_{\alpha,c}}{\sum_c \left(\mathrm{TP}_{\alpha,c} + \mathrm{FN}_{\alpha,c} + \mathrm{FP}_{\alpha,c}\right)}, \tag{4.8}$$

where c runs over sequences, so crowded sequences weigh more. For the adaptive formulation the module also provides a streaming CLEAR matching that processes one frame at a time while keeping the matching state (the previous assignment and each object's last tracker identity). By construction its windowed counts sum to the batch counts of the sequence; a smoke test in the code base asserts this equality for a full 900-frame sequence tracked from synthetic detections derived from the ground truth.

4.9 Reinforcement-Learning Environment Design

Both formulations are Gymnasium environments [62] solved with the PPO implementation of Stable-Baselines3 [54], [61] (Figure 4.10). Formulation A asks whether a policy can choose a good parameter vector for a scene from a short look at the detector output, a one-shot decision. Formulation B asks whether a policy can adjust the parameters continuously as the scene evolves, a sequential decision problem. The trained agents are referred to as PPO-contextual (formulation A) and PPO-adaptive (formulation B).

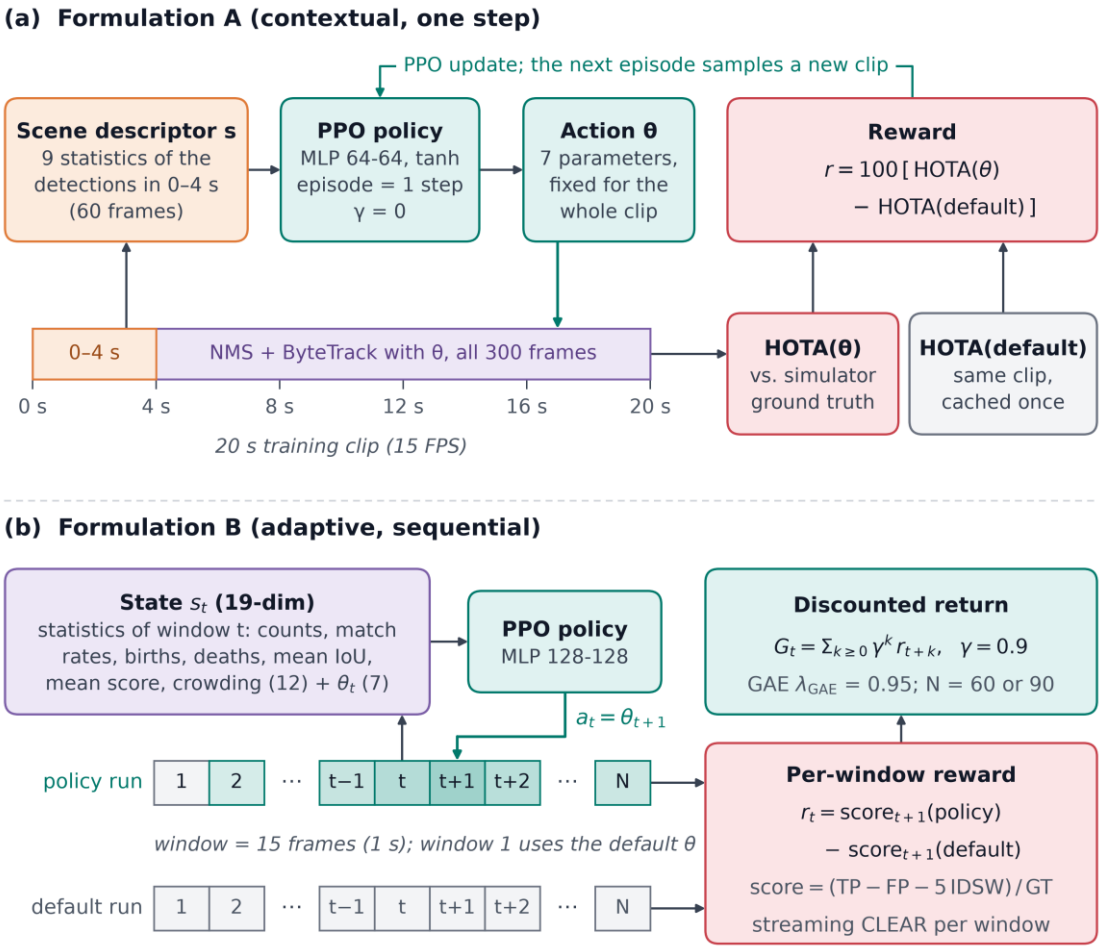


Figure 4.10. RL formulations. (a) Formulation A, contextual: a descriptor of the first 4 s of detections is mapped to one θ for the whole 20 s clip, rewarded by the HOTA gain over the defaults (γ = 0). (b) Formulation B, adaptive: every 1 s the policy observes window statistics and the current θ and sets θ for the next window, rewarded by the windowed score difference to a default run (γ = 0.9).

4.9.1 Formulation A: Contextual Parameter Selection

An episode is one 20 s clip of 300 frames with a single decision. The observation is a nine-dimensional descriptor (Table 4.7) of the detections in the first 4 s (60 frames), after default NMS at 0.7 and a score floor of 0.1. Its features, scaled to roughly the unit interval, describe how many pedestrians are detected, how confident and how large the detections are, how crowded the scene is and how stable the detections are over time. It uses no ground truth, so the policy could in principle run on unlabelled video.

Table 4.7. Scene descriptor of formulation A.

No.	Feature	Definition (over the 60 observation frames)
1	Detection count	Mean detections per frame / 100
2	High-score fraction	Fraction of detections with score ≥ 0.5
3	Mean score	Mean detection score
4	Score spread	2 × standard deviation of the scores

No.	Feature	Definition (over the 60 observation frames)
5	Box size	4 × mean box height / image height
6	Crowding	2 × mean IoU of each detection with its most-overlapping neighbour
7	Overlap fraction	2 × fraction of detections overlapping another with IoU > 0.3
8	Temporal stability	Mean IoU of each detection with its best match in the previous frame
9	Count variation	Coefficient of variation of the per-frame count

The action $\boldsymbol{u} \in [-1,1]^7$ is mapped by Equation (4.7) and kept fixed for the whole clip, including the observed first 4 s. The reward is the HOTA gain over the defaults on the same clip, in HOTA points:

$$r = 100\,[\mathrm{HOTA}_c(\boldsymbol{\theta}) - \mathrm{HOTA}_c(\boldsymbol{\theta}_0)]. \qquad (4.9)$$

The default score is cached per clip. Because it does not depend on the action, it leaves the optimal policy of Equation (3.50) unchanged while removing much of the reward variation due to clip difficulty, the role of a baseline in policy gradients [172]. Since the action affects no later state, the problem is a contextual bandit [57]. PPO is used with $\gamma = 0$ and a generalised advantage estimation (GAE) parameter $\lambda_{\mathrm{GAE}} = 1$ [173], for which the advantage of Equation (3.53) reduces to $\hat{A} = r - V_{\boldsymbol{\psi}}(\boldsymbol{s})$, so the critic learns the expected gain of the current policy in that context and acts as a context-dependent baseline. Actor and critic are multilayer perceptrons (MLP) with two hidden layers of 64 tanh units, and the Gaussian policy has a state-independent log standard deviation initialised at −0.7, a standard deviation of about 0.5 in the cube; sampled actions are clipped to the cube. Each of the 24 parallel environments draws clips from its own random permutation of all 312 training clips. With the budget of Section 4.9.3 an environment makes about 1.7 passes through its permutation, so every clip is seen approximately equally often and conditions are mixed within each rollout. At deployment the deterministic mean action is used. On a test sequence the descriptor is computed from the first 4 s of the whole sequence and the selected $\boldsymbol{\theta}$ is applied to the whole sequence, so a practical system would buffer the first 4 s of video.

This test-time rule is a limitation of the design for the dynamic test sequences. Their first 15 s always run at the low arrival rate (Equation (4.2)), so in these sequences $\boldsymbol{\theta}$ is always chosen from a low-density view and then kept through the high-density pulse. Formulation A cannot react to the change of density by construction; reacting to it is the purpose of formulation B.

4.9.2 Formulation B: Adaptive Online Control

Here the sequence is divided into 1 s windows of 15 frames. The first window uses $\boldsymbol{\theta}_0$, and at the end of each window the agent chooses $\boldsymbol{\theta}$ for the next. The parameters change on the running tracker without a reset: tracks, Kalman states and identity counters persist, and only the thresholds, the lost-track retention and the NMS mask change, so a decision affects the tracks carried into later windows. The 19-dimensional observation consists of 12 window statistics (Table 4.8) and the current $\boldsymbol{\theta}$ in unit coordinates. The current parameters are included because the statistics depend on them; the number of high-score detections, for example, reflects both the scene and τ_{high}.

Table 4.8. Observation of formulation B.

No.	Statistic	Definition (mean over the 15 frames of the window)
1	Detections	Detections after NMS at the current threshold / 100
2	High-score detections	Number of detections with $s > \tau_{\text{high}}$, divided by 100
3	Low-score detections	Number of detections with $\tau_{\text{low}} < s < \tau_{\text{high}}$, divided by 100
4	First-association match rate	First-association matches / high-score detections
5	Second-association match rate	Second-association matches / low-score detections
6	Births	New tracks per frame
7	Lost tracks	Lost-track pool size / 50
8	Tracked tracks	Confirmed tracked tracks / 100
9	Deaths	Removed tracks per frame
10	Match quality	Mean IoU of first-association matches
11	Mean score	Mean detection score
12	Crowding	2 × mean IoU of each detection with its most-overlapping neighbour
13–19	Current parameters	$\boldsymbol{u}$ of the current $\boldsymbol{\theta}$

HOTA cannot be the per-window reward, because its association accuracy rests on a global alignment of whole trajectories and does not decompose over time. The reward is therefore built from CLEAR counts, which are additive over frames. Using the streaming matching, each window's counts give a score, and the reward is the policy run's score minus that of a default-parameter run of the same sequence:

$$\text{score}_t = \frac{\text{TP}_t - \text{FP}_t - 5\,\text{IDSW}_t}{\max(1, \text{GT}_t)}, \qquad r_t = \text{score}_{t+1}(\pi) - \text{score}_{t+1}(\boldsymbol{\theta}_0), \qquad (4.10)$$

with counts summed over window t and GT_t the number of evaluated ground-truth objects; the decision taken at the end of window t sets $\boldsymbol{\theta}$ for window $t+1$ and is rewarded

with that window's score difference. Since $\mathrm{GT}_t = \mathrm{TP}_t + \mathrm{FN}_t$, the score is a windowed MOTA in which identity switches weigh five times (Section 3.4.4). Switches are rarer than false positives or misses but damage trajectories more lastingly, and the weight steers the reward towards the association quality that HOTA and IDF1 reward. The default run's window scores are pre-computed once per sequence and do not depend on the actions, so subtracting them does not change the optimal policy. The caveat lies elsewhere: once the policy run departs from the default run, the two carry different tracks, and r_t measures the accumulated difference between the two runs in window $t + 1$, which reflects all earlier decisions and not only the latest one; crediting the individual decisions is left to the discounted return. An episode is a whole training sequence of 60 or 90 windows, and thus 59 or 89 decisions, drawn in random order from the 96 training sequences. The discount $\gamma = 0.9$ gives an effective horizon of about ten windows, matching the few seconds over which a parameter change influences the tracks. Actor and critic are MLPs with two hidden layers of 128 tanh units, the log standard deviation starts at −1.0 (a standard deviation of about 0.37), and the deterministic mean action is used at deployment.

4.9.3 PPO Hyper-Parameters and Budget Matching

Table 4.9 lists the PPO settings. Both formulations use the clipped surrogate objective (clipping range 0.2), Adam with a learning rate of 3×10^{-4}, ten epochs per update, advantage normalisation per minibatch and no entropy bonus. Training uses 24 parallel environment processes. A rollout of formulation A holds 24 × 16 = 384 clip evaluations in three minibatches of 128, and one of formulation B holds 24 × 128 = 3,072 decisions in six minibatches of 512. Because the environment step dominates the cost, the small networks are trained on the CPU. Each formulation is trained with three seeds.

Table 4.9. PPO hyper-parameters (Stable-Baselines3).

Hyper-parameter	Formulation A (contextual)	Formulation B (adaptive)
Episode	One 20 s clip, one decision	One training sequence, a decision every 1 s
Observation / action dimension	9 / 7	19 / 7
Actor and critic	MLP 64-64, tanh	MLP 128-128, tanh
Initial log standard deviation	−0.7	−1.0
Environments × steps per rollout	24 × 16 = 384	24 × 128 = 3,072
Minibatch size, epochs	128, 10	512, 10
Learning rate, clipping range	3×10^{-4}, 0.2	3×10^{-4}, 0.2
Discount γ, GAE λ_{GAE}	0, 1	0.9, 0.95
Entropy / value coefficient, gradient clip	0 / 0.5, 0.5	0 / 0.5, 0.5
Training budget per seed	12,800 clip evaluations, budget-matched (13,056 executed);	300,000 decisions (98 rollouts, 301,056 executed); not budget-

Hyper-parameter	Formulation A (contextual)	Formulation B (adaptive)
	extended run 51,200 (51,456 executed)	matched
Seeds, deployment	3, deterministic mean action	3, deterministic mean action

The budget of formulation A is matched to the baselines in clip evaluations, the dominant cost of every method. One seed of a per-condition HPO method over all 16 conditions makes 200 objective evaluations of four clips each per condition:

$$N_{\text{clip}} = 16 \times 200 \times 4 = 12{,}800, \qquad (4.11)$$

and formulation A is trained for 12,800 environment steps, each one clip evaluation. Stable-Baselines3 completes its last rollout, so the executed count is rounded up to a multiple of 384, namely 13,056 steps (34 rollouts), 2 % above the nominal budget. The 312 default-parameter evaluations behind the reward baseline of Equation (4.9) are not counted; they would add about 2.4 %. An extended run with four times the budget, 51,200 nominal and 51,456 executed steps (134 rollouts), shows how far the policy improves with more experience. Formulation B is not budget-matched, because no baseline solves the same sequential problem; it is trained for 300,000 environment steps (decisions) per seed, which Stable-Baselines3 completes to 98 rollouts of 3,072, or 301,056 steps, and its learning curves are reported in Chapter 5. At test time the trained actors are evaluated as small NumPy forward passes in the same worker processes as the baselines.

4.10 Baseline Optimisers and Fairness

The baselines are the classical HPO methods of Chapter 2, applied per condition, complemented by a global TPE that tunes one parameter vector for all conditions. For each of the 16 conditions q the objective is f_q of Equation (3.59), the mean HOTA over the four fixed clips of Section 4.5, and each seed may evaluate it 200 times per condition (800 clip evaluations). Random search samples uniformly in the unit cube [28]. TPE [27] is Optuna's sampler with 20 random start-up trials and the multivariate option. CMA-ES [29] is Optuna's sampler based on the `cmaes` library, started at the centre with a step size of 0.2 in Optuna's internal [0, 1] coordinates, that is, 0.4 in $\boldsymbol{u}$; both use Optuna's ask-and-tell interface [30]. Each method runs with three seeds per condition and returns the best $\boldsymbol{\theta}$ evaluated. A three-level grid (lower bound, centre, upper bound) evaluates all $3^7 = 2{,}187$ points per condition, or 8,748 clip evaluations, as a high-budget reference that is not budget-matched. A global TPE, also with three seeds, searches one $\boldsymbol{\theta}$ for all conditions; its objective is the mean HOTA over the first HPO clip of each of the 16 conditions, so its 200

evaluations cost 3,200 clip evaluations per seed. It represents the common practice of tuning one configuration for a whole deployment. Table 4.10 summarises the methods.

Table 4.10. Optimisation methods, budgets and test-time selection rules.

Method	Search	Budget per seed	Seeds	Selection at test time
Default	–	–	–	$\boldsymbol{\theta}_0$
Grid search	$3^7 = 2{,}187$ points	8,748 clips per condition	1	Oracle condition label
Random search	Uniform in $[-1,1]^7$	800 clips per condition	3	Oracle condition label
TPE	Optuna, 20 start-up trials, multivariate	800 clips per condition	3	Oracle condition label
CMA-ES	Optuna/cmaes, centre start, step 0.4 in $\boldsymbol{u}$	800 clips per condition	3	Oracle condition label
Global TPE	As TPE, first HPO clip of each of the 16 conditions	3,200 clips (200 × 16)	3	One $\boldsymbol{\theta}$ for all sequences
PPO-contextual (formulation A)	Policy, reward of Equation (4.9), 312 clips	12,800 clips (extended 51,200; executed 13,056 and 51,456)	3	Scene descriptor of the first 4 s
PPO-adaptive (formulation B)	Policy, reward of Equation (4.10), 96 sequences	300,000 decisions (not budget-matched)	3	Online window statistics

The comparison is fair in the following sense. All methods use the same cached YOLOv8s detections, NMS masks, tracker, evaluation code, parameter ranges and normalisation, tune only on the training splits, are evaluated on the same unseen test sequences, and have budgets expressed in clip evaluations where comparable; the grid search and formulation B are not budget-matched. In one respect the comparison deliberately favours the per-condition baselines. At test time they select $\boldsymbol{\theta}$ with the oracle condition label, the true scenario and density (or "dynamic") of the test sequence, whereas the RL policies must infer the conditions from detector and tracker output. In another respect it disadvantages them, because their objective rests on only four clips per condition, which their best parameters may partly overfit, while formulation A sees 312 clips but must serve all conditions with one policy.

Results are reported as the mean and standard deviation over the three seeds of the combined test metrics (Section 4.8); the deterministic default and the single-seed grid search are reported as single values. Whether improvements exceed the sequence-to-sequence variation is tested per sequence. For each method and test sequence, the sequence's HOTA is averaged over the method's seeds, and the HOTA of the deterministic default run is subtracted. Paired two-sided Wilcoxon signed-rank tests [63] on these

differences are run separately for the static test split ($n = 48$ pairs) and the dynamic test split ($n = 16$ pairs). Within each split, the p-values of the seven methods compared with the default are adjusted with Holm's step-down procedure [64], and a difference is called significant at a family-wise level $\alpha = 0.05$. To answer whether the PPO agents match the baselines, each PPO agent is also compared with each baseline, and the adaptive agent with the contextual one, by the same paired test applied to the differences of their per-sequence gains, with Holm's procedure applied over all of these pairs within a split. In addition, a sensitivity analysis on the training split varies each parameter over its range with the others fixed at $\boldsymbol{\theta}_0$ and maps response surfaces for selected parameter pairs, characterising the landscape the optimisers face (Chapter 5).

4.11 Implementation and Computational Setup

The Python package `ptrack` mirrors the layers of Figure 4.1 (modules `pedsim`, `render`, `bytetrack`, `metrics`, `pipeline`, `rl_env` and `data`). One script per experiment drives the stages, and a test directory holds the cross-checks of the metrics against TrackEval and py-motmetrics and the smoke test of the RL environments. Table 4.11 lists the hardware and software.

Table 4.11. Hardware and software.

Component	Specification
CPU, memory	Intel Core i9-14900KF (24 cores / 32 threads), 64 GB RAM
GPU	NVIDIA GeForce RTX 3060, 12 GB
Operating system	Windows 11 Pro
Language and numerics	Python 3.14.7, NumPy 2.5.3, SciPy 1.18.1
Deep learning and detection	PyTorch 2.14.1 (CUDA 13.0), Ultralytics 8.4.184
Reinforcement learning	Stable-Baselines3 2.9.0, Gymnasium 1.4.0
Hyper-parameter optimisation	Optuna 5.0.0 (TPE; CMA-ES via `cmaes` 0.13.1)
Image and video processing	OpenCV 5.0.0, FFmpeg 9.0.2 (libx264)
Metric verification	TrackEval 1.0.dev1, py-motmetrics 1.4.0
Simulation modelling toolbox	AnyLogic 8 with Pedestrian Library (model specification only, Appendix A; not used to generate the experimental data)

The workload follows the memoisation structure of Section 4.1. Data generation runs one sequence per process (simulation, rendering, ground truth and encoding) in a pool of 28 processes, longest sequences first, with numerical libraries limited to one thread per process to avoid oversubscription. Detection runs once per model and sequence on the GPU in half precision and batches of 16, storing detections, frame indices and NMS masks in one compressed file per model and sequence. Everything else runs on the CPU. Independent clip evaluations (grid search, sensitivity analysis, final evaluation) are

distributed over 28 worker processes. TPE and CMA-ES are sequential, so different methods, conditions and seeds run in parallel, one per worker. PPO uses 24 environment processes and two threads for the network update. Workers cache loaded detections and ground truth, so repeated evaluations of a clip cost only tracking and scoring. All seeds are fixed, and all results are written to JSON files from which the figures and tables of Chapter 5 are generated.

4.12 Chapter Summary

This chapter has described the system used to study RL-based parameter optimisation of a pedestrian tracking pipeline. Four scenarios (corridor, street crossing, bottleneck and station plaza) are specified as AnyLogic Pedestrian Library models with code templates, and the data were generated by a headless Python co-simulation with the same geometry, flows, pedestrian attributes and camera poses. Its social-force dynamics were verified, with the elliptical specification and the lateral evasion term, complemented by the bound on backward walking of Equation (4.3), reducing backward walking from 28–74 % of the pedestrians in counter-flow to at most 1.2 %, and calibrated against Weidmann's fundamental diagram. A virtual pinhole camera renders articulated pedestrians with sensor effects and H.264 compression, and an instance-identifier buffer yields exact amodal boxes, visibility ratios and a MOTChallenge-style ignore protocol. The 256 sequences are divided into disjoint splits for detector training, tuning and testing. A fine-tuned YOLOv8 runs once over all sequences, and post-hoc NMS masks, the only approximation of the memoisation, make the NMS threshold tunable together with six ByteTrack parameters. The tracker reproduces the official ByteTrack up to the documented IoU convention, the evaluation matches TrackEval exactly, and since the parameters cannot influence the pedestrians, all expensive stages are memoised. A contextual and an adaptive RL formulation are compared with grid search, random search, TPE, CMA-ES and a global TPE, with the budget of the contextual agent matched to the baselines and a protocol that even gives the per-condition baselines the oracle condition label. Chapter 5 reports the results.

Chapter 5 Experiments and Results

This chapter reports the experiments carried out with the system of Chapter 4. All data were generated by the Python co-simulation of the AnyLogic-specified scenarios (Section 4.3), rendered by the virtual camera (Section 4.4) and processed by the real, fine-tuned YOLOv8 detector, the ByteTrack re-implementation and the evaluation module of Sections 4.6 to 4.8. The chapter follows the research questions of Section 1.3. Section 5.1 summarises the protocol, Section 5.2 evaluates the transfer learning of the detector (RQ1), and Section 5.3 characterises how the tracking accuracy depends on the seven parameters and on the scenario (RQ2). Section 5.4 reports the optimisation runs on the training objective, Section 5.5 the results of all methods on the held-out test splits (RQ3 and RQ4), and Section 5.6 analyses the learned policies. Sections 5.7 to 5.9 examine the influence of the detector, the remaining errors and the computational cost, and Section 5.10 summarises the findings. Interpretation beyond what is needed to read the results is left to Chapter 6.

Three conventions hold throughout. HOTA, DetA, AssA, MOTA and IDF1 are given in per cent, and differences between them in percentage points ("points"). Results of stochastic methods are reported as the mean ± standard deviation over three seeds of the combined metrics of Section 4.8, whereas the deterministic default and the single-seed grid search are single values. The test splits were used only for evaluation: the detector evaluation of Section 5.2 and the final tracking evaluation of Section 5.5 with the analyses derived from it. No method, budget or parameter was chosen on them.

5.1 Experimental Setup and Protocol

The experiments ran on the workstation of Table 4.11 (Intel Core i9-14900KF, 32 threads, 64 GB; NVIDIA GeForce RTX 3060, 12 GB; Windows 11 Pro). Detector training and inference used the GPU; tracking, evaluation, the HPO baselines and the PPO training ran on the CPU, distributed over worker processes as described in Section 4.11.

Each stage used its own split of the dataset (Table 4.2). The YOLOv8 models were fine-tuned on the detector-training split and validated on the validation split. The sensitivity analysis, the HPO baselines and both PPO formulations used only the training and dynamic-training splits: the per-condition baselines optimise the mean HOTA over four fixed 20 s clips per condition, global TPE the mean over the first HPO clip of each of the 16 conditions, PPO-contextual all 312 training clips and PPO-adaptive the 96 full

training sequences (Section 4.5). The final evaluation used the static-density test split (48 sequences of 60 s, four per static condition) and the dynamic-density test split (16 sequences of 90 s, four per scenario). Unless stated otherwise, all tracking experiments use the cached detections of the fine-tuned YOLOv8s.

Eight methods are compared (Table 4.10). Default applies the parameter vector $\boldsymbol{\theta}_0$ of Table 4.6. Grid search evaluates the $3^7 = 2{,}187$ points of a three-level grid per condition as a high-budget reference with one seed. Random search, TPE and CMA-ES each have 200 objective evaluations, or 800 clip evaluations, per condition and seed. TPE (global) searches one $\boldsymbol{\theta}$ for all conditions with 200 evaluations of 16 clips each. Grid search and the three per-condition optimisers select their parameters at test time with the oracle condition label of the test sequence. PPO-contextual (formulation A, Section 4.9.1) is trained for a budget of 12,800 clip evaluations per seed, the budget of one seed of a per-condition optimiser over all 16 conditions (Equation (4.11)); 13,056 steps were executed. PPO-contextual with four times the budget (51,200 nominal, 51,456 executed steps) shows the effect of more experience. PPO-adaptive (formulation B, Section 4.9.2) is trained for 300,000 decisions per seed (301,056 executed) and is not budget-matched. Both PPO agents are deployed with the deterministic mean action and choose their parameters from detector and tracker output only.

Each test sequence was tracked with the parameters chosen by each method and seed, and the metrics were combined over the sequences of a split as in TrackEval's COMBINED row (Equation (4.8)). Significance was assessed per sequence (Section 4.10): for each method the sequence HOTA was averaged over the seeds, the default HOTA was subtracted, and two-sided Wilcoxon signed-rank tests were run on these differences separately for the static ($n = 48$) and dynamic ($n = 16$) test splits, with Holm's correction over the compared methods within a split at a family-wise level of 0.05. Each PPO agent was also compared with each baseline, and PPO-adaptive with PPO-contextual, by the same paired test, with Holm's correction over all these pairs within a split.

5.2 Transfer Learning of the YOLOv8 Detector (RQ1)

RQ1 asks how far transfer learning on automatically labelled simulated pedestrians closes the domain gap of a COCO-pretrained YOLOv8. This section answers the detection part of the question on the validation split, on the unseen test split, and by relating the detector's confidence to pedestrian visibility, which determines how detections are

distributed over ByteTrack's two association stages. The effect of the detector on tracking accuracy is reported in Section 5.7.

5.2.1 Training Behaviour

YOLOv8n, YOLOv8s and YOLOv8m were fine-tuned for 25 epochs each with the settings of Table 4.5. Figure 5.1 shows the training curves. The box loss falls steadily for all three models, with a visible step after epoch 20, when mosaic augmentation is switched off. The validation loss stays below the training loss throughout. The validation mAP@0.5 saturates within a few epochs, whereas mAP@0.5:0.95, which also rewards precise localisation, is still rising slowly at epoch 25. Larger models reach lower losses and higher mAP@0.5:0.95 at every epoch. The best checkpoint was that of the last epoch for all three models (training-log mAP@0.5:0.95 of 0.9081, 0.9328 and 0.9455 for YOLOv8n, YOLOv8s and YOLOv8m), so a longer schedule might have improved localisation slightly further.

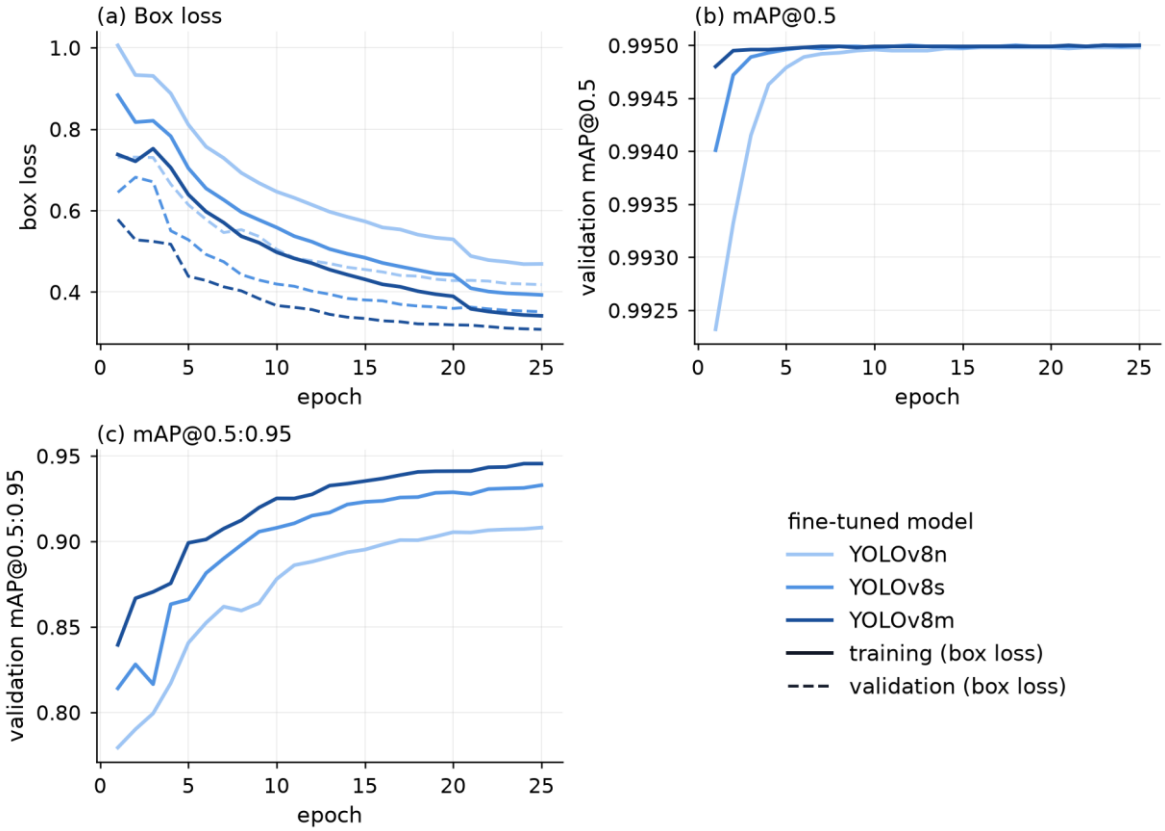


Figure 5.1. Transfer learning of YOLOv8n, YOLOv8s and YOLOv8m on the detector-training split. (a) Box loss on the training data (solid) and the validation data (dashed); (b) validation mAP@0.5 and (c) validation mAP@0.5:0.95 per epoch. Colour shade indicates the model size, from light (YOLOv8n) to dark (YOLOv8m). Note the narrow vertical range of panel (b).

Table 5.1 compares the fine-tuned models with the COCO-pretrained weights evaluated zero-shot (class “person”) on the validation split; the values come from an Ultralytics validation run of the best checkpoints and differ slightly from the training-log values. Fine-tuning raises mAP@0.5 to 99.5 % for all three models and mAP@0.5:0.95 by 37.6, 35.4 and 31.8 points for YOLOv8n, s and m, and precision and recall both exceed 99 %. The pretrained detectors thus already find most rendered pedestrians at a loose IoU of 0.5 but miss about a sixth to a quarter of them and localise their boxes poorly. The ranking of the model sizes is unchanged by fine-tuning, and the gap between the smallest and the largest model shrinks from 9.7 to 3.9 points of mAP@0.5:0.95. Training time grows from 29.1 min (YOLOv8n) to 53.7 min (YOLOv8s) and 119.4 min (YOLOv8m, with half the batch size; Table 4.5).

Table 5.1. Detection accuracy on the validation split (Ultralytics validation, every tenth frame, labels with visibility ≥ 0.2) of the COCO-pretrained models evaluated zero-shot and of the fine-tuned models, with number of epochs and training time.

Model	Weights	Epochs	Training time (min)	mAP@0.5 (%)	mAP@0.5: 0.95 (%)	Precision (%)	Recall (%)
YOLOv8n	COCO, zero-shot	–	–	88.2	53.9	86.4	77.0
YOLOv8n	Fine-tuned	25	29.1	99.5	91.5	99.7	99.3
YOLOv8s	COCO, zero-shot	–	–	91.1	58.6	89.0	81.3
YOLOv8s	Fine-tuned	25	53.7	99.5	94.0	99.8	99.6
YOLOv8m	COCO, zero-shot	–	–	92.2	63.6	91.2	84.3
YOLOv8m	Fine-tuned	25	119.4	99.5	95.4	99.9	99.7

5.2.2 Accuracy on the Unseen Test Split

The validation split is small (1,080 images) and was used to select the checkpoint, so the detectors were evaluated again on all 43,200 frames of the test split. This evaluation differs from Table 5.1 and is not directly comparable with it: it applies the post-hoc NMS at the default threshold of 0.7 used in tracking (Section 4.6); AP is computed COCO-style with 101-point interpolation over pedestrians with visibility of at least 0.2 that lie at least half inside the image; and AP is also computed per visibility band, with pedestrians outside the band treated as ignore regions while unmatched detections count as false positives in every band. Recall and precision are reported at a confidence of at least 0.25 and an IoU of 0.5. Table 5.2 lists the results.

Table 5.2. Detection accuracy on the test split per model and visibility band (post-hoc NMS at 0.7; COCO-style AP with 101-point interpolation; AP averaged over IoU 0.50:0.95; recall and precision at confidence ≥ 0.25 and IoU 0.5; pedestrians at least half inside the image; pedestrians outside the band ignored).

Model	Visibility band	Ground-truth boxes	AP50 (%)	AP75 (%)	AP (%)	Recall (%)	Precision (%)
YOLOv8s zero-shot	0.2–1.0	1,490,579	83.4	61.8	54.7	70.7	94.3
	0.2–0.5	92,715	25.9	0.2	5.0	31.4	37.8
	0.5–0.8	222,710	38.7	1.4	9.1	30.9	58.2
	0.8–1.0	1,175,154	93.7	74.8	64.3	84.9	94.1
YOLOv8n fine-tuned	0.2–1.0	1,490,579	98.8	95.9	88.4	99.4	97.3
	0.2–0.5	92,715	80.0	50.9	46.0	97.8	69.4
	0.5–0.8	222,710	93.0	77.9	65.6	98.7	86.2
	0.8–1.0	1,175,154	98.9	98.0	91.8	100.0	97.3
YOLOv8s fine-tuned	0.2–1.0	1,490,579	98.9	96.9	90.8	99.5	97.9
	0.2–0.5	92,715	81.7	56.8	49.6	98.4	72.0
	0.5–0.8	222,710	93.9	80.9	69.8	98.9	87.5
	0.8–1.0	1,175,154	98.9	98.0	93.6	100.0	97.5
YOLOv8m fine-tuned	0.2–1.0	1,490,579	98.9	97.0	92.1	99.7	98.1
	0.2–0.5	92,715	83.8	60.9	53.1	99.0	73.4
	0.5–0.8	222,710	94.5	81.8	72.7	98.9	88.1
	0.8–1.0	1,175,154	99.9	98.1	94.6	100.0	97.6

Over all pedestrians with visibility of at least 0.2, the fine-tuned YOLOv8s reaches an AP of 90.8 % against 54.7 % zero-shot and a recall of 99.5 % against 70.7 %, which confirms the validation results on more than a million unseen boxes. The fine-tuned models differ much less in AP50 (98.8 % to 98.9 %) than in AP (88.4 % to 92.1 %), that is, mainly in localisation precision.

The visibility bands show where the domain gap lies. For nearly fully visible pedestrians, 1,175,154 of the 1,490,579 boxes, the zero-shot model is serviceable (AP50 93.7 %), but for partly occluded pedestrians it collapses (AP50 38.7 % in the band 0.5 to 0.8 and 25.9 % in the band 0.2 to 0.5, with an AP75 of 0.2 %): a detector trained on COCO's mostly visible persons fails on the occlusions typical of crowded scenes and does not match the amodal ground truth. After fine-tuning, YOLOv8s recalls 98.4 % of the pedestrians with visibility between 0.2 and 0.5 at a confidence of 0.25, so transfer learning closes most of the gap for occluded pedestrians too, but not all of it: AP in the lowest band (49.6 %) remains far below that of the highest band (93.6 %), because the amodal box of a mostly hidden pedestrian must be inferred from a small visible part. The low band

precision (72.0 %) partly reflects the evaluation rule, since the same unmatched detections weigh more heavily against the few true positives of a small band. Model size matters most at low visibility: from YOLOv8n to YOLOv8m, AP rises by 7.1 points in the band 0.2 to 0.5 but by 2.8 points in the band 0.8 to 1.0.

Figure 5.2 summarises the transfer-learning gain on the validation split, and Figure 5.3 shows it on two test frames.

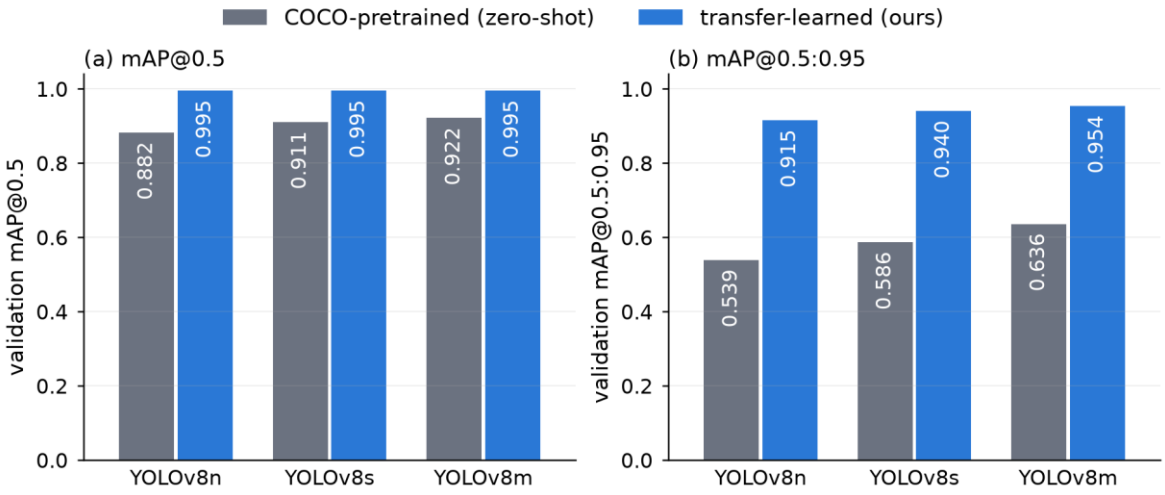


Figure 5.2. Gain from transfer learning on the validation split: (a) mAP@0.5 and (b) mAP@0.5:0.95 of the COCO-pretrained models evaluated zero-shot (grey) and of the transfer-learned models (blue) for YOLOv8n, YOLOv8s and YOLOv8m.

The gain is much larger for mAP@0.5:0.95 than for mAP@0.5 (Figure 5.2). The frames of Figure 5.3, from a high-density bottleneck and a high-density corridor test sequence, show the consequence for ByteTrack's two stages. The zero-shot model places a large share of its boxes below the default high-score threshold of 0.5, misses pedestrians in the dense parts of both frames and covers occluded pedestrians only partly; the transfer-learned model places almost all boxes above 0.5 and covers the dense far end of the corridor. With the zero-shot detector, many correct detections would reach the tracker only through the second association, which cannot start new tracks; with the fine-tuned detector, nearly all of them enter the first association.

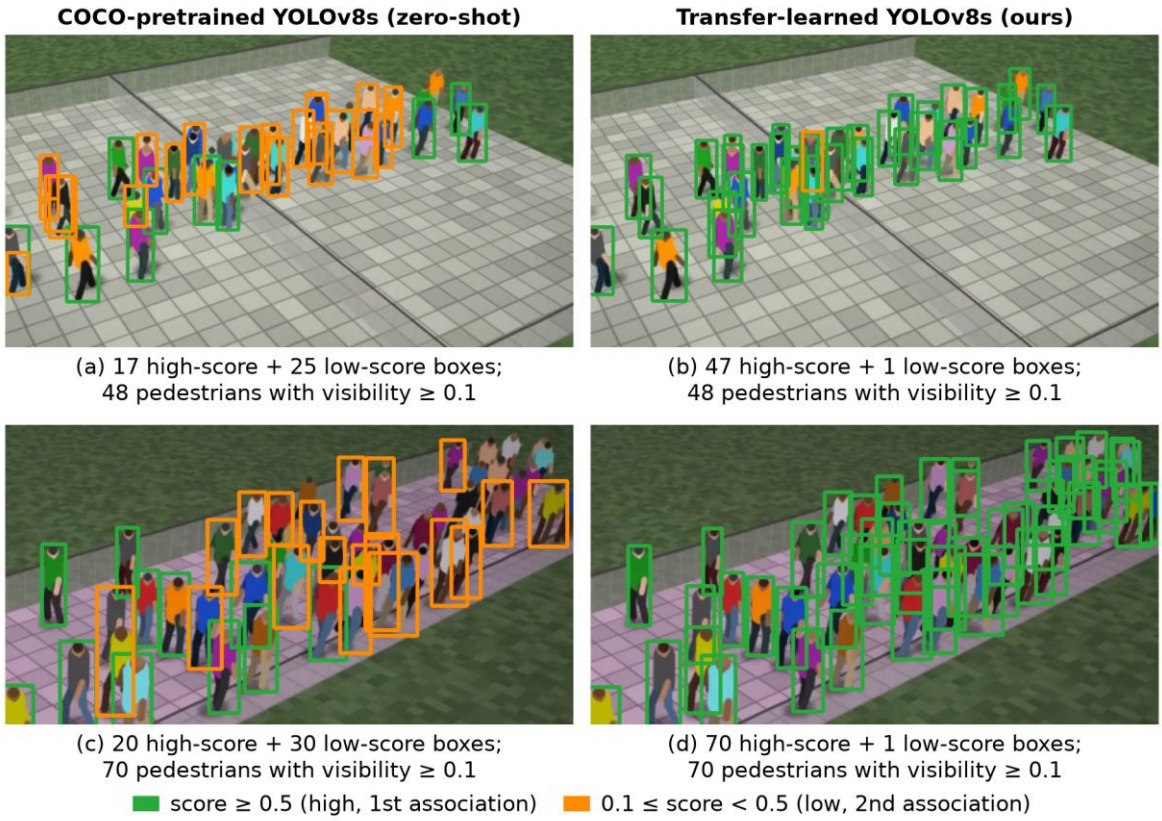


Figure 5.3. Detections of the COCO-pretrained YOLOv8s evaluated zero-shot (left) and of the transfer-learned YOLOv8s (right) on the same test frames of a high-density bottleneck sequence (top) and a high-density corridor sequence (bottom), after NMS at 0.7. Green boxes have a confidence of at least 0.5 and enter ByteTrack's first association under the default parameters, orange boxes a confidence between 0.1 and 0.5 and enter the second association. The panel labels give the numbers of high- and low-score boxes and of pedestrians with visibility of at least 0.1 in the whole frame; the images are crops.

5.2.3 Confidence, Visibility and the Two-Stage Association

ByteTrack's second association exists because occluded objects tend to receive low confidence scores (Section 3.2). Whether the fine-tuned detector shows this behaviour determines how much τ_{low} and θ_{m2} can matter. The confidences of the fine-tuned YOLOv8s were therefore analysed on every third frame of the test split, after NMS at 0.7 and a confidence floor of 0.05; detections were matched greedily in order of confidence to pedestrians at least half inside the image at IoU ≥ 0.5, and the true positives were grouped by the visibility of the matched pedestrian. Figure 5.4 shows the result.

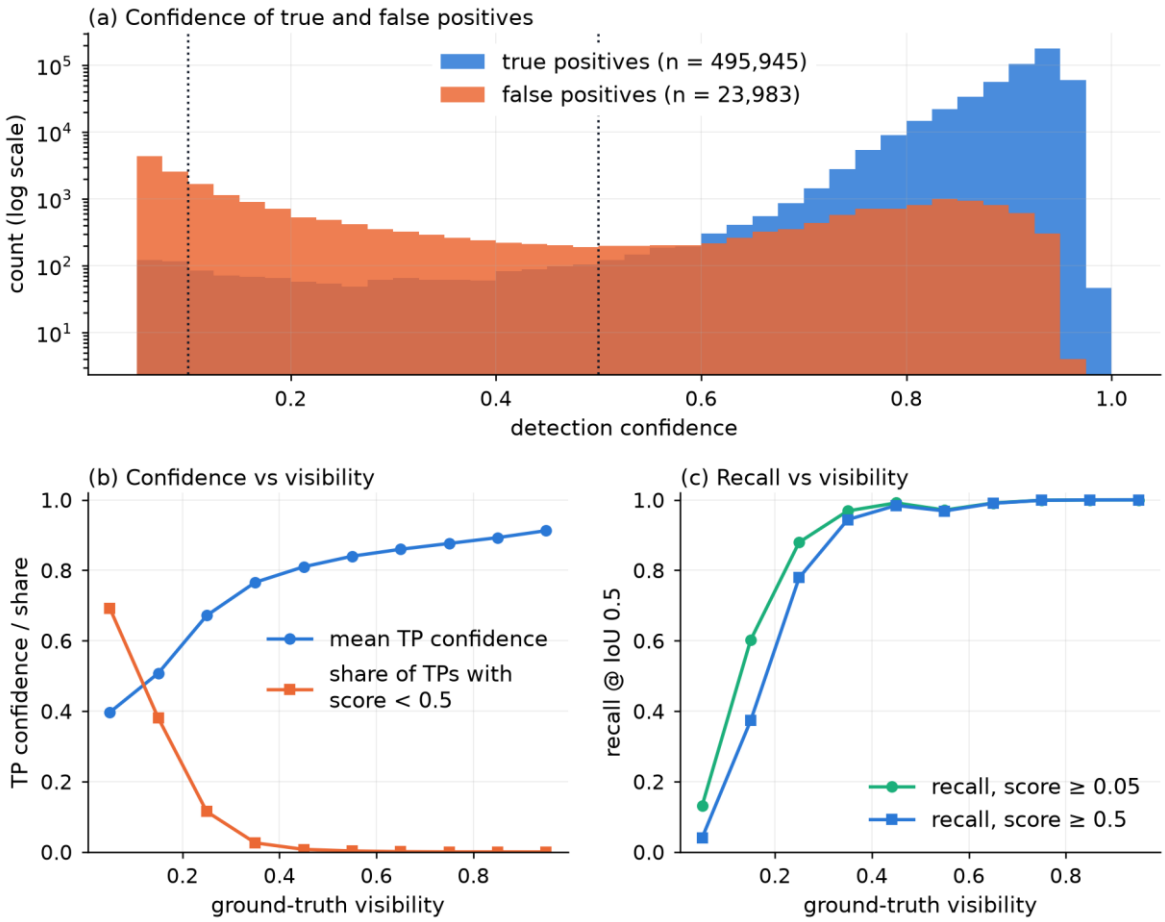


Figure 5.4. Confidence of the transfer-learned YOLOv8s on the test split (every third frame, NMS at 0.7, confidence ≥ 0.05, matching at IoU ≥ 0.5). (a) Histograms of the confidence of true positives (blue) and false positives (orange) on a logarithmic scale, with dotted lines at the default low-score threshold 0.1 and high-score threshold 0.5; (b) mean confidence of true positives and share of true positives with a confidence below 0.5 per visibility bin; (c) recall per visibility bin counting detections with confidence ≥ 0.05 (green) and ≥ 0.5 (blue).

True positives are concentrated at high confidence, mostly above 0.8 (Figure 5.4a). False positives are most frequent at the lowest confidences, fall to a minimum around 0.5 to 0.6 and then rise to a second mode at 0.8 to 0.9: some false positives receive high confidence and pass any plausible high-score threshold τ_{high}. Below about 0.6 false positives outnumber true positives in every bin, so a detection between the two default thresholds is more likely to be wrong than right. Confidence falls as visibility decreases (Figure 5.4b): the mean confidence of true positives is 0.913 at visibility 0.9 to 1.0, 0.672 at 0.2 to 0.3 and 0.396 below 0.1, and the share of true positives below 0.5 rises from below 1 % above a visibility of 0.4 to 11.5 %, 38.0 % and 69.2 % in the three lowest bins. Counting all detections above 0.05, recall is at least 96.9 % for visibility above 0.3 but 88.0 %, 60.2 % and 13.2 % in the three lowest bins; counting only detections above 0.5 it is 77.9 %, 37.3 % and 4.1 % (Figure 5.4c).

The fine-tuned detector therefore behaves as ByteTrack assumes: the low-score detections recovered by the second association are mainly strongly occluded pedestrians, with visibility below about 0.3. Such pedestrians are rare, however: the analysis contains 7,056 evaluated boxes below a visibility of 0.3 against 199,212 above it, and only 0.8 % to 14.6 % of the pedestrians per condition have a visibility below 0.5 (Table 4.4). The second association can thus affect only a small share of the matches, which anticipates the weak influence of τ_{low} and θ_{m2} in Section 5.3. It also explains why pedestrians with visibility between 0.1 and 0.2, which are evaluated in tracking but were not positive training examples (Section 4.6), are found only in about three of five cases.

5.3 Parameter Sensitivity and Scenario Dependence (RQ2)

RQ2 asks how sensitive the HOTA of the pipeline is to its seven parameters and whether the best settings differ between scenarios and densities enough to justify scene-adaptive tuning. Three analyses on the training split answer it: one-at-a-time sweeps, two-dimensional response surfaces, and the best parameter vectors found per condition by TPE.

5.3.1 One-at-a-Time Sensitivity

Each parameter was varied over eleven equally spaced values covering its range (Table 4.6), with the other six held at $\boldsymbol{\theta}_0$, on two training clips per static condition, 24 clips in all. Figure 5.5 shows the mean HOTA per scenario (two clips × three densities) and over all clips. The span of the mean HOTA over all clips (largest minus smallest value of a sweep) is 5.3 points for θ_{nms}, 2.2 points for θ_{m1}, 1.6 points for τ_{new} and 0.7 points for τ_{high}, and only 0.1 to 0.2 points for τ_{low}, θ_{m2} and n_{buf}.

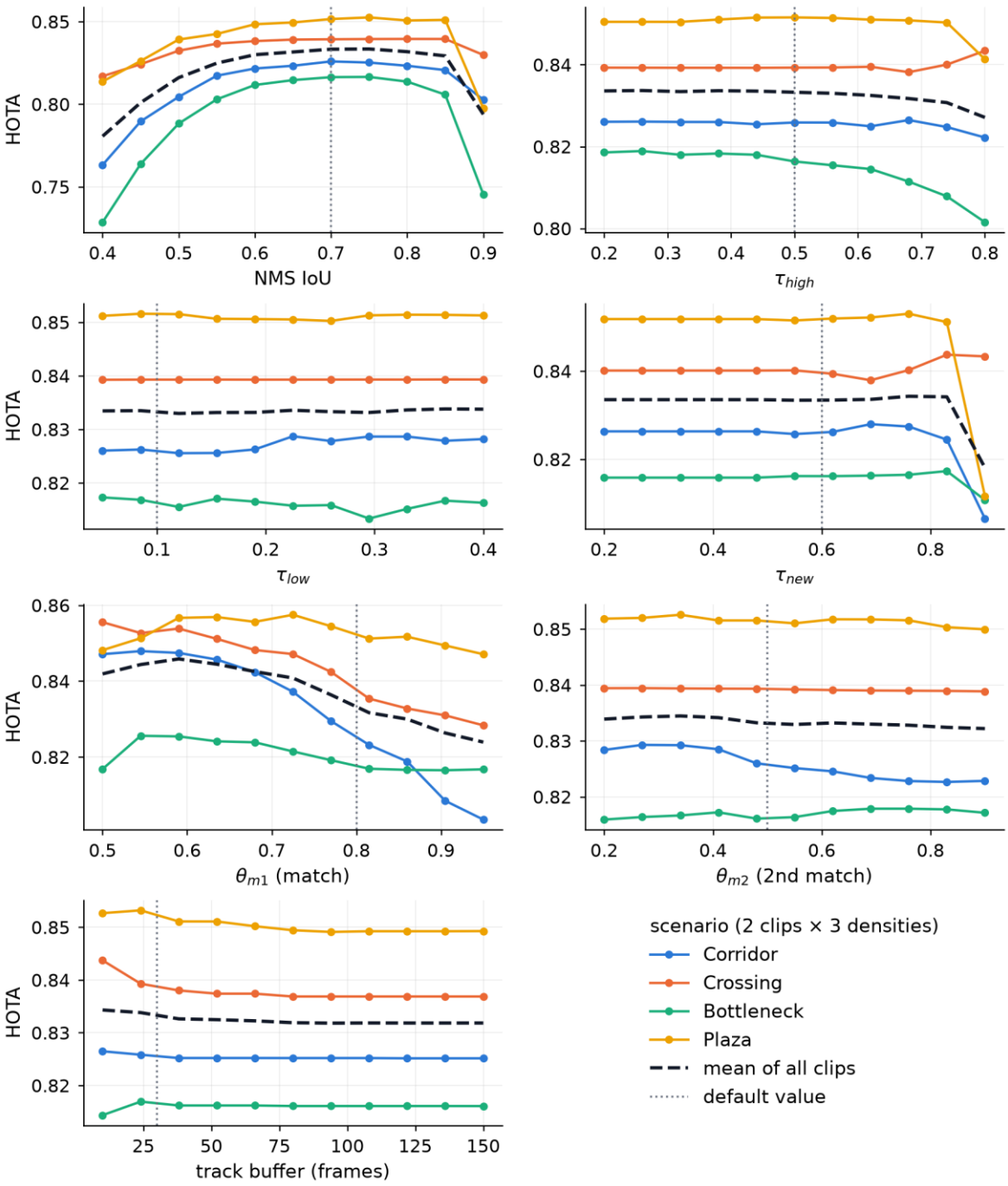


Figure 5.5. One-at-a-time sensitivity of HOTA on the training split. Each panel varies one parameter over eleven values with the others at their defaults; coloured lines are the mean over the six clips of a scenario (two clips per density level), the dashed line the mean over all 24 clips, and the dotted vertical line the default value.

The NMS threshold has the largest span mostly because of its extremes: low thresholds suppress neighbouring pedestrians in crowds and 0.9 leaves duplicates, so HOTA drops to a mean of 78.1 % at the lowest value. Between about 0.6 and 0.85 the curve is flat, and its maximum at 0.75 (83.3 %) is no higher, at the reported precision, than the value at the default of 0.7; the parameter matters mainly in that a poor value is costly.

The first-association cost limit θ_{m1} is the only parameter whose sweep improves clearly on the default: the mean HOTA rises from 83.2 % at 0.8 to 84.6 % at 0.59, by 1.4 points, with about two points in the corridor and the crossing, much less in the plaza and the bottleneck, where the curve falls again at the lower end. τ_{new} is flat up to about 0.83 and then drops sharply, most in the plaza and the corridor. τ_{high} lowers the mean HOTA gradually as it rises, mostly because of the bottleneck; the other scenarios are flat until about 0.75. τ_{low}, θ_{m2} and n_{buf} change the mean HOTA by at most 0.2 points, consistent with Section 5.2.3. Within single scenarios they have slightly larger effects below one point (short buffers are best in the crossing and the plaza, whereas the shortest is worst in the bottleneck). The scenario curves are largely parallel, and the differences in level between scenarios, from the bottleneck (lowest) to the plaza (highest), exceed most parameter effects.

5.3.2 Response Surfaces

To reveal interactions, two pairs were mapped on a 9 × 9 grid, with the other five parameters at $\boldsymbol{\theta}_0$, on three high-density training clips per scenario: τ_{high} with θ_{m1}, which decide which detections enter the first association and how they are matched (Figure 5.6), and n_{buf} with τ_{new}, which govern track termination and birth (Figure 5.7). Each panel has its own colour scale, so shades are comparable only within a panel.

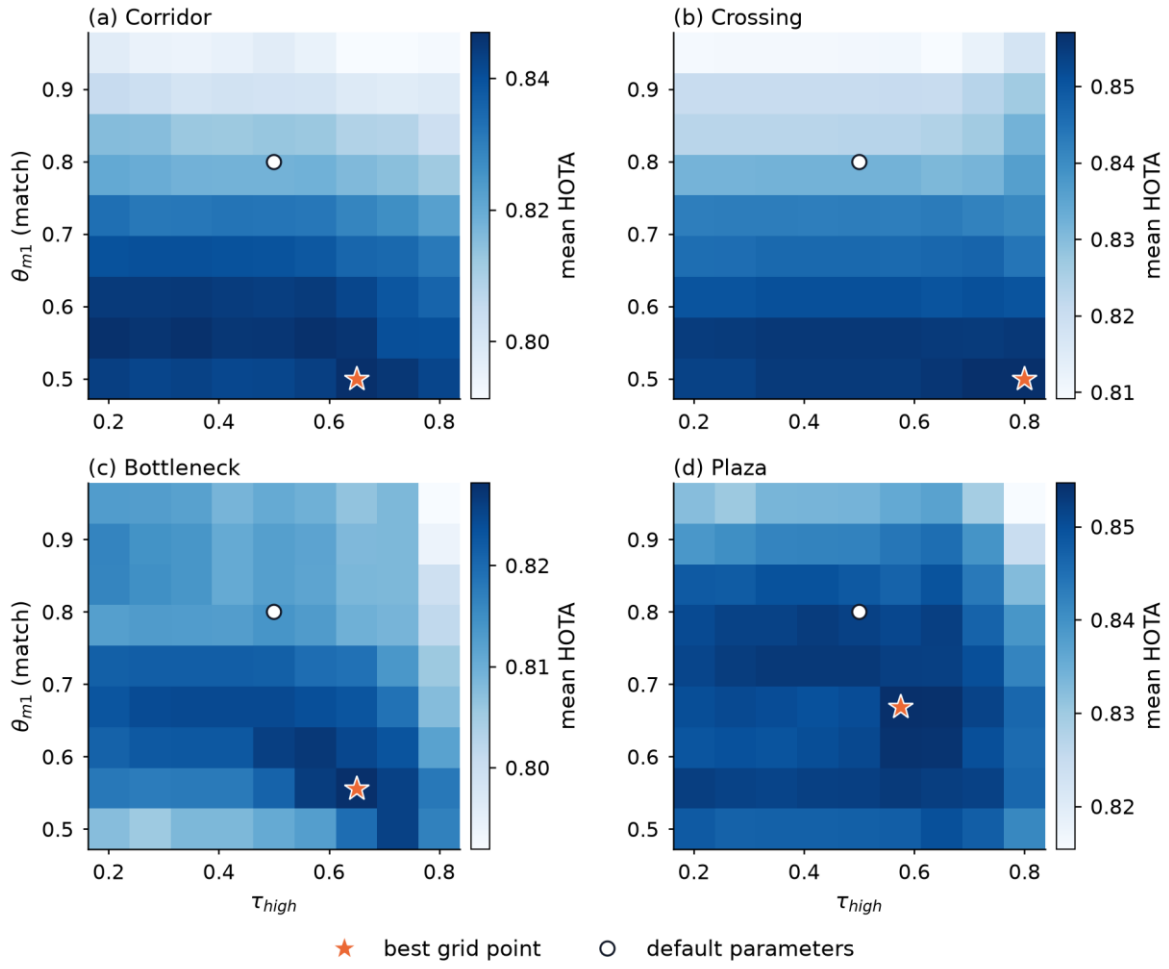


Figure 5.6. Response surfaces of the mean HOTA over three high-density training clips per scenario as a function of the high-score threshold τ_{high} and the first-association cost limit θ_{m1}, with the other parameters at their defaults: (a) corridor, (b) crossing, (c) bottleneck, (d) plaza. The star marks the best grid point and the circle the default; each panel has its own colour scale.

In the corridor and the crossing the surface of Figure 5.6 is dominated by θ_{m1}: HOTA increases almost monotonically as the limit is tightened towards 0.5 and changes little with τ_{high}, and the default lies in a clearly inferior band. In the bottleneck the best region is a diagonal ridge, with the best point at $\tau_{\text{high}} = 0.65$ and a limit of about 0.55; the lowest limit is worse again and $\tau_{\text{high}} = 0.8$ is poor at every limit. In the plaza the surface is flat over a wide central region that includes the default. The two parameters thus interact in the bottleneck and the plaza but act almost separately in the corridor and the crossing, and the best point differs between scenarios.

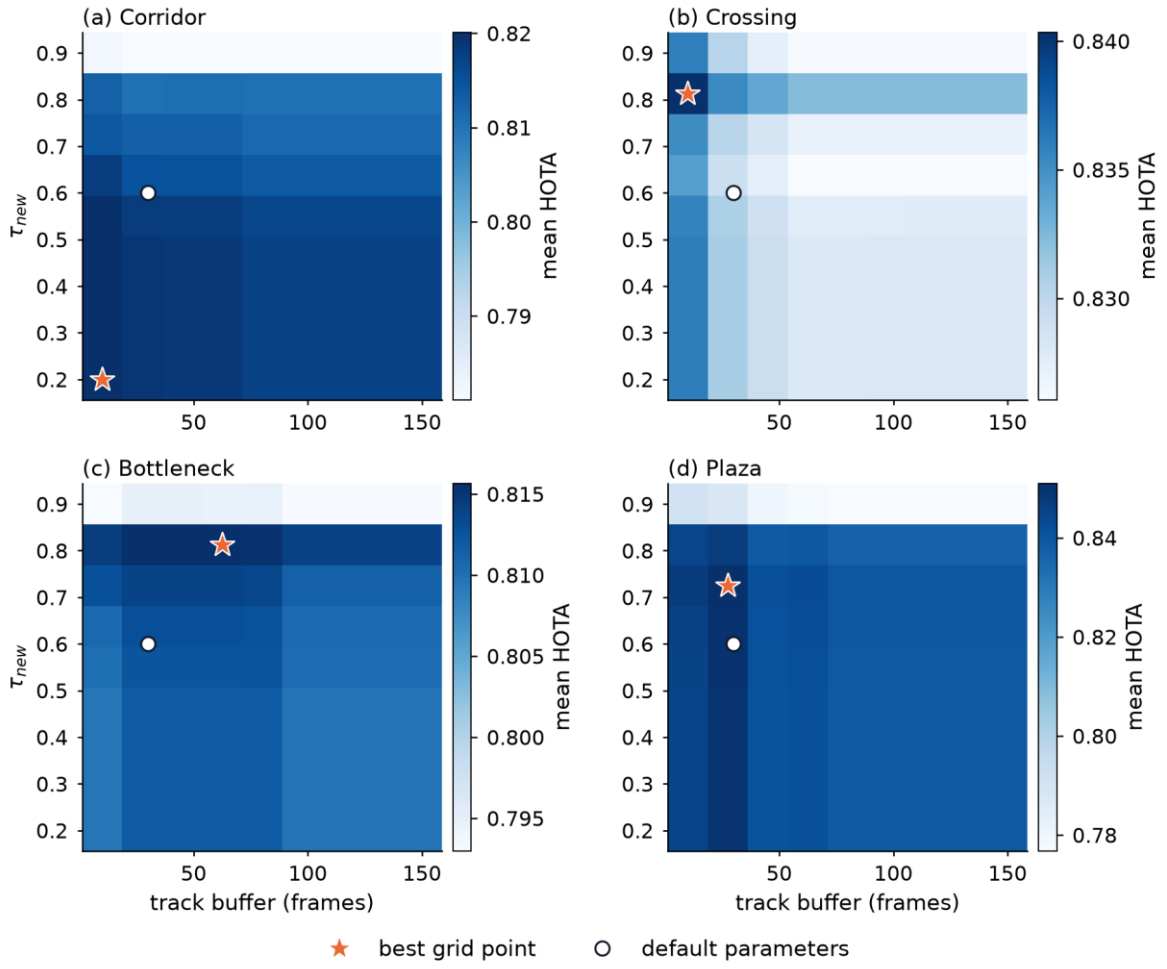


Figure 5.7. Response surfaces of the mean HOTA over three high-density training clips per scenario as a function of the track buffer n_{buf} (frames at 30 FPS) and the new-track threshold τ_{new}, with the other parameters at their defaults: (a) corridor, (b) crossing, (c) bottleneck, (d) plaza. The star marks the best grid point and the circle the default; each panel has its own colour scale.

Figure 5.7 shows flatter surfaces for track birth and termination; note the narrow colour ranges of the crossing and bottleneck panels (about 1.5 and 2.3 points). A new-track threshold of 0.9 is poor everywhere. Below it, the best region differs: low thresholds with the shortest buffer in the corridor, a threshold of about 0.8 with the shortest buffer in the crossing (an isolated optimum on an otherwise nearly level surface), about 0.8 with a buffer of about 60 frames in the bottleneck, and short buffers at almost any threshold in the plaza. A plausible reading is that where pedestrians queue and occlude each other for longer, as at the bottleneck gate, keeping lost tracks longer helps to re-attach them, whereas where pedestrians cross the view quickly a short buffer avoids re-attaching a lost track to the wrong person.

5.3.3 Scenario Dependence of the Optimum

The best full parameter vectors per condition are those found by the per-condition HPO runs on the objective of Section 4.10, the mean HOTA over the four fixed clips of a condition. Table 5.3 lists, for each of the 16 conditions and for the global objective, the default objective value, the best objective value found by TPE (seed with the best objective) and its parameter vector, ordered by the best objective value. The grid-search optimum is compared with these results in Section 5.4.

Table 5.3. Best parameter vector per condition found by TPE on the training objective (mean HOTA over the four HPO clips of a condition; for the global row, over the first HPO clip of each of the 16 conditions), seed with the best objective, ordered by the best objective value. Default and best objective in per cent; gain in points computed from the values rounded to three decimals. The last row gives the default parameters.

Condition	**Default (%)**	**Best (%)**	**Gain (points)**	θ_{nms}	τ_{high}	τ_{low}	τ_{new}	θ_{m1}	θ_{m2}	n_{buf} **(frames)**
Bottleneck, high	81.1	82.3	1.2	0.745	0.234	0.168	0.714	0.678	0.250	49.5
Bottleneck, dynamic	80.2	82.8	2.6	0.858	0.442	0.235	0.757	0.564	0.287	90.3
Bottleneck, medium	82.0	84.0	2.0	0.727	0.600	0.169	0.237	0.504	0.317	46.8
Corridor, high	81.4	84.4	3.0	0.703	0.272	0.235	0.669	0.559	0.418	128.1
Bottleneck, low	81.9	85.0	3.1	0.738	0.332	0.228	0.900	0.506	0.474	109.5
Corridor, medium	82.6	85.2	2.6	0.859	0.564	0.359	0.322	0.520	0.245	66.4
Corridor, dynamic	82.8	85.4	2.6	0.770	0.658	0.296	0.202	0.503	0.288	25.2
Global (all 16 conditions)	83.8	85.6	1.8	0.802	0.244	0.255	0.781	0.538	0.313	46.5
Crossing, high	83.3	85.7	2.4	0.746	0.375	0.282	0.716	0.520	0.447	10.2
Plaza, high	85.1	86.1	1.0	0.714	0.636	0.291	0.233	0.545	0.459	20.9
Crossing, medium	83.7	86.2	2.5	0.789	0.430	0.384	0.234	0.538	0.209	10.8
Crossing,	84.2	86.2	2.0	0.773	0.262	0.198	0.825	0.519	0.600	142.8

Condition	Default (%)	Best (%)	Gain (points)	θ_{nms}	τ_{high}	τ_{low}	τ_{new}	θ_{m1}	θ_{m2}	n_{buf} (frames)
dynamic										
Corridor, low	82.9	86.4	3.5	0.589	0.287	0.332	0.874	0.534	0.383	48.2
Crossing, low	84.7	86.8	2.1	0.648	0.799	0.064	0.379	0.579	0.230	27.5
Plaza, medium	86.4	86.9	0.5	0.685	0.432	0.339	0.611	0.629	0.375	27.7
Plaza, dynamic	86.4	87.5	1.1	0.652	0.291	0.337	0.301	0.511	0.863	23.5
Plaza, low	88.5	89.3	0.8	0.864	0.745	0.363	0.289	0.552	0.334	67.3
Default $\boldsymbol{\theta}_0$	–	–	–	0.70	0.50	0.10	0.60	0.80	0.50	30

TPE improves the objective over the default in every condition, by 0.5 to 3.5 points: most in the corridor (2.6 to 3.5) and the crossing (2.0 to 2.5), less in the bottleneck (1.2 to 3.1) and least in the plaza (0.5 to 1.1), where the default is already best. The gain does not follow density in a uniform way. These gains are measured on the clips the optimiser selected for and are therefore optimistic; Section 5.5 gives the unbiased comparison.

Two components are consistent across conditions. θ_{m1} lies between 0.503 and 0.678 in all 17 rows, far below the default of 0.8, in agreement with Sections 5.3.1 and 5.3.2, and τ_{low} lies above the default of 0.1 in 16 of the 17 rows, although it alone changes HOTA very little. The NMS threshold stays mostly in the flat region between about 0.6 and 0.85 found in Section 5.3.1 (16 of 17 values; only 0.589 lies below it, and 0.858 to 0.864 sit at its edge). The other components are scattered over most of their ranges without a recognisable relation to scenario or density; even corridor medium and corridor high differ widely in τ_{high}, τ_{new} and n_{buf}. Given the flat sweeps of these parameters, this scatter is consistent with an objective that is nearly insensitive to them near the optimum and suggests that the choice among almost equivalent values is influenced by the noise of four clips, although the scenario-specific optima of Figure 5.7 show that the scene also plays a role; Chapter 6 returns to this. The globally tuned vector shares the strict cost limit (0.538) and the raised τ_{low} (0.255), and its gain of 1.8 points on its own, differently defined objective is of the same order as the per-condition gains.

The answer to RQ2 from the training data is therefore twofold. HOTA is sensitive mainly to the NMS threshold, whose default is adequate but whose extremes are costly, and to the first-association cost limit, whose default is too loose in every scenario; the other

parameters change HOTA by less than one point when varied alone, although they interact in the bottleneck and the plaza. The best settings do differ between scenarios, but the one-at-a-time sweep suggests that a strict first-association limit alone yields about 1.4 of the roughly 2 points gained by full tuning, although the clips differ, and the scenario-specific part lies largely in parameters to which the objective is weakly sensitive. Whether scene-adaptive tuning pays off on unseen sequences is the subject of Sections 5.4 and 5.5, and Chapter 6 discusses the consequences for the comparison between RL and classical optimisation.

5.4 Optimisation on the Training Objective

Before the methods are compared on unseen sequences, this section reports how they behave on the quantities they actually optimise: the per-condition objective f_q of Equation (3.59) for the baseline optimisers, the reward of Equation (4.9) for PPO-contextual and the return of Equation (4.10) for PPO-adaptive. These results show whether each optimiser converged within its budget and how much of the improvement available on the training data it found. They are not evidence of generalisation, because the baselines' objective rests on four fixed clips per condition (Section 4.5) and the selected parameters may partly overfit them; generalisation is the subject of Section 5.5.

5.4.1 Baseline Optimisers

For every condition q and seed, the best objective value found after k evaluations was recorded and the default value $f_q(\boldsymbol{\theta}_0)$ subtracted, giving the best-so-far gain in HOTA points. Figure 5.8 shows this gain averaged over the 16 conditions and three seeds for Random search, TPE and CMA-ES. All three curves rise steeply during the first 25 evaluations and then flatten. Random search starts lowest, because its first sample is drawn uniformly in the cube and is on average worse than the default, but it overtakes the other two within the first ten evaluations, reaching about 1.6 points while TPE is still in its 20 random start-up trials and CMA-ES is still contracting from the centre of the cube. TPE and CMA-ES catch up at about 20 to 25 evaluations, from then on lie above Random search, and keep improving slowly until the end of the budget, where CMA-ES ends slightly above TPE and both slightly above Random search. The shaded bands show the standard error of the mean over the 48 condition–seed runs; they are narrow because they describe the uncertainty of the average curve, not the spread between conditions (Table

5.4). Between about 8 and 15 evaluations the Random search band lies above the TPE band; after about 25 evaluations the bands of the three optimisers overlap or nearly touch.

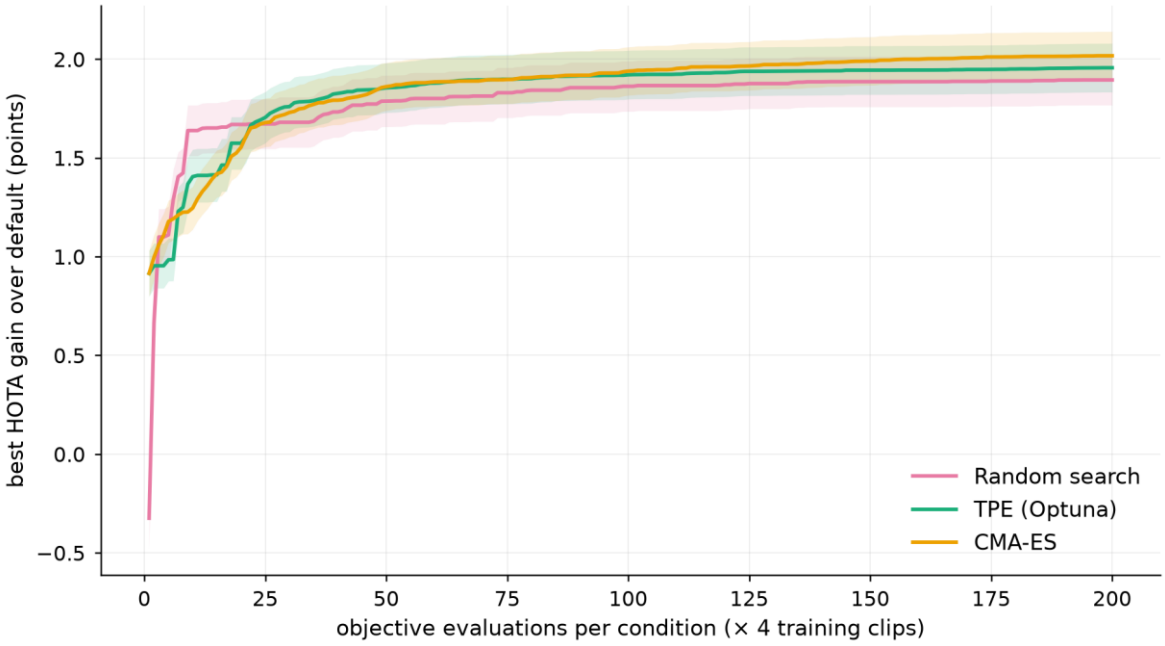


Figure 5.8. Convergence of the baseline optimisers on the training objective. Best HOTA gain over the default found after a given number of objective evaluations (each evaluation scores four training clips), averaged over the 16 conditions and three seeds; shaded bands show the standard error of the mean over the 48 condition–seed runs. Pink: Random search; green: TPE; orange: CMA-ES.

Table 5.4 condenses the curves into the final gain after 200 evaluations and the number of evaluations needed to reach 95 % of that final gain. The final gains are 2.0 points for CMA-ES and 1.9 points for Random search and TPE, each with a standard deviation of about 0.8 to 0.9 points that reflects mainly the differences between conditions rather than between seeds; for example, in TPE's best seed the gain ranges from about 0.5 points (plaza, medium density, 0.864 to 0.869) to about 3.5 points (corridor, low density, 0.829 to 0.864) (Table 5.3). Random search and TPE reach 95 % of their final gain after about 53 and 55 evaluations, CMA-ES after about 80; the large standard deviations of 39 to 47 evaluations show that this budget point varies strongly between conditions and seeds. CMA-ES needs longer because its curve is still rising slightly at 200 evaluations; it ends marginally, about a tenth of a HOTA point, above the other two. The differences between the three optimisers on the training objective are thus about a tenth of a HOTA point, far smaller than their spread. Global TPE optimises a different objective (one clip from each of the 16 conditions with a single $\boldsymbol{\theta}$), so its training gain is not comparable and is not included; its best objective value of 0.856 against 0.838 for the default on the same clips (Table 5.3) corresponds to a gain of 1.8 points.

Table 5.4. Gain of the baseline optimisers on the training objective f_q (mean HOTA over the four objective clips of a condition), mean $\pm$ standard deviation over 16 conditions × 3 seeds. The grid search evaluates all 3^7 points once and has no convergence curve.

Method	Budget per condition and seed (clip evaluations)	Gain over default after 200 evaluations (points)	Evaluations to 95 % of final gain
Random search	800	1.9 ± 0.9	53.4 ± 47.5
TPE	800	1.9 ± 0.8	55.4 ± 39.1
CMA-ES	800	2.0 ± 0.9	80.1 ± 47.4
Grid search (reference)	8,748	[[PENDING:grid]]	–

The grid search, with eleven times the budget of the other per-condition optimisers but only three levels per parameter, serves as a reference for how much of the attainable gain the budget-matched optimisers capture. Its gain on the training objective is [[PENDING:grid]] points. Section 5.3 showed that the response is dominated by the NMS threshold and the first-association cost limit and is otherwise flat; a plateau reached after about 50 evaluations is consistent with that landscape.

5.4.2 Training of the PPO Agents

Figure 5.9 shows the learning curves of the two PPO formulations. For PPO-contextual (panel a) the curve is the rolling mean of the HOTA achieved by the sampled actions on the training clips, so it includes the exploration noise of the Gaussian policy and is a lower bound on the quality of the deterministic policy used at test time. It rises from about 83.0 % to about 84.3 % within the first 4,000 steps and then creeps upward to about 84.5 % with fluctuations of a few tenths of a point; the range over the three seeds is narrow throughout. Averaged over the first and last 10 % of training, the HOTA of the sampled actions increased from 83.3 % to 84.5 % for seeds 0 and 2 and from 83.5 % to 84.5 % for seed 1, an improvement of 1.0 to 1.2 points. Each budget-matched run took between 7.3 and 8.1 minutes of wall-clock time for its 13,056 executed steps on 24 worker processes. The curve has not clearly levelled off at the end of the budget, which motivated the extended run with four times the budget; its HOTA over the first and last 10 % of training is [[PENDING:ext]], and its wall-clock time is [[PENDING:ext]] minutes per seed.

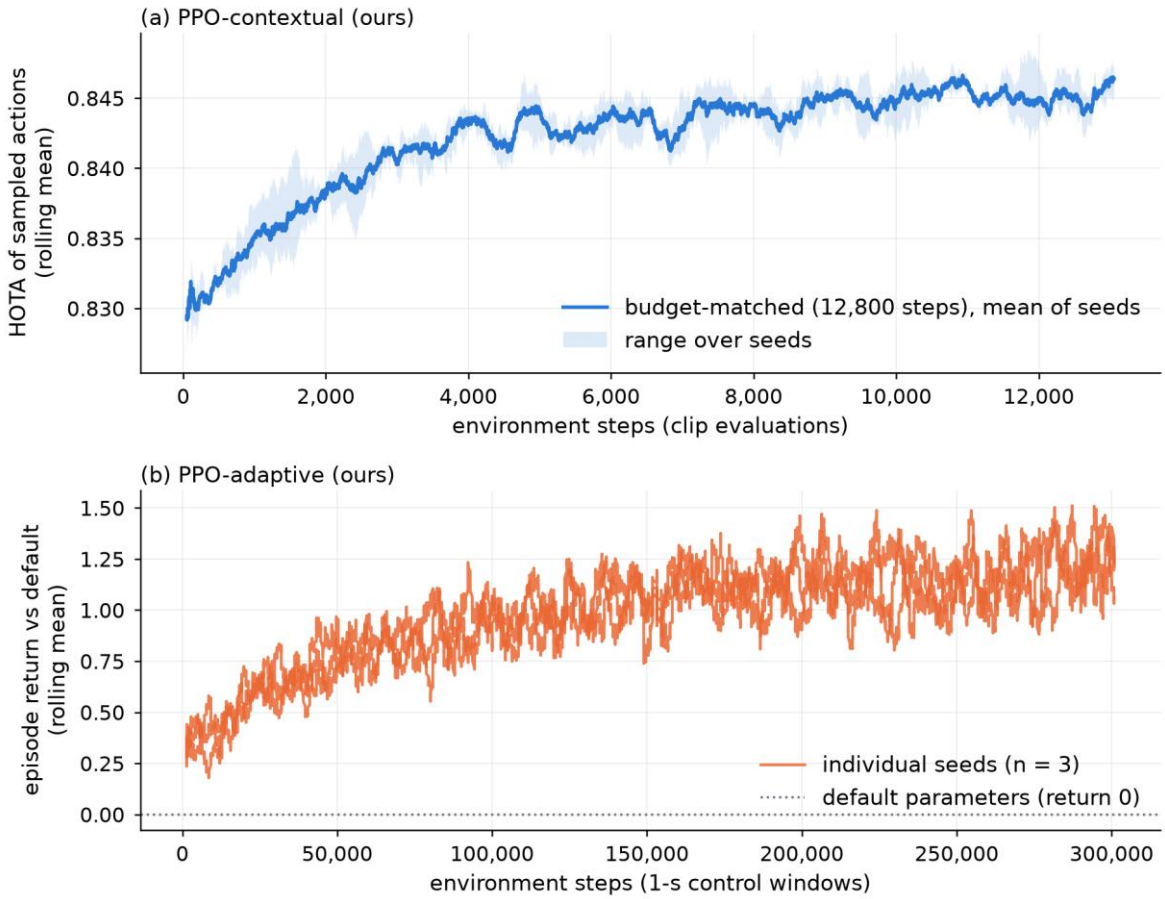


Figure 5.9. Learning curves of the PPO agents. (a) PPO-contextual, budget-matched run: rolling mean (window 384 steps, one rollout) of the HOTA of the sampled actions on the training clips, mean of three seeds (line) and range over seeds (band). (b) PPO-adaptive: rolling mean (50 episodes) of the episode return, the sum over windows of the score difference to a default run (Equation (4.10)), for the three seeds individually; the dotted line marks the return of the default parameters (zero).

The learning curves of the two formulations are not directly comparable, because PPO-contextual is scored by clip HOTA and PPO-adaptive by the windowed, identity-weighted score of Equation (4.10). For PPO-adaptive (panel b) the episode return is the summed score difference to a default run of the same sequence, so a return of zero means no improvement over $\boldsymbol{\theta}_0$. All three seeds are above zero from the start, since the initial policy, centred in the cube, already differs from the default in a favourable direction, and their returns rise from 0.50 to 0.53 over the first 10 % of the episodes to between 1.14 and 1.23 over the last 10 %, more than doubling. The curves rise fastest during the first 100,000 steps and level off after about 150,000, with large episode-to-episode fluctuations that come from the varying length and difficulty of the training sequences. Each seed completed between 4,515 and 4,519 episodes in 13.1 to 13.7 minutes of wall-clock time. Both agents therefore learned stably and with little variation between seeds; whether what they learned transfers to the test sequences is examined next.

5.5 Results on the Test Splits (RQ3, RQ4)

This section answers RQ3 and RQ4 on the held-out test splits, which were used exactly once. Every method was applied with its test-time selection rule of Table 4.10: the per-condition baselines with the oracle condition label, global TPE with its single parameter vector, PPO-contextual with the descriptor of the first 4 s of each sequence and PPO-adaptive online. The metrics are the combined metrics of Section 4.8 over the 48 static-density test sequences and, separately, over the 16 dynamic-density test sequences, reported as mean ± standard deviation over the three seeds of each method.

5.5.1 Overall Accuracy

Table 5.5 lists the results on the static-density test split. Every tuning method improved on the default parameters by about two HOTA points. The default reached 81.15 % HOTA. The three per-condition baselines reached between 83.52 % (Random search) and 83.58 % (CMA-ES), gains of 2.37 to 2.43 points; global TPE reached 83.42 % (+2.27 points), PPO-contextual 83.29 % (+2.14 points) and PPO-adaptive 83.20 % (+2.05 points). The spread among all six tuned methods is thus 0.38 points, less than a fifth of their common gain over the default. The seed standard deviations are small, between 0.03 and 0.19 points, with global TPE and Random search the least reproducible and the two PPO agents and CMA-ES the most. The grid-search reference reached [[PENDING:grid]] % HOTA, and PPO-contextual with four times the budget [[PENDING:ext]] %.

Table 5.5. Tracking accuracy on the static-density test split (48 sequences, 43,200 frames), combined metrics. Tuned methods: mean ± standard deviation over three seeds (DetA and AssA: mean); counts are seed means rounded to integers. Default and grid search are deterministic single runs.

Method	HOTA (%)	DetA (%)	AssA (%)	MOTA (%)	IDF1 (%)	IDSW	FP	FN	Tracker IDs
Default	81.15	84.01	78.42	95.54	91.54	508	32,760	33,514	6,631
Grid search	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]
Random search	83.52 ± 0.12	85.10	82.01	97.21 ± 0.09	94.69 ± 0.20	1,339	17,910	22,543	10,303
TPE	83.53 ± 0.05	85.07	82.05	97.15 ± 0.02	94.66 ± 0.14	1,457	17,747	23,448	10,767
CMA-ES	83.58 ± 0.03	85.12	82.10	97.22 ± 0.08	94.75 ± 0.08	1,279	17,594	22,687	10,290
TPE (global)	83.42 ± 0.19	85.10	81.82	97.21 ± 0.08	94.49 ± 0.25	1,259	14,434	26,027	9,825
PPO-contextual	83.29 ± 0.04	85.16	81.49	97.24 ± 0.01	94.03 ± 0.07	2,193	17,349	21,812	12,292
PPO-	[[PEND	[[PEND	[[PEND	[[PEND	[[PEND	[[PEND	[[PEND	[[PEND	[[PEND

Method	HOTA (%)	DetA (%)	AssA (%)	MOTA (%)	IDF1 (%)	IDSW	FP	FN	Tracker IDs
contextual (4x budget)	ING:ext]]	ING:ext]]	ING:ext]]	ING:ext]]	ING:ext]]	ING:ext]]	ING:ext]]	ING:ext]]	ING:ext]]
PPO-adaptive	83.20 ± 0.03	85.15	81.34	97.25 ± 0.05	94.16 ± 0.04	911	16,938	23,274	8,916

The decomposition of HOTA shows where the gain comes from. Detection accuracy rose by 1.06 to 1.15 points (from 84.01 % to between 85.07 % and 85.16 %) and association accuracy by 2.92 to 3.68 points (from 78.42 % to between 81.34 % and 82.10 %). The two PPO agents have the highest DetA of all methods but the lowest AssA, and their lower HOTA is entirely due to association. The CLEAR and identity metrics move in the same direction: MOTA rose from 95.54 % to between 97.15 % and 97.25 %, where the PPO agents are best, and IDF1 from 91.54 % to between 94.03 % and 94.75 %, where the per-condition baselines are best and PPO-contextual is lowest.

The error counts reveal the trade-off behind these gains. Every tuned method roughly halved the false positives, from 32,760 to between about 14,400 (global TPE) and 17,900 (Random search), and reduced the misses from 33,514 to between about 21,800 (PPO-contextual) and 26,000 (global TPE). At the same time all of them made more identity switches than the default, between 1.8 times (PPO-adaptive, about 911 against 508) and 4.3 times as many (PPO-contextual, about 2,193); the per-condition baselines and global TPE made 2.5 to 2.9 times as many. The number of tracker identities grew from 6,631, close to the 6,485 ground-truth identities of the split, to between about 8,900 and 12,300, so the tuned trackers fragment trajectories into more identities. Because identity switches are a small count compared with false positives and misses, they barely affect MOTA, and HOTA, which weighs detection and association equally over all localisation thresholds, still rises. PPO-adaptive is the only method whose training signal penalises identity switches explicitly (weight 5 in Equation (4.10)), and it has the fewest identity switches and tracker identities of all tuned methods. The mostly-tracked count changed little (6,319 for the default, between about 6,374 and 6,402 for the tuned methods), whereas the mostly-lost count fell from 39 to between about 15 and 18. Track fragmentations (Frag) rose with the identity switches, from 2,200 for the default to between about 2,376 (PPO-adaptive) and 3,485 (PPO-contextual). Section 5.8 analyses this trade-off in more detail.

Table 5.6 lists the same metrics for the dynamic-density test split, in which the arrival rate rises and falls during each sequence. The picture is the same as on the static split. The default reached 80.50 % HOTA; the tuned methods reached between 82.48 % (PPO-

contextual, +1.98 points) and 82.76 % (TPE, +2.26 points). CMA-ES (82.74 %), global TPE (82.70 %) and Random search (82.63 %) lie in between, as does PPO-adaptive with 82.57 % (+2.07 points). The grid-search reference reached [[PENDING:grid]] % and PPO-contextual with four times the budget [[PENDING:ext]] %. Identity switches again rose, from 264 to between about 489 (PPO-adaptive) and 1,198 (PPO-contextual), while false positives fell from 16,358 to between about 7,200 and 8,400 and misses from 17,412 to between about 11,200 and 13,500. Fragmentations rose from 1,050 to between about 1,124 (PPO-adaptive) and 1,701 (PPO-contextual).

Table 5.6. Tracking accuracy on the dynamic-density test split (16 sequences, 21,600 frames), combined metrics; conventions as in Table 5.5. The per-condition baselines use the parameters tuned for the dynamic condition of the scenario.

Method	HOTA (%)	DetA (%)	AssA (%)	MOTA (%)	IDF1 (%)	IDSW	FP	FN	Tracker IDs
Default	80.50	84.07	77.11	95.41	90.14	264	16,358	17,412	2,818
Grid search	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]
Random search	82.63 ± 0.25	85.19	80.18	97.12 ± 0.01	92.96 ± 0.32	680	8,365	12,317	4,511
TPE	82.76 ± 0.06	85.26	80.37	97.17 ± 0.03	93.08 ± 0.07	870	7,927	12,226	4,911
CMA-ES	82.74 ± 0.04	85.23	80.36	97.15 ± 0.09	92.98 ± 0.09	832	8,003	12,290	4,883
TPE (global)	82.70 ± 0.25	85.16	80.34	97.12 ± 0.11	93.01 ± 0.36	634	7,244	13,496	4,356
PPO-contextual	82.48 ± 0.06	85.29	79.79	97.22 ± 0.02	92.40 ± 0.09	1,198	8,272	11,172	5,652
PPO-contextual (4x budget)	[[PENDING:ext]]	[[PENDING:ext]]	[[PENDING:ext]]	[[PENDING:ext]]	[[PENDING:ext]]	[[PENDING:ext]]	[[PENDING:ext]]	[[PENDING:ext]]	[[PENDING:ext]]
PPO-adaptive	82.57 ± 0.03	85.18	80.07	97.19 ± 0.04	92.86 ± 0.06	489	8,336	12,059	3,936

Figure 5.10 summarises the three headline metrics of both tables as a dot plot with the seed standard deviation as error bars and the default as a dotted reference line. In every panel the default lies far to the left of a tight cluster formed by all tuned methods. In HOTA (panels a and d) the two PPO agents lie at the left edge of the cluster; in MOTA (panels b and e) they lie at its right edge; in IDF1 (panels c and f) PPO-contextual is clearly the lowest tuned method and PPO-adaptive lies between it and the baselines. The figure does not yet contain the grid-search and extended PPO-contextual rows; it will be regenerated with the same layout once these results are available.

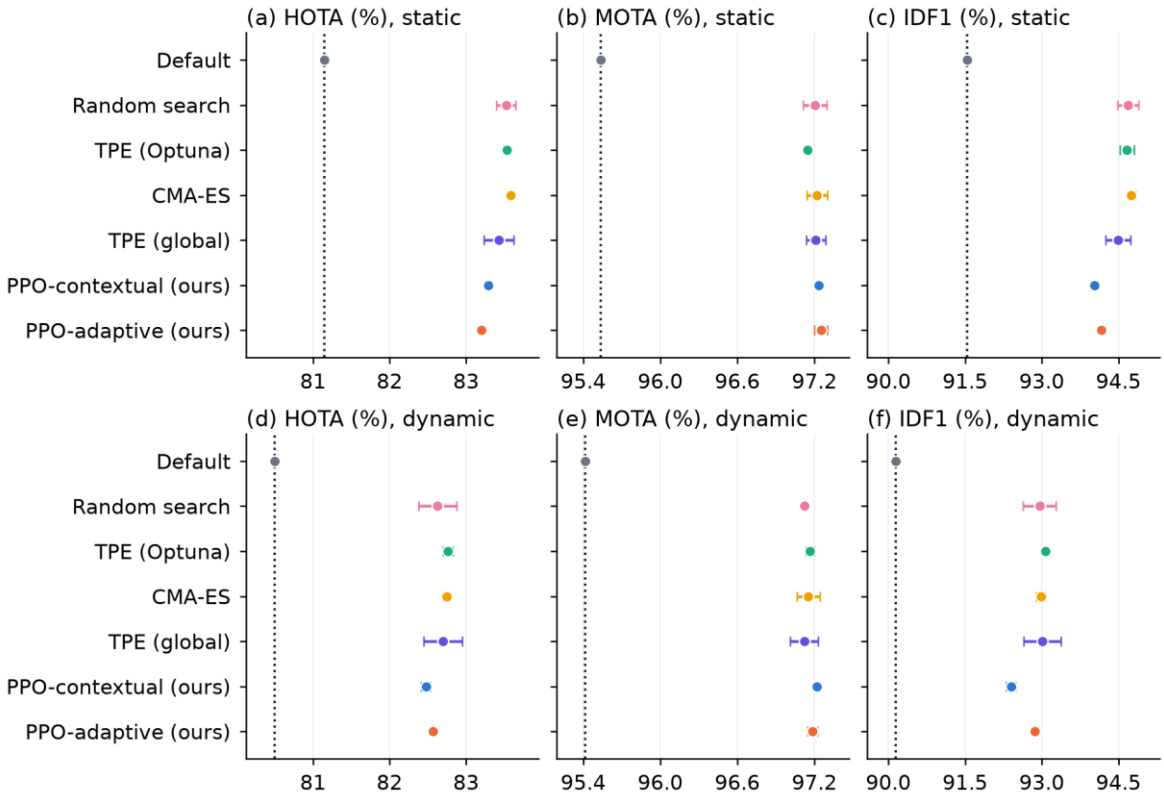


Figure 5.10. Test accuracy of all methods. Top row: static-density test split (48 sequences); bottom row: dynamic-density test split (16 sequences). (a, d) HOTA, (b, e) MOTA, (c, f) IDF1, all in %. Dots: mean over three seeds; error bars: one standard deviation; dotted line: default parameters.

For RQ3, these results show that PPO-contextual, with the same budget of clip evaluations as each per-condition baseline and without access to the condition label, came within 0.23 to 0.29 points HOTA of the per-condition baselines and within 0.13 points of global TPE on the static split. Whether these differences exceed the variation between sequences is tested in Section 5.5.3. For RQ4, re-tuning the parameters every second did not raise HOTA above that of fixed parameters on the dynamic split: PPO-adaptive (82.57 %) lies 0.17 to 0.19 points below TPE and CMA-ES and 0.09 points above PPO-contextual.

5.5.2 Per-Condition Gains

The combined metrics average over very different scenes. Figure 5.11 breaks the HOTA gain over the default down by condition, for the twelve static conditions (top) and the four dynamic conditions (bottom); each cell is the gain of the seed-averaged combined HOTA over the sequences of that condition. The gain depends far more on the condition than on the method. It is largest in the corridor, where it grows with density from 3.2 to 3.9 points at low density to 5.2 points for every method except PPO-adaptive (4.1 points) at

high density, and in the crossing (2.0 to 3.2 points). It is smaller in the bottleneck, where it shrinks with density from 2.4 to 2.8 points at low to at most 0.9 points at high density, and smallest in the plaza, where no method gains more than 1.7 points. The default parameters are thus furthest from the optimum in the corridor, whose default HOTA at high density (77.96 %) is the lowest of all static conditions, and closest to it in the plaza.

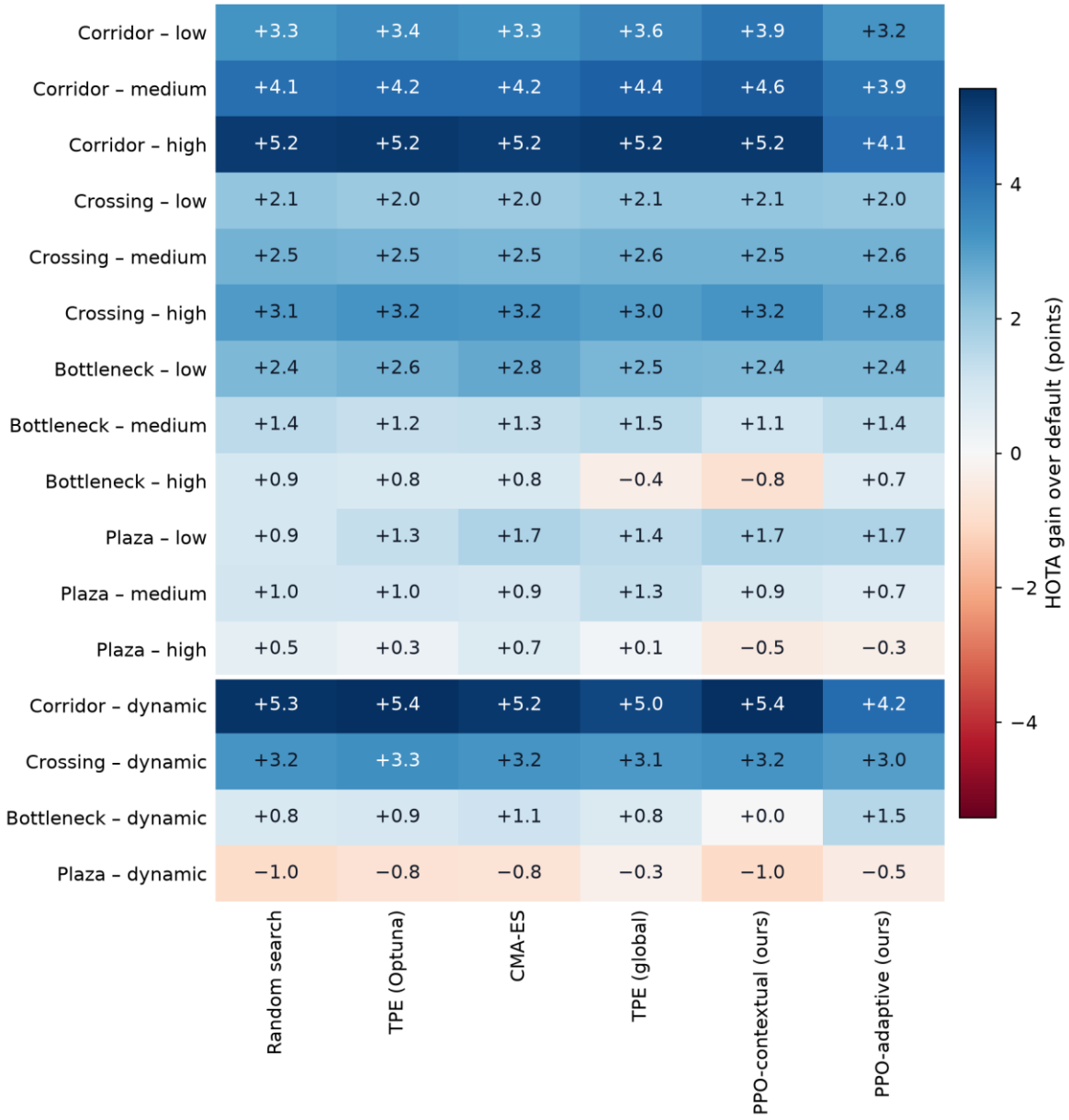


Figure 5.11. HOTA gain over the default (points) per test condition and method. Rows: the twelve static conditions (top block) and the four dynamic conditions (bottom block); columns: methods. Each cell is the difference between the seed-averaged combined HOTA of the method and that of the default over the test sequences of the condition; blue cells mark improvements, red cells deteriorations.

Negative gains occur in only a few cells, and they concentrate on the high-density bottleneck and plaza conditions, two of the most heavily occluded conditions (Table 4.4),

and on the dynamic plaza. On bottleneck–high, PPO-contextual lost 0.83 points and global TPE 0.36 points, while the per-condition baselines, which use parameters tuned for exactly this condition, gained 0.81 to 0.94 points and PPO-adaptive 0.68 points. On plaza–high, PPO-contextual lost 0.49 and PPO-adaptive 0.32 points, while the per-condition baselines gained 0.33 to 0.75 points. On plaza–dynamic every method lost HOTA, between 0.26 points (global TPE) and 1.05 points (PPO-contextual). The PPO-contextual policy is therefore at a disadvantage exactly where the best parameters deviate most from those of the other conditions; Section 5.6 shows that its parameters vary only little between scenes. Conversely, it achieved the largest gains of all methods on corridor–low and corridor–medium (3.94 and 4.55 points), on crossing–high (3.20 points), on plaza–low (1.72 points) and on corridor–dynamic (5.42 points), so its lower overall HOTA results from a few conditions with losses rather than from a uniform shortfall.

The dynamic conditions are the relevant ones for RQ4. On bottleneck–dynamic PPO-adaptive gained 1.54 points, more than any other method (CMA-ES 1.13, PPO-contextual 0.02), and on plaza–dynamic it lost less than the per-condition baselines (−0.45 against −0.76 to −1.00 points). On corridor–dynamic, however, it gained 4.16 points against 4.95 to 5.42 points for the other methods, and on crossing–dynamic 3.00 against 3.13 to 3.31 points. The online control thus helped clearly only in the bottleneck; in the plaza it still lost HOTA, less than the per-condition baselines but more than global TPE (−0.26 points), and in the corridor and the crossing it fell behind all fixed-parameter methods. These effects nearly cancel in the combined result.

5.5.3 Per-Sequence Gains and Significance

Because the combined metrics give crowded sequences more weight and say nothing about the consistency of an improvement, the gain was also evaluated per sequence. Figure 5.12 shows the distribution of the per-sequence HOTA gain over the default (seed-averaged) for each tuned method, pooled over the 64 static and dynamic test sequences. All six boxes lie clearly to the right of zero with medians between about 2.1 and 2.3 points, and the interquartile ranges overlap almost completely. PPO-contextual has the widest distribution: its lower quartile is the lowest and its upper quartile the highest of all methods, and its lowest gain is a loss of about 3 points, whereas the baselines and PPO-adaptive lose at most about 2.2 points on any sequence.

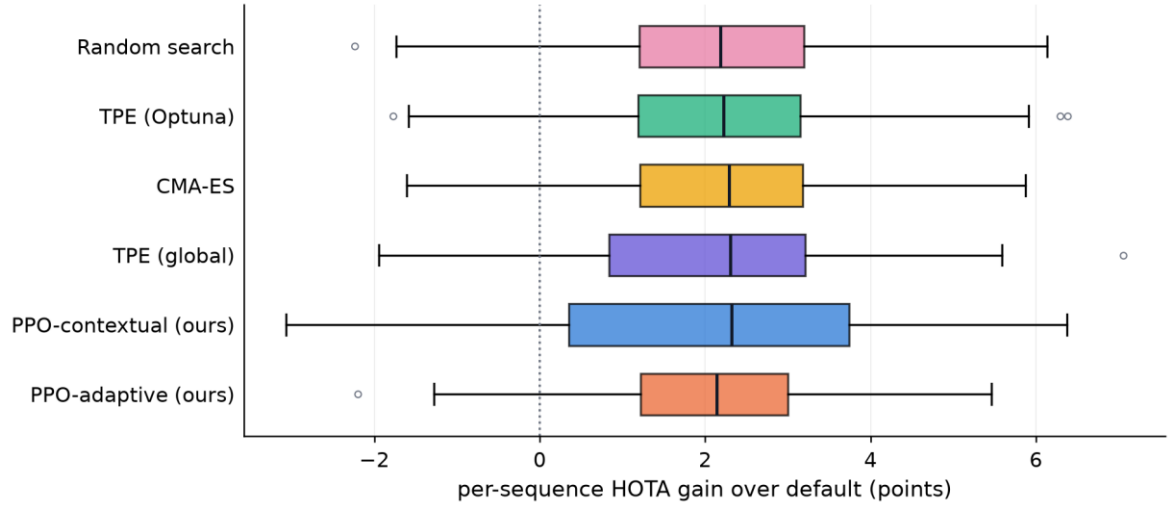


Figure 5.12. Distribution of the per-sequence HOTA gain over the default (points) for each tuned method, pooled over the 48 static and 16 dynamic test sequences; each sequence's HOTA is averaged over the three seeds before the default is subtracted. Boxes: interquartile range with median; whiskers: 1.5 times the interquartile range; circles: outliers; dotted line: no gain.

Table 5.7 tests, separately for each split, whether the per-sequence gains differ from zero, using the paired two-sided Wilcoxon signed-rank test with Holm's adjustment over the methods of a split (Section 4.10). On the static split all methods gained on average between 2.11 points (PPO-adaptive) and 2.37 points (CMA-ES), improved between 83 % (PPO-contextual, 40 of 48 sequences) and 96 % (CMA-ES, 46 of 48) of the sequences, and are significant with Holm-adjusted $p < 0.001$. On the dynamic split, with only 16 sequences, the mean gains lie between 1.94 points (PPO-contextual) and 2.25 points (TPE), and all are significant after adjustment as well, PPO-contextual with the largest adjusted p-value (0.034), having improved 11 of the 16 sequences. The grid-search reference row will be added to this table, and to the family of the Holm adjustment, once it is available.

Table 5.7. Per-sequence HOTA gain over the default (seed-averaged), two-sided Wilcoxon signed-rank test against zero and Holm-adjusted p-values within each split (family-wise $\alpha = 0.05$). All listed results are significant.

Split	Method	n	Mean gain (points)	Median gain (points)	Sequences improved (%)	Wilcoxon p	Holm p
Static	Grid search	48	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]
Static	Random search	48	+2.29	+2.19	93.8	< 0.001	< 0.001
Static	TPE	48	+2.32	+2.28	91.7	< 0.001	< 0.001
Static	CMA-ES	48	+2.37	+2.31	95.8	< 0.001	< 0.001
Static	TPE (global)	48	+2.28	+2.35	87.5	< 0.001	< 0.001
Static	PPO-contextual	48	+2.20	+2.37	83.3	< 0.001	< 0.001
Static	PPO-	48	+2.11	+2.19	89.6	< 0.001	< 0.001

Split	Method	n	Mean gain (points)	Median gain (points)	Sequences improved (%)	Wilcoxon p	Holm p
	adaptive						
Dynamic	Grid search	16	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]	[[PENDING:grid]]
Dynamic	Random search	16	+2.11	+1.87	87.5	0.005	0.013
Dynamic	TPE	16	+2.25	+2.10	81.2	0.003	0.013
Dynamic	CMA-ES	16	+2.24	+2.02	81.2	0.003	0.013
Dynamic	TPE (global)	16	+2.19	+1.94	81.2	0.003	0.013
Dynamic	PPO-contextual	16	+1.94	+1.60	68.8	0.034	0.034
Dynamic	PPO-adaptive	16	+2.09	+2.04	87.5	0.001	0.005

The mean per-sequence gains in Table 5.7 differ slightly from the differences of the combined HOTA in Tables 5.5 and 5.6 because every sequence counts equally here. The ranking is nevertheless similar: CMA-ES and TPE at the top and PPO-adaptive at the bottom on the static split, TPE and CMA-ES at the top and PPO-contextual at the bottom on the dynamic split. On the static split PPO-contextual has the highest median gain (2.37 points) but the lowest share of improved sequences, consistent with the wide distribution in Figure 5.12.

Table 5.8 answers the central comparative question of RQ3 and RQ4: whether the PPO agents differ from the baselines. It reports the per-sequence difference between the HOTA gains of a PPO agent and a baseline (positive values favour the PPO agent), tested with the same paired Wilcoxon test and Holm's adjustment over all pairs within a split.

Table 5.8. Pairwise comparison of the PPO agents with the baselines and with each other: per-sequence difference of the seed-averaged HOTA gains (points; positive = PPO agent better), two-sided Wilcoxon signed-rank test, Holm adjustment over all pairs within a split. Bold: significant at family-wise $\alpha = 0.05$.

Split	PPO agent	Compared with	Mean difference (points)	Median difference (points)	PPO better (%)	Wilcoxon p	Holm p
Static	PPO-contextual	Random search	−0.09	+0.07	56.2	0.851	1.000
Static	PPO-contextual	TPE	−0.12	−0.04	47.9	0.479	1.000
Static	PPO-contextual	CMA-ES	−0.17	−0.09	47.9	0.436	1.000
Static	PPO-contextual	TPE (global)	−0.09	−0.04	45.8	0.401	1.000
Static	PPO-adaptive	Random search	−0.18	−0.13	33.3	0.033	0.200
Static	PPO-adaptive	TPE	−0.21	−0.11	29.2	0.001	**0.009**
Static	PPO-adaptive	CMA-ES	−0.26	−0.11	35.4	0.002	**0.012**

Split	PPO agent	Compared with	Mean difference (points)	Median difference (points)	PPO better (%)	Wilcoxon p	Holm p
Static	PPO-adaptive	TPE (global)	−0.17	−0.08	31.2	0.009	0.063
Static	PPO-adaptive	PPO-contextual	−0.09	−0.10	37.5	0.100	0.500
Dynamic	PPO-contextual	Random search	−0.16	−0.06	50.0	0.348	1.000
Dynamic	PPO-contextual	TPE	−0.31	−0.22	25.0	0.021	0.193
Dynamic	PPO-contextual	CMA-ES	−0.30	−0.09	43.8	0.231	1.000
Dynamic	PPO-contextual	TPE (global)	−0.24	−0.13	50.0	0.231	1.000
Dynamic	PPO-adaptive	Random search	−0.02	−0.02	50.0	0.900	1.000
Dynamic	PPO-adaptive	TPE	−0.16	−0.10	43.8	0.433	1.000
Dynamic	PPO-adaptive	CMA-ES	−0.16	−0.01	50.0	0.495	1.000
Dynamic	PPO-adaptive	TPE (global)	−0.10	−0.11	31.2	0.323	1.000
Dynamic	PPO-adaptive	PPO-contextual	+0.14	+0.06	50.0	0.669	1.000

On the static split PPO-contextual is statistically indistinguishable from all four baselines. Its mean differences are small and negative (−0.09 to −0.17 points), its median difference to Random search is even positive, it is better than the baseline on 46 % to 56 % of the sequences, and no unadjusted p-value is below 0.4. This holds although the per-condition baselines select their parameters with the oracle condition label, which the policy must infer from the first 4 s of detector output. PPO-adaptive, in contrast, is slightly but significantly worse than TPE (−0.21 points, Holm $p = 0.009$) and CMA-ES (−0.26 points, Holm $p = 0.012$) on the static split, and better than them on fewer than 36 % of the sequences; its differences to Random search and global TPE are of similar size but not significant after adjustment, and its difference to PPO-contextual (−0.09 points) is not significant. On static sequences, where there is nothing to adapt to, the online controller thus costs a little accuracy compared with well-tuned fixed parameters.

On the dynamic split no pairwise difference is significant. PPO-contextual lies on average 0.16 to 0.31 points below the baselines, with the largest difference to TPE (unadjusted $p = 0.021$, Holm $p = 0.193$). PPO-adaptive lies 0.02 to 0.16 points below the baselines and 0.14 points above PPO-contextual, which it beats on half of the 16 sequences ($p = 0.669$). With 16 sequences the tests have little power, and differences of a few tenths of a point cannot be resolved; the per-condition analysis of Section 5.5.2 shows that the sign of the difference between PPO-adaptive and the fixed-parameter methods depends on

the scenario. For RQ4 the answer on this test set is therefore negative: online re-tuning every second did not improve HOTA over fixed parameters tuned for the dynamic condition or over the contextual policy. Its measurable benefit lies in the identity-switch count (about 489, against 634 to 1,198; Table 5.6) and the number of tracker identities, not in IDF1, which at 92.86 % is slightly below that of the baselines; this follows from the identity-switch weight of its reward rather than from better HOTA. For RQ3, PPO-contextual matched the budget-matched classical optimisers within the resolution of the tests while using no oracle information, but did not exceed them; its comparison with the high-budget grid search is [[PENDING:grid]], and the effect of four times the training budget is [[PENDING:ext]]. Chapter 6 discusses why reinforcement learning did not outperform the classical optimisers on this problem, in particular the flat, largely shared optimum reflected in the small difference between global TPE and per-condition tuning.

5.6 Analysis of the Learned Policies

Sections 5.4 and 5.5 evaluated the two agents by their scores. This section examines what they actually do, that is, which parameter vectors $\boldsymbol{\theta}$ they select and how these depend on the observed scene. The analysis explains why PPO-contextual behaves almost like a single tuned configuration and why the adaptive controller improves its own window score without improving HOTA over fixed parameters.

5.6.1 Parameters Selected by PPO-Contextual

For every test sequence the deterministic policy of each seed was applied to the scene descriptor of the first 4 s (Section 4.9.1), and the resulting $\boldsymbol{\theta}$ was recorded. Table 5.9 lists the mean over sequences and seeds for each scenario and density level, together with the default $\boldsymbol{\theta}_0$ and the ranges of Table 4.6. Figure 5.13 shows the same means together with the range over seeds and sequences, with the default as a dotted line.

Table 5.9. Mean parameters selected by PPO-contextual (deterministic policy, budget-matched run) per test condition, averaged over test sequences and three seeds, compared with the default $\boldsymbol{\theta}_0$ and the search ranges.

Split	Condition	θ_{nms}	τ_{high}	τ_{low}	τ_{new}	θ_{m1}	θ_{m2}	n_{buf} (frames)
Range (Table 4.6)		[0.40, 0.90]	[0.20, 0.80]	[0.05, 0.40]	[0.20, 0.90]	[0.50, 0.95]	[0.20, 0.90]	[10, 150]
Default $\boldsymbol{\theta}_0$		0.700	0.500	0.100	0.600	0.800	0.500	30.0
Static test	corridor	0.741	0.590	0.241	0.549	0.500	0.445	53.1

Split	Condition	θ_{nms}	τ_{high}	τ_{low}	τ_{new}	θ_{m1}	θ_{m2}	n_{buf} (frames)
Static test	low corridor medium	0.730	0.561	0.241	0.534	0.500	0.446	53.8
Static test	corridor high	0.716	0.529	0.241	0.518	0.534	0.449	55.1
Static test	crossing low	0.739	0.591	0.241	0.549	0.500	0.446	53.3
Static test	crossing medium	0.734	0.581	0.241	0.544	0.500	0.446	53.5
Static test	crossing high	0.723	0.554	0.241	0.528	0.505	0.447	54.1
Static test	bottleneck low	0.739	0.594	0.241	0.551	0.500	0.446	53.4
Static test	bottleneck medium	0.728	0.561	0.241	0.535	0.501	0.448	54.4
Static test	bottleneck high	0.716	0.535	0.240	0.522	0.537	0.450	55.4
Static test	plaza low	0.731	0.587	0.241	0.547	0.500	0.449	53.9
Static test	plaza medium	0.726	0.574	0.241	0.540	0.500	0.449	54.2
Static test	plaza high	0.715	0.552	0.241	0.527	0.512	0.451	55.0
Dynamic test	corridor dynamic	0.736	0.576	0.241	0.542	0.500	0.445	53.5
Dynamic test	crossing dynamic	0.741	0.595	0.241	0.551	0.500	0.445	53.1
Dynamic test	bottleneck dynamic	0.739	0.590	0.241	0.550	0.500	0.446	53.4
Dynamic test	plaza dynamic	0.734	0.592	0.241	0.549	0.500	0.448	53.6

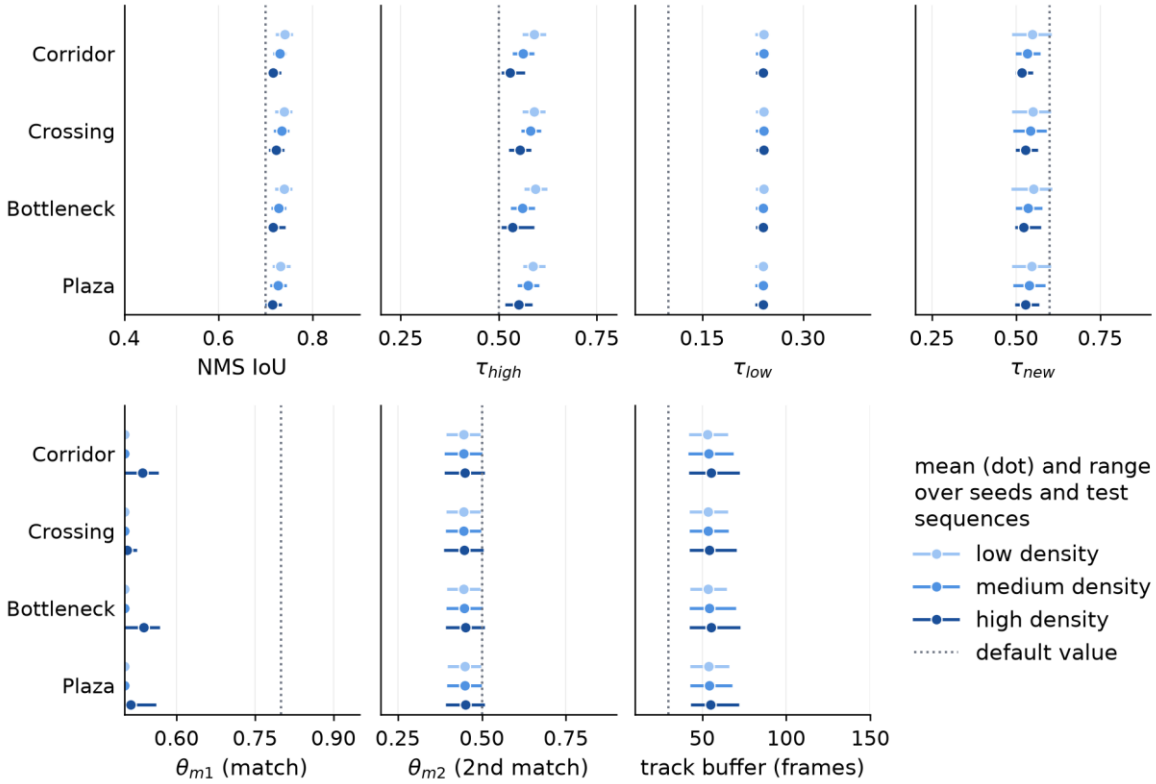


Figure 5.13. Parameters selected by PPO-contextual on the test splits. Each panel shows one component of $\boldsymbol{\theta}$; rows are the four scenarios and the three markers per scenario the low, medium and high static density levels (light to dark blue). Dots are means and horizontal bars the range over the three seeds and the test sequences of the condition; the dotted vertical line marks the default value. The horizontal axes span the search ranges of Table 4.6.

Three properties stand out. First, the policy moves every parameter away from the default in the same direction for all conditions. It raises the NMS threshold slightly (means 0.715 to 0.741, which the post-hoc NMS grid of Section 4.6 maps to 0.70 or 0.75), raises the high-score threshold τ_{high} from 0.50 to between 0.529 and 0.595, lowers the new-track threshold τ_{new} from 0.60 to between 0.518 and 0.551, lowers the second-association limit θ_{m2} from 0.50 to about 0.45, raises the low-score threshold τ_{low} from 0.10 to 0.24 and extends the track buffer from 30 to between 53 and 55 frames. With the lost-track retention of Section 4.7.1 a buffer of 53 to 55 retains lost tracks for about 1.8 s instead of 1 s.

Second, the largest change concerns the first-association cost limit θ_{m1}. The default of 0.80 accepts a first-stage match down to a fused IoU of about 0.2; the policy chooses the lower bound of the range, 0.50, for all low- and medium-density and all dynamic conditions, and only in the high-density conditions does its mean rise slightly above the bound (0.505 to 0.537). Since sampled actions are clipped to the cube (Section 4.9.1), a mean exactly at 0.500 with zero spread means that the policy's mean action is at or beyond

the bound, so the policy would have chosen a still stricter limit if the range had allowed it. The policy thus saturates at a range bound in one of seven dimensions. The optimisers point the same way: the best per-condition θ_{m1} found by TPE lies between 0.503 and 0.678 in all 16 conditions, and the one-at-a-time sensitivity analysis of Section 5.3 found its best value at 0.59, the parameter with the second-largest HOTA span after the NMS threshold. The other six parameters stay well inside their ranges; in particular $\tau_{\mathrm{low}} = 0.24$ is near the middle of [0.05, 0.40].

Third, the dependence on the scene is weak and almost entirely a function of density. From low to high density the policy lowers θ_{nms} by about 0.02, lowers τ_{high} by about 0.04 to 0.06 and τ_{new} by about 0.02 to 0.03, and raises θ_{m1} and n_{buf} slightly, consistently in all four scenarios. Differences between scenarios at the same density are mostly about 0.01 and at most about 0.025 (τ_{high} at high density), smaller than the spread within a condition visible in Figure 5.13, and τ_{low} and θ_{m2} are practically constant. The policy has therefore learned one shared configuration with a small density correction rather than scenario-specific settings. This is consistent with Section 5.5, where global TPE, which uses a single $\boldsymbol{\theta}$ for all conditions, was statistically indistinguishable from the per-condition baselines and from PPO-contextual, and with the scattered per-condition TPE optima of Section 5.3 for the insensitive parameters (for example a best track buffer anywhere between 10 and 143 frames): where the objective is flat, a condition-specific choice carries little information.

The four dynamic-test rows resemble the static low-density rows, the corridor row being slightly lower (for example τ_{high} between 0.576 and 0.595 against 0.587 to 0.594). This is the expected consequence of the test-time rule of Section 4.9.1: the descriptor is computed in the first 4 s, which in a dynamic sequence always fall into the low-rate phase of Equation (4.2), so the policy configures the tracker for low density and keeps that configuration through the pulse. The configurations it chooses for high density differ only moderately from this, which limits the cost of the rule; Section 5.5 nevertheless showed that PPO-contextual had the smallest gain of all tuned methods on the dynamic split, with no gain in the bottleneck and a loss in the plaza.

Finally, the strict first-association limit offers a plausible explanation for the large number of identity switches of PPO-contextual reported in Tables 5.5 and 5.6 and analysed in Section 5.8. A track whose predicted box overlaps its pedestrian's detection with an IoU below about 0.5, as happens after abrupt changes of direction or partial occlusion in a

crowd, is no longer matched in the first stage, and may be lost and re-initialised under a new identity. Chapter 6 returns to this point.

5.6.2 Behaviour of PPO-Adaptive on a Dynamic Sequence

PPO-adaptive changes $\boldsymbol{\theta}$ every second. To show what it does within a sequence, the deterministic policy of seed 0 was run on the dynamic test sequence bottleneck_dynamic_200 while the parameters and the window scores of Equation (4.10) were recorded, and the default parameters were run on the same sequence for comparison. Figure 5.14 shows the trace.

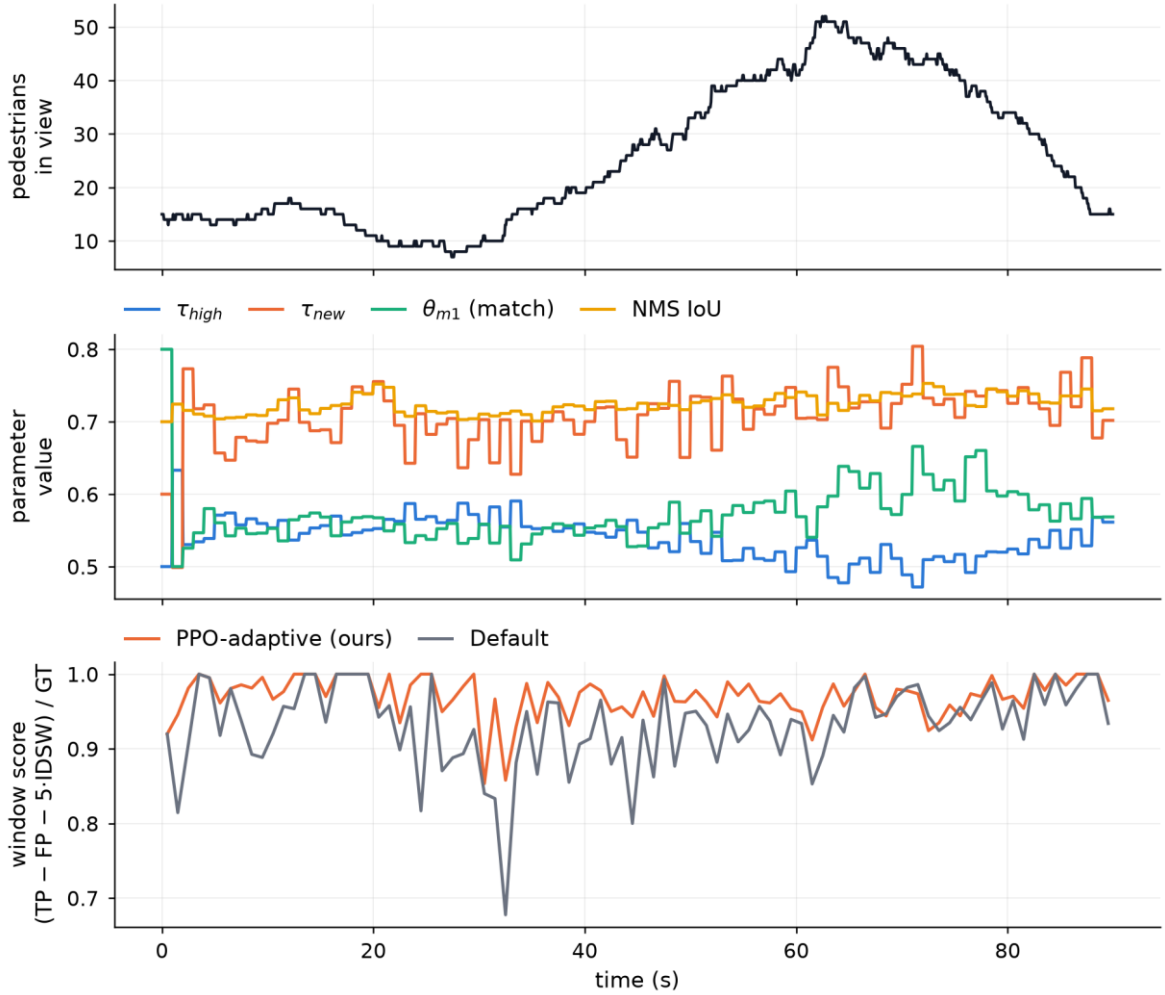


Figure 5.14. PPO-adaptive (seed 0) on the dynamic-density test sequence bottleneck_dynamic_200 (90 s, 90 windows of 1 s). Top: number of evaluated pedestrians in view per frame. Middle: four of the seven parameters chosen by the policy (τ_{high}, τ_{new}, θ_{m1} and the NMS IoU threshold); the first window uses the default. Bottom: window score $(\text{TP} - \text{FP} - 5\,\text{IDSW})/\text{GT}$ of Equation (4.10) for the policy run and for the default run.

The number of pedestrians in view follows the rush-hour pulse of Equation (4.2) with a delay, because pedestrians need time to reach the field of view and to pass the gate: it is

low during the first 30 s, rises to its maximum after about 60 s and falls again towards the end. The parameters react to this pattern. Over the sequence the policy kept τ_{high} between 0.47 and 0.63, τ_{new} between 0.50 and 0.80 and θ_{m2} between 0.37 and 0.58. The first window always uses the default $\boldsymbol{\theta}_0$, so the extreme values 0.10 of τ_{low}, 30 frames of n_{buf}, 0.70 of θ_{nms} and 0.80 of θ_{m1} are defaults, not choices of the policy; after the first window the policy keeps τ_{low} between 0.18 and 0.26, n_{buf} between about 56 and 87 frames, θ_{nms} between 0.70 and 0.75 and θ_{m1} between the lower bound 0.50 and about 0.67. During the high-density phase the policy lowers τ_{high} and raises θ_{m1}, that is, it admits more detections to the first association and relaxes the overlap it requires, and it returns towards its low-density settings as the crowd thins. For τ_{high} and θ_{m1} the direction agrees with the density correction of PPO-contextual in Table 5.9, with a larger amplitude. For τ_{new} it does not: the adaptive agent keeps it mostly above the default (about 0.63 to 0.80 in Figure 5.14), whereas PPO-contextual lowers it below the default (0.518 to 0.551), and its window-to-window changes are partly erratic.

The policy's mean window score on this sequence is 0.9696 against 0.9322 for the default run, a difference of 0.0374. The bottom panel shows where the difference arises: the default run has several sharp drops, to between about 0.80 and 0.85 and once to about 0.68 at about 32 s, mainly before the density peak, which the policy run avoids or reduces (it also dips, to about 0.86, around 30 to 33 s), while in the windows of the pulse the two curves are close. The improvement in the reward thus comes from avoided error bursts in individual windows rather than a systematic advantage at high density. This sequence is an illustration, not a test: it belongs to the bottleneck dynamic condition, the only dynamic condition in which PPO-adaptive gained more over the default than every baseline, 1.54 HOTA points against 0.91 for TPE and 1.13 for CMA-ES (Section 5.5.2). Over the whole dynamic split the adaptive agent did not improve on fixed parameters: its HOTA of 82.57 % is below those of TPE (82.76 %), CMA-ES (82.74 %) and global TPE (82.70 %), and none of the pairwise differences is significant (Section 5.5.3). The trace therefore shows that the controller reacts to density in a sensible direction and improves the windowed quantity it was trained on, but this does not translate into a better sequence-level HOTA than a well-tuned fixed $\boldsymbol{\theta}$. Section 5.8 shows the one systematic effect of its reward, the low number of identity switches.

5.7 Influence of the Detector on Tracking

All tuning and RL experiments use the fine-tuned YOLOv8s (Section 4.6). Since the tracker parameters act on the detector's boxes and scores, their optimal values and the benefit of tuning may depend on the detector. This section examines how strongly the tracking accuracy depends on the detector, and whether parameters tuned or a policy trained with YOLOv8s transfer to other detectors.

The study tracks the 48 sequences of the static test split with the detections of four models: the fine-tuned YOLOv8n, YOLOv8s and YOLOv8m and the COCO-pretrained YOLOv8s applied zero-shot with the class "person" (Section 5.2). For every model the detections are produced and cached with the same inference settings (confidence floor 0.05, loose NMS at 0.9, post-hoc NMS masks; Section 4.6), so that only the detector differs. Each detector's detections are tracked with three parameter choices. The first is the default $\boldsymbol{\theta}_0$. The second is the global TPE configuration, taken from the seed with the best objective value, which was tuned with YOLOv8s detections and is applied unchanged. The third is the budget-matched PPO-contextual policy, also trained with YOLOv8s, of all three seeds. The policy is not retrained: it observes the scene descriptor of Table 4.7 computed from the new detector's detections in the first 4 s of each sequence, and its $\boldsymbol{\theta}$ is applied to the whole sequence as in Section 4.9.1. The YOLOv8s rows of Table 5.10 therefore reproduce the static-test rows of Table 5.5 (Default 81.15 %, TPE (global) 83.42 %, PPO-contextual 83.29 % HOTA) and serve as a consistency check of the pipeline. This is a test of cross-detector transfer, because a different detector changes the distribution of the descriptor (for example the confidence statistics) and the response of the tracker to $\boldsymbol{\theta}$. Metrics are combined over the 48 sequences as in Section 4.8, and the PPO-contextual values are means over the three seeds. Figure 5.15 shows HOTA and IDF1 for each detector and parameter choice, and Table 5.10 lists the metrics.

Figure 5.15. Influence of the detector on tracking accuracy on the static test split. Grouped bars of HOTA (left) and IDF1 (right) for the zero-shot YOLOv8s and the fine-tuned YOLOv8n, YOLOv8s and YOLOv8m, each tracked with the default parameters, the global TPE parameters and the PPO-contextual policy (mean over three seeds); TPE and the policy were tuned or trained with the fine-tuned YOLOv8s.

Figure 5.15. Influence of the detector on tracking accuracy on the static test split. Grouped bars of HOTA (left) and IDF1 (right) for the zero-shot YOLOv8s and the fine-tuned YOLOv8n, YOLOv8s and YOLOv8m, each tracked with the default parameters, the global TPE parameters and the PPO-contextual policy (mean over three seeds); TPE and the policy were tuned or trained with the fine-tuned YOLOv8s.

Table 5.10. Tracking accuracy on the static test split (48 sequences, combined metrics) with different detectors. TPE (global) and PPO-contextual were tuned or trained with the fine-tuned YOLOv8s and are applied without re-tuning; PPO-contextual values are means over three seeds.

Detector	Parameters	HOTA (%)	DetA (%)	AssA (%)	MOTA (%)	IDF1 (%)	IDSW
YOLOv8s zero-shot	Default	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]
YOLOv8s zero-shot	TPE (global)	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]
YOLOv8s zero-shot	PPO-contextual	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]
YOLOv8n	Default	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]
YOLOv8n	TPE (global)	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]
YOLOv8n	PPO-contextual	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]
YOLOv8s	Default	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]
YOLOv8s	TPE (global)	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]
YOLOv8s	PPO-contextual	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]
YOLOv8m	Default	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]
YOLOv8m	TPE (global)	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]
YOLOv8m	PPO-contextual	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]	[[PENDING:detstudy]]

The effect of the detector on tracking is large compared with the effect of the tracker parameters. With default parameters, HOTA ranges from [[PENDING:detstudy]] % with the zero-shot YOLOv8s to [[PENDING:detstudy]] % with the fine-tuned YOLOv8m, so transfer learning raises the tracking HOTA by [[PENDING:detstudy]] points for the YOLOv8s architecture, whereas tuning the parameters with YOLOv8s detections gains about 2 points (Section 5.5). Among the fine-tuned models the differences are [[PENDING:detstudy]], in line with the ordering of the detection AP in Table 5.2.

The parameters tuned with YOLOv8s transfer to the other fine-tuned detectors [[PENDING:detstudy]]: global TPE changes HOTA relative to the default by [[PENDING:detstudy]] points with YOLOv8n and by [[PENDING:detstudy]] points with

YOLOv8m, and the PPO-contextual policy by [[PENDING:detstudy]] and [[PENDING:detstudy]] points. For the zero-shot detector, whose scores and recall differ most from those of the training detector (Section 5.2.3), the change is [[PENDING:detstudy]] points with global TPE and [[PENDING:detstudy]] points with PPO-contextual. [[PENDING:detstudy]] The study therefore indicates [[PENDING:detstudy]], a point that Chapter 6 takes up in the discussion of RQ1.

5.8 Error Analysis

The aggregate scores of Section 5.5 hide which kinds of error the tuning removes and which it introduces. This section decomposes the HOTA gain over localisation thresholds and into detection and association, relates it to the CLEAR error counts, and shows typical frames.

5.8.1 Decomposition of HOTA

HOTA is the mean of $\mathrm{HOTA}_\alpha = \sqrt{\mathrm{DetA}_\alpha \, \mathrm{AssA}_\alpha}$ over the localisation thresholds α (Section 4.8). Figure 5.16 shows HOTA_α for the default, TPE and the two PPO agents, averaged over the 48 static test sequences, and the pair of DetA and AssA for every sequence and seed.

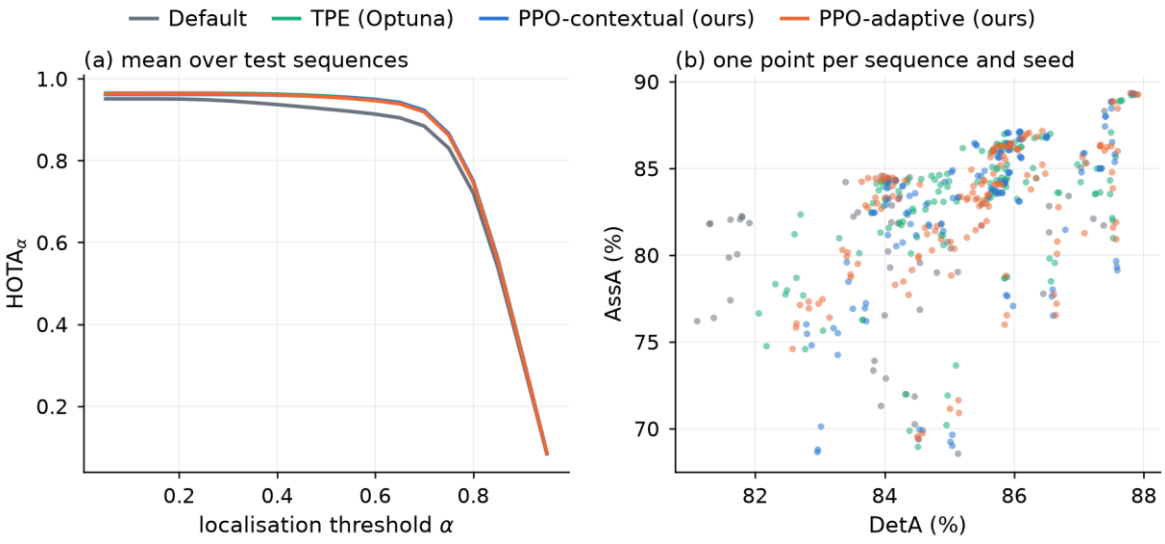


Figure 5.16. Decomposition of HOTA on the static test split for Default, TPE and the two PPO agents. (a) HOTA_α over the localisation threshold α, mean over the 48 test sequences (and the three seeds of each tuned method). (b) DetA against AssA, one point per test sequence and seed (one point per sequence for the deterministic default).

In panel (a) the three tuned methods produce almost identical curves, which lie above the default curve for all thresholds up to about $\alpha = 0.8$; above that the curves converge and

at $\alpha = 0.95$ they coincide. The gain is therefore not a gain in localisation precision: the tracked boxes are not placed more accurately. It comes from more correctly detected and associated pedestrians at loose and moderate overlap requirements, where the default loses objects through false positives, misses and broken trajectories. The steep fall of all curves above $\alpha \approx 0.7$ is common to all methods and is set by the detector's box accuracy.

Panel (b) separates detection and association. The default points are spread over the whole DetA range (about 81 to 87 %), and the tuned methods shift the cloud as a whole to the right and upwards; on the hardest sequences, however, some tuned points reach a lower AssA (down to about 69 %) than the lowest default points (about 71 %), so tuning lowers AssA on a few sequences; the combined values in Table 5.5 quantify this. On the static split TPE raises DetA from 84.01 % to 85.07 % (+1.06 points) and AssA from 78.42 % to 82.05 % (+3.63 points); PPO-contextual raises them to 85.16 % (+1.15 points) and 81.49 % (+3.07 points), and PPO-adaptive to 85.15 % (+1.14 points) and 81.34 % (+2.92 points). The HOTA gain of about 2 points thus comes more from association than from detection, and the PPO agents reach the highest DetA but a lower AssA than TPE, CMA-ES (82.10 %) and random search (82.01 %), which accounts for their slightly lower HOTA. The scatter also shows a group of sequences with AssA around 70 % for every method, including the tuned ones, so the hardest sequences remain hard regardless of the parameters.

5.8.2 The Trade-Off between FP/FN and Identity Switches

The CLEAR counts of Tables 5.5 and 5.6 reveal what the gain in DetA and AssA costs. All relative changes below are computed from the seed means of these tables.

Every tuned method removes about half of the false positives (between 45.3 % and 55.9 % on the static and between 48.9 % and 55.7 % on the dynamic split) and about a quarter to a third of the misses (between 22.3 % and 34.9 % and between 22.5 % and 35.8 %, respectively). On the static split the default produces 32,760 false positives and 33,514 misses; TPE reduces them to about 17,747 and 23,448, and PPO-contextual to about 17,349 and 21,812, so the sum of the two falls from 66,274 to about 41,195 (−37.8 %) and 39,161 (−40.9 %), respectively. The reduction of false positives presumably follows mainly from the higher score thresholds and the stricter matching, which stop low-confidence boxes from starting or extending tracks; the reduction of misses from the longer track buffer and the larger low-score threshold, which keep occluded pedestrians' tracks

alive and associate their weak detections. MOTA, which counts these errors, rises from 95.54 % to between 97.15 % and 97.25 %, and the number of mostly lost trajectories falls from 39 to about 15 to 18.

The price is a several-fold increase of identity switches. The default has only 508 identity switches on the static split. The per-condition baselines have 2.5 to 2.9 times as many (about 1,279 to 1,457), PPO-contextual 4.3 times as many (about 2,193), and the number of tracker identities rises from 6,631 (for 6,485 ground-truth identities) to between about 9,825 (global TPE) and 12,292 (PPO-contextual) for the fixed-parameter methods and PPO-contextual, 1.48 to 1.85 times the default (8,916, or 1.34 times, for PPO-adaptive). The tuned parameters therefore split many trajectories into several tracks, and fragmentations rise accordingly (from 2,200 to about 3,485 for PPO-contextual). The dynamic split shows the same pattern, with 264 identity switches for the default, 2.4 to 3.3 times as many for the baselines and 4.5 times as many (about 1,198) for PPO-contextual. Because false positives and misses remain far more frequent than identity switches (about 18 times for PPO-contextual and 130 times for the default), the net effect on the error count and on HOTA is clearly positive: on the static split the roughly 27,100 fewer FP and FN of PPO-contextual outweigh about 1,685 additional identity switches. IDF1 still rises (from 91.54 % to 94.03 % for PPO-contextual and 94.66 % for TPE), because IDF1 measures the share of correctly identified detections and is dominated by the many more matched detections. The increase of AssA in Section 5.8.1 has a similar explanation: AssA is computed over the matched detections, and the many short identity breaks apparently reduce it less than the default loses through associations to false or poorly overlapping boxes. For applications in which the number of distinct identities matters, for example counting pedestrians, the tuned configurations are nevertheless worse than the default, which Chapter 6 discusses.

The methods differ mainly in where they place themselves on this trade-off. Global TPE removes the most false positives (−55.9 %) but the fewest misses (−22.3 %). PPO-contextual removes the most misses and has by far the most identity switches, consistent with its strict first-association limit (Section 5.6.1). PPO-adaptive has the fewest identity switches of all tuned methods, 911 on the static and 489 on the dynamic split, 1.8 times the default in both cases, and the fewest tracker identities (8,916 on the static split, 1.34 times the default). This is the one systematic effect of its reward, which weights each identity switch five times (Equation (4.10)): it trades a little HOTA (Section 5.5) for markedly

better identity preservation. The objective of the other methods, HOTA, does not penalise identity switches explicitly, and they exploit the trade-off fully.

5.8.3 Qualitative Comparison

Figure 5.17 shows the per-frame outcome of the default and of PPO-contextual on two crowded static test sequences, bottleneck_high_100 and corridor_high_101. To avoid selecting favourable frames by eye, the frame is chosen by a fixed rule: in each sequence, all frames are ranked by the reduction of the error count (false positives plus missed pedestrians, CLEAR matching at IoU ≥ 0.5 with the ignore protocol of Section 4.4.3) from the default to PPO-contextual, and the frame at the 90th percentile of this reduction is shown. Both selected frames (42 and 40) lie in the first 3 s of their sequences, where track initialisation dominates, so they illustrate the start-up phase rather than steady-state tracking. The parameters are those chosen by the PPO-contextual policy for the respective sequence.

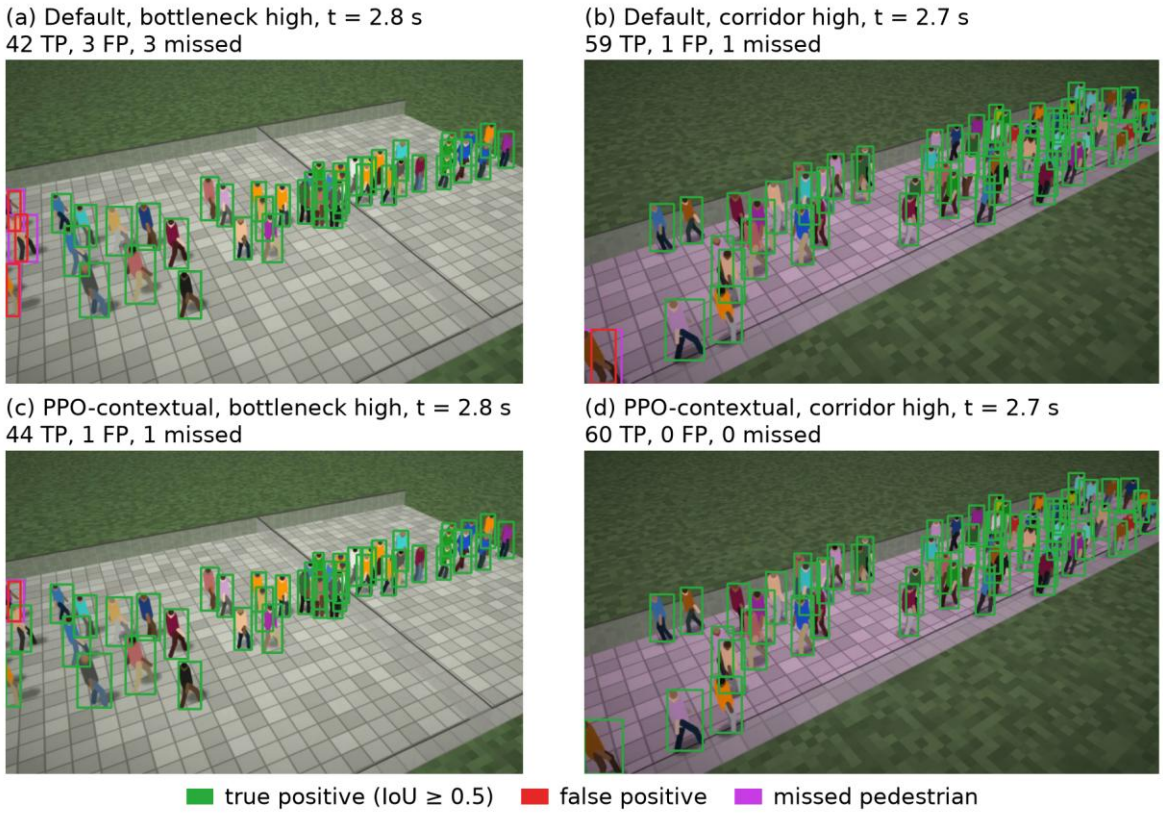


Figure 5.17. Default (top) and PPO-contextual (bottom) on two high-density static test sequences, at the frame at the 90th percentile of the per-frame reduction of FP plus missed pedestrians: (a, c) bottleneck_high_100 at t = 2.8 s (frame 42), (b, d) corridor_high_101 at t = 2.7 s (frame 40). Green: true-positive track boxes (IoU ≥ 0.5); red: false-positive track boxes; magenta: ground-truth boxes of missed pedestrians. Counts per frame are given in the panel titles.

In the bottleneck frame the default produces 42 true positives, 3 false positives and 3 missed pedestrians, and PPO-contextual 44, 1 and 1, a reduction of four errors. The errors of the default lie at the left image border, where pedestrians enter the field of view partly cut off and partly occluded, and where its track boxes do not overlap the ground truth sufficiently. In the corridor frame the default has 59 true positives, 1 false positive and 1 missed pedestrian at the lower-left border, and PPO-contextual 60, 0 and 0. Because of the percentile rule these frames show a large improvement by construction; the sequence-level statistics put them in context. Over the 900 frames of bottleneck_high_100, PPO-contextual reduces the error count by 1.35 per frame on average, reduces it in 60.8 % of the frames and increases it in 7.7 %. In corridor_high_101 the mean reduction is 0.38 per frame, with 37.1 % of the frames improved and 19.3 % worse. The visible effect of tuning is thus a modest, frequent removal of single errors, often at the image border or in occlusion clusters, rather than a dramatic change in a few frames. Identity switches are not visible in single frames and are covered by Section 5.8.2.

5.9 Computational Cost

The memoisation of Chapter 4 separates the expensive one-off stages (simulation, rendering, detection) from the cheap stage that is repeated for every parameter evaluation (tracking and evaluation of cached detections). Table 5.11 lists the measured run times on the workstation of Table 4.11.

Table 5.11. Measured run times. Detection times are means over the processed videos; training times are wall-clock times.

Stage	Unit	Hardware	Time
Simulation and rendering	per 60 s sequence	CPU	[[PENDING:timing]]
YOLOv8n transfer learning	25 epochs	GPU	29.1 min
YOLOv8s transfer learning	25 epochs	GPU	53.7 min
YOLOv8m transfer learning	25 epochs	GPU	119.4 min
Detection, YOLOv8n	per frame (batch 16, FP16, 960 px)	GPU	11.69 ms
Detection, YOLOv8s	per frame (batch 16, FP16, 960 px)	GPU	24.30 ms
Detection, YOLOv8m	per frame (batch 16, FP16, 960 px)	GPU	14.49 ms
Detection, YOLOv8s zero-shot	per frame (batch 16, FP16, 960 px)	GPU	17.18 ms
Tracking and evaluation of cached detections	per 20 s clip (300 frames)	CPU, 1 process	[[PENDING:timing]]
HPO run (random search, TPE or CMA-ES)	per condition and seed (200 evaluations)	CPU	[[PENDING:timing]]

Stage	Unit	Hardware	Time
Grid search	per condition (2,187 evaluations)	CPU	[[PENDING:timing]]
PPO-contextual training (budget-matched)	per seed (13,056 steps)	CPU, 24 environments	7.3–8.1 min (mean 7.7 min)
PPO-contextual training (4x budget)	per seed (51,456 steps)	CPU, 24 environments	[[PENDING:ext]]
PPO-adaptive training	per seed (301,056 steps)	CPU, 24 environments	13.1–13.7 min (mean 13.5 min)
Policy inference	per decision	CPU	[[PENDING:timing]]

Transfer learning of the detectors took between half an hour and two hours for 25 epochs; YOLOv8m was trained with batch 8 because batch 16 exceeded the GPU memory. The detection times are means over the videos each model processed, which differ in number (127 videos for YOLOv8s, the detector of all experiments, but 13 for YOLOv8m and 48 for the other two models) and were measured in separate runs. They are therefore not directly comparable across models; in particular, the lower time of YOLOv8m than of YOLOv8s does not reflect the relative cost of the architectures. All models run well below the 66.7 ms per frame of the 15 FPS video, so detection could run in real time on this GPU. Simulation and rendering take [[PENDING:timing]] per 60 s sequence.

Once detections are cached, one evaluation of a parameter vector on a 20 s clip takes [[PENDING:timing]], so a per-condition HPO run of 200 evaluations on four clips takes [[PENDING:timing]] and the grid search [[PENDING:timing]] per condition. Training PPO-contextual with the budget of 13,056 clip evaluations took 7.7 min per seed on average with 24 parallel environments, and PPO-adaptive, with 301,056 decisions, 13.5 min per seed; with four times the budget, PPO-contextual took [[PENDING:ext]]. Since each update of the small policy networks is cheap, these times are expected to be dominated by the environment, that is, by tracking and evaluation; Table 5.11 is to confirm this ([[PENDING:timing]]). At deployment the PPO-contextual policy needs one forward pass per sequence after the 4 s descriptor window, and PPO-adaptive one per second; a forward pass takes [[PENDING:timing]], which is negligible compared with detection. In terms of computing cost the RL agents have no disadvantage at test time, whether their training is cheaper or dearer than the classical optimisers at equal budget can only be judged once the HPO run times of Table 5.11 are available ([[PENDING:timing]]), although both spend most of their effort in the same tracking and evaluation step; the advantage of a trained contextual policy, selecting parameters for a new scene without any search, is only relevant if it selects better parameters, which Section 5.5 did not show.

5.10 Summary

Transfer learning on the synthetic frames turned a COCO-pretrained YOLOv8 into a near-saturated detector on the synthetic test split, with the remaining errors concentrated on strongly occluded pedestrians (Section 5.2). The tracking accuracy of YOLOv8 + ByteTrack depends on the parameters mainly through the NMS threshold and the first-association limit, and its optimum varies only weakly between scenarios and densities (Section 5.3). On the test splits every tuned method improved HOTA over the default by about 2 points, for example from 81.15 % to 83.58 % with CMA-ES on the static split, and all gains over the default are significant. Among the tuned methods the differences are a few tenths of a point. PPO-contextual (83.29 %) is statistically indistinguishable from random search, TPE, CMA-ES and global TPE, although the per-condition baselines use the oracle condition label; PPO-adaptive (83.20 %) is slightly but significantly below TPE and CMA-ES on the static split; and online re-tuning every second did not improve on fixed parameters on the dynamic split (82.57 % against 82.76 % for TPE). The grid-search reference [[PENDING:grid]], and PPO-contextual with four times the budget [[PENDING:ext]].

The analyses of this chapter explain these results. PPO-contextual learned essentially one configuration with a small density correction, close to the configuration found by the optimisers and saturated at the lower bound of the first-association limit, which is expected when a single configuration is nearly optimal everywhere (Section 5.6). The adaptive controller reacts to density in a sensible direction and improves its windowed reward, but not the sequence-level HOTA. The gain of tuning is a gain in detection and association at loose overlap thresholds: about half of the false positives and a quarter to a third of the misses disappear, at the cost of 2.5 to 4.3 times as many identity switches on the static split; PPO-adaptive, whose reward penalises identity switches, limits this increase to a factor of 1.8 (Section 5.8). The influence of the detector on tracking is [[PENDING:detstudy]] (Section 5.7), and the computing cost of training and applying the policies is reported in Section 5.9 ([[PENDING:timing]]). Chapter 6 interprets these findings with respect to the research questions RQ1 to RQ4.

References

[1] D. Helbing, L. Buzna, A. Johansson, and T. Werner, “Self-organized pedestrian crowd dynamics: Experiments, simulations, and design solutions,” *Transportation Science*, vol. 39, no. 1, pp. 1–24, 2005, doi: 10.1287/trsc.1040.0108.

[2] D. Helbing and A. Johansson, “Pedestrian, crowd and evacuation dynamics,” in *Encyclopedia of Complexity and Systems Science*, R. A. Meyers, Ed., New York: Springer, 2009, pp. 6476–6495. doi: 10.1007/978-0-387-30440-3_382.

[3] D. Helbing, I. Farkas, and T. Vicsek, “Simulating dynamical features of escape panic,” *Nature*, vol. 407, no. 6803, pp. 487–490, 2000, doi: 10.1038/35035023.

[4] D. Helbing and P. Mukerji, “Crowd disasters as systemic failures: Analysis of the Love Parade disaster,” *EPJ Data Science*, vol. 1, no. 1, p. 7, 2012, doi: 10.1140/epjds7.

[5] G. Ananthanarayanan *et al.*, “Real-time video analytics: The killer app for edge computing,” *Computer*, vol. 50, no. 10, pp. 58–67, 2017, doi: 10.1109/MC.2017.3641638.

[6] A. Fuller, Z. Fan, C. Day, and C. Barlow, “Digital twin: Enabling technologies, challenges and open research,” *IEEE Access*, vol. 8, pp. 108952–108971, 2020, doi: 10.1109/ACCESS.2020.2998358.

[7] F. Dembski, U. Wössner, M. Letzgus, M. Ruddat, and C. Yamu, “Urban digital twins for smart cities and citizens: The case study of Herrenberg, Germany,” *Sustainability*, vol. 12, no. 6, p. 2307, 2020, doi: 10.3390/su12062307.

[8] A. Seyfried, B. Steffen, W. Klingsch, and M. Boltes, “The fundamental diagram of pedestrian movement revisited,” *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2005, no. 10, p. P10002, 2005, doi: 10.1088/1742-5468/2005/10/P10002.

[9] A. Johansson, D. Helbing, and P. K. Shukla, “Specification of the social force pedestrian model by evolutionary adjustment to video tracking data,” *Advances in Complex Systems*, vol. 10, no. supp02, pp. 271–288, 2007, doi: 10.1142/S0219525907001355.

[10] T. Li, B. Xu, W. Lu, Z. Chen, S. Zhang, and F. Xia, “The parameter calibration of social force model for pedestrian flow simulation based on YOLOv5,” *Sensors*, vol. 24, no. 15, p. 5011, 2024, doi: 10.3390/s24155011.

[11] C. Niu, W. Wang, H. Guo, and K. Li, “Emergency evacuation simulation study based on improved YOLOv5s and Anylogic,” *Applied Sciences*, vol. 13, no. 9, p. 5812, 2023, doi: 10.3390/app13095812.

[12] G. Ciaparrone, F. Luque Sánchez, S. Tabik, L. Troiano, R. Tagliaferri, and F. Herrera, “Deep learning in video multi-object tracking: A survey,” *Neurocomputing*, vol. 381, pp. 61–88, 2020, doi: 10.1016/j.neucom.2019.11.023.

[13] W. Luo, J. Xing, A. Milan, X. Zhang, W. Liu, and T.-K. Kim, “Multiple object tracking: A literature review,” *Artificial Intelligence*, vol. 293, p. 103448, 2021, doi: 10.1016/j.artint.2020.103448.

[14] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788. doi: 10.1109/CVPR.2016.91.

[15] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, “A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS,” *Machine Learning and Knowledge Extraction*, vol. 5, no. 4, pp. 1680–1716, 2023, doi: 10.3390/make5040083.

[16] T.-Y. Lin *et al.*, “Microsoft COCO: Common objects in context,” in *Computer Vision – ECCV 2014*, in Lecture Notes in Computer Science, vol. 8693. Springer, 2014, pp. 740–755. doi: 10.1007/978-3-319-10602-1_48.

[17] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8.” GitHub repository, 2023. Available: https://github.com/ultralytics/ultralytics

[18] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960, doi: 10.1115/1.3662552.

[19] H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955, doi: 10.1002/nav.3800020109.

[20] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and realtime tracking,” in *2016 IEEE International Conference on Image Processing (ICIP)*, 2016, pp. 3464–3468. doi: 10.1109/ICIP.2016.7533003.

[21] Y. Zhang *et al.*, “ByteTrack: Multi-object tracking by associating every detection box,” in *Computer Vision – ECCV 2022*, in Lecture Notes in Computer Science, vol. 13682. Springer, 2022, pp. 1–21. doi: 10.1007/978-3-031-20047-2_1.

[22] Y. Zhang *et al.*, “ByteTrack: Official code repository.” GitHub repository (tools/track.py, yolox/tracker/byte_tracker.py, yolox/evaluators/mot_evaluator.py); now hosted at github.com/FoundationVision/ByteTrack, 2021. Accessed: Oct. 23, 2026. [Online]. Available: https://github.com/ifzhang/ByteTrack

[23] P. Dendorfer *et al.*, “MOT20: A benchmark for multi object tracking in crowded scenes.” arXiv preprint arXiv:2003.09003, 2020. Available: https://arxiv.org/abs/2003.09003

[24] Ultralytics, “Multi-object tracking with Ultralytics YOLO.” Ultralytics documentation and GitHub repository (ultralytics/cfg/trackers/bytetrack.yaml, botsort.yaml; ultralytics/trackers/byte_tracker.py; ultralytics/engine/model.py), 2023. Available: https://docs.ultralytics.com/modes/track/

[25] T. Fleck and J. M. Zöllner, “Tuning multi object tracking systems using Bayesian optimization,” in *2021 IEEE 24th International Conference on Information Fusion (FUSION)*, 2021, pp. 1–8. doi: 10.23919/FUSION49465.2021.9626895.

[26] M. Adžemović, “Revisiting multi-object tracking baselines: Hyperparameter optimization with multi-fidelity greedy coordinate search.” arXiv preprint arXiv:2609.12261, 2026. doi: 10.48550/arXiv.2609.12261.

[27] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyper-parameter optimization,” in *Advances in Neural Information Processing Systems (NIPS)*, 2011, pp. 2546–2554. Available: https://papers.nips.cc/paper/2011/hash/86e8f7ab32cfd12577bc2619bc635690-Abstract.html

[28] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012, Available: https://jmlr.org/papers/v13/bergstra12a.html

[29] N. Hansen, “The CMA evolution strategy: A tutorial.” arXiv preprint arXiv:1604.00772, 2016. Available: https://arxiv.org/abs/1604.00772

[30] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2019, pp. 2623–2631. doi: 10.1145/3292500.3330701.

[31] K. Bernardin and R. Stiefelhagen, “Evaluating multiple object tracking performance: The CLEAR MOT metrics,” *EURASIP Journal on Image and Video Processing*, vol. 2008, pp. 1–10, 2008, doi: 10.1155/2008/246309.

[32] E. Ristani, F. Solera, R. Zou, R. Cucchiara, and C. Tomasi, “Performance measures and a data set for multi-target, multi-camera tracking,” in *Computer Vision – ECCV 2016 Workshops*, in Lecture Notes in Computer Science, vol. 9914. Springer, 2016, pp. 17–35. doi: 10.1007/978-3-319-48881-3_2.

[33] J. Luiten *et al.*, “HOTA: A higher order metric for evaluating multi-object tracking,” *International Journal of Computer Vision*, vol. 129, no. 2, pp. 548–578, 2021, doi: 10.1007/s11263-020-01375-2.

[34] A. Milan, L. Leal-Taixé, I. Reid, S. Roth, and K. Schindler, “MOT16: A benchmark for multi-object tracking.” arXiv preprint arXiv:1603.00831, 2016. Available: https://arxiv.org/abs/1603.00831

[35] P. Dendorfer *et al.*, “MOTChallenge: A benchmark for single-camera multiple target tracking,” *International Journal of Computer Vision*, vol. 129, no. 4, pp. 845–881, 2021, doi: 10.1007/s11263-020-01393-0.

[36] P. Kohl, A. Specker, A. Schumann, and J. Beyerer, “The MTA dataset for multi target multi camera pedestrian tracking by weighted distance aggregation,” in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2020, pp. 4489–4498. doi: 10.1109/CVPRW50498.2020.00529.

[37] M. Fabbri *et al.*, “MOTSynth: How can synthetic data help pedestrian detection and tracking?” in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 10829–10839. doi: 10.1109/ICCV48922.2021.01067.

[38] R. Delussu, L. Putzu, and G. Fumera, “Synthetic data for video surveillance applications of computer vision: A review,” *International Journal of Computer Vision*, vol. 132, no. 10, pp. 4473–4509, 2024, doi: 10.1007/s11263-024-02102-x.

[39] A. Gaidon, Q. Wang, Y. Cabon, and E. Vig, “Virtual worlds as proxy for multi-object tracking analysis,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4340–4349. doi: 10.1109/CVPR.2016.470.

[40] M. Fabbri, F. Lanzi, S. Calderara, A. Palazzi, R. Vezzani, and R. Cucchiara, “Learning to detect and track visible and occluded body joints in a virtual world,” in *Computer Vision – ECCV 2018*, in Lecture Notes in Computer Science, vol. 11208. Springer, 2018, pp. 450–466. doi: 10.1007/978-3-030-01225-0_27.

[41] H. Hattori, V. N. Boddeti, K. Kitani, and T. Kanade, “Learning scene-specific pedestrian detectors without real data,” in *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3819–3827. doi: 10.1109/CVPR.2015.7299006.

[42] H. Hattori, N. Lee, V. N. Boddeti, F. Beainy, K. M. Kitani, and T. Kanade, “Synthesizing a scene-specific pedestrian detector and pose estimator for static video surveillance,” *International Journal of Computer Vision*, vol. 126, no. 9, pp. 1027–1044, 2018, doi: 10.1007/s11263-018-1077-3.

[43] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30. doi: 10.1109/IROS.2017.8202133.

[44] J. Tremblay *et al.*, “Training deep networks with synthetic data: Bridging the reality gap by domain randomization,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018. doi: 10.1109/CVPRW.2018.00143.

[45] S. J. Pan and Q. Yang, “A survey on transfer learning,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010, doi: 10.1109/TKDE.2009.191.

[46] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, “How transferable are features in deep neural networks?” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014. Available: https://proceedings.neurips.cc/paper_files/paper/2014/hash/532a2f85b6977104bc93f8580abbb330-Abstract.html

[47] D. Helbing and P. Molnár, “Social force model for pedestrian dynamics,” *Physical Review E*, vol. 51, no. 5, pp. 4282–4286, 1995, doi: 10.1103/PhysRevE.51.4282.

[48] X. Chen, M. Treiber, V. Kanagaraj, and H. Li, “Social force models for pedestrian traffic – state of the art,” *Transport Reviews*, vol. 38, no. 5, pp. 625–653, 2018, doi: 10.1080/01441647.2017.1396265.

[49] A. Borshchev, *The big book of simulation modeling: Multimethod modeling with AnyLogic 6*. AnyLogic North America, 2013. Available: https://www.anylogic.com/resources/books/big-book-of-simulation-modeling/

[50] The AnyLogic Company, “Pedestrian Library: Library reference guide.” AnyLogic Help (online documentation), 2026. Accessed: Oct. 23, 2026. [Online]. Available: https://anylogic.help/library-reference-guides/pedestrian-library/index.html

[51] P.-O. Siebers, Y. Deng, J. Thaler, H. Schnädelbach, and E. Özcan, “Proposal of a design pattern for embedding the concept of social forces in human centric simulation models,” in *Proceedings of the Operational Research Society Simulation Workshop 2018 (SW18)*, A. Anagnostou, R. Meskarian, and D. Robertson, Eds., 2018. Available: https://people.cs.nott.ac.uk/pszeo/docs/publications/SW2018.pdf

[52] The AnyLogic Company, “Alpyne: A Python library for executing AnyLogic models.” GitHub repository and PyPI package anylogic-alpyne, 2025. Available: https://github.com/the-anylogic-company/Alpyne

[53] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*, 2nd ed. in Adaptive Computation and Machine Learning. Cambridge, MA: MIT Press, 2018. Available: https://mitpress.mit.edu/9780262039246/reinforcement-learning/

[54] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms.” arXiv preprint arXiv:1707.06347, 2017. Available: https://arxiv.org/abs/1707.06347

[55] X. Dong, J. Shen, W. Wang, Y. Liu, L. Shao, and F. Porikli, “Hyperparameter optimization for tracking with continuous deep Q-learning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 518–527. doi: 10.1109/CVPR.2018.00061.

[56] N. Ruiz, S. Schulter, and M. Chandraker, “Learning to simulate,” in *International Conference on Learning Representations (ICLR)*, 2019. doi: 10.48550/arXiv.1810.02513.

[57] J. Langford and T. Zhang, “The Epoch-Greedy algorithm for multi-armed bandits with side information,” in *Advances in Neural Information Processing Systems (NIPS)*, 2007, pp. 817–824. Available: https://papers.nips.cc/paper/2007/hash/4b04a686b0ad13dce35fa99fa4161c65-Abstract.html

[58] L. Li, W. Chu, J. Langford, and R. E. Schapire, “A contextual-bandit approach to personalized news article recommendation,” in *Proceedings of the 19th International Conference on World Wide Web (WWW)*, 2010, pp. 661–670. doi: 10.1145/1772690.1772758.

[59] U. Weidmann, “Transporttechnik der Fussgänger: Transporttechnische Eigenschaften des Fussgängerverkehrs (Literaturauswertung),” ETH Zürich, Institut für Verkehrsplanung (IVT), IVT Schriftenreihe 90, 1993. doi: 10.3929/ethz-b-000242008.

[60] J. Luiten and A. Hoffhues, “TrackEval.” GitHub repository, 2020. Available: https://github.com/JonathonLuiten/TrackEval

[61] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021, Available: https://jmlr.org/papers/v22/20-1364.html

[62] M. Towers *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” in *Advances in Neural Information Processing Systems 38, Datasets and Benchmarks Track*, 2025, pp. 163114–163129. doi: 10.52202/085713-4916.

[63] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945, doi: 10.2307/3001968.

[64] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979, Available: https://www.jstor.org/stable/4615733

[65] R. Girshick, J. Donahue, T. Darrell, and J. Malik, “Rich feature hierarchies for accurate object detection and semantic segmentation,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 580–587. doi: 10.1109/CVPR.2014.81.

[66] R. Girshick, “Fast R-CNN,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 1440–1448. doi: 10.1109/ICCV.2015.169.

[67] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2017, doi: 10.1109/TPAMI.2016.2577031.

[68] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask R-CNN,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988. doi: 10.1109/ICCV.2017.322.

[69] Z. Cai and N. Vasconcelos, “Cascade R-CNN: Delving into high quality object detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 6154–6162. doi: 10.1109/CVPR.2018.00644.

[70] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature pyramid networks for object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 936–944. doi: 10.1109/CVPR.2017.106.

[71] W. Liu *et al.*, “SSD: Single shot MultiBox detector,” in *Computer Vision – ECCV 2016*, in Lecture Notes in Computer Science, vol. 9905. Springer, 2016, pp. 21–37. doi: 10.1007/978-3-319-46448-0_2.

[72] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2999–3007. doi: 10.1109/ICCV.2017.324.

[73] H. Law and J. Deng, “CornerNet: Detecting objects as paired keypoints,” in *Computer Vision – ECCV 2018*, in Lecture Notes in Computer Science, vol. 11218. Springer, 2018, pp. 765–781. doi: 10.1007/978-3-030-01264-9_45.

[74] Z. Tian, C. Shen, H. Chen, and T. He, “FCOS: Fully convolutional one-stage object detection,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 9626–9635. doi: 10.1109/ICCV.2019.00972.

[75] X. Zhou, D. Wang, and P. Krähenbühl, “Objects as points.” arXiv preprint arXiv:1904.07850, 2019. doi: 10.48550/arXiv.1904.07850.

[76] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, “YOLOX: Exceeding YOLO series in 2021.” arXiv preprint arXiv:2107.08430, 2021. doi: 10.48550/arXiv.2107.08430.

[77] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-end object detection with transformers,” in *Computer Vision – ECCV 2020*, in Lecture Notes in Computer Science, vol. 12346. Springer, 2020, pp. 213–229. doi: 10.1007/978-3-030-58452-8_13.

[78] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, “Deformable DETR: Deformable transformers for end-to-end object detection,” in *International Conference on Learning Representations (ICLR)*, 2021. Available: https://arxiv.org/abs/2010.04159

[79] Y. Zhao *et al.*, “DETRs beat YOLOs on real-time object detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 16965–16974. doi: 10.1109/CVPR52733.2024.01605.

[80] A. Wang *et al.*, “YOLOv10: Real-time end-to-end object detection,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, pp. 107984–108011. doi: 10.52202/079017-3429.

[81] J. Redmon and A. Farhadi, “YOLO9000: Better, faster, stronger,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 6517–6525. doi: 10.1109/CVPR.2017.690.

[82] J. Redmon and A. Farhadi, “YOLOv3: An incremental improvement.” arXiv preprint arXiv:1804.02767 (Tech Report), 2018. doi: 10.48550/arXiv.1804.02767.

[83] K. He, X. Zhang, S. Ren, and J. Sun, “Spatial pyramid pooling in deep convolutional networks for visual recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 37, no. 9, pp. 1904–1916, 2015, doi: 10.1109/TPAMI.2015.2389824.

[84] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal speed and accuracy of object detection.” arXiv preprint arXiv:2004.10934, 2020. doi: 10.48550/arXiv.2004.10934.

[85] C.-Y. Wang, H.-Y. M. Liao, Y.-H. Wu, P.-Y. Chen, J.-W. Hsieh, and I.-H. Yeh, “CSPNet: A new backbone that can enhance learning capability of CNN,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2020, pp. 1571–1580. doi: 10.1109/CVPRW50498.2020.00203.

[86] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, “Path aggregation network for instance segmentation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 8759–8768. doi: 10.1109/CVPR.2018.00913.

[87] G. Jocher, “Ultralytics YOLOv5.” GitHub repository, 2020. doi: 10.5281/zenodo.3908559.

[88] C. Li *et al.*, “YOLOv6: A single-stage object detection framework for industrial applications.” arXiv preprint arXiv:2209.02976 (technical report), 2022. doi: 10.48550/arXiv.2209.02976.

[89] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 7464–7475. doi: 10.1109/CVPR52729.2023.00721.

[90] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, “YOLOv9: Learning what you want to learn using programmable gradient information,” in *Computer Vision – ECCV 2024*, in Lecture Notes in Computer Science, vol. 15089. Springer, 2024, pp. 1–21. doi: 10.1007/978-3-031-72751-1_1.

[91] G. Jocher and J. Qiu, “Ultralytics YOLO11.” GitHub repository, 2024. Available: https://github.com/ultralytics/ultralytics

[92] Y. Tian, Q. Ye, and D. Doermann, “YOLOv12: Attention-centric real-time object detectors,” in *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025, pp. 87151–87175. doi: 10.52202/085713-2627.

[93] M. Lei *et al.*, “YOLOv13: Real-time object detection with hypergraph-enhanced adaptive visual perception.” arXiv preprint arXiv:2506.17733, 2025. doi: 10.48550/arXiv.2506.17733.

[94] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, and M. E. Kalfaoglu, “Ultralytics YOLO26: Unified real-time end-to-end vision models.” arXiv preprint arXiv:2606.03748, 2026. doi: 10.48550/arXiv.2606.03748.

[95] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, “Distance-IoU loss: Faster and better learning for bounding box regression,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 7, pp. 12993–13000, 2020, doi: 10.1609/aaai.v34i07.6999.

[96] H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid, and S. Savarese, “Generalized intersection over union: A metric and a loss for bounding box regression,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 658–666. doi: 10.1109/CVPR.2019.00075.

[97] X. Li *et al.*, “Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 21002–21012. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/f0bda020d2470f2e74990a07a607ebd9-Abstract.html

[98] C. Feng, Y. Zhong, Y. Gao, M. R. Scott, and W. Huang, “TOOD: Task-aligned one-stage object detection,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 3490–3499. doi: 10.1109/ICCV48922.2021.00349.

[99] S. Shao *et al.*, “CrowdHuman: A benchmark for detecting human in a crowd.” arXiv preprint arXiv:1805.00123, 2018. doi: 10.48550/arXiv.1805.00123.

[100] F. Zhuang *et al.*, “A comprehensive survey on transfer learning,” *Proceedings of the IEEE*, vol. 109, no. 1, pp. 43–76, 2021, doi: 10.1109/JPROC.2020.3004555.

[101] S. Kornblith, J. Shlens, and Q. V. Le, “Do better ImageNet models transfer better?” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 2656–2666. doi: 10.1109/CVPR.2019.00277.

[102] P. Dollár, C. Wojek, B. Schiele, and P. Perona, “Pedestrian detection: An evaluation of the state of the art,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 34, no. 4, pp. 743–761, 2012, doi: 10.1109/TPAMI.2011.155.

[103] S. Zhang, R. Benenson, and B. Schiele, “CityPersons: A diverse dataset for pedestrian detection,” in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 4457–4465. doi: 10.1109/CVPR.2017.474.

[104] J. Munkres, “Algorithms for the assignment and transportation problems,” *Journal of the Society for Industrial and Applied Mathematics*, vol. 5, no. 1, pp. 32–38, 1957, doi: 10.1137/0105003.

[105] R. Jonker and A. Volgenant, “A shortest augmenting path algorithm for dense and sparse linear assignment problems,” *Computing*, vol. 38, no. 4, pp. 325–340, 1987, doi: 10.1007/BF02278710.

[106] N. Wojke, A. Bewley, and D. Paulus, “Simple online and realtime tracking with a deep association metric,” in *2017 IEEE International Conference on Image Processing (ICIP)*, 2017, pp. 3645–3649. doi: 10.1109/ICIP.2017.8296962.

[107] J. Cao, J. Pang, X. Weng, R. Khirodkar, and K. Kitani, “Observation-centric SORT: Rethinking SORT for robust multi-object tracking,” in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 9686–9696. doi: 10.1109/CVPR52729.2023.00934.

[108] G. Maggiolino, A. Ahmad, J. Cao, and K. Kitani, “Deep OC-SORT: Multi-pedestrian tracking by adaptive re-identification,” in *2023 IEEE International Conference on Image Processing (ICIP)*, 2023, pp. 3025–3029. doi: 10.1109/ICIP49359.2023.10222576.

[109] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, “BoT-SORT: Robust associations multi-pedestrian tracking.” arXiv preprint arXiv:2206.14651, 2022. Available: https://arxiv.org/abs/2206.14651

[110] Y. Du *et al.*, “StrongSORT: Make DeepSORT great again,” *IEEE Transactions on Multimedia*, vol. 25, pp. 8725–8737, 2023, doi: 10.1109/TMM.2023.3240881.

[111] Y. Zhang, C. Wang, X. Wang, W. Zeng, and W. Liu, “FairMOT: On the fairness of detection and re-identification in multiple object tracking,” *International Journal of Computer Vision*, vol. 129, no. 11, pp. 3069–3087, 2021, doi: 10.1007/s11263-021-01513-4.

[112] X. Zhou, V. Koltun, and P. Krähenbühl, “Tracking objects as points,” in *Computer Vision – ECCV 2020*, in Lecture Notes in Computer Science, vol. 12349. Springer, 2020, pp. 474–490. doi: 10.1007/978-3-030-58548-8_28.

[113] P. Sun *et al.*, “TransTrack: Multiple object tracking with transformer.” arXiv preprint arXiv:2012.15460, 2020. Available: https://arxiv.org/abs/2012.15460

[114] P. Sun *et al.*, “DanceTrack: Multi-object tracking in uniform appearance and diverse motion,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 20961–20970. doi: 10.1109/CVPR52688.2022.02032.

[115] P. Bergmann, T. Meinhardt, and L. Leal-Taixé, “Tracking without bells and whistles,” in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 941–951. doi: 10.1109/ICCV.2019.00103.

[116] Z. Wang, L. Zheng, Y. Liu, Y. Li, and S. Wang, “Towards real-time multi-object tracking,” in *Computer Vision – ECCV 2020*, in Lecture Notes in Computer Science, vol. 12356. Springer, 2020, pp. 107–122. doi: 10.1007/978-3-030-58621-8_7.

[117] F. Zeng, B. Dong, Y. Zhang, T. Wang, X. Zhang, and Y. Wei, “MOTR: End-to-end multiple-object tracking with transformer,” in *Computer Vision – ECCV 2022*, in Lecture Notes in Computer Science, vol. 13687. Springer, 2022, pp. 659–675. doi: 10.1007/978-3-031-19812-0_38.

[118] Y. Zhang, T. Wang, and X. Zhang, “MOTRv2: Bootstrapping end-to-end multi-object tracking by pretrained object detectors,” in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 22056–22065. doi: 10.1109/CVPR52729.2023.02112.

[119] R. Gao, J. Qi, and L. Wang, “Multiple object tracking as ID prediction,” in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 27883–27893. doi: 10.1109/CVPR52734.2025.02596.

[120] L. Leal-Taixé, A. Milan, I. Reid, S. Roth, and K. Schindler, “MOTChallenge 2015: Towards a benchmark for multi-target tracking.” arXiv preprint arXiv:1504.01942, 2015. Available: https://arxiv.org/abs/1504.01942

[121] R. L. Hughes, “A continuum theory for the flow of pedestrians,” *Transportation Research Part B: Methodological*, vol. 36, no. 6, pp. 507–535, 2002, doi: 10.1016/S0191-2615(01)00015-7.

[122] L. F. Henderson, “The statistics of crowd fluids,” *Nature*, vol. 229, no. 5284, pp. 381–383, 1971, doi: 10.1038/229381a0.

[123] S. P. Hoogendoorn and P. H. L. Bovy, “Gas-kinetic modeling and simulation of pedestrian flows,” *Transportation Research Record: Journal of the Transportation Research Board*, vol. 1710, no. 1, pp. 28–36, 2000, doi: 10.3141/1710-04.

[124] D. C. Duives, W. Daamen, and S. P. Hoogendoorn, “State-of-the-art crowd motion simulation models,” *Transportation Research Part C: Emerging Technologies*, vol. 37, pp. 193–209, 2013, doi: 10.1016/j.trc.2013.02.005.

[125] X. Zheng, T. Zhong, and M. Liu, “Modeling crowd evacuation of a building based on seven methodological approaches,” *Building and Environment*, vol. 44, no. 3, pp. 437–445, 2009, doi: 10.1016/j.buildenv.2008.04.002.

[126] A. Schadschneider, W. Klingsch, H. Klüpfel, T. Kretz, C. Rogsch, and A. Seyfried, “Evacuation dynamics: Empirical results, modeling and applications,” in *Encyclopedia of Complexity and Systems Science*, R. A. Meyers, Ed., Springer, 2009, pp. 3142–3176. doi: 10.1007/978-0-387-30440-3_187.

[127] C. Burstedde, K. Klauck, A. Schadschneider, and J. Zittartz, “Simulation of pedestrian dynamics using a two-dimensional cellular automaton,” *Physica A: Statistical Mechanics and its Applications*, vol. 295, no. 3–4, pp. 507–525, 2001, doi: 10.1016/S0378-4371(01)00141-8.

[128] M. J. Seitz and G. Köster, “Natural discretization of pedestrian movement in continuous space,” *Physical Review E*, vol. 86, no. 4, p. 046108, 2012, doi: 10.1103/PhysRevE.86.046108.

[129] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha, “Reciprocal n-body collision avoidance,” in *Robotics Research: The 14th International Symposium ISRR*, vol. 70, in Springer Tracts in Advanced Robotics, vol. 70., Berlin, Heidelberg: Springer, 2011, pp. 3–19. doi: 10.1007/978-3-642-19457-3_1.

[130] M. Moussaïd, D. Helbing, and G. Theraulaz, “How simple rules determine pedestrian behavior and crowd disasters,” *Proceedings of the National Academy of Sciences*, vol. 108, no. 17, pp. 6884–6888, 2011, doi: 10.1073/pnas.1016507108.

[131] A. Kirchner and A. Schadschneider, “Simulation of evacuation processes using a bionics-inspired cellular automaton model for pedestrian dynamics,” *Physica A: Statistical Mechanics and its Applications*, vol. 312, no. 1–2, pp. 260–276, 2002, doi: 10.1016/S0378-4371(02)00857-9.

[132] A. Borshchev and A. Filippov, “From system dynamics and discrete event to practical agent based modeling: Reasons, techniques, tools,” in *Proceedings of the 22nd International Conference of the System Dynamics Society*, Oxford, England, Jul. 2004. Available: https://faculty.sites.iastate.edu/tesfatsi/archive/tesfatsi/SystemDynDiscreteEventABMCompared.BorshchevFilippov04.pdf

[133] J. Li, Y. Yang, and Q. Zhao, “Study on passenger flow simulation in urban subway station based on Anylogic,” *Journal of Software*, vol. 9, no. 1, pp. 140–146, 2014, doi: 10.4304/jsw.9.1.140-146.

[134] F. Li, S. Chen, X. Wang, and F. Feng, “Pedestrian evacuation modeling and simulation on metro platforms considering panic impacts,” *Procedia - Social and Behavioral Sciences*, vol. 138, pp. 314–322, 2014, doi: 10.1016/j.sbspro.2014.07.209.

[135] M. Geng, J. Gao, and Z. Zeng, “A pilot study for the service level of the metro station renovation project based on passenger detection and simulation techniques,” *Promet – Traffic&Transportation*, vol. 38, no. 3, pp. 699–715, 2026, doi: 10.7307/ptt.v38i3.1117.

[136] H. Vermuyten, J. Beliën, L. De Boeck, G. Reniers, and T. Wauters, “A review of optimisation models for pedestrian evacuation and design problems,” *Safety Science*, vol. 87, pp. 167–178, 2016, doi: 10.1016/j.ssci.2016.04.001.

[137] The AnyLogic Company, “AnyLogic Pypeline: A custom library for AnyLogic that allows you to call Python within a running model.” GitHub repository, 2025. Available: https://github.com/the-anylogic-company/AnyLogic-Pypeline

[138] J. Marín, D. Vázquez, D. Gerónimo, and A. M. López, “Learning appearance in virtual scenarios for pedestrian detection,” in *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 2010, pp. 137–144. doi: 10.1109/CVPR.2010.5540218.

[139] Y. Cabon, N. Murray, and M. Humenberger, “Virtual KITTI 2.” arXiv preprint arXiv:2001.10773, 2020. doi: 10.48550/arXiv.2001.10773.

[140] S. R. Richter, V. Vineet, S. Roth, and V. Koltun, “Playing for data: Ground truth from computer games,” in *Computer Vision – ECCV 2016*, in Lecture Notes in Computer Science, vol. 9906. Springer, 2016, pp. 102–118. doi: 10.1007/978-3-319-46475-6_7.

[141] S. R. Richter, Z. Hayder, and V. Koltun, “Playing for benchmarks,” in *2017 IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2232–2241. doi: 10.1109/ICCV.2017.243.

[142] G. Ros, L. Sellart, J. Materzynska, D. Vázquez, and A. M. López, “The SYNTHIA dataset: A large collection of synthetic images for semantic segmentation of urban scenes,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 3234–3243. doi: 10.1109/CVPR.2016.352.

[143] M. Johnson-Roberson, C. Barto, R. Mehta, S. N. Sridhar, K. Rosaen, and R. Vasudevan, “Driving in the Matrix: Can virtual worlds replace human-generated annotations for real world tasks?” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 746–753. doi: 10.1109/ICRA.2017.7989092.

[144] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, “CARLA: An open urban driving simulator,” in *Proceedings of the 1st Annual Conference on Robot Learning (CoRL)*, in Proceedings of Machine Learning Research, vol. 78. PMLR, 2017, pp. 1–16. Available: https://proceedings.mlr.press/v78/dosovitskiy17a.html

[145] S. Shah, D. Dey, C. Lovett, and A. Kapoor, “AirSim: High-fidelity visual and physical simulation for autonomous vehicles,” in *Field and Service Robotics (FSR 2017)*, in Springer Proceedings in Advanced Robotics, vol. 5. Springer, 2018, pp. 621–635. doi: 10.1007/978-3-319-67361-5_40.

[146] S. Borkman *et al.*, “Unity Perception: Generate synthetic data for computer vision.” arXiv preprint arXiv:2107.04259, 2021. doi: 10.48550/arXiv.2107.04259.

[147] A. Prakash *et al.*, “Structured domain randomization: Bridging the reality gap by context-aware synthetic data,” in *2019 International Conference on Robotics and Automation (ICRA)*, 2019, pp. 7249–7255. doi: 10.1109/ICRA.2019.8794443.

[148] A. Shrivastava, T. Pfister, O. Tuzel, J. Susskind, W. Wang, and R. Webb, “Learning from simulated and unsupervised images through adversarial training,” in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2242–2251. doi: 10.1109/CVPR.2017.241.

[149] J. Hoffman *et al.*, “CyCADA: Cycle-consistent adversarial domain adaptation,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, in Proceedings of Machine Learning Research, vol. 80. PMLR, 2018, pp. 1989–1998. Available: https://proceedings.mlr.press/v80/hoffman18a.html

[150] Y. Chen, W. Li, C. Sakaridis, D. Dai, and L. Van Gool, “Domain adaptive faster R-CNN for object detection in the wild,” in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 3339–3348. doi: 10.1109/CVPR.2018.00352.

[151] A. Kar *et al.*, “Meta-Sim: Learning to generate synthetic datasets,” in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 4550–4559. doi: 10.1109/ICCV.2019.00465.

[152] S. I. Nikolenko, *Synthetic data for deep learning*. in Springer Optimization and Its Applications, no. 174. Cham: Springer, 2021. doi: 10.1007/978-3-030-75178-4.

[153] W. Zhao, J. P. Queralta, and T. Westerlund, “Sim-to-real transfer in deep reinforcement learning for robotics: A survey,” in *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, 2020, pp. 737–744. doi: 10.1109/SSCI47803.2020.9308468.

[154] Q. Wang, J. Gao, W. Lin, and Y. Yuan, “Learning from synthetic data for crowd counting in the wild,” in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 8190–8199. doi: 10.1109/CVPR.2019.00839.

[155] S. E. Ebadi *et al.*, “PeopleSansPeople: A synthetic data generator for human-centric computer vision.” arXiv preprint arXiv:2112.09290, 2021. doi: 10.48550/arXiv.2112.09290.

[156] P. Foszner *et al.*, “CrowdSim2: An open synthetic benchmark for object detectors,” in *Proceedings of the 18th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISIGRAPP 2023) – Volume 5: VISAPP*, SciTePress, 2023, pp. 676–683. doi: 10.5220/0011692500003417.

[157] G. Varol *et al.*, “Learning from synthetic humans,” in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 4627–4635. doi: 10.1109/CVPR.2017.492.

[158] X. Sun and L. Zheng, “Dissecting person re-identification from the viewpoint of viewpoint,” in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 608–617. doi: 10.1109/CVPR.2019.00070.

[159] E. Cheung, T. K. Wong, A. Bera, X. Wang, and D. Manocha, “LCrowdV: Generating labeled videos for simulation-based crowd behavior learning,” in *Computer Vision – ECCV 2016 Workshops*, in Lecture Notes in Computer Science, vol. 9914. Springer, 2016, pp. 709–727. doi: 10.1007/978-3-319-48881-3_50.

[160] M. Staniszewski *et al.*, “Application of crowd simulations in the evaluation of tracking algorithms,” *Sensors*, vol. 20, no. 17, p. 4960, 2020, doi: 10.3390/s20174960.

[161] P. Foszner *et al.*, “Development of a realistic crowd simulation environment for fine-grained validation of people tracking methods,” in *Proceedings of the 18th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISIGRAPP 2023)*, SciTePress, 2023, pp. 222–229. doi: 10.5220/0011691500003417.

[162] F. Hutter, L. Kotthoff, and J. Vanschoren, Eds., *Automated machine learning: Methods, systems, challenges*. in The Springer Series on Challenges in Machine Learning. Cham: Springer, 2019. doi: 10.1007/978-3-030-05318-5.

[163] J. Snoek, H. Larochelle, and R. P. Adams, “Practical Bayesian optimization of machine learning algorithms,” in *Advances in Neural Information Processing Systems (NIPS)*, 2012, pp. 2951–2959. Available: https://papers.nips.cc/paper/2012/hash/05311655a15b75fab86956663e1819cd-Abstract.html

[164] N. Hansen and A. Ostermeier, “Completely derandomized self-adaptation in evolution strategies,” *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001, doi: 10.1162/106365601750190398.

[165] I. Loshchilov and F. Hutter, “CMA-ES for hyperparameter optimization of deep neural networks.” arXiv preprint arXiv:1604.07269, 2016. Available: https://arxiv.org/abs/1604.07269

[166] M. Jaderberg *et al.*, “Population based training of neural networks.” arXiv preprint arXiv:1711.09846, 2017. Available: https://arxiv.org/abs/1711.09846

[167] F. Madrigal, C. Maurice, and F. Lerasle, “Hyper-parameter optimization tools comparison for multiple object tracking applications,” *Machine Vision and Applications*, vol. 30, no. 2, pp. 269–289, 2019, doi: 10.1007/s00138-018-0984-1.

[168] D. P. Chau, M. Thonnat, and F. Brémond, “Automatic parameter adaptation for multi-object tracking,” in *Computer Vision Systems (ICVS 2013)*, in Lecture Notes in Computer Science, vol. 7963. Springer, 2013, pp. 244–253. doi: 10.1007/978-3-642-39402-7_25.

[169] L. V. Ma, M. I. Hussain, J. Park, J. Kim, and M. Jeon, “Adaptive confidence threshold for ByteTrack in multi-object tracking,” in *2023 12th International Conference on Control, Automation and Information Sciences (ICCAIS)*, 2023, pp. 370–374. doi: 10.1109/ICCAIS59597.2023.10382403.

[170] M. L. Puterman, *Markov decision processes: Discrete stochastic dynamic programming*. in Wiley Series in Probability and Statistics. John Wiley & Sons, 1994. doi: 10.1002/9780470316887.

[171] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.

[172] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992, doi: 10.1007/BF00992696.

[173] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” in *International Conference on Learning Representations (ICLR)*, 2016. Available: https://arxiv.org/abs/1506.02438

[174] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, “Trust region policy optimization,” in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, in Proceedings of Machine Learning Research, vol. 37. PMLR, 2015, pp. 1889–1897. Available: https://proceedings.mlr.press/v37/schulman15.html

[175] T. P. Lillicrap *et al.*, “Continuous control with deep reinforcement learning,” in *International Conference on Learning Representations (ICLR)*, 2016. Available: https://arxiv.org/abs/1509.02971

[176] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, in Proceedings of Machine Learning Research, vol. 80. PMLR, 2018, pp. 1861–1870. Available: https://proceedings.mlr.press/v80/haarnoja18b.html

[177] B. Zoph and Q. V. Le, “Neural architecture search with reinforcement learning,” in *International Conference on Learning Representations (ICLR)*, 2017. Available: https://openreview.net/forum?id=r1Ue8Hcxg

[178] K. Li and J. Malik, “Learning to optimize,” in *International Conference on Learning Representations (ICLR)*, 2017. Available: https://openreview.net/forum?id=ry4Vrt5gl

[179] C. Huang, S. Lucey, and D. Ramanan, “Learning policies for adaptive tracking with deep feature cascades,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 105–114. doi: 10.1109/ICCV.2017.21.

[180] S. Yun, J. Choi, Y. Yoo, K. Yun, and J. Y. Choi, “Action-decision networks for visual tracking with deep reinforcement learning,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1349–1358. doi: 10.1109/CVPR.2017.148.

[181] M. Stephan, L. Servadei, J. Arjona-Medina, A. Santra, R. Wille, and G. Fischer, “Scene-adaptive radar tracking with deep reinforcement learning,” *Machine Learning with Applications*, vol. 8, p. 100284, 2022, doi: 10.1016/j.mlwa.2022.100284.

[182] Z. Zhao, H. Chen, and W. Zhang, “Reinforcement learning based hyperparameter optimization for irregular shape extended object tracking,” *Physica Scripta*, vol. 100, no. 7, p. 075247, 2025, doi: 10.1088/1402-4896/ade5d6.

[183] Y. Xiang, A. Alahi, and S. Savarese, “Learning to track: Online multi-object tracking by decision making,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 4705–4713. doi: 10.1109/ICCV.2015.534.

[184] L. Ren, J. Lu, Z. Wang, Q. Tian, and J. Zhou, “Collaborative deep reinforcement learning for multi-object tracking,” in *Computer Vision – ECCV 2018*, in Lecture Notes in Computer Science, vol. 11207. Springer, 2018, pp. 605–621. doi: 10.1007/978-3-030-01219-9_36.

[185] P. Rosello and M. J. Kochenderfer, “Multi-agent reinforcement learning for multi-object tracking,” in *Proceedings of the 17th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2018, pp. 1397–1404. Available: https://www.ifaamas.org/Proceedings/aamas2018/pdfs/p1397.pdf

[186] D. F. Crouse, “On implementing 2D rectangular assignment algorithms,” *IEEE Transactions on Aerospace and Electronic Systems*, vol. 52, no. 4, pp. 1679–1696, 2016, doi: 10.1109/TAES.2016.140952.

[187] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proceedings of the 3rd international conference on learning representations (ICLR)*, San Diego, CA, USA, 2015. Available: https://arxiv.org/abs/1412.6980

[188] Optuna Developers, “Optuna samplers: TPESampler and CmaEsSampler.” Optuna API reference documentation and source code (optuna/samplers/_tpe/sampler.py, optuna/samplers/_cmaes.py), version 5.0.0, 2026. Accessed: Oct. 23, 2026. [Online]. Available: https://optuna.readthedocs.io/en/stable/reference/samplers/index.html

[189] T. Lozano-Pérez and M. A. Wesley, “An algorithm for planning collision-free paths among polyhedral obstacles,” *Communications of the ACM*, vol. 22, no. 10, pp. 560–570, 1979, doi: 10.1145/359156.359164.

[190] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959, doi: 10.1007/BF01386390.

[191] R. Hartley and A. Zisserman, *Multiple view geometry in computer vision*, 2nd ed. Cambridge: Cambridge University Press, 2004. doi: 10.1017/CBO9780511811685.

[192] T. Wiegand, G. J. Sullivan, G. Bjøntegaard, and A. Luthra, “Overview of the H.264/AVC video coding standard,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 13, no. 7, pp. 560–576, 2003, doi: 10.1109/TCSVT.2003.815165.

Appendix A AnyLogic Model Specification and Code

This appendix specifies the four pedestrian scenarios as an AnyLogic 8 Pedestrian Library model and gives code templates that connect such a model to the detection–tracking–reinforcement-learning pipeline. The Java and Python listings are templates written against the public AnyLogic 8 and Alpyne interfaces; method names of the presentation and 3D API can differ slightly between AnyLogic releases and should be checked against the installed version. No AnyLogic run supplied data for this thesis: the data of Chapter 5 were generated by the Python co-simulation (Section 4.3), which implements the same layouts, flows, pedestrian attributes and camera poses with its own calibrated social-force dynamics, and the thesis does not establish that trajectories from the AnyLogic runtime and from the co-simulation are equivalent. The converter in Listing A.2 maps AnyLogic exports to the pipeline's video and ground-truth format.

A.1 Model Elements

Table A.1 lists the agent types, parameters and other elements of the model, and Table A.2 the Pedestrian Library blocks of the flowcharts in Figure 4.2 with their key properties; geometry, arrival rates and camera poses are those of Table 4.1.

Table A.1. Elements of the AnyLogic model PedTrackSim (agent types Main and Ped).

AnyLogic element	Name	Setting
Agent type	Main	space markup, flowchart, logging event, parameters
Agent type	Ped (extends Pedestrian)	variables h (body height, m), shirt, pants (colour indices) for the renderer; lastX, lastY (last logged position, m) for the finite-difference velocity; logged (flag for the first logged frame)
Population	peds	custom population filled by PedSource (Add pedestrians to: custom population)
Parameter	scenario	int, 1-4 (corridor, crossing, bottleneck, plaza)
Parameter	arrivalRate	double, pedestrians per second (low / medium / high, Table 4.1)
Parameter	seed	int, used as fixed random seed of the experiment
Parameter	dynamicDensity	boolean, true for the dynamic-density runs (rate from rateAt(t) instead of arrivalRate)
Parameter	rateLow, rateHigh	double, pedestrians per second, low and high rate of the scenario used by rateAt(t) (Table 4.1)
Parameter	spaceHeightM	double, height of the space in metres (6, 24, 16 or 22), used to flip

AnyLogic element	Name	Setting
Parameter	captureFrames	AnyLogic's downward y axis boolean, also capture the 3D window to PNG files
Parameter	captureX, captureY	int, screen position in pixels of the 3D window for captureFrame()
Text File	trajFile	mode write, file name traj__.csv
Event	logEvent	timeout, cyclic, first occurrence 30 s (warm-up), recurrence 1/15 s
Camera (3D)	cam	position and look-at point of Table 4.1, converted to pixels at 10 px per metre
3D Window	window3d	bound to cam, 1280 × 720 px

Table A.2. Pedestrian Library blocks and their key properties.

Block	Key properties
PedSource	Arrivals defined by rate = share × arrivalRate (one source per entrance; dynamic-density runs: set_rate(share × rateAt(t))); appears at the entrance target line; comfortable speed uniform(1.0, 1.6) m/s; diameter uniform(0.4, 0.5) m; new pedestrian of type Ped
PedGoTo	mode reach target; target = gate line (bottleneck), waiting area (plaza) or exit line; the exit is the opposite end of the space (corridor, crossing, bottleneck) or, in the plaza, one of the three other openings, drawn per pedestrian with equal probability
PedSelectOutput	probabilities 0.25 (waiting area) / 0.75 (direct), plaza only
PedWait	delay uniform(10, 30) s inside the rectangular area waitArea
PedSink	removes pedestrians at the exit line

A.2 Java Code of the Agent Type Main

The code of Listing A.1 is to be pasted into the corresponding properties of the agent types Main and Ped and of the block PedSource: the additional class code, the start-up and destroy actions of Main, the action of the cyclic event logEvent, the arrival rate and on-enter action of PedSource, and the variables of Ped.

Listing A.1. Java code snippets for the AnyLogic model (code/anylogic/Main_snippets.java).

```
// ======================================================================
// AnyLogic 8 - code snippets for agent type `Main` of model "PedTrackSim"
// (template: paste into the corresponding element properties)
// ======================================================================

// ----------------------------------------------------------------------
// Additional class code of Main
// ----------------------------------------------------------------------
int frameIdx = 0;                          // frame counter (15 FPS)
static final double PX_PER_M = 10.0;       // default space scale: 10 pixels per metre
java.awt.Robot robot;                      // for optional 3D-window capture
```

```
/** Arrival rate for the dynamic-density scenario (low -> high -> low). */
double rateAt(double t) {
    double lo = rateLow, hi = rateHigh, tt = t - 30.0;   // 30 s warm-up
    double s;
    if (tt < 15) s = 0;
    else if (tt < 35) s = (tt - 15) / 20.0;
    else if (tt < 60) s = 1;
    else if (tt < 75) s = 1 - (tt - 60) / 15.0;
    else s = 0;
    return lo + (hi - lo) * s;
}

/** Captures the 3D window region of the running model to a PNG file. */
void captureFrame(int k) {
    try {
        if (robot == null) robot = new java.awt.Robot();
        // screen rectangle of window3d (adjust to your display / window layout)
        java.awt.Rectangle r = new java.awt.Rectangle(captureX, captureY, 1280, 720);
        java.awt.image.BufferedImage img = robot.createScreenCapture(r);
        javax.imageio.ImageIO.write(img, "png",
                new java.io.File(String.format("frames/%06d.png", k)));
    } catch (Exception e) {
        traceln("capture failed: " + e);
    }
}

// ---------------------------------------------------------------------
// Main -> On startup
// ---------------------------------------------------------------------
trajFile.println("frame,time,id,x,y,vx,vy,h,shirt,pants");

// ---------------------------------------------------------------------
// Event `logEvent`: Timeout, Cyclic, first occurrence 30 s, recurrence 1/15.0 s
// ---------------------------------------------------------------------
frameIdx++;
for (Ped p : peds) {
    double x = p.getX() / PX_PER_M, y = p.getY() / PX_PER_M;     // metres
    // AnyLogic's y axis points down on screen; the thesis uses a y-up frame:
    double yUp = spaceHeightM - y;
    // first logged frame: zero velocity
    if (!p.logged) { p.lastX = x; p.lastY = yUp; p.logged = true; }
    // finite-difference velocity
    double vx = (x - p.lastX) * 15.0, vy = (yUp - p.lastY) * 15.0;
    p.lastX = x; p.lastY = yUp;
    trajFile.println(frameIdx + "," + time() + "," + p.getId() + "," + x + "," + yUp + ","
                     + vx + "," + vy + "," + p.h + "," + p.shirt + "," + p.pants);
}
if (captureFrames) captureFrame(frameIdx);

// ---------------------------------------------------------------------
// pedSource -> Arrivals defined by: Rate; one PedSource per entrance with
// Arrival rate = share * (dynamicDensity ? rateLow : arrivalRate)
// (share = 0.5 corridor, 0.25 crossing and plaza, 1 bottleneck; Table 4.1).
// Dynamic-density runs: AnyLogic changes the rate at run time with set_rate(),
// e.g. in the action of logEvent, for every source:
// if (dynamicDensity) pedSource.set_rate(share * rateAt(time()));
// ---------------------------------------------------------------------

// ---------------------------------------------------------------------
// pedSource -> On enter (body height and clothing indices for the renderer)
// ---------------------------------------------------------------------
// AnyLogic normal(sigma, mean)
ped.h = Math.min(1.95, Math.max(1.50, normal(0.09, 1.70)));
ped.shirt = uniform_discr(0, 19);
ped.pants = uniform_discr(0, 9);

// ---------------------------------------------------------------------
// Ped agent type: variables
// ---------------------------------------------------------------------
// double h; int shirt; int pants; double lastX; double lastY; boolean logged;
```

```
// 3D animation: use a person shape from the 3D Objects palette and set the
// colour of the shirt/pants material from `shirt`/`pants` (palette arrays).

// ---------------------------------------------------------------------
// Main -> On destroy
// ---------------------------------------------------------------------
trajFile.close();
```

A.3 Converting AnyLogic Exports to the Pipeline Format

Listing A.2 converts the trajectory CSV written by logEvent into the sequence format of the pipeline. The converter always renders the video from the AnyLogic trajectories with the virtual camera of Section 4.4, which also computes the ground truth (amodal boxes and visibility ratios) for exactly these frames, so that the same detector, tracker and evaluation code can be applied; frames captured from the AnyLogic 3D window (captureFrames) are not read by the converter, because this ground truth would not match them.

Listing A.2. Converter from AnyLogic trajectory exports to the pipeline format (code/anylogic/anylogic_to_ptrack.py).

```
"""Convert an AnyLogic trajectory export (Main_snippets.java -> trajFile) into the
pipeline's sequence format, rendering the virtual-camera video from the trajectories.

    python anylogic_to_ptrack.py traj_1_7.csv --scenario corridor \
        --name corridor_anylogic_007 --split test

Output: data/seqs/<split>/<name>.mp4 and .npz (same layout as experiments/gen_data.py),
after which detect.py / evaluate_test.py run unchanged.
"""
import argparse
import sys
from pathlib import Path

import numpy as np
import pandas as pd

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from ptrack import FPS  # noqa: E402
from ptrack.data import DATA  # noqa: E402
from ptrack.render import Renderer, VideoWriter  # noqa: E402


def frames_from_csv(df):
    out = []
    phase = {}
    for f, g in df.groupby("frame", sort=True):
        sp = np.hypot(g.vx.values, g.vy.values)
        hd = np.stack([g.vx.values, g.vy.values], 1) / np.maximum(sp[:, None], 1e-6)
        hd[sp < 0.15] = (1.0, 0.0)
        ph = np.array([phase.get(i, 0.0) for i in g.id.values])
        ph = (ph + 2 * np.pi * sp / FPS / (0.75 * g.h.values)) % (2 * np.pi)
        phase.update(dict(zip(g.id.values, ph)))
        style = np.stack([g.shirt.values, g.pants.values,
                          g.id.values % 5, g.id.values % 7], 1).astype(int)
        out.append(dict(id=g.id.values.astype(int), x=g.x.values, y=g.y.values,
                        vx=g.vx.values, vy=g.vy.values, r=np.full(len(g), 0.225),
                        h=g.h.values, hx=hd[:, 0], hy=hd[:, 1], phase=ph, style=style,
                        speed=sp, waiting=np.zeros(len(g), np.int8)))
```

```
    return out


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("csv")
    ap.add_argument("--scenario", required=True)
    ap.add_argument("--name", required=True)
    ap.add_argument("--split", default="test")
    a = ap.parse_args()
    frames = frames_from_csv(pd.read_csv(a.csv))
    out = DATA / "seqs" / a.split
    out.mkdir(parents=True, exist_ok=True)
    rend = Renderer(a.scenario, seed=0)
    vw = VideoWriter(out / f"{a.name}.mp4")
    G = {k: [] for k in ("frame", "id", "x1", "y1", "x2", "y2",
                         "vis", "inside", "wx", "wy", "depth")}
    for f, fr in enumerate(frames, 1):
        img, gt = rend.render(fr)
        vw.write(img)
        n = len(gt["id"])
        G["frame"].append(np.full(n, f)); G["id"].append(gt["id"])
        for j, k in enumerate(("x1", "y1", "x2", "y2")):
            G[k].append(gt["box"][:, j])
        for k in ("vis", "inside", "wx", "wy", "depth"):
            G[k].append(gt[k])
    vw.close()
    np.savez_compressed(out / f"{a.name}.npz",
                        **{k: np.concatenate(v) for k, v in G.items()},
                        n_frames=len(frames), fps=FPS, scn=a.scenario,
                        dens="anylogic", seed=0)
    print("wrote", out / f"{a.name}.mp4", len(frames), "frames")


if __name__ == "__main__":
    main()
```

A.4 Closed-Loop Connection with Alpyne

For closed-loop use, an AnyLogic Reinforcement Learning experiment is exported and driven from Python with Alpyne (Listing A.3). The configuration (scenario, arrival rate, seed) is set at reset; the model runs until its stop condition, a 30 s warm-up plus a 60 s (static) or 90 s (dynamic-density) recording, i.e. 90 s or 120 s of model time, and reports the path of the trajectory file as observation. As listed, the configuration does not set dynamicDensity, rateLow and rateHigh, so the template generates static-density sequences; dynamic-density runs require these parameters in the configuration and in the reset call of Listing A.3, and spaceHeightM must be set to the height of the selected scenario's space (Table A.1). Because the agent of this thesis acts on the perception pipeline and not on the pedestrians, each AnyLogic run is generated once and reused for many reinforcement-learning episodes.

Listing A.3. Alpyne bridge template (code/anylogic/alpyne_bridge.py).

```
"""Bridge between an exported AnyLogic RL experiment and the RL environments.

Template for Alpyne (https://github.com/the-anylogic-company/Alpyne).  The
```

```
AnyLogic model is exported via its Reinforcement Learning experiment
("Export to Alpyne"); each `reset(**configuration)` runs one pedestrian
scenario, whose trajectory CSV is converted to video + ground truth, run
through YOLO once, and then served to `ContextualTuningEnv` /
`AdaptiveTuningEnv` for many episodes (the agent's detector/tracker parameters
do not affect the pedestrians, so the expensive stages are memoised).
"""
import subprocess
import sys
from pathlib import Path

HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE.parent))


def generate_with_anylogic(model_jar, scenario_id, scenario_name, rate, seed,
                           split="train"):
    from alpyne.sim import AnyLogicSim           # pip install alpyne (AnyLogic 8.8+)
    sim = AnyLogicSim(model_jar, engine_overrides=dict(seed=seed))
    status = sim.reset(scenario=scenario_id, arrivalRate=rate, seed=seed)
    while not status.stop:                       # the model has no actions; just advance
        status = sim.take_action()
    csv = status.observation["csvPath"]
    name = f"{scenario_name}_al{int(rate * 10):02d}_{seed:03d}"
    subprocess.run([sys.executable, str(HERE / "anylogic_to_ptrack.py"), csv,
                    "--scenario", scenario_name, "--name", name, "--split", split],
                   check=True)
    subprocess.run([sys.executable, str(HERE.parent / "experiments" / "detect.py"),
                    "--model", "yolov8s", "--splits", split], check=True)
    return split, name


if __name__ == "__main__":
    print(generate_with_anylogic("ModelExported/model.jar", 1, "corridor", 1.6, 7))
```

Appendix B Software Structure and Reproducibility

The complete source code accompanies this thesis. Table B.1 lists the modules and Table B.2 the experiment and analysis scripts in the order in which they are executed. Every random process (simulation, rendering, data splits, optimisers and PPO) is seeded, and all result files referred to in Chapter 5 are produced by these scripts.

Table B.1. Library modules (package ptrack).

Module	Content
pedsim.py	social-force simulator of the four AnyLogic scenarios (elliptical specification II, evasion, contacts, navigation)
render.py	virtual surveillance camera, articulated pedestrian figures, instance-ID ground truth, H.264 writer
bytetrack.py	ByteTrack re-implementation with exposed parameters (verified against the official code)
metrics.py	HOTA, CLEAR-MOT and Identity metrics (verified bit-exact against TrackEval)
pipeline.py	parameter vector, detection-to-tracking run with optional online controller, observation features
rl_env.py	Gymnasium environments for the contextual and the adaptive formulation, streaming CLEAR reward
data.py	loading of sequences, ground truth and cached detections; post-hoc NMS

Table B.2. Experiment and analysis scripts (code/experiments, code/figures).

Script	Purpose
calib_fd.py, calib_sf.py	calibration of the social-force parameters (Weidmann fundamental diagram, bounce and stuck statistics)
gen_data.py	simulation, rendering and ground truth of all 256 sequences
yolo_dataset.py, train_yolo.py	YOLO dataset export, zero-shot evaluation and transfer learning
detect.py, det_eval.py	detection on all videos (cached with NMS keep-masks), detection AP
run_defaults.py	default-parameter reference scores (reward baselines)
sensitivity.py	one-at-a-time sweeps and two-dimensional response surfaces
hpo_baselines.py	grid search, random search, TPE, CMA-ES and global TPE
train_rl_contextual.py, train_rl_adaptive.py	PPO training of PPO-contextual and PPO-adaptive (formulations A and B)
evaluate_test.py, detector_study.py, adaptive_trace.py	final evaluation on the unseen test splits
timing.py, timing_yolo.py	run-time measurements of the tracking pipeline and of YOLOv8 inference speed
figures/fig_data.py	dataset statistics (results/dataset_stats.json) and dataset figures
figures/fig_detection.py, fig_results.py, fig_qualitative.py, results_facts.py	result figures and tables of Chapter 5 (results/tables), paired Wilcoxon signed-rank tests with Holm correction
tests/*.py	metric cross-checks against TrackEval and py-motmetrics; smoke test of the RL environments (streaming and batch CLEAR counts)